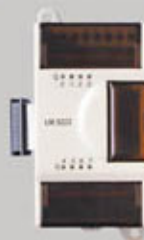




HOLLiAS-LEC G3系列小型一体化PLC

软件手册

北京和利时系统工程股份有限公司
杭州和利时自动化有限公司 PLC事业部



北京和利时系统工程股份有限公司
杭州和利时自动化有限公司 PLC事业部

地 址：北京市海淀区西三旗建材城中路10号
邮 编：100096
电 话：(010)82922200-2431/2430/2428/2425
传 真：(010)82928687/82923980
网 址：<http://www.hollysys.com>
技术支持邮箱：PLC@hollysys.com

前言

简介

PowerPro 软件是和利时公司专为 HOLLiAS-LEC G3 系列小型一体化 PLC 所开发的基于 Windows 的编程工具。PowerPro 软件具有控制方案编辑功能和仿真调试功能，是 HOLLiAS-LEC G3 系列小型一体化 PLC 的硬件配置和软件编程的标准软件包。

使用标准

PowerPro 软件集成了 IEC61131-3 的 5 种标准编程语言，运行在 Windows 操作系统下，并与 Windows 的图形界面和面向对象的操作原理相匹配。

手册信息

- 软件的安装指南
- 软件的使用入门
- 软件的编程环境
- 创建和管理工程
- IEC 编程基础
- 变量的声明与分类
- 库的概念与制作
- 管理 POU
- 视图与报警的配置
- 与显示屏的连接
- 程序实例

应用范围

本手册的内容适用于 PowerPro V2.0 软件。

如何使用本手册

如果初次使用 PowerPro 软件，需要通读手册。对于有经验的用户，可以通过目录查找相关信息。

在线帮助

集成在软件中的在线帮助是本手册的补充。

相关信息

- 安装光盘提供 PowerPro 编程软件。
- 硬件信息请参考《HOLLiAS-LEC G3 系列小型一体化 PLC 硬件手册》。
- 指令信息请参考《HOLLiAS-LEC G3 系列小型一体化 PLC 指令与功能块手册》。
- 有关技术支持和产品订货的相关事宜，请与和利时公司联系。

版权申明

本手册内容，包括文字、图表、标志、标识、商标、产品型号、软件程序和版面设计等，均受《中华人民共和国著作权法》、《中华人民共和国商标法》和《中华人民共和国专利法》的保护，并受与之相适用的国际公约中有关著作权、商标权、专利权或其他财产所有权法律的保护，为北京和利时系统工程股份有限公司专属所有或持有。

本手册仅供商业用户阅读和查询。在未得到北京和利时系统工程股份有限公司特别授权的情况下，无论出于何种原因和目的，均不得用任何电子或机械的方法，以任何形式复制和传递本手册的内容。否则本公司将依法追究法律责任。

我们已经核对本手册中的内容和图表与所述的硬件设备相符，但是误差难以避免，并不能保证完全一致。同时，我们会定期对手册的内容和图表进行检查、修改与维护，恕不另行通知。

北京和利时系统工程股份有限公司保留全部权利。

1993-2005 Copyright HollySys

HOLLiAS-LEC、HollySys、和利时



的字样和徽标均为北京和利时系统工程股份有限公司的商标或注册商标。

Microsoft 和 Windows 是微软公司在美国和/或其他国家分支机构的商标或注册商标。

手册中涉及到的其他商标或注册商标属于他们各自的拥有者。

联系方式

北京和利时系统工程股份有限公司

杭州和利时自动化有限公司

北京市海淀区西三旗建材城中路 10 号

邮政编码：100096

电话：(+86) 010-82922200

传真：(+86) 010-82928687

网址：<http://www.hollysys.com>

技术服务邮箱：PLC@hollysys.com

版本：2005 年 12 月

目 录

第 1 章	安装指南	1
1.1	软件安装	1
1.2	软件卸载	8
1.3	安装目标	8
1.4	注意事项	11
第 2 章	使用入门	12
2.1	硬件连接	12
2.2	启动软件	13
2.3	新建工程	14
2.4	设置通讯参数	16
2.5	编写程序	18
2.6	编译	24
2.7	在线调试	25
2.8	仿真调试	27
第 3 章	编程环境	28
3.1	主界面	28
3.1.1	标题栏	29
3.1.2	对象组织器	30
3.1.3	工作区域	30
3.1.4	消息窗口	31
3.1.5	状态栏	31
3.2	菜单列表	31
3.2.1	文件菜单	31
3.2.2	编辑菜单	32
3.2.3	工程菜单	33
3.2.4	插入菜单	36
3.2.5	其它菜单	37
3.2.6	在线菜单	39
3.2.7	窗口菜单	40
3.2.8	帮助菜单	41
3.3	快捷工具	41
3.3.1	文件工具	41
3.3.2	调试工具	41
3.3.3	编辑工具	42
3.3.4	编程工具	42
3.4	对象组织器	43
3.4.1	程序	43

3.4.2	数据类型	44
3.4.3	视图	45
3.4.4	资源	45
第 4 章	创建工程	46
4.1	目标设置	46
4.2	创建主程序	49
4.3	硬件模块配置	50
4.3.1	配置CPU模块	50
4.3.2	添加扩展模块	51
4.3.3	插入扩展模块	52
4.4	扩展模块参数设置	52
4.4.1	数字量模块	54
4.4.2	模拟量输入模块	55
4.4.3	模拟量输出模块	57
4.5	保存工程	59
第 5 章	IEC编程基础	60
5.1	POU的基本概念	60
5.1.1	POU的类型	60
5.1.2	POU的调用	61
5.1.3	POU的组成	61
5.1.4	主程序PLC_PRG	62
5.2	梯形图LD	62
5.2.1	光标位置	63
5.2.2	操作说明	63
5.3	功能块图FBD	66
5.3.1	光标位置	66
5.3.2	操作说明	67
5.4	指令列表IL	70
5.4.1	操作说明	70
5.4.2	程序举例	70
5.5	结构化文本ST	73
5.5.1	ST表达式	73
5.5.2	ST指令	73
5.6	顺序功能图SFC	79
5.6.1	基本概念	79
5.6.2	操作说明	82
5.7	连续功能图CFC	86
5.7.1	CFC编辑器	86
5.7.2	操作说明	86
5.8	IEC运算符	89
第 6 章	变量	90
6.1	操作数	90

6.1.1	常量	90
6.1.2	变量	92
6.2	变量的声明	92
6.2.1	声明编辑器	92
6.2.2	声明格式	94
6.2.3	AT声明	95
6.2.4	初始化	96
6.2.5	自动声明变量	96
6.2.6	表格声明变量	97
6.3	变量的数据类型	98
6.3.1	标准数据类型	98
6.3.2	自定义数据类型	100
6.4	全局变量的声明	102
6.4.1	在全局变量中声明	103
6.4.2	在变量配置中声明	103
6.4.3	在PLC配置中的声明	104
6.5	局部变量的声明	104
6.5.1	声明中间变量	104
6.5.2	声明输入变量	104
6.5.3	声明输出变量	105
6.5.4	声明输入/输出变量	105
6.5.5	声明保留型变量	105
6.5.6	声明常数	105
第7章	库	106
7.1	库的概念	106
7.1.1	库管理器	106
7.1.2	添加库	107
7.1.3	删除库	108
7.2	库的分类	109
7.2.1	标准库	110
7.2.2	应用库	111
7.2.3	系统库	112
7.2.4	IEC动作库	112
7.3	库的制作	113
第8章	管理POU	116
8.1	管理POU菜单	116
8.1.1	建立文件夹	116
8.1.2	转换语言	117
8.2	创建POU	118
8.2.1	创建程序	118
8.2.2	创建函数	118
8.2.3	创建动作	120
8.3	调用POU	121

8.3.1	调用程序	122
8.3.2	调用功能块	123
8.3.3	调用函数	125
8.4	任务配置	128
8.4.1	配置任务	128
8.4.2	系统事件	129
8.4.3	任务调用程序	133
第 9 章	管理工程	134
9.1	管理工程菜单	134
9.1.1	打印工程文件	135
9.1.2	导入导出工程	137
9.1.3	合并工程	138
9.1.4	比较工程	139
9.1.5	用户口令	140
9.2	工程选项设置	144
9.2.1	下装与保存	145
9.2.2	用户信息	146
9.2.3	编辑器	146
9.2.4	窗口	148
9.2.5	颜色	149
9.2.6	目录	149
9.2.7	日志	150
9.2.8	编译	151
9.2.9	口令	152
第 10 章	编译与调试	154
10.1	编译	154
10.2	下装	155
10.2.1	设备安装与连接	155
10.2.2	建立通信连接	156
10.2.3	程序下装	157
10.3	显示参考数据	158
10.3.1	查看实例	158
10.3.2	查看调用树	159
10.3.3	查看交叉引用列表	159
10.3.4	查看	160
10.4	调试	161
10.4.1	进入调试状态	161
10.4.2	退出调试状态	162
10.4.3	运行程序	162
10.4.4	停止程序	162
10.4.5	复位	162
10.4.6	断点	163
10.4.7	单步	165

10.4.8	单循环.....	165
10.4.9	变量输入值.....	165
10.4.10	变量强制值.....	165
10.4.11	查看调用栈.....	167
10.4.12	显示流控制.....	167
10.4.13	监视与接收管理器.....	168
第 11 章 视图.....		170
11.1	创建视图文件.....	170
11.2	视图编辑工具.....	172
11.3	视图编辑方法.....	174
11.3.1	绘制视图.....	174
11.3.2	布置视图.....	174
11.3.3	对象列表.....	177
11.3.4	使用键盘.....	178
11.3.5	占位符列表.....	179
11.3.6	视图设置.....	179
11.4	视图属性配置.....	181
11.4.1	属性配置方法.....	181
11.4.2	视图对象的属性.....	182
11.5	视图静态属性.....	183
11.5.1	形状.....	183
11.5.2	文本.....	183
11.5.3	线宽.....	185
11.5.4	颜色.....	185
11.5.5	工具提示文本.....	186
11.5.6	安全属性.....	186
11.5.7	位图属性.....	187
11.5.8	视图属性.....	188
11.5.9	组框架属性.....	189
11.5.10	角度.....	190
11.6	视图编程.....	191
11.6.1	编程属性.....	191
11.6.2	视图库.....	192
11.7	视图动态属性.....	195
11.7.1	文本变量.....	195
11.7.2	颜色变量.....	195
11.7.3	绝对运动.....	196
11.7.4	相对运动.....	196
11.7.5	变量.....	197
11.7.6	输入.....	197
11.8	表格.....	198
11.9	趋势图.....	199
11.10	报警表.....	200

11.11	ActiveX控件	201
11.12	视图举例	202
第 12 章 报警配置		204
12.1	报警概述	204
12.2	报警类	205
12.3	报警组	210
12.4	报警文件	212
12.5	报警举例	213
第 13 章 连接显示屏		216
13.1	连接EasyView显示屏	216
13.1.1	物理连接	216
13.1.2	PLC侧参数设置	216
13.1.3	显示屏侧参数设置	217
13.1.4	地址映射	218
13.2	连接Proface显示屏	220
13.2.1	物理连接	220
13.2.2	PLC侧参数设置	220
13.2.3	显示屏侧参数设置	221
13.3	连接Hitech显示屏	224
13.3.1	物理连接	224
13.3.2	PLC侧参数设置	224
13.3.3	显示屏侧参数设置	225
13.4	连接GT01 显示屏	227
13.4.1	物理连接	227
13.4.2	PLC侧参数设置	227
13.4.3	显示屏侧参数设置	228
13.5	连接POP显示屏	229
13.5.1	物理连接	229
13.5.2	PLC侧参数设置	230
13.5.3	显示屏侧参数设置	230
第 14 章 程序实例		232
14.1	LD程序实例	232
14.2	FBD程序实例	234
14.3	SFC程序实例	237
第 15 章 附录		242
15.1	PLC的基本概念	242
15.1.1	PLC的工作过程	242
15.1.2	CPU的扫描周期	243
15.1.3	CPU的工作方式	244
15.1.4	PLC的运行方式	244
15.1.5	PowerPro的工作方式	244

15.1.6	PowerPro存储区分配	245
15.1.7	PowerPro数据存储格式	245
15.1.8	采样滤波	246
15.1.9	中断	246
15.2	PowerPro输入提示功能	247
15.3	PowerPro文件存储目录	248
15.4	PowerPro使用注意事项	249
15.5	PowerPro键盘命令	251
15.6	扩展模块参数设置	253
15.6.1	4 通道热电偶输入模块LM3311	253
15.6.2	4 通道热电阻输入模块LM3312	255
15.6.3	8 通道模拟量输入模块LM3313	256
15.6.4	8 通道热敏电阻输入模块LM3314	257
15.6.5	Profibus-DP从站模块LM3401	258
15.6.6	以太网模块LM3403	263
15.7	常见问题解答	265

第1章 安装指南

本章主要介绍 PowerPro V2.0 软件的安装过程，并对目标安装进行详细的描述。

1.1 软件安装

在装有中文 Windows2000 操作系统及其 SP4 的计算机上，将 PowerPro V2.0 软件安装光盘插入光驱，在以下路径中找到安装程序：\PowerPro\Setup.exe。双击打开此安装程序，出现 PowerPro V2.0 安装界面，点击“Next”按钮，如图 1-1-1 所示。

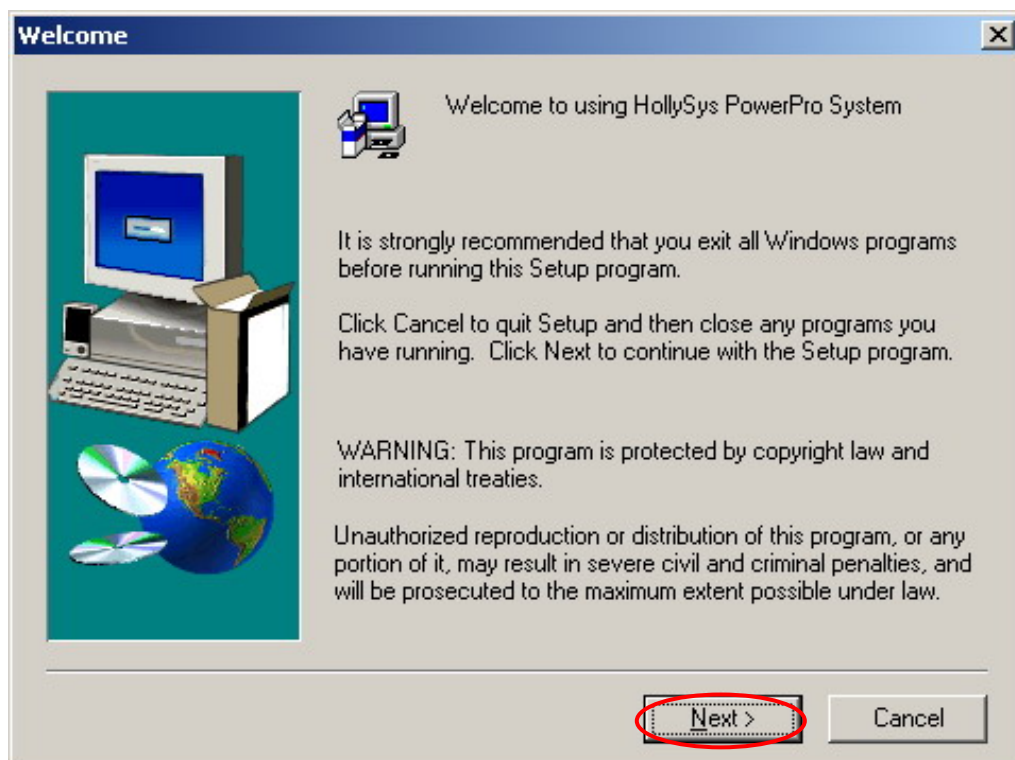


图 1-1-1 安装步骤（1）

同意版权协议，点击“**Yes**”按钮，如图 1-1-2 所示。

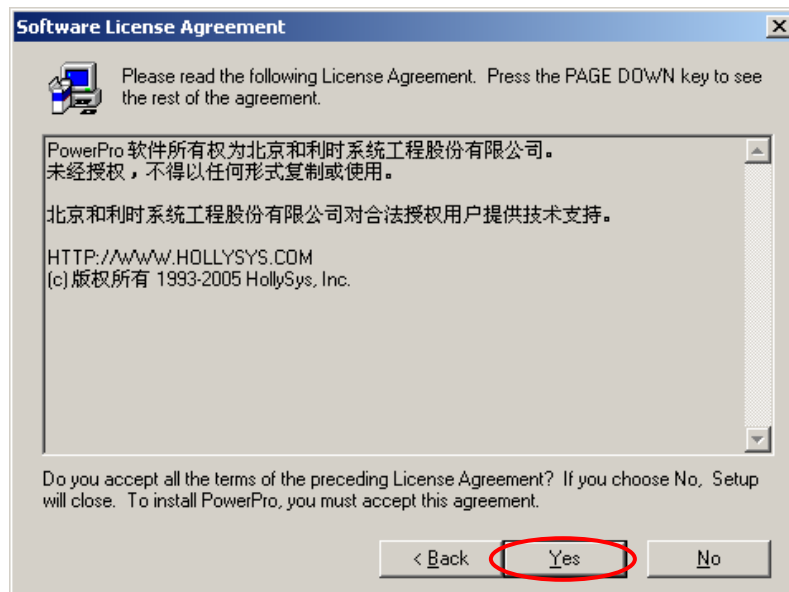


图 1-1-2 安装步骤 (2)

选择安装路径。默认的安装路径为 D:\Hollysys\PowerPro，建议不要修改。如果需要修改，可点击“Browse”按钮选择其它的路径。点击“Next”按钮开始安装，如图 1-1-3 所示。

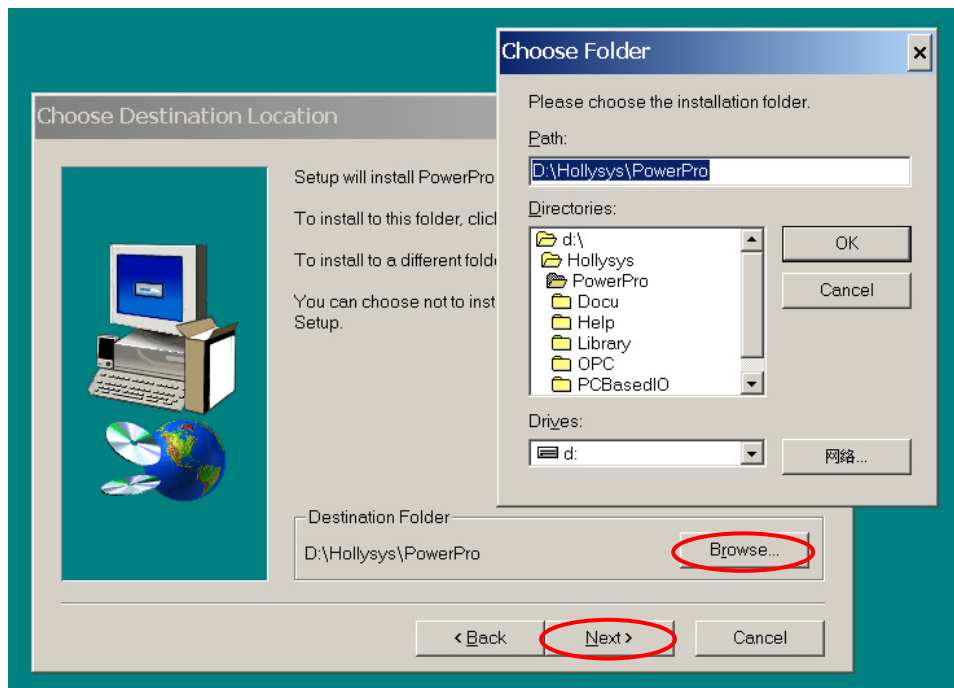


图 1-1-3 安装步骤 (3)

界面提示安装 MSXML 3.0，点击“确定”按钮，如图 1-1-4 所示。



图 1-1-4 安装步骤（4）

进入 MSXML 3.0 安装向导，点击“Next”按钮，如图 1-1-5 所示。

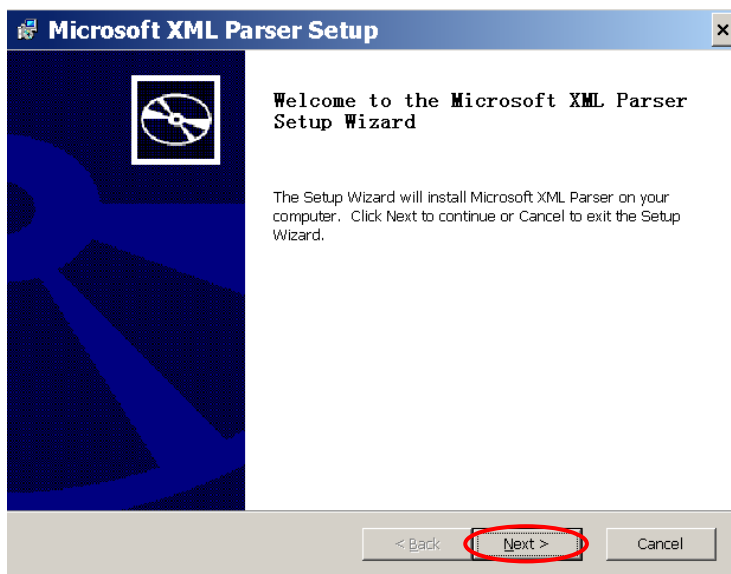


图 1-1-5 安装步骤（5）

同意版权协议，选择“I accept the terms in the License Agreement”，点击“Next”按钮，如图 1-1-6 所示。



图 1-1-6 安装步骤（6）

填写 User Name (用户名) 和 Organization (单位), 点击“Next”按钮, 如图 1-1-7 所示。

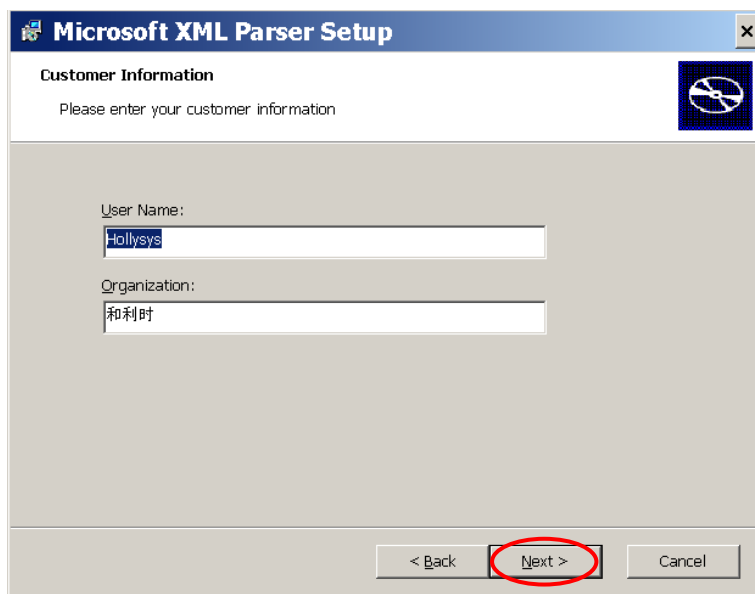


图 1-1-7 安装步骤 (7)

开始安装, 点击“Install”按钮, 如图 1-1-8 所示。

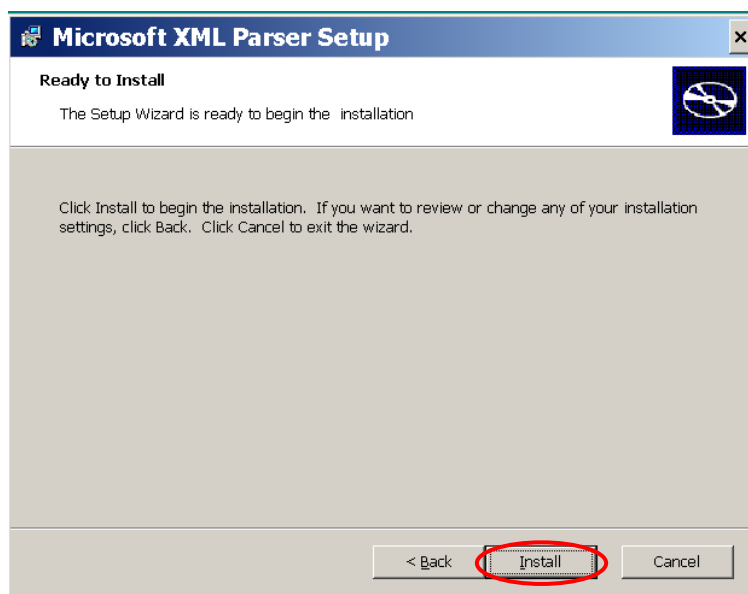


图 1-1-8 安装步骤 (8)

安装完成后，点击“Finish”按钮，关闭 MSXML 3.0 安装程序，如图 1-1-9 所示。

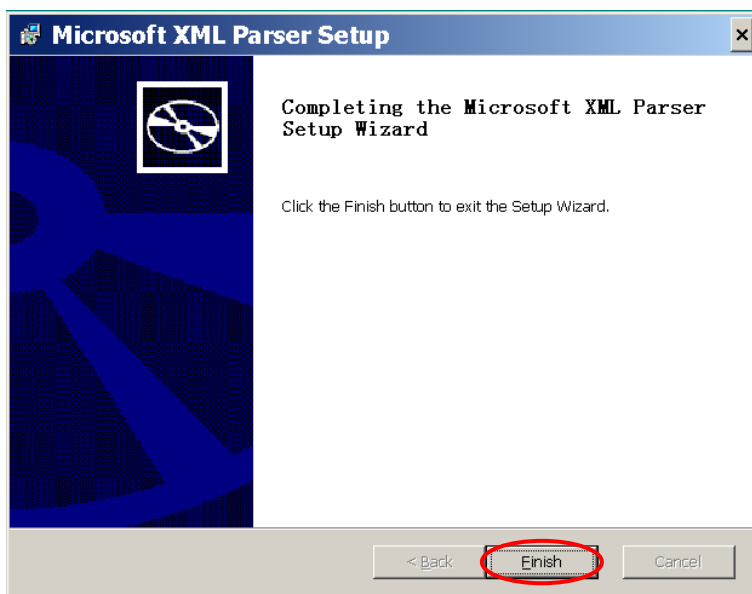


图 1-1-9 安装步骤（9）

然后，界面提示安装 MSXML 3.0 SP4，点击“确定”按钮，如图 1-1-10 所示。



图 1-1-10 安装步骤（10）

进入 MSXML 3.0 SP4 安装向导，点击“Next”按钮，如图 1-1-11 所示。

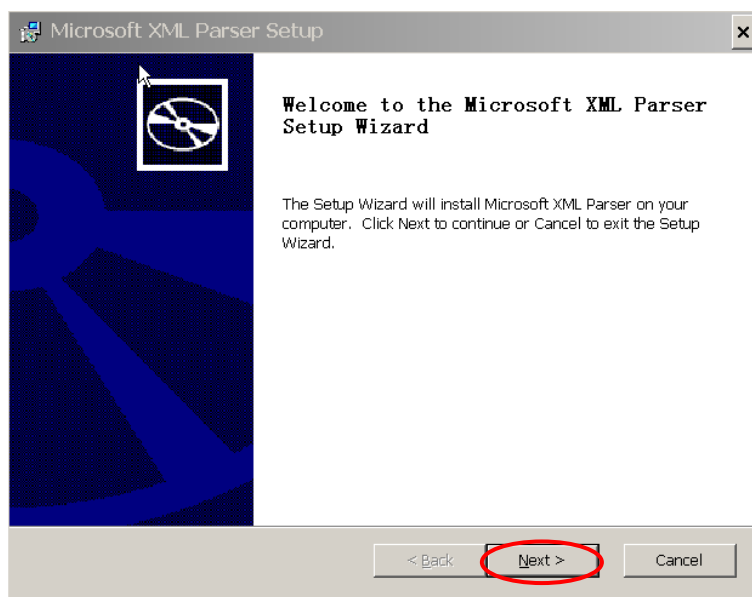


图 1-1-11 安装步骤（11）

同意版权协议，选择“I accept the terms in the License Agreement”，点击“Next”按钮，如图 1-1-12 所示。

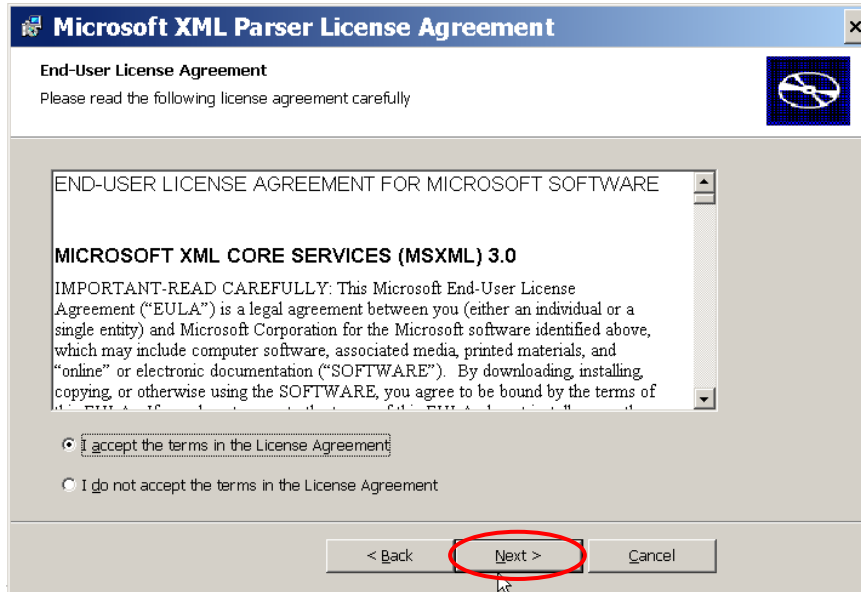


图 1-1-12 安装步骤（12）

填写 User Name（用户名）和 Organization（单位），点击“Next”按钮，如图 1-1-13 所示。

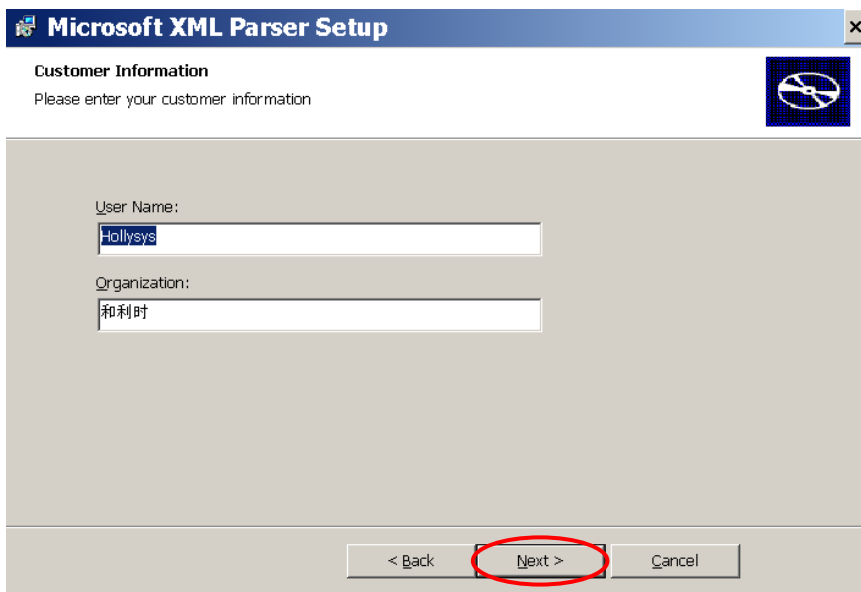


图 1-1-13 安装步骤（13）

开始安装，点击“Install”按钮，如图 1-1-14 所示。

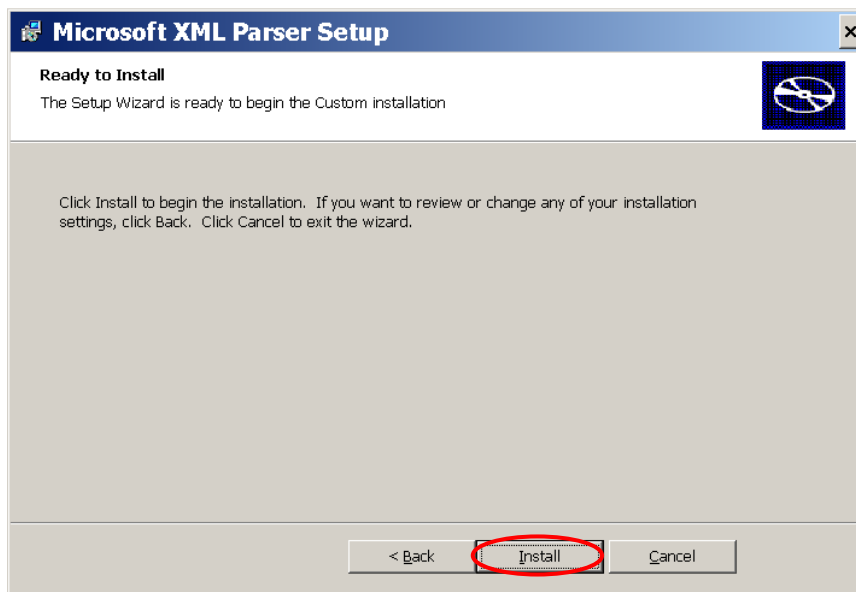


图 1-1-14 安装步骤（14）

安装完成后，点击“Finish”按钮，关闭 MSXML 3.0 SP4 安装程序，如图 1-1-15 所示。

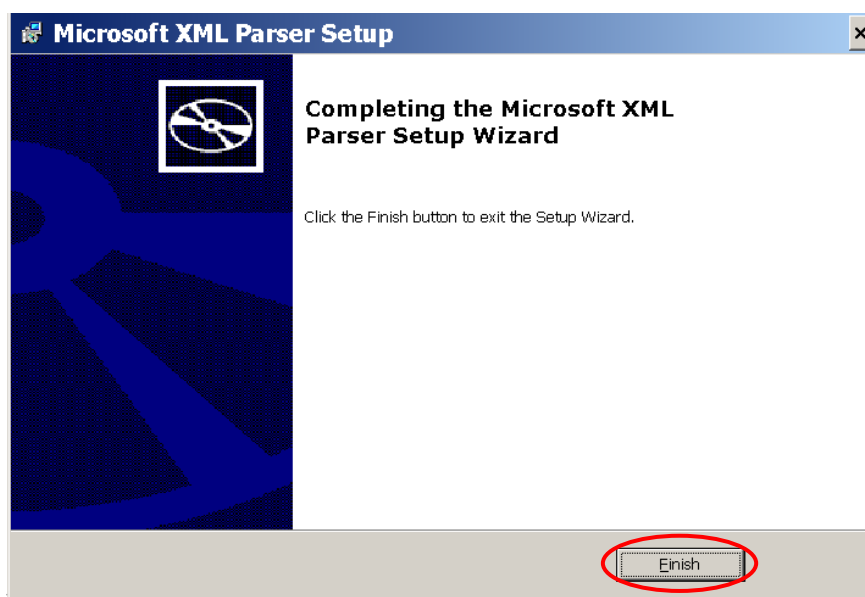


图 1-1-15 安装步骤（15）

最后点击“Finish”按钮，PowerPro V2.0 安装完毕，如图 1-1-16 所示。

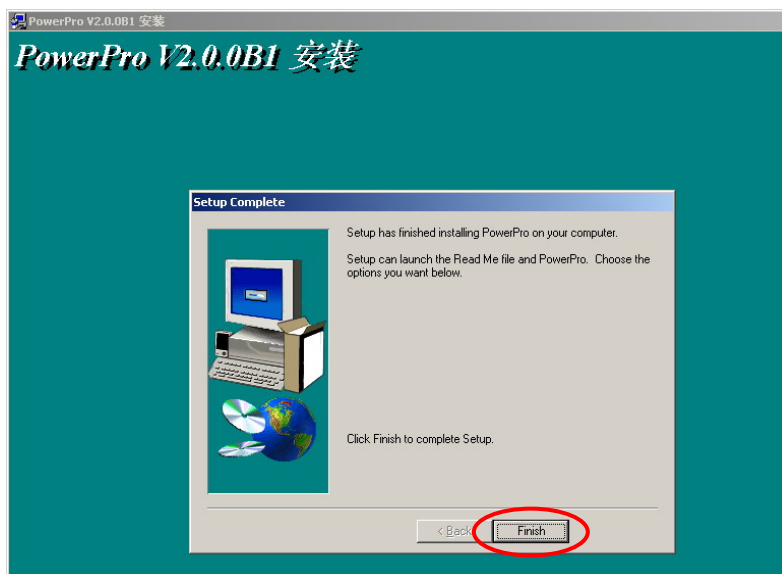


图 1-1-16 安装步骤（16）

1.2 软件卸载

如果计算机中已经安装了低版本的 PowerPro 软件，当安装新版本的 PowerPro 软件时，需要将旧版本的 PowerPro 软件卸载。在“控制面板”/“添加/删除程序”中选择 PowerPro，点击“更改/删除”便可对该程序进行卸载，如图 1-2-1 所示。

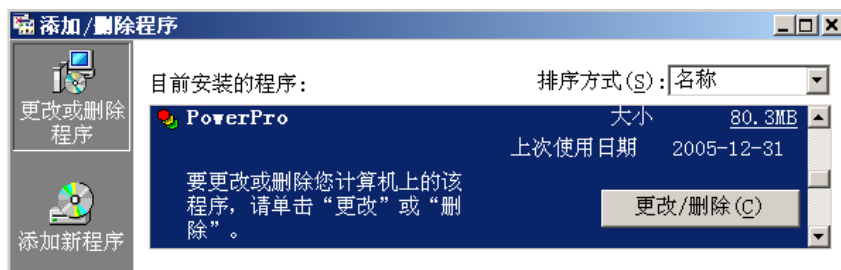


图 1-2-1 卸载程序

1.3 安装目标

PowerPro 软件是 PLC 控制方案的开发平台。在使用 PowerPro 软件之前，必须首先要进行“安装目标”，为 PLC 模块选择软件运行的平台。由于安装内容对所有工程通用，因此，使用 PowerPro 软件之前只需要进行一次“安装目标”即可。

“安装目标”的具体步骤如下所述。

在桌面点击“开始”/“程序”/“Hollysys PowerPro”/“安装目标”，如图 1-3-1 所示。



图 1-3-1 安装目标（1）

弹出“安装目标”窗口“InstallTarget”，如图 1-3-2 所示。点击“Open...”按钮，在弹出的窗口中选择其中的“C16x_hollysys.tnf”文件，点击“打开”按钮，窗口自动关闭。

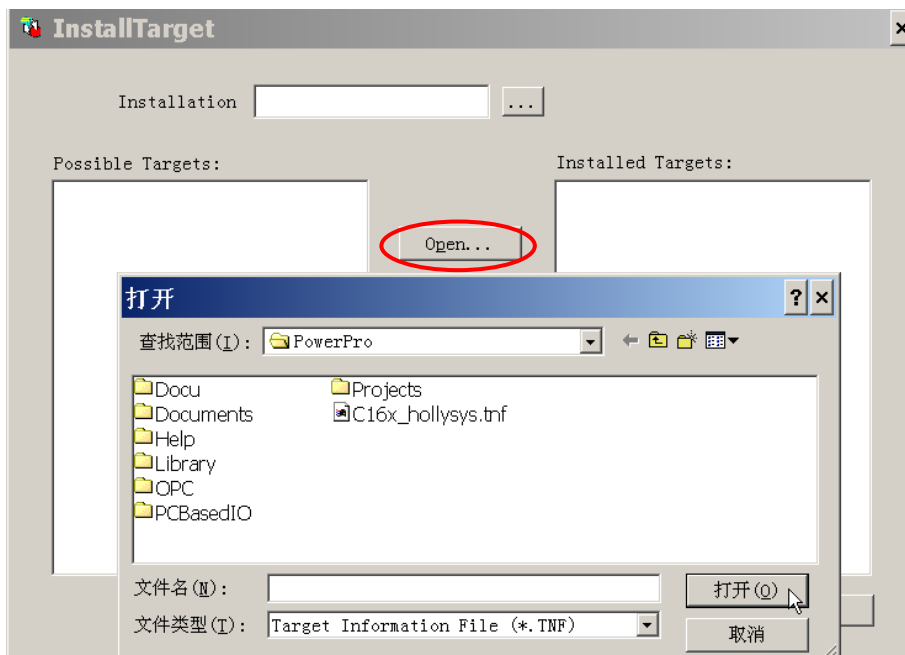


图 1-3-2 安装目标（2）

这时，在“InstallTarget”窗口左侧的可用目标“Possible Targets”窗口中，产生一个目标“Hollysys PLC”，双击该目标或点击该目标前面的加号可以将其打开，其中包括两项内容：

- HollySys CoDeSys SP for C16x 为 LM3104/LM3105/LM3106/LM3107 非扩容 CPU 模块所选用的目标设置。
- HollySys CoDeSys SP for C16x Extend 为 LM3106/LM3107/LM3108/LM3109 大容量 CPU 模块所选用的扩展目标设置。

选中该目标“Hollysys PLC”，点击“Install”按钮，如图 1-3-3 所示。

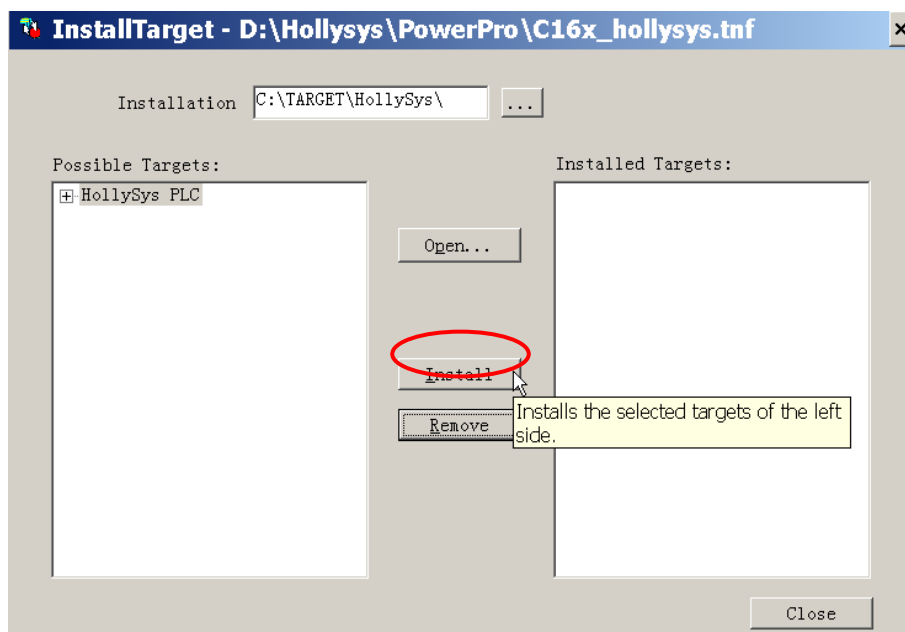


图 1-3-3 安装目标（3）

此时，在右侧已安装目标窗口“Installed Targets”中生成同样的“Hollysys PLC”目标文件。点击“Close”按钮，安装目标结束，关闭此窗口，如图 1-3-4 所示。

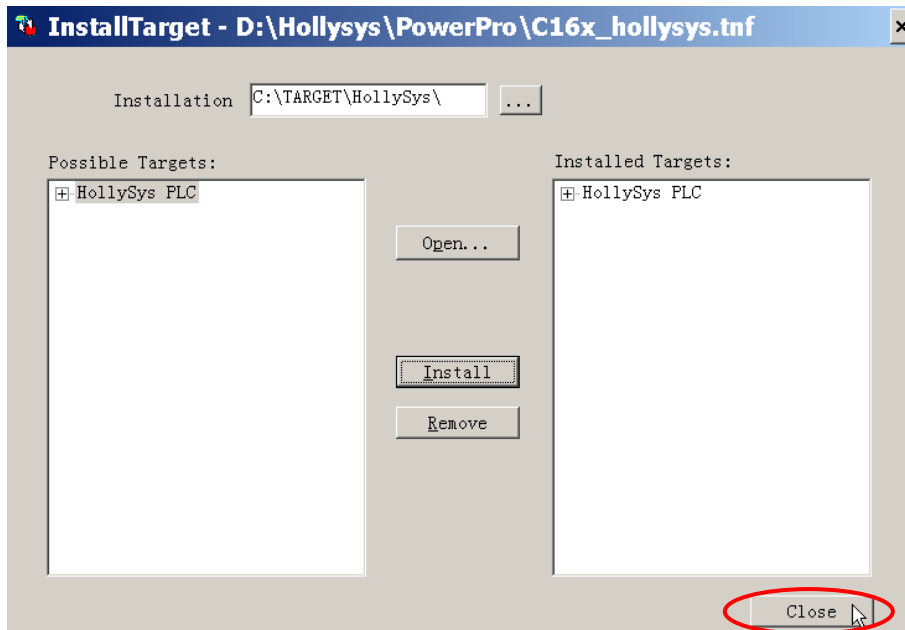


图 1-3-4 安装目标（4）

1.4 注意事项

- 正确安装中文 Windows2000 操作系统并安装 SP4。系统的分辨率设置为 1024*768，字体设置为小字体。
- 一定要按照步骤安装 MSXML3.0 及其补丁程序 MSXML3.0SP4。否则会导致一些不正常的运行情况发生，例如无法调用系统事件等。
- 如果在安装过程中出现错误提示，退出程序重新启动计算机后，再进行安装。
- 如果存在补丁程序，请按照相关说明安装补丁。
- 如果需要在同一台计算机上同时安装和利时公司的 SmartPro 和 PowerPro，应当先安装 SmartPro，然后再安装 PowerPro，否则 PowerPro 中的 COM 口不能通讯。如果先前安装了 SmartPro，则无须卸载该软件，可直接安装 PowerPro。
- PowerPro V2.0 首次安装完毕后，应执行的步骤是：在 Windows“开始”/“程序”菜单的 Hollysys PowerPro 程序组中执行“安装目标”；运行“控制方案生成系统”；在“工程”/“选项”的“Editor”中选择“自动声明”和“自动格式化”。
- 软件卸载前要注意，一定要先退出 Gateway.exe 程序，再进行软件的卸载。如果未退出 Gateway.exe 程序，或在卸载过程中由于出现问题而没有完全卸载，则需重启计算机。在桌面的右下角有此 Gateway.exe 程序的图标 ，点击右键，选择“Exit”即可退出该可执行程序。



本章小结

本章主要介绍了 PowerPro 软件的安装和卸载，并介绍了目标安装和一些注意事项。

第2章 使用入门

本章的主要内容是，通过编写简单的程序，对 PowerPro 的编程方式进行介绍，使初学者对 PowerPro 的编程有一个初步的认识。

本章示例程序的主要功能是，在一定的时间间隔内，使开关不断地进行交替通断。在编程前，首先需要确定硬件配置。对于此程序，无需太多的 I/O 点数，仅需配置一个 CPU 模块即可满足要求，这里选择带有 24 点 I/O 的 CPU 模块 LM3107。

2.1 硬件连接

➤ 所需设备

已经安装 PowerPro 软件且带有 RS232 串口的 PC 机一台。

CPU 模块 LM3107（这里选用扩容的 LM3107 模块）一个。

220V 交流电源线一根。

连接 PC 机与 CPU 模块的 RS232 通讯接口专用编程电缆一根。

➤ 选择 CPU 模块

选择 LM3107 模块，按照图 2-1-1 所示的方式连接电源线。注意，当电源线连接好之后，应该把端子盖扣好，以免造成不必要的人身伤害或设备损坏！

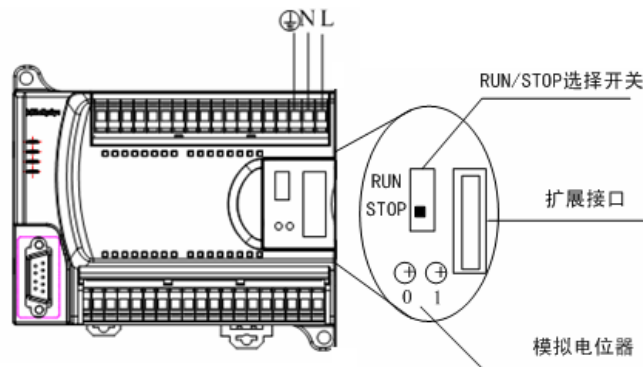


图 2-1-1 连接电源线

电源线连接之后，先不要接通电源。在检查所有电缆连接无误后，再接通电源，并确认 CPU 模块面板上的 RUN 指示灯点亮并显示正常，以保证 PLC 可靠运行。

HOLLiAS-LEC G3 系列 CPU 模块具有两种工作方式可供选择，具体设置由“RUN/STOP”选择开关来完成，详细说明见表 2-1-1 所示。

如果在 RUN 方式下，不能向 CPU 模块装载程序时，则必须把 CPU 模块置于 STOP 方式下才能进行装载。

表 2-1-1 “RUN/STOP 选择开关”设置说明

开关状态	说明
RUN	CPU 处于运行方式，执行用户程序
STOP	CPU 不执行用户程序，此时用户可以向 CPU 装载用户程序。

➤ 建立 PC 通讯

通过配套的编程电缆，将 CPU 模块连接到 PC 机，建立数据传递通道，如图 2-1-2 所示。由于 CPU 模块的 RS232 通讯接口是非隔离的，所以编程电缆的连接应该在 PLC 上电之前进行。

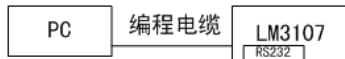


图 2-1-2 连接编程电缆

2.2 启动软件

如果第一次使用 PowerPro 软件，一定要进行目标安装。上一章已经介绍了目标安装的方法。

在 Windows 的“开始”/“程序”菜单中，点击“HollySys PowerPro”/“控制方案生成系统”，启动 PowerPro 软件，如图 2-2-1 所示。

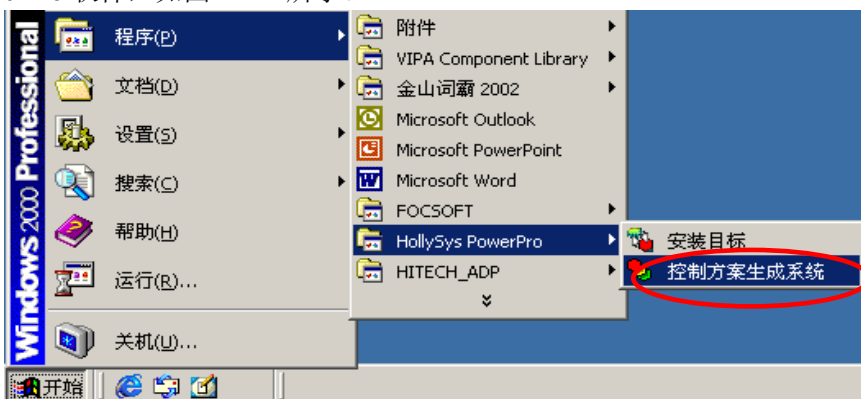


图 2-2-1 启动 PowerPro 软件

2.3 新建工程

➤ 目标设置

在主界面中点击“文件”/“新建”菜单，或在工具栏中点击“”按钮，随之出现“目标设置”对话框。在“配置”栏中选中“Hollysys CoDeSys SP for C16x Extend”，此目标为 LM3106/LM3107/LM3108/LM3109 大容量模块所选用的扩展目标设置。点击“确认”按钮，如图 2-3-1 所示。如果所使用的模块为非扩容模块，则需选择“Hollysys CoDeSys SP for C16x”，此目标为 LM3104/LM3105/LM3106/LM3107 非扩容模块所选用的目标设置。

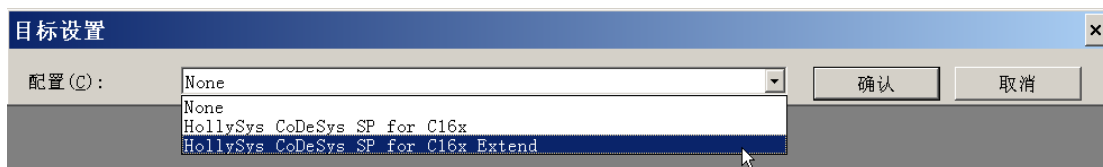


图 2-3-1 选择目标

随后弹出“目标设置”窗口，默认设置已能满足绝大多数应用需求，点击“确认”按钮即可，如图 2-3-2 所示。



图 2-3-2 目标设置界面

➤ 创建 POU

这里以“LD”语言为例，POU 语言选择“LD”，如图 2-3-3 所示。

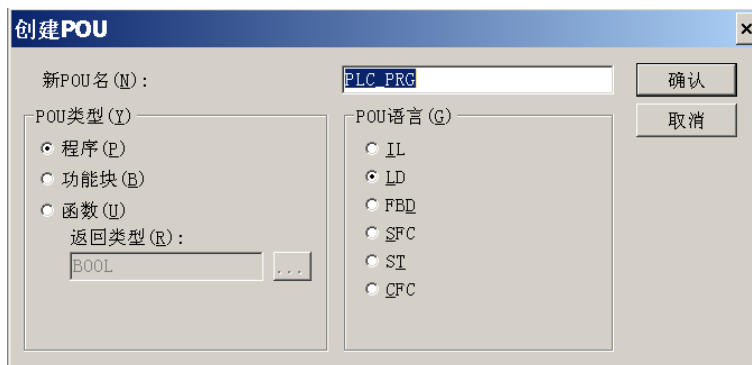


图 2-3-3 创建 POU

点击“确认”按钮，则会出现如图 2-3-4 所示界面。

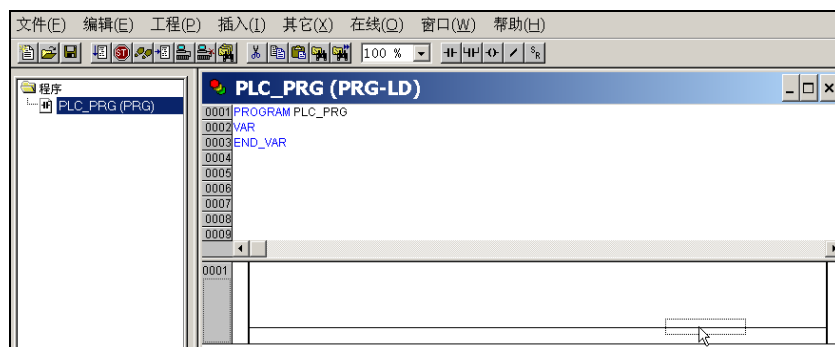


图 2-3-4 工作区域

➤ PLC 配置

本章示例程序以 LM3107 CPU 模块为例介绍 PLC 的配置。在“资源”选项卡中双击“PLC 配置”，在“PLC Configuration”上点击鼠标右键，选择“Append Subelement”中的“LM3107”，进行 PLC 的配置，如图 2-3-5 所示。

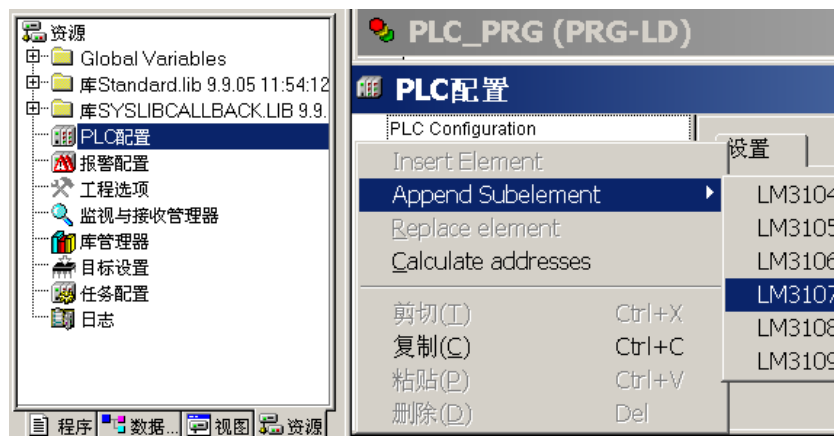


图 2-3-5 PLC 配置

2.4 设置通讯参数

在“在线”菜单中选定“通讯参数”，弹出“Communication Parameters”通讯参数对话框，如图 2-4-1 所示。

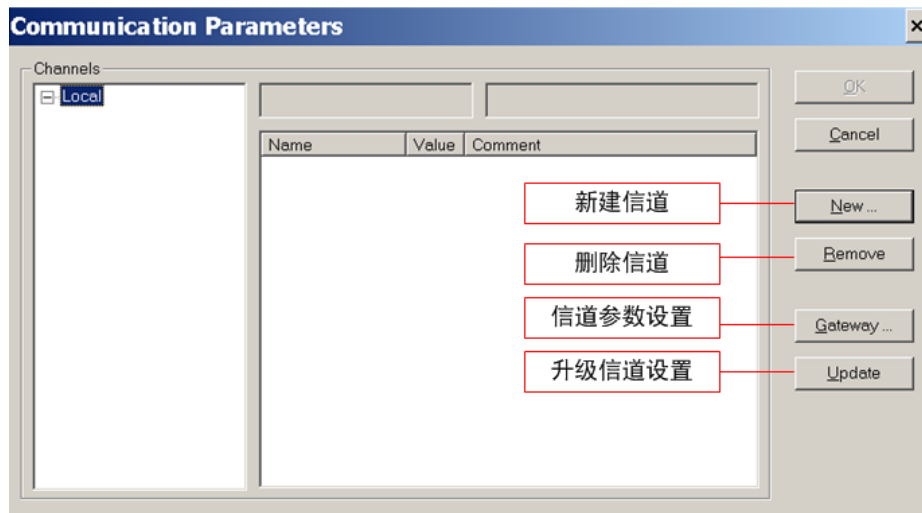


图 2-4-1 通讯参数对话框

点击“Gateway”按钮进入信道参数设置，确认“Connection”信道参数设置为“Local”，如图 2-4-2 所示。



图 2-4-2 通道参数设置

点击“OK”按钮后返回“Communication Parameters”通讯参数对话框，选择“New”按钮添加新信道，出现如图 2-4-3 所示的对话框。

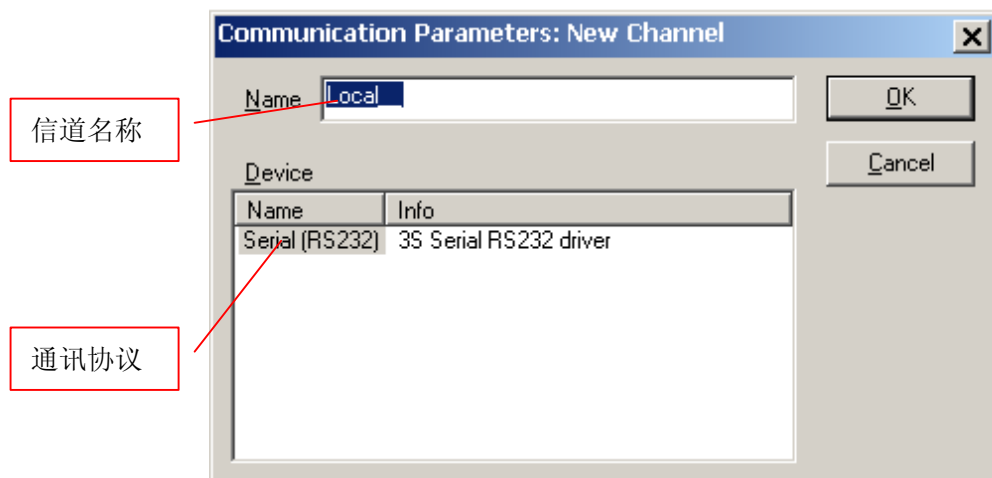


图 2-4-3 添加新信道

其中信道名称默认为“Local_”，通信协议使用缺省的 RS232 协议。点击“OK”按钮后返回“Communication Parameters”通讯参数对话框，如图 2-4-4 所示。连续双击“Baudrate”中“Value”的对应值，可以改变通讯速率，使其值设为“38400”，点击“OK”按钮确认。这样，本地计算机与 CPU 模块之间的通信连接便建立完成。

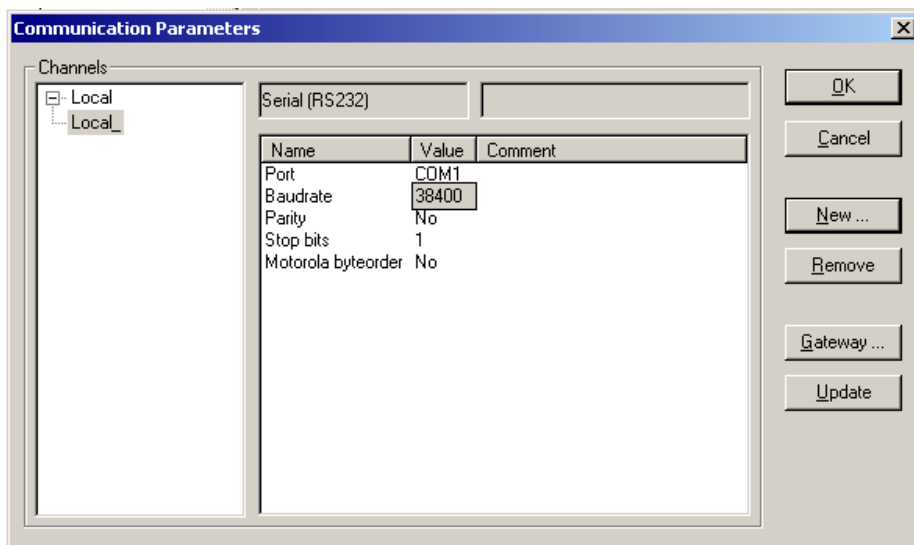


图 2-4-4 通讯速率设置

2.5 编写程序

PLC 配置完成后,可以开始进行 PLC 程序的编写。本例是一个简单的定时器应用程序。本程序主要目的是产生一个“1s 断 2s 通”的脉冲信号。

在工具栏里点击表示触点的“串联”按钮“”，便在程序区域内的 0001 节中出现了一个串联触点“”，如图 2-5-1 所示。

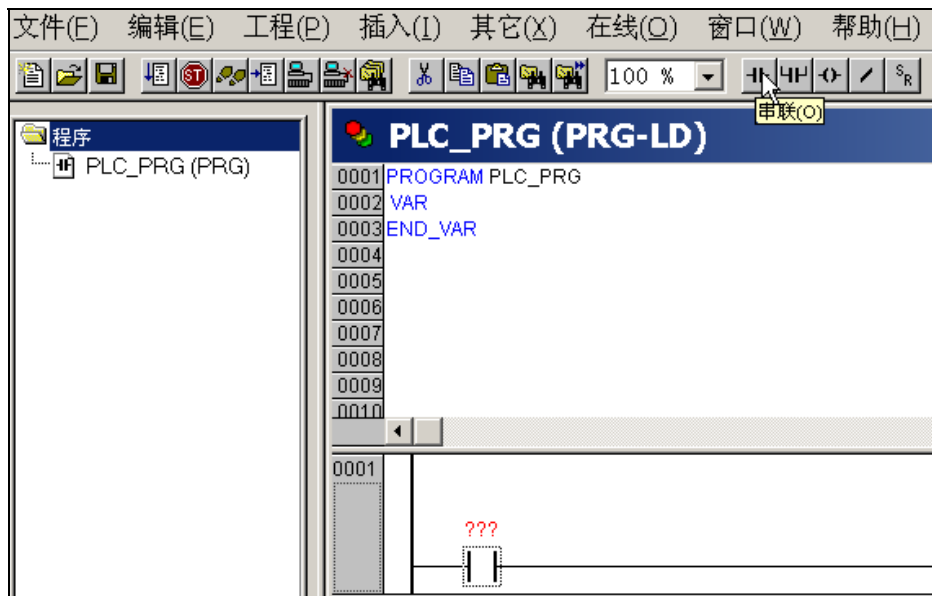


图 2-5-1 程序编写步骤 (1)

串联触点的标记文本缺省值为“???”。点击此文本,输入“I.0”,如图 2-5-2 所示。

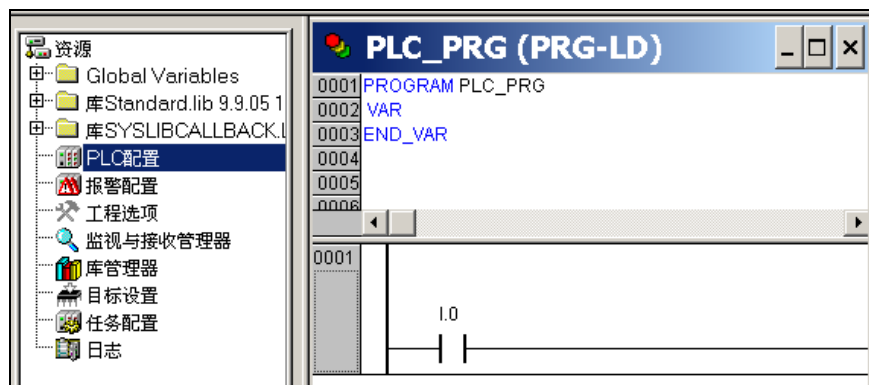


图 2-5-2 程序编写步骤 (2)

在“I.0”触点后点击鼠标右键，选择“功能块”，如图 2-5-3 所示。

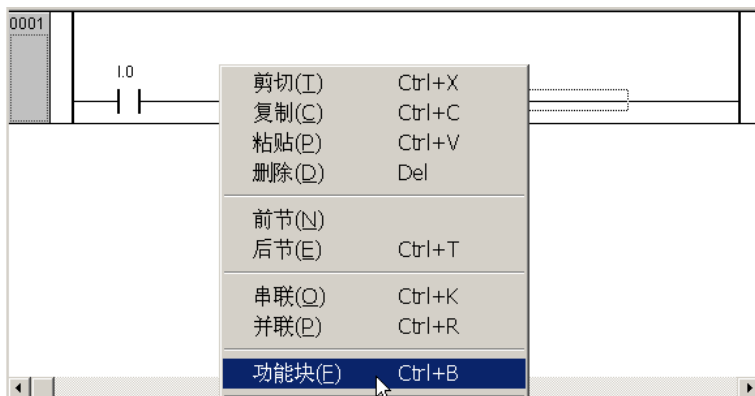


图 2-5-3 程序编写步骤（3）

点击“功能块”，则会弹出如图 2-5-4 所示的对话框，选择“TON（FB）”。

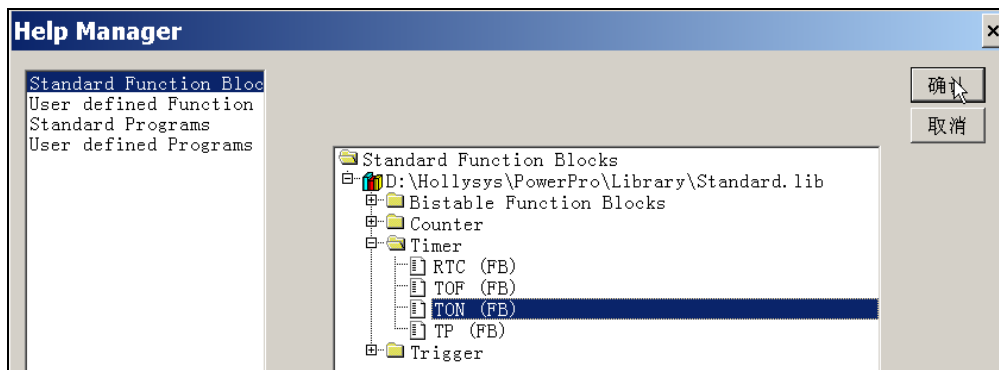


图 2-5-4 程序编写步骤（4）

双击“TON（FB）”，在“???”光标所在位置处，输入 T1 后回车，则会弹出如图 2-5-5 所示的对话框，选择默认类型为“TON”，点击“确认”按钮。

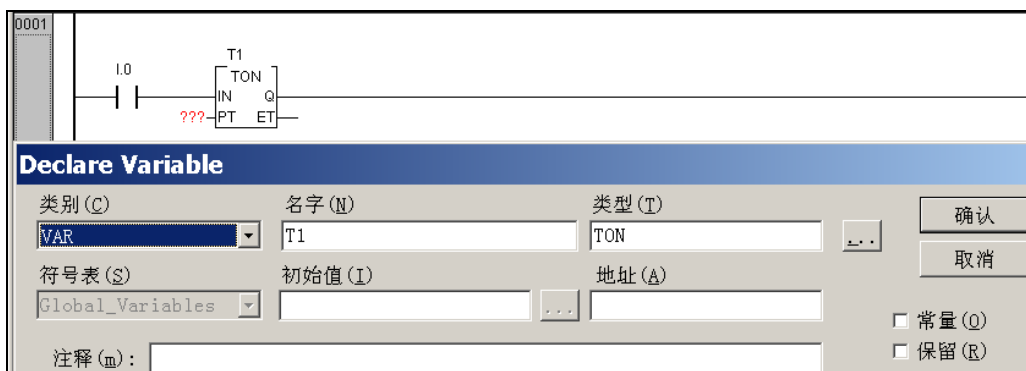


图 2-5-5 程序编写步骤（5）

则在梯形图中增加了一个以 T1 命名的通电延时定时器“TON”。在“PT”的标记文本“???”处，点击文本，输入表示延时 1S 的常量“T#1S”。在“ET”处输入变量“ET”，此变量为时间类型变量，类型选择时间类型“TIME”，如图 2-5-6 所示。点击“确认”按钮返回。

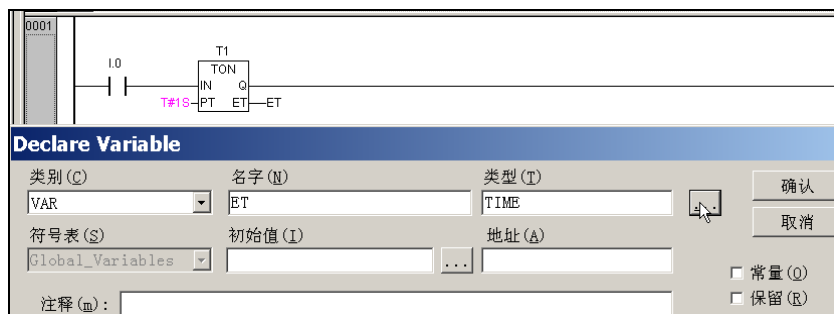


图 2-5-6 程序编写步骤（6）

当光标位置位于“T1”后时，在工具栏中选择表示输出线圈的按钮“”，则会在光标处出现如图 2-5-7 所示的输出线圈“”。

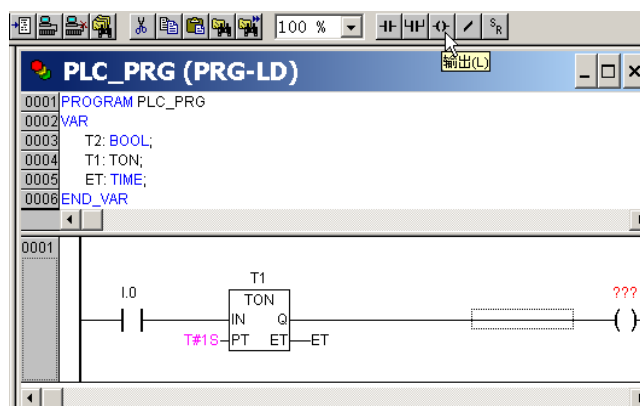


图 2-5-7 程序编写步骤（7）

在相应的标记文本“???”处填入变量名“M”，其类型为“BOOL”，如图 2-5-8 所示。点击“确认”按钮返回。

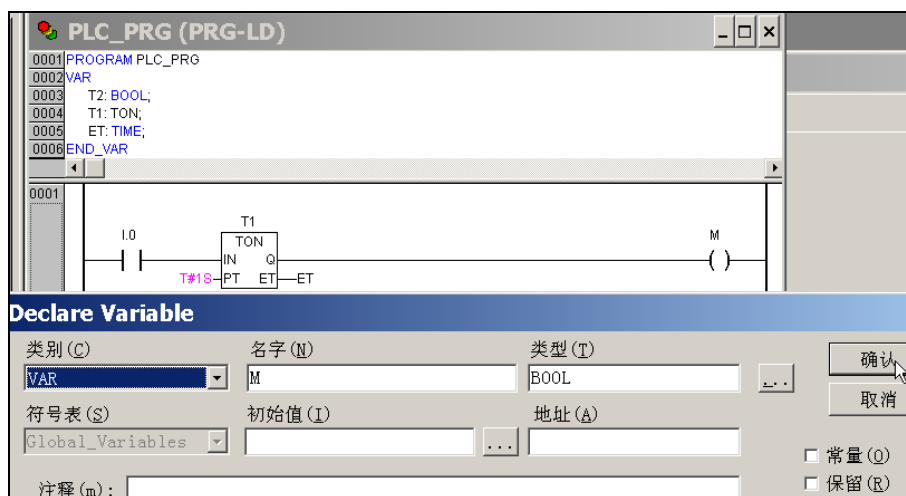


图 2-5-8 程序编写步骤（8）

在工作区域上点击鼠标右键，选择“后节”，如图 2-5-9 所示，进行 0002 节的程序编写。

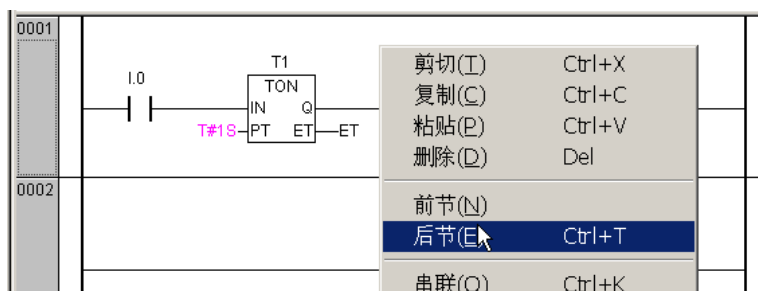


图 2-5-9 程序编写步骤（9）

0002 节的梯形图如图 2-5-10 所示。其编写方法与 0001 节相同。

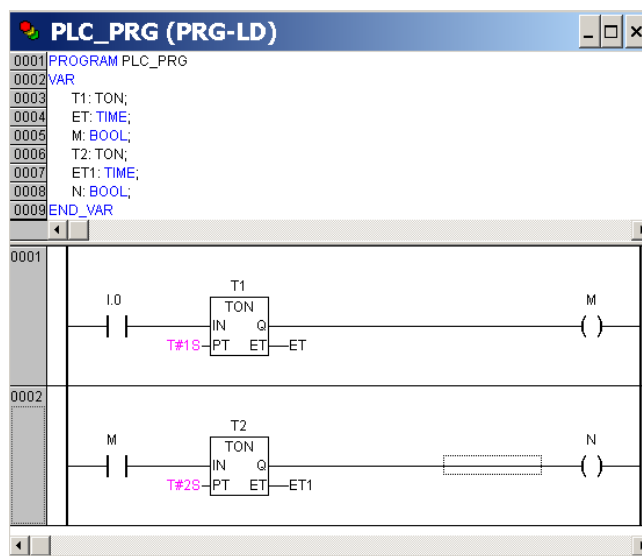


图 2-5-10 程序编写步骤（10）

为了实现开关的交替通断，需要在“T1”前添加“N”开关。选中“T1”，点击工具栏中的“串联”快捷按钮，或点击鼠标右键选择“串联”，添加开关如图 2-5-11 所示。

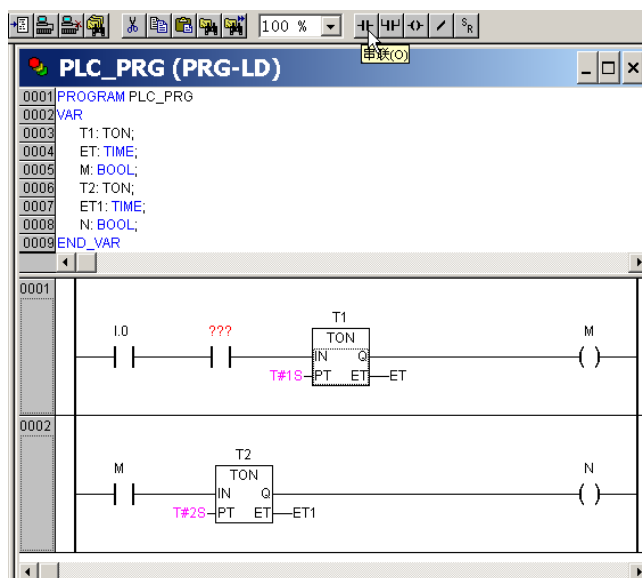


图 2-5-11 程序编写步骤（11）

点击触点标记文本“???”，输入“N”。在工具栏中选择“反向”，便添加了一个取反开关，如图 2-5-12 所示。

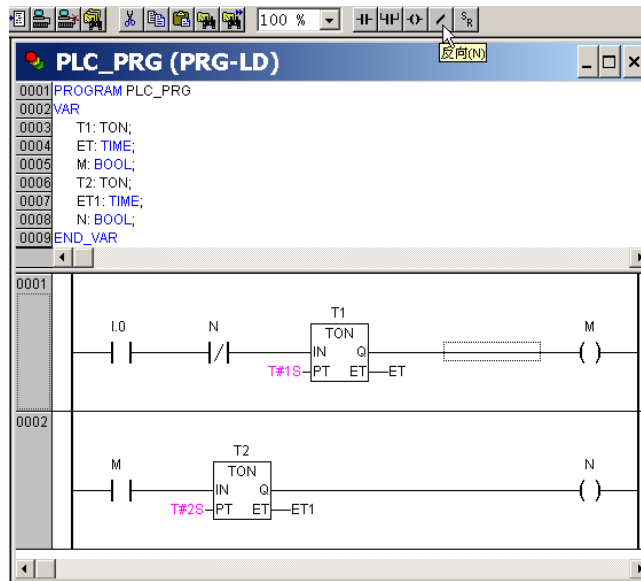


图 2-5-12 程序编写步骤（12）

为了将 M 的值从 Q.0 输出，可以增加 0003 节。在 0003 节中添加触点“M”，并在其后添加线圈。在工具栏里选择输出按钮 ，并将其定义为“Q.0”，如图 2-5-13 所示。

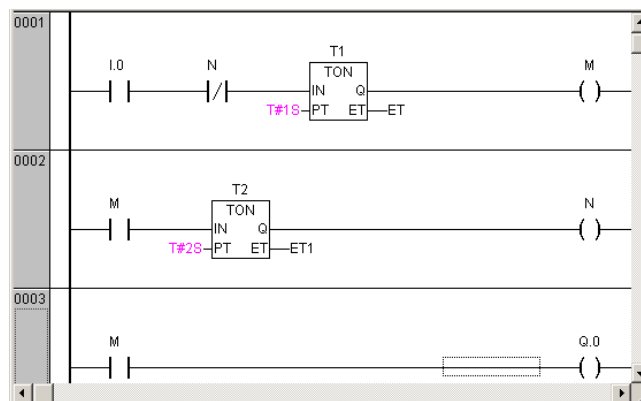


图 2-5-13 程序编写步骤（13）

为了使程序简单易读，一般会在程序中加入相应的注释。例如在 0001 节中，点击右键，选择“注释”，如图 2-5-14 所示。

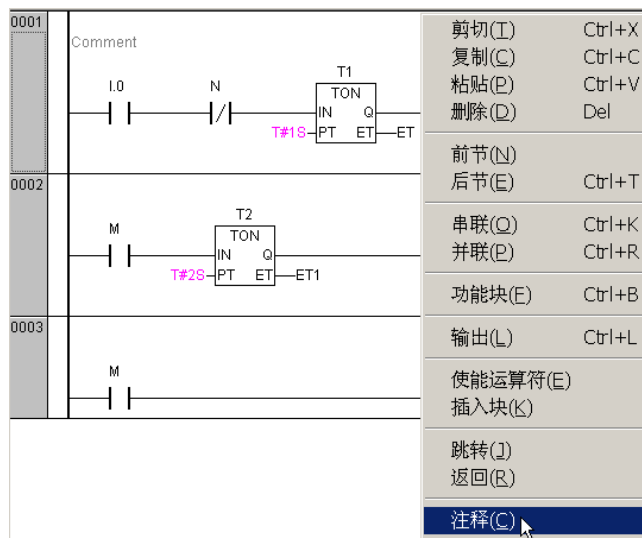


图 2-5-14 程序编写步骤（14）

双击“Comment”，输入相应的注释内容，如图 2-5-15 所示。

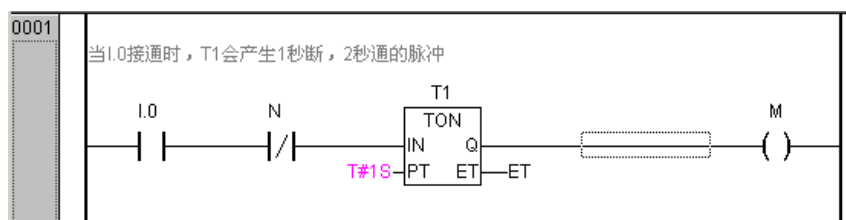


图 2-5-15 程序编写步骤（15）

注释添加完毕，如图 2-5-16 所示。

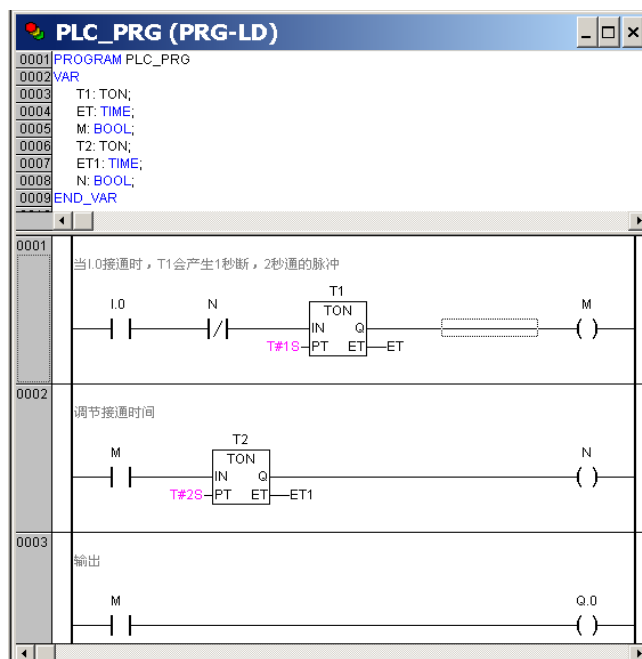


图 2-5-16 程序编写步骤（16）

2.6 编译

程序编写完毕，应该对其进行编译。打开“工程”菜单，选择“全部编译”，如图 2-6-1 所示。

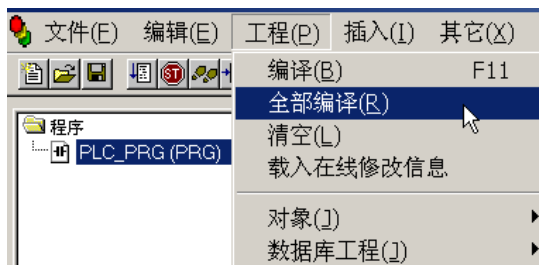


图 2-6-1 编译过程（1）

则在消息窗口会显示如图 2-6-2 所示的信息。

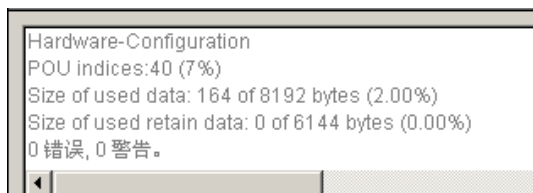


图 2-6-2 编译过程（2）

在“工程”菜单下选择“查看”/“未使用变量”，则会在消息窗口显示如图 2-6-3 所示的信息。“No unused variables found”表示程序中没有未使用的变量。

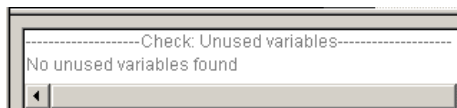


图 2-6-3 编译过程（3）

如果程序中出现了未使用的变量，如图 2-6-4 所示，如果有一个多余的变量 a，则会在消息窗口中显示信息：PLC_PRG(9): a。这表示本程序中检查到了一个未使用的变量 a。

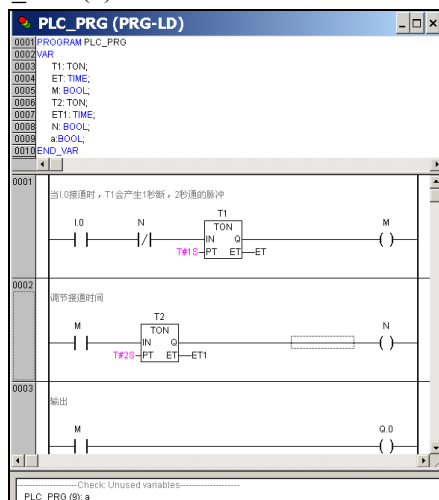


图 2-6-4 编译过程（4）

对于“查看”/“未使用变量”的应用，应该习惯性地编译通过后进行一下自检。如果存在未使用的变量，建议删除未使用的变量，有助于程序的正确运行。

另外，也可以直接设置。在“资源”选项卡中，选择“工程选项”/“Build”/“自动检查”，选中“未用变量”，在编译时会自动检查未使用的变量。

编译结束后，执行“在线”/“登录”命令进入调试状态。调试分为在线调试和仿真模式两种方式。在后面的介绍中，将按照在线调试和仿真模式两种方式分别介绍程序的运行情况。

2.7 在线调试

程序下装到 PLC 的 CPU 模块中，称为在线调试。编译通过的目标文件在进行下装时，会将全部目标文件下装到 CPU 模块中，同时将 CPU 模块复位，所有变量返回到初始状态。

在“在线”菜单中选择“登录”，建立本地计算机与 PLC 的 CPU 模块的连接，出现系统下装提示信息，如图 2-7-1 所示。

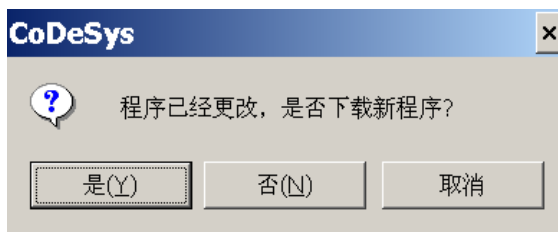


图 2-7-1 下装提示信息

点击“是”按钮，将新程序下装到 PLC 的 CPU 模块中。当出现如图 2-7-2 所示的创建启动工程提示信息时，点击“是”按钮，下装结束。确保 PLC 断电后再上电，运行此下装工程。

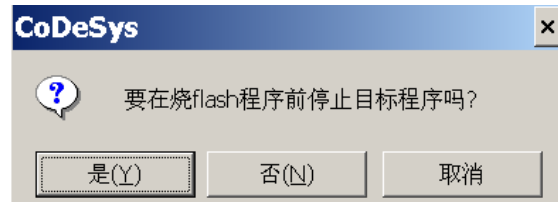


图 2-7-2 创建启动工程提示信息

程序下装结束后，在菜单栏中打开“在线”下拉菜单，选择“运行”，或者直接按“F5”功能键，均可实现程序的运行，如图 2-7-3 所示。

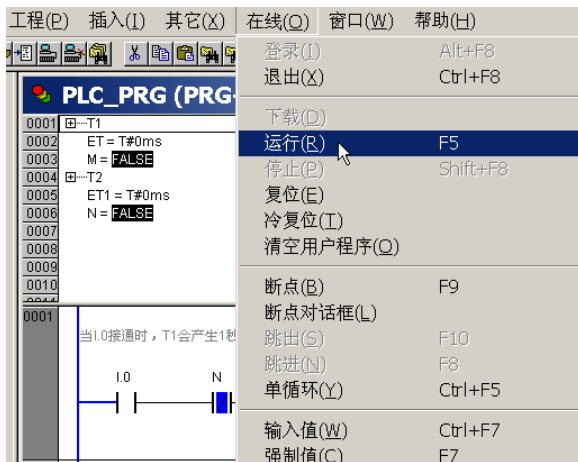


图 2-7-3 程序运行

双击“I.0”，按“F7”功能键强制输入值后，使 I.0 闭合，程序开始运行，如图 2-7-4 所示。从运行的结果可见，Q.0 通道灯输出“1s 断 2s 通”的脉冲信号。

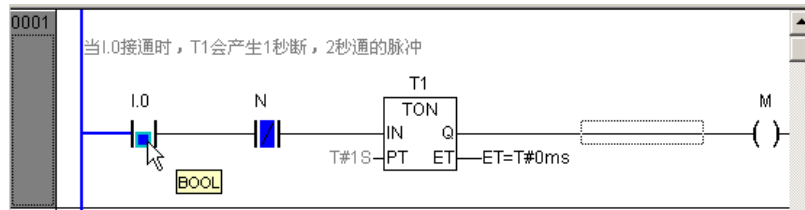


图 2-7-4 强制程序运行

程序运行情况如下所述。程序启动后，当运行时间 $t < 1s$ 时，Q.0 尚未闭合，PLC 上的通道灯 Q.0 不亮，如图 2-7-5 所示。

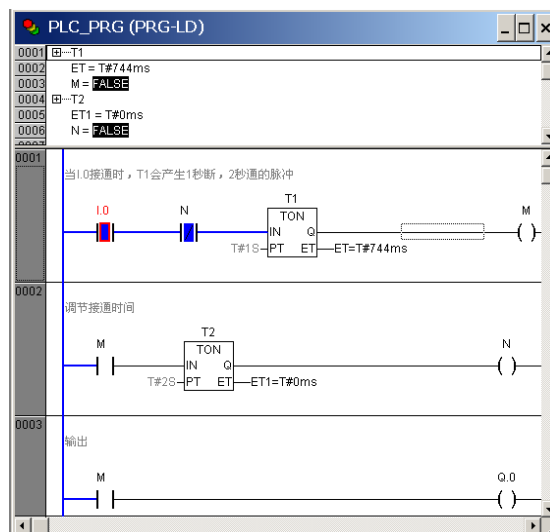


图 2-7-5 程序运行情况（1）

程序运行一段时间后，当运行时间 $1s < t < 3s$ 时，Q.0 闭合，PLC 上的通道灯 Q.0 亮，如图 2-7-6 所示。

当运行时间 $3s < t < 4s$ 时，触点“M”断开，PLC 上的通道灯 Q.0 灭。如此反复，Q.0 通道灯会出现“1s 亮，并且持续 2s 后再灭，间隔 1s 后又亮”的现象。

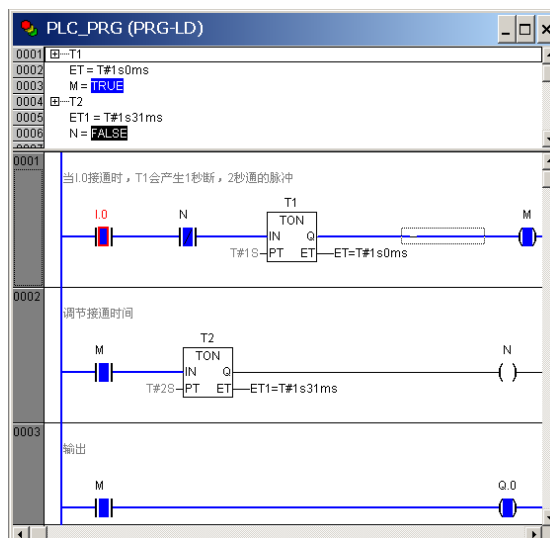


图 2-7-6 程序运行情况（2）

2.8 仿真调试

上述程序实现了在线模式下的运行。如果没有连接 PLC 的 CPU 模块，在本地计算机模拟运行用户程序，称为仿真模式。在菜单栏中打开“在线”下拉菜单，选择“仿真模式”，便进入了仿真模式下的程序运行过程。在“仿真模式”下，选择登录，如图 2-8-1 所示。“在线”/“仿真模式”被选中（出现“√”），登录时便会进入仿真模式。

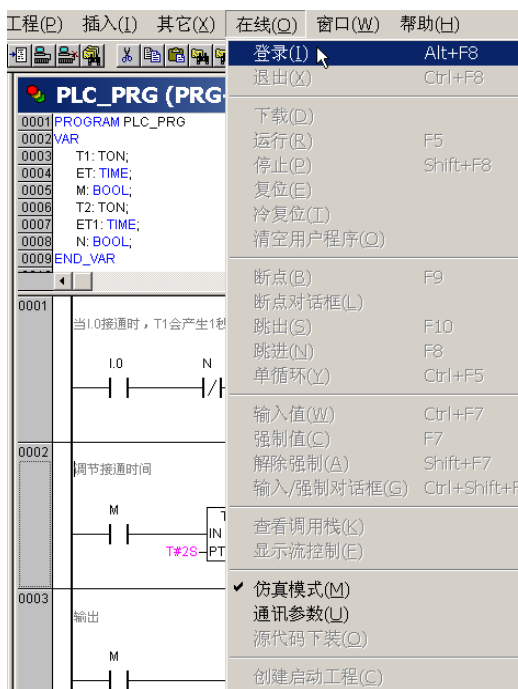


图 2-8-1 仿真模式

双击“I.0”，按“CTRL+F7”组合功能键输入值，或按“F7”功能键强制值，使 I.0 闭合，再按“F5”功能键，使程序运行，如图 2-8-2 所示。仿真模式下程序运行情况与在线模式下相同。

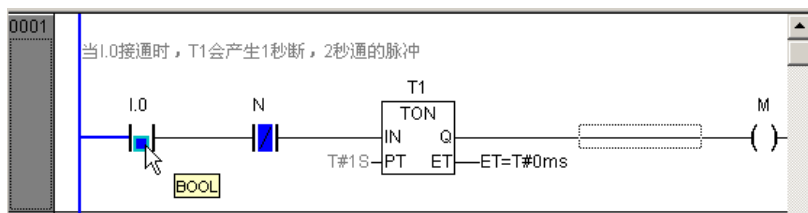


图 2-8-2 仿真模式下程序运行



本章小结

本章介绍了一个产生“1s 断 2s 通”脉冲信号的例子，编程语言使用梯形图。并介绍了编程前的准备工作，例如启动软件、新建工程和建立通讯连接等。简单描述了程序的编译、下装和运行等一些常用的操作。对于熟悉编程语言的用户，通过此章的学习，可以较快地掌握 PowerPro 软件的使用。

第3章 编程环境

至此，我们对 PowerPro 软件的编程有了初步的认识。为了使编程得心应手，需要对 PowerPro 的编程环境有进一步的了解。

打开“开始”/“程序”菜单，点击“HollySys PowerPro”/“控制方案生成系统”，启动 PowerPro 软件后，便进入了 PowerPro 软件的编程环境。

3.1 主界面

启动 PowerPro 软件，进入如图 3-1-1 所示的编程环境主界面。

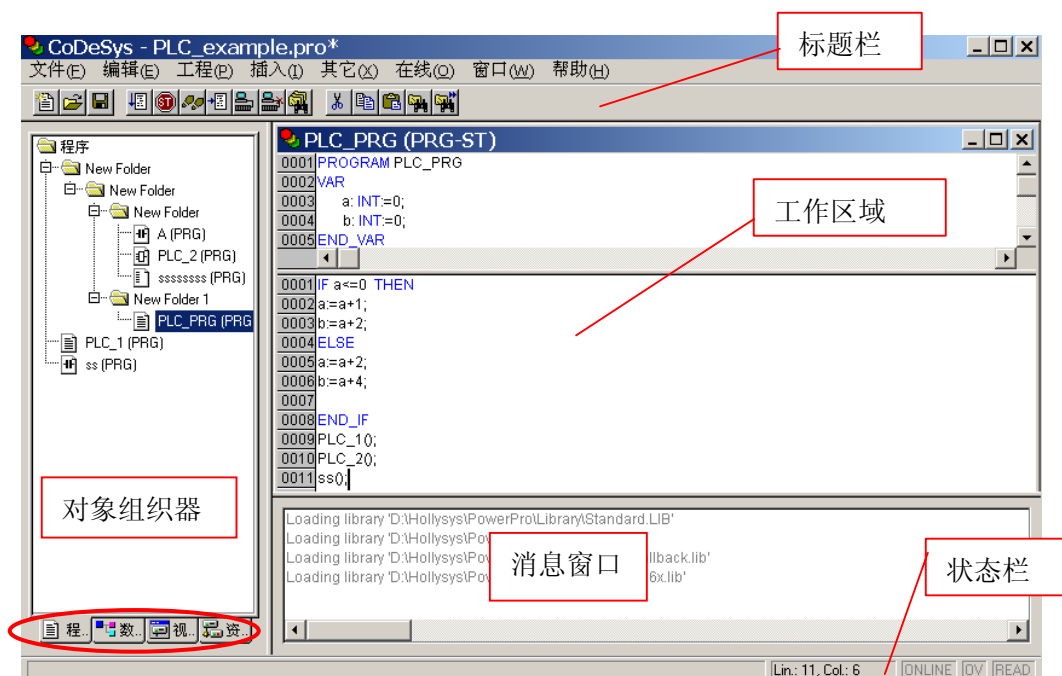


图 3-1-1 主界面

PowerPro 软件主界面中主要有下列组件：

- 标题栏：包括菜单条（操作所需的各种菜单命令）和工具条（可选），部分菜单命令可以通过工具条的快捷图标来快速选择。
- 对象组织器：由程序、数据类型、视图和资源四个选项卡组成。
- 工作区域：包含所有的编辑器窗口和管理器窗口。
- 消息窗口：显示编译、查错或比较结果等相关信息。
- 状态栏：显示当前工程和当前命令的相关信息。

3.1.1 标题栏

PowerPro 软件运行后，顶端标题栏如图 3-1-2 所示，主要包括菜单条（“文件”、“编辑”、“工程”、“插入”、“其它”、“在线”、“窗口”、“帮助”）和工具条（可选），工具条上的快捷图标可以方便地实现一些常用操作。



图 3-1-2 标题栏

首先介绍工具条中的快捷图标。想要知道各个快捷图标的名称，可以把鼠标指针移至快捷图标上，对应的快捷图标的名称就会出现在提示框中，如图 3-1-2 所示。菜单命令和快捷图标变灰，表示该功能在当前窗口禁用。

-  新建工程：新建一个工程。
-  打开文件或工程：打开已经存在的文件或工程。
-  保存：保存文件。
-  运行：启动登录程序的运行。
-  停止：停止程序的运行。
-  登录：进入调试状态。
-  退出：退出调试状态，回到程序编写状态。
-  剪切：将选中的部分剪切到剪贴板。
-  复制：将选中的部分复制到剪贴板。
-  粘贴：将剪贴板中的部分粘贴到当前窗口。
-  查找：在当前窗口查找所需目标。
-  查找下一项：查找下一个目标。

3.1.2 对象组织器

主界面左侧的竖条窗口称为对象组织器，由“程序”、“数据类型”、“视图”和“资源”四个选项卡组成，包含了一个工程所必需的基本对象，如图 3-1-3 所示。

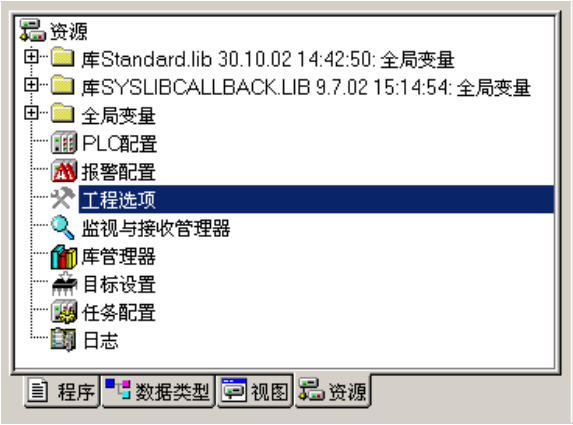


图 3-1-3 对象组织器

3.1.3 工作区域

工作区域位于对象组织器窗口的右边，所有的编辑器窗口和管理器窗口都在这个区域打开。在对象组织器的“程序”选项中，工作区域主要指程序、函数和功能块的编辑器窗口。所有 POU 的编辑器都包含声明部分和代码部分，其中代码部分是 POU 的主体，如图 3-1-4 所示。声明部分用于定义局部变量，有文本和表格两种显示形式。代码部分用于编写控制算法。由于选择的编程语言不同，编辑环境也会有所不同。根据编程语言的特点，编程语言可以分为图形编辑语言和文本编辑语言两大类。LD、SFC、FBD 和 CFC 语言的编辑器属于图形编辑器。IL 和 ST 语言的编辑器属于文本编辑器，包含了 Windows 文本编辑器的所有通用功能。

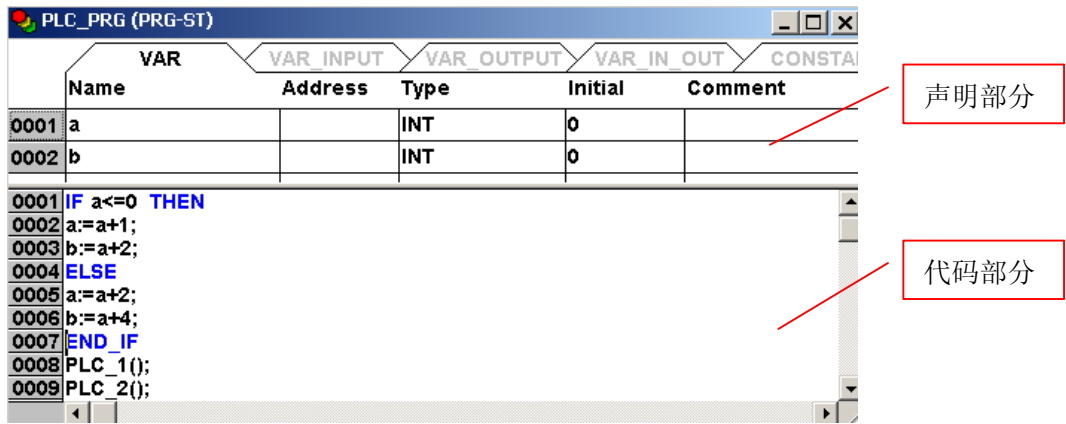


图 3-1-4 程序工作区域

3.1.4 消息窗口

消息窗口位于编辑器的下方，用于实时显示程序关于编译、错误、警告或比较的消息，如图 3-1-5 所示。双击消息窗口中的任一条消息，可以自动跳到编辑器中的相关行，以便查找相关信息。通过“编辑”/“后错误”（F4 功能键）和“编辑”/“前错误”（Shift+F4 组合功能键）命令可以在错误消息行中快速跳转。消息窗口的显示是可选的。当“窗口”下拉菜单里“信息”命令前出现选中符号（出现“√”），则消息窗口打开，否则消息窗口关闭。

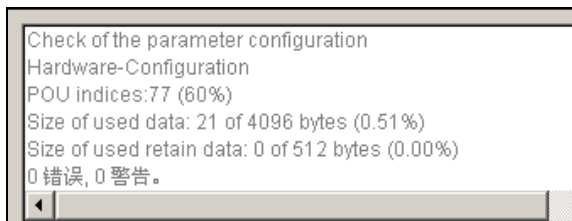


图 3-1-5 消息窗口

3.1.5 状态栏

状态栏位于主窗口边框的底部，用于显示有关当前工程和当前命令的信息。如果某项选中，相关的信息就以黑色的文本出现在状态栏中。

3.2 菜单列表

3.2.1 文件菜单

文件[F]菜单如图 3-2-1 所示，具体功能如下所述。

文件(F)	
新建(N)	
从模板中新建(I)	
打开(O)	Ctrl+O
关闭(C)	
保存(S)	Ctrl+S
另存为(A)	
保存/邮寄文档(H)	
打印(P)	Ctrl+P
打印机设置(I)	
退出(E)	Alt+F4

图 3-2-1 文件菜单

- 文件[F]/新建[N]：创建一个新工程。
- 文件[F]/从模板中新建[T]：用已有的模版来创建新工程。打开此项会弹出对话框，选择相应路径下已有模版的工程，点击“打开”按钮，便可以在相应模板下创建工程。PLC 不支持“从 PLC 打开工程”和“从源代码管理器打开工程”这两项功能。
- 文件[F]/打开[O]：打开一个工程。
- 文件[F]/关闭[C]：关闭当前打开的工程。

- 文件[F]/保存[S]: 对当前打开的工程进行保存。
- 文件[F]/另存为[A]: 将当前工程以新的文件名或路径保存。
- 文件[F]/保存/邮寄文档[H]: 可以自动保存文档并将其压缩为 ZIP 文件进行邮寄。
- 文件[F]/打印[P]: 打印当前窗口内容。
- 文件[F]/打印机设置[T]: 对打印参数进行设置。打开此项会弹出对话框, 可以选择打印机, 对页面大小、份数和方向等参数进行设置, 还可以对打印质量和打印布局进行设置。
- 文件[F]/退出[E]: 退出 PowerPro 软件。

3.2.2 编辑菜单

编辑[E]菜单如图 3-2-2 所示, 具体功能如下所述。

编辑(E)	工程(P)	插入(I)
恢复(U)		Ctrl+Z
重复(E)		Ctrl+Y
剪切(I)		Ctrl+X
复制(C)		Ctrl+C
粘贴(P)		Ctrl+V
删除(D)		Del
查找(E)		
查找下一个(N)	F3	
替换(R)	Ctrl+H	
提示输入(A)	F2	
自动定义	Shift+F2	
后错误	F4	
前错误	Shift+F4	
宏		

图 3-2-2 编辑菜单

- 编辑[E]/恢复[U]: 撤消上一次操作。
- 编辑[E]/重复[E]: 重复上一次操作。
- 编辑[E]/剪切[T]: 把所选内容复制到剪贴板, 并从当前位置删除。
- 编辑[E]/复制[C]: 把所选内容复制到剪贴板, 但不删除所选内容。
- 编辑[E]/粘贴[P]: 把剪贴板上的内容粘贴到当前位置。
- 编辑[E]/删除[D]: 删除当前所选内容。
- 编辑[E]/查找[F]: 在当前编辑器中查找某一文本。
- 编辑[E]/查找下一个[N]: 查找与最后一次所查文本相同的内容。
- 编辑[E]/替换[R]: 将查找到的目标替换成所需内容。
- 编辑[E]/提示输入[A]: 可以快速输入相关内容。在编辑窗口中的当前光标位置, 按 F2 功能键, 会自动弹出当前位置可以插入的待选项, 例如运算符、函数、功能块和变量类型等列表。在左边的列表中选择输入类型, 在右边的列表中选择期望的输入, 选择所需输入的内容, 点击“确认”按钮, 则所选内容便被输入。
- 编辑[E]/自动定义: 弹出变量定义对话框。
- 编辑[E]/后错误: 自前向后查找并显示消息窗口的提示错误或警告。
- 编辑[E]/前错误: 自后向前查找并显示消息窗口的提示错误或警告。
- 编辑[E]/宏: PLC 不支持此项功能。

3.2.3 工程菜单

工程[P]菜单如图 3-2-3 所示，具体功能如下所述。

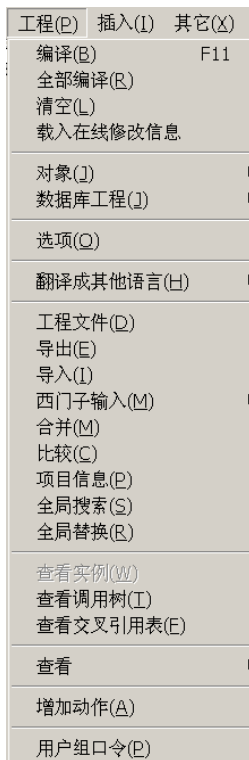


图 3-2-3 工程菜单

- 工程[P]/编译[B]：对当前工程的程序变化部分进行编译。
- 工程[P]/全部编译[R]：对当前工程的所有程序进行全部编译。
- 工程[P]/清空[L]：清除以前产生的下装文件。“清空”的目的是确保系统在下次进行编译时，重新建立下装文件。执行“清空”不影响 PLC 中的用户程序。每执行一次“清空”后再进行“编译”或“全部编译”，则在下次下装程序时，无论用户程序改变与否，均会出现提示：“The program has changed! Download the new program?”，根据相应的提示完成下装。这与“在线/清空用户程序”有所不同。“在线/清空用户程序”清除 PLC 中的程序，重新初始化 PLC 系统，而“工程[P]/清空[L]”清除以前产生的下装文件，确保系统在下次进行编译时，重新建立下装文件。
- 载入在线修改信息：在线修改功能表示程序的某些部分可以与 PLC 交换，而不会干扰 PLC 的运行，所有的数据都尽可能地保持不变。PLC 不支持此项功能。
- 工程[P]/对象[J]：对所选中的对象实现删除、添加、重命名、转换、复制、编辑和属性等操作，如图 3-2-4 所示。

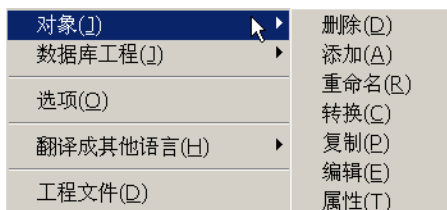


图 3-2-4 对象子菜单

删除：删除当前程序。

添加：在程序组中添加新程序，选择程序语言并命名。

重命名：给当前的程序更换名称。

转换：将当前程序转换成其它语言。例如可以将当前的 LD 语言转换为 IL 或 FBD 语言，如图 3-2-5 所示。

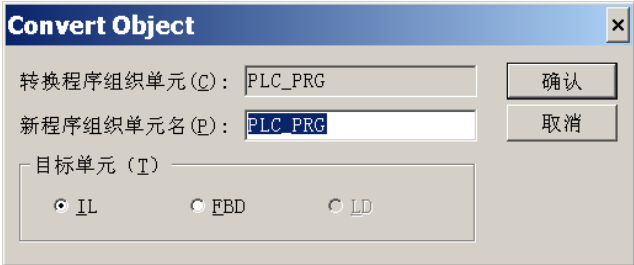


图 3-2-5 语言转换

复制：将当前程序另取名，成为一段新程序。

编辑：打开所选中的程序编辑窗口，也可双击程序名。

属性：设定用户对当前程序的操作权限，如图 3-2-6 所示。

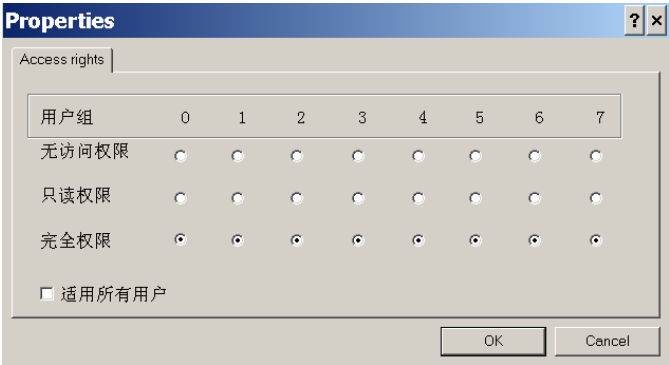


图 3-2-6 属性对话框

- 工程[P]/数据库工程[J]：PLC 不支持此项功能。
 - 工程[P]/选项[O]：对当前工程的参数进行设置，例如存储方式、路径或密码等。具体用法在第 9 章进行详细说明。
 - 工程[P]/翻译成其他语言[H]：对语言进行设置，如图 3-2-7 所示。
- 创建翻译文件：为当前工程创建一个.tlt 后缀的文件。
- 翻译工程：将当前工程按目标语言进行翻译。
- 查看已翻译过的工程：可以查看曾经翻译过的工程文件。
- 锁定翻译：一旦锁定翻译，翻译工程选项便呈现灰色，不可对该工程进行翻译操作。只有当再次选择锁定翻译时，才会允许翻译工程。

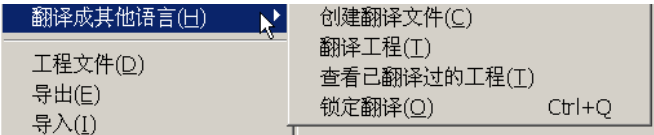


图 3-2-7 语言对话框

- 工程[P]/工程文件[D]: 打印输出当前工程中包含的所有文件或部分文件。
- 工程[P]/导出[E]: 将当前工程中所选中的程序导出, 生成一个*.EXP 文件。
- 工程[P]/导入[I]: 将*.EXP 文件导入当前工程, 常用于不同工程之间的程序导入。
- 工程[P]/西门子输入[M]: 包括“从 SEQ 符号文件输入”和“从 S5 工程文件输入”两个子菜单, 如图 3-2-8 所示。此命令可以从 Siemens STEP5/ STEP7 中导入全局变量。“从 SEQ 符号文件输入”命令应当在“从 S5 工程文件输入”之前被输入。这样, 当从程序导入时, 符号名可以采用绝对地址创建。PLC 不支持此项功能。



图 3-2-8 西门子输入

- 工程[P]/合并[M]: 将其它工程中的所需内容合并到当前工程。
- 工程[P]/比较[C]: 将当前工程与其它工程中的“程序”、“数据类型”、“视图”、“资源”进行比较, 并将比较结果列于信息窗口。
- 工程[P]/项目信息[P]: 列出当前工程的相关信息, 例如名称和保存路径等。
- 工程[P]/全局搜索[S]: 选择在当前工程的部分对象中搜索指定内容。
- 工程[P]/全局替换[R]: 选择在当前工程的部分对象中搜索, 并替换指定内容。
- 工程[P]/查看实例[W]: 打开所要查看的功能块实例。如果想要查看实例, 首先将编译好且无错误的程序进行在线登录, 然后再进行查看。
- 工程[P]/查看调用树[T]: 显示调用树窗口。必须首先对工程进行全部编译, 然后才能查看。
- 工程[P]/查看交叉引用表[F]: 查看当前工程中交叉引用的变量和程序等。
- 工程[P]/查看: 可以查看与变量有关的信息, 如图 3-2-9 所示。

未使用变量: 工程自动检查有无未使用变量, 如果没有未使用变量, 则会在消息窗口显示如下提示: No unused variables found (没有未使用变量)。

重叠内存区: 工程自动检查重叠内存区, 如果没有重叠内存区, 则会在消息窗口显示如下提示: No variables with overlapping memory area found (没有重叠内存区)。

同时访问: 工程自动检查有无同时访问, 如果没有同时访问, 则会在消息窗口显示如下提示: No concurrent accesses found (没有同时访问情况)。

多路写输出: 工程自动检查多路写输出, 如果没有多路写输出, 则会在消息窗口显示如下提示: No outputs found which are written to at more than one location (没有多路写输出)。



图 3-2-9 查看子菜单

在后面的章节中会介绍, 在“资源”选项卡中的“工程选项”/“build”/“自动检查”中的内容, 与“工程”菜单栏里“查看”的内容相同, 而且具体功能也类似。唯一不同的是, “工程/查看”只有在编译后才可以通过“查看”选择要查看的内容, 例如未使用的变量等, 且只能一一查看。而“自动检查”则可以选择多项, 在编译时一起实现自动检查。

- 工程[P]/增加动作[A]: 在当前程序中增加程序分支。
- 工程[P]/用户组口令[P]: 定义用户口令。在下次打开工程时需写入相应的口令。如果口令一致, 允许对其进行操作, 否则无法进行操作。

3.2.4 插入菜单

在编写程序时，使用插入工具可以定义变量，调用功能块、运算符和函数。不同的编程语言和当前位置，插入菜单所提示的待插入项目是不相同的。下面以“LD”语言为例，分几种不同的情况来介绍“插入”菜单的不同使用方法。

首先，光标处于工作区域的变量声明区所弹出的下拉菜单，与光标处于工作区域的程序区域所弹出的下拉菜单，所显示的内容是不同的，如图 3-2-10 所示。这里可插入一些 LD 语言中的功能块、输出、注释以及声明关键字、变量类型等内容。至于在具体的程序中如何插入各个选项，在后面的章节中有详细的介绍。



图 3-2-10 “插入”菜单（1）

其次，在“资源”选项卡中“PLC 配置”下，此时的“插入”是对模块的插入，先插入程序中所需的 CPU 模块，再依次插入相应的扩展模块，如图 3-2-11 所示。如果 CPU 的 I/O 点数可以满足工程的需求，则不需添加任何扩展模块。

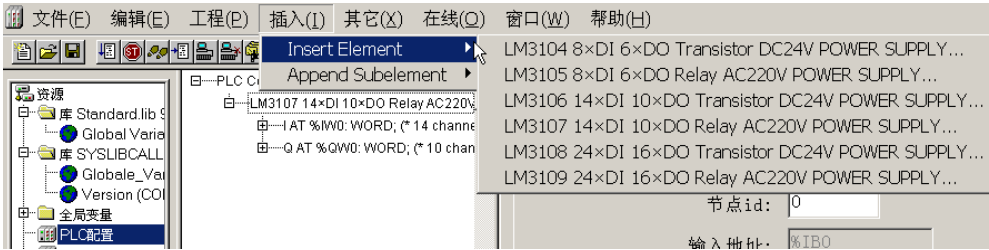


图 3-2-11 “插入”菜单（2）

再次，在“资源”选项卡中的“监视与接收管理器”下，选择“插入”菜单下的“新建监视列表”，或在“监视与接收管理器”列表区的左区点击鼠标右键，从弹出的菜单中选择“增加监视列表”，并为列表输入合适的名称，即可插入新的监视列表，如图 3-2-12 所示。

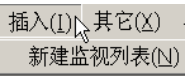


图 3-2-12 “插入”菜单（3）

3.2.5 其它菜单

使用不同的语言进行编程，或在不同的当前位置下，“其它”菜单所显示的选项不相同。下面以“LD”语言为例，分几种不同的情况来介绍“其它”菜单。

首先，在“资源”选项卡中的“程序”选项下，“其它”菜单如图 3-2-13 所示。

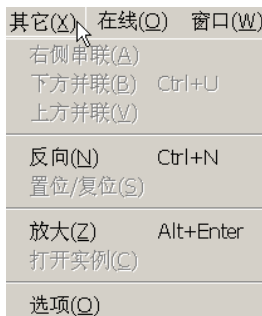


图 3-2-13 “其它”菜单（1）

- 反向：表示触点和线圈取非。如果线圈取非，则取非以后的值会被保存到对应的逻辑变量中。如果触点取非，则只有当逻辑值是 FALSE 时才能连通。
- 置位/复位：线圈可以定义成置位或者复位状态。用线圈符号（S）表示一个置位线圈。一旦设置为 TRUE 值，置位线圈将一直保持为 TRUE，直到被复位。用线圈符号（R）表示一个复位线圈。一旦设置为 FALSE 值，复位线圈将一直保持为 FALSE，直到被重新置位。
- 放大：快捷键为“Alt+Enter”。在梯形图中，选中某个功能块，使用“放大”，则会弹出相应功能块的“帮助”文件，即“库管理器”，从而了解该功能块的应用。
- 选项：在梯形图中，使用“选项”菜单，则会弹出如图 3-2-14 所示的“功能块梯形图选项”对话框，可以进行梯形图的相关参数设置。

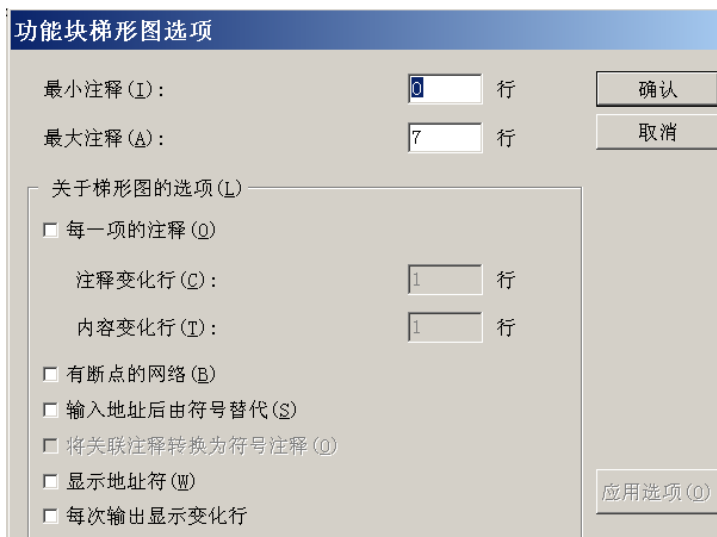


图 3-2-14 “其它”菜单（2）

其次，在“资源”选项卡中的“PLC 配置”选项下，“其它”菜单如图 3-2-15 所示。

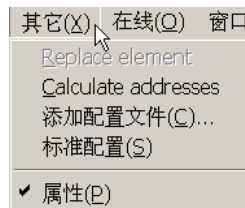


图 3-2-15 “其它”菜单（3）

- **Replace element:** 替换 CPU 模块。PLC 软件不支持此项功能。
- **Calculate addresses:** 自动计算地址。
- **添加配置文件:** 可以添加*.eds、*.gsd、*.cfg 等配置文件。本软件无须添加任何配置文件。
- **标准配置:** 选择此项，会弹出如图 3-2-16 所示的对话框。点击“是”按钮，恢复默认配置。否则不改变当前配置。
- **属性:** 当“属性”前出现“√”，即在激活属性的情况下，会弹出基本参数、模块参数和通道参数等项目栏，否则不会显示此类信息。

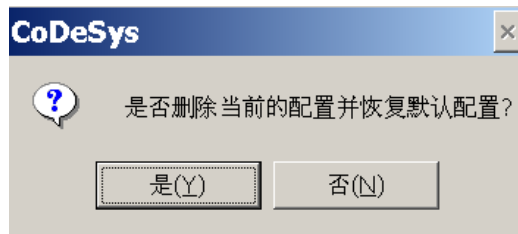


图 3-2-16 “其它”菜单（4）

再次，在“资源”选项卡中的“监视与接收管理器”选项下，“其它”菜单如图 3-2-17 所示。

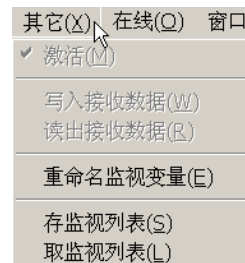


图 3-2-17 “其它”菜单（5）

- **激活:** 点击该项，使“激活”之前出现“√”，激活监视列表，才可以监控列表中的变量。
- **写入接收数据:** 将变量的值改写为设置的变量值。
- **读出接收数据:** 读取变量的当前值。
- **重命名监视变量:** 改变监视列表的名称。
- **存监视列表:** 保存监视列表，扩展名为*.wtc。
- **取监视列表:** 重新载入已保存的扩展名为*.wtc 的监视列表。

3.2.6 在线菜单

“在线”菜单是用于程序下装和调试的一组工具，如图 3-2-18 所示。

在线(O)	
登录(I)	Alt+F8
退出(X)	Ctrl+F8
下载(D)	
运行(R)	F5
停止(P)	Shift+F8
复位(E)	
冷复位(T)	
清空用户程序(O)	
断点(B)	F9
断点对话框(L)	
跳出(S)	F10
跳进(N)	F8
单循环(Y)	Ctrl+F5
输入值(W)	Ctrl+F7
强制值(C)	F7
解除强制(A)	Shift+F7
输入/强制对话框(G)	Ctrl+Shift+F7
查看调用栈(K)	
显示流控制(E)	
✓ 仿真模式(M)	
通讯参数(U)	
源代码下装(Q)	
创建启动工程(C)	
写文件到PLC(W)	
从PLC中读取文件(R)	

图 3-2-18 “在线”菜单

- 在线[O]/登录[I]：进入调试状态。
- 在线[O]/退出[X]：退出调试状态，切换到程序编辑状态。
- 在线[O]/下载[D]：把工程装载到 PLC 中。
- 在线[O]/运行[R]：启动程序，进入运行状态。
- 在线[O]/停止[P]：停止程序的运行。
- 在线[O]/复位[E]：停止程序的运行，变量置为初始值。retain 保留型维持当前值。
- 在线[O]/冷复位[T]：停止程序的运行，重新初始化所有变量。
- 在线[O]/清空用户程序[O]：清除 PLC 中的程序，重新初始化 PLC 系统。注意，这与“工程/清空”菜单有所不同。“在线/清空用户程序”是指清除 PLC 中的程序，重新初始化 PLC 系统。而“工程/清空”菜单清除以前产生的下装文件，确保系统在下次进行编译时，重新建立新的下装文件。
- 在线[O]/断点[B]：在当前位置设置一个断点，或删除已有的断点。如果程序运行后到达断点，则程序终止，相应得程序段会以红色背景显示。为了继续程序的运行，可用“在线/运行”、“在线/跳进”或“在线/跳出”命令。
- 在线[O]/断点对话框[L]：编辑整个工程中的断点。
- 在线[O]/跳出[S]：单步执行程序，程序在执行之后停止。
- 在线[O]/跳进[N]：如果在当前位置是函数或功能块，则程序将执行到被调用程序的第一条指令。在其它情况下，与“在线/跳出”命令一样。
- 在线[O]/单循环[Y]：程序执行一次循环之后停止运行。

- 在线[O]/输入值[W]: 调试时修改变量值。
- 在线[O]/强制值[C]: 同样用于调试时对变量赋值。在每个循环结束之后, 被强制的变量都被写入强制值, 直到执行“解除强制”指令为止。
- 在线[O]/解除强制[A]: 终止变量的强制命令。
- 在线[O]/输入/强制对话框[G]: 允许对多个变量写入新值, 并同时输入到 PLC 中。对于“在线/输入值”, 变量只被写一次, 而且变量允许立刻被其它程序赋值。对于“在线/强制值”, 变量在每一个循环之后被写入强制值, 直到“解除强制”命令为止。
- 在线[O]/查看调用栈[K]: 在仿真模式下, 显示在调用堆栈里运行程序的列表。
- 在线[O]/显示流控制[F]: 查看程序的运行流程。
- 在线[O]/仿真模式[M]: 如果仿真模式被选择, 则选中符号“√”将出现在菜单项的前面。在仿真模式下, 用户程序运行在操作系统平台下的本地计算机内。此模式可用来检查工程。如果不运行于仿真模式, 那么可以直接将程序运行于 PLC 中。
- 在线[O]/通讯参数[U]: 设置本地计算机和一个或多个 PLC 模块的通信参数。
- 在线[O]/源代码下装[O]: PLC 不支持此项功能。
- 在线[O]/创建启动工程[C]: 将下装到 PLC 中的工程作为 PLC 运行的默认工程。所谓启动工程, 是指保存在 PLC 的 flash 中且上电后可以运行的用户程序。如果创建启动工程的代码与上次下装的代码不同, 系统会给出相应提示: “当前代码与上次下装代码不匹配! 是否继续?”, 如图 3-2-19 所示。一般情况选择“是”, 从而创建启动工程。在离线模式下, 将会产生*.ri 文件。
- 在线[O]/写文件到 PLC[W]: PLC 不支持对文件的操作, 故不支持此项功能。
- 在线[O]/从 PLC 中读取文件[R]: PLC 不支持对文件的操作, 故不支持此项功能。

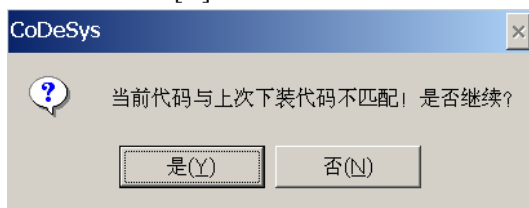


图 3-2-19 创建启动工程

3.2.7 窗口菜单

窗口[W]菜单如图 3-2-20 所示, 具体功能如下所述。

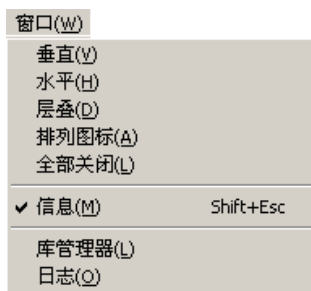


图 3-2-20 “窗口”菜单

- 窗口[W]/垂直[V]: 所有窗口在工作区垂直排列, 不重叠, 且填充整个工作区。
- 窗口[W]/水平[H]: 所有窗口在工作区水平排列, 不重叠, 且填充整个工作区。
- 窗口[W]/层叠[D]: 所有窗口在工作区串联排列, 一个叠一个, 依次错开。
- 窗口[W]/排列图标[A]: 在工作区底部的一行内, 顺次排列所有的最小化窗口。
- 窗口[W]/全部关闭[L]: 关闭所有打开的窗口。

- 窗口[W]/信息[M]: 显示/关闭消息窗口。
- 窗口[W]/库管理器[L]: 打开库管理器窗口, 添加或删除库函数。
- 窗口[W]/日志[O]: 弹出打开日志窗口。

3.2.8 帮助菜单

帮助[H]菜单如图 3-2-21 所示, 提供本软件的相关帮助, 具体功能如下所述。



图 3-2-21 “帮助”菜单

- 帮助[H]/内容[C]: 弹出帮助主题, 并将帮助的相关项目列出, 以供方便、快捷地查找帮助项目。
- 帮助[H]/搜索[S]: 弹出对话框, 提示输入所需查找项目的关键字。
- 帮助[H]/关于[A]: 软件名称及版本信息。

3.3 快捷工具

首先介绍工具条中的快捷图标。如果想要知道各个快捷图标工具按钮的名称, 可以把鼠标指针移至快捷图标工具按钮上, 对应快捷图标工具按钮的名称就会出现在提示框中。菜单命令和工具按钮变灰, 表示该功能在当前窗口禁用。

3.3.1 文件工具

-  新建工程: 新建一个工程。
-  打开文件/工程: 打开已经存在的文件/工程。
-  保存: 保存文件。

3.3.2 调试工具

-  运行: 启动登录程序的运行。
-  停止: 停止程序的运行。
-  跳出: 单步执行程序, 程序在执行之后停止。“跳出”命令遇到功能块或函数时, 会跳过功能块执行下一条语句。
-  断点: 在程序中设置进程停止的地方。

-  登录：进入调试状态。
-  退出：退出调试状态，回到程序编辑状态。
-  全局搜索：在整个工程中查找所需目标。

3.3.3 编辑工具

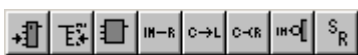
-  剪切：将选中的部分剪切到剪贴板。
-  复制：将选中的部分复制到剪贴板。
-  粘贴：将剪贴板中的部分粘贴到当前窗口。
-  查找：可以在当前编辑器中查找某一文本。打开“查找”对话框，在查找内容字段中输入要查找的字符序列。此外，还可以确定正在查找的文本是否全字匹配，或者是否区分大小写，从当前光标位置向上或是向下查找等。
-  查找下一个：继续查找最近的一次“编辑”/“查找”的文本。

3.3.4 编程工具

编程工具在不同编程语言下的表示形式是不相同的。按照编程语言种类的不同，编程工具的具体功能如下所述。

- LD 语言：

依次为串联、并联、输出、反向、置位/复位等功能的操作符。

- FBD 语言：

依次为输入、输出、赋值、跳转、返回、运算符、反向、置位/复位等功能的操作符。

- SFC 语言：

依次为前步转移、后步转移、右选择分支、左选择分支、右并行分支、左并行分支、跳转、转移跳转、使用 IEC 步等功能的操作符。

- CFC 语言：

依次为输入、输出、块、跳转、标记、返回、注释、反向、置位/复位、使能输入输出、创建宏、输入引脚、输出引脚、退出所有宏、退出当前宏、跳转到宏等功能的操作符。

对于 IL 和 ST 语言，无此类编程工具。

对于具体快捷编程工具的应用，将在后面的章节中详细介绍。

3.4 对象组织器

主界面左侧的竖条窗口称为对象组织器，由“程序”、“数据类型”、“视图”和“资源”等四个选项卡组成，包含了一个工程所必需的基本对象，如图 3-4-1 所示。

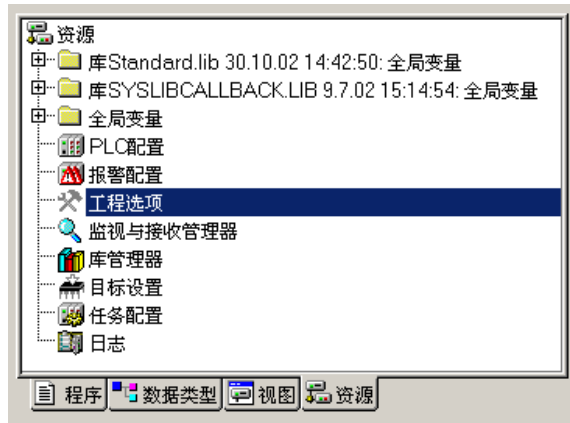


图 3-4-1 对象组织器（1）

3.4.1 程序

“程序”选项的功能是建立 POU（程序组织单元），用来编写用户程序。在对象组织器“程序”选项卡的空白区域点击右键，弹出如图 3-4-2 所示的菜单，具体功能如下所述。



图 3-4-2 程序

- 添加：在程序组中添加新程序，选择程序语言并给新程序命名。
- 重命名：给当前程序更换名称。
- 编辑：打开所选中的程序编辑窗口，也可双击程序名。
- 复制：将当前程序另赋予新名称，成为一段新程序。
- 删除：删除当前程序。
- 转化：可以实现五种语言的程序间的转化。
- 属性：可以设置不同用户组的使用权限。
- 数据库工程：PLC 不支持此项功能。
- 添加动作：可以在当前选定的程序或功能块下创建一个动作。在弹出的对话框中填入动作名，并选择动作实现的语言。“动作”代表一个进一步的功能实现，这种功能也可由其他语言所创建的一般功能块来实现。动作隶属于一个功能块或程序，可以像调用功能块一样来调用功能块的动作。动作调用的格式为：<程序名>.<动作名>或<实例名>.<动作名>。
- 新文件夹：可以自动生成“NewFolder”文件夹。如果某个文件夹被选中，新建文件夹“New Folder”将在其下层创建。另外，在该文件被选中时，点击右键，选择“重命名”还可以对文件夹的名称进行修改。
- 扩展节点：选择“扩展节点”，节点由“”变为“”，将文件夹内的内容进行扩展。
- 合拢节点：选择“合拢节点”，节点由“”变为“”，将已扩展的文件内进行合拢。
- 查看实例：此命令可打开和显示当前功能块的实例列表，双击功能块也可以打开该列表。另外，选择“工程”/“查看实例”命令，也同样可以打开该列表。应当注意的是，该命令只有“在线模式”下才有效，即工程编译成功，通过“在线”/“登录”下装到模块中。
- 显示调用树：与“工程”/“查看调用树”命令相同。可以在一个新窗口中显示当前对象调用程序、函数、功能块的树型结构，直观地指出当前 POU 与工程中其它 POU 的先后调用关系。工程必须通过编译，该命令才有效。
- 保存为模板：系统会自动将当前工程中定义的变量、工程选项中所作的一些设置的修改等，作为模板进行保存。
- 不参与编译：选择此项，当前程序、功能块或函数变为绿色。在编译过程中，并不参与程序的编译。如果想取消此项，需再次选择该项，将“不参与编译”前的“√”删除，便可取消该项，此时，当前程序、功能块或函数由绿色变为黑色。

3.4.2 数据类型

在“数据类型”选项卡中，和系统标准数据类型一样，用户可以定义自己的数据类型，生成结构型、枚举型或引用型等数据类型。例如，在“数据类型”选项卡中点击右键，添加数据类型，定义添加的数据类型“A”，如图 3-4-3 所示。关于数据类型的详细定义，请参见本手册第 6 章。

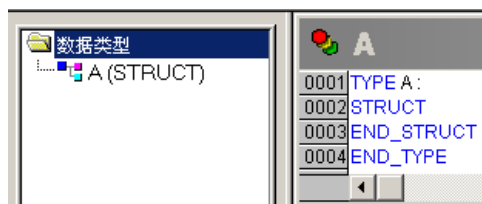


图 3-4-3 数据类型

3.4.3 视图

“视图”选项用于建立视图，观察工程变量，如图 3-4-4 所示。关于“视图”的具体应用，请参见本手册第 11 章。



图 3-4-4 视图

3.4.4 资源

“资源”选项用来定义全局变量、配置硬件模块和组织工程，如图 3-4-5 所示。

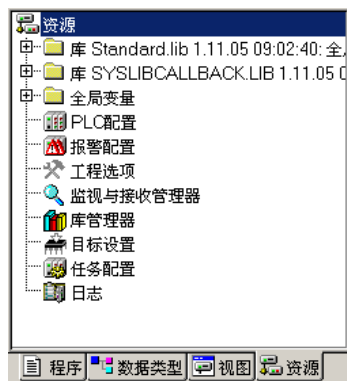


图 3-4-5 资源

- “全局变量”：用于定义在整个工程中均能使用的全局变量。
- “PLC 配置”：用于系统 PLC 硬件配置。
- “报警配置”：用于配置报警各项参数。
- “工程选项”：用于配置当前工程参数，如存储方式、路径或密码等。
- “监视与接收管理器”：用于“在线模式”下，集中监控同一工程不同程序的变量值。
- “库管理器”：与“窗口”菜单下“库管理器”的内容一致，均可用来显示所有和当前工程连接的库及添加所需的库。
- “目标设置”：用于设定目标平台和查看存储区域分配等。
- “任务配置”：用于创建任务，调用程序。
- “日志”：用于显示工程日志信息。



本章小结

本章介绍了 PowerPro 软件的编程环境，主要包括主界面及其各个组件、菜单列表及其子菜单等。

第4章 创建工程

在介绍了 PowerPro 软件的编程环境之后，本章将介绍如何创建一个工程。工程包含 PLC 程序中的所有对象，包括 POU、数据类型、资源和算法等。创建一个新工程的顺序是可以灵活掌握的，基本步骤主要包括目标设置、创建主程序、硬件配置和保存工程等。

4.1 目标设置

在主界面中点击“文件”/“新建”菜单，或在工具栏中点击“”按钮，随之出现“目标设置”对话框。在“配置”栏中选中“HollySys CoDeSys SP for C16x”或“Hollysys CoDeSys SP for C16x Extend”（此目标为 LM3106/LM3107/LM3108/LM3109 大容量模块所选用的扩展目标设置），点击“确认”按钮，如图 4-1-1 所示。

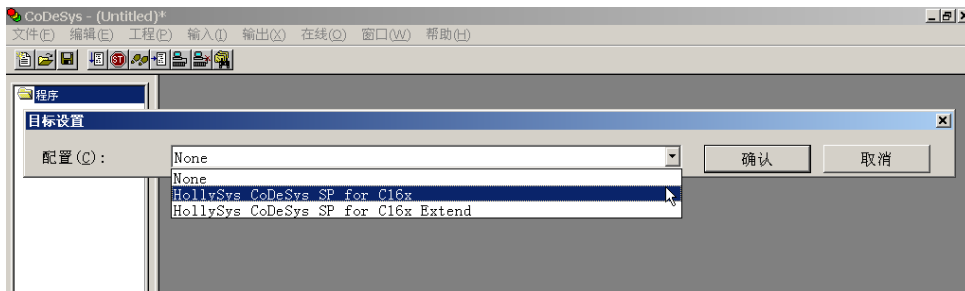


图 4-1-1 目标设置

随后弹出“目标设置”窗口。默认设置已能满足绝大多数应用需求。下面对目标设置中的各项逐一说明。

➤ 目标平台

第一项为目标平台，如图 4-1-2 所示。目标平台主要给出了 HOLLiAS-LEC G3 系列 PLC 所采用的 CPU 平台、对代码的编译以及库函数的设置。其中，选中复选框“HEX”表示可以将用户程序生成相应的 Hex 文件。一般情况下，直接点击“确认”按钮，对设置不做修改。

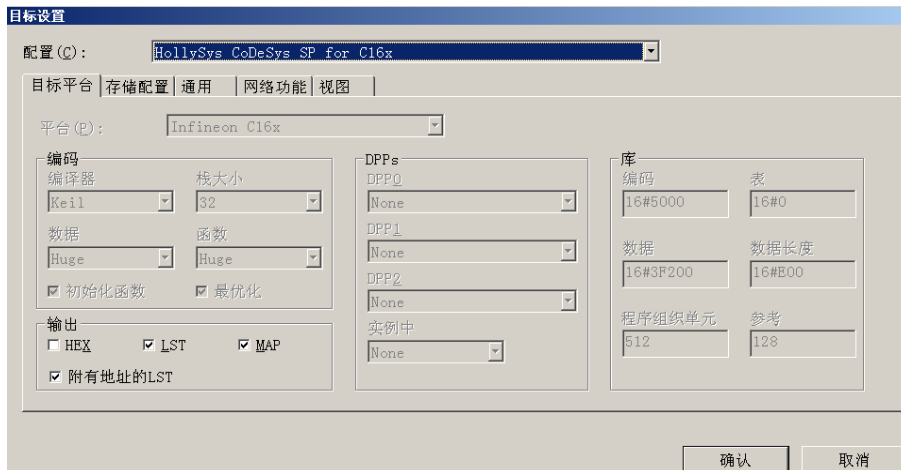


图 4-1-2 目标平台

➤ 存储配置

第二项为数据区的分配。非扩容与扩容 CPU 模块的主要差别在于数据区的内存分配大小不同，如图 4-1-3 和 4-1-4 所示。该设置说明了基本数据类型在 CPU 模块中的起始地址（绝对地址）及其大小。详细说明见表 4-1-1 所述。

在“存储设置”选项卡中包括一系列的数据类型，例如代码、全局、内存、输入、输出和保留等。另外还包括该数据类型所在区的绝对地址和大小等参数的设置。在一般情况下，不作任何修改，选择“确认”按钮即可。

对于数据内存总数、每段大小和数据段最大数这三个参数，满足一定的关系，表示为：每段大小与数据段最大数的乘积不大于数据内存总数。

数据内存总数是指 CPU 为用户数据（不包括代码段的用户程序）所提供的总空间。此参数为一常量。非扩容 CPU 是从 16#3E000 开始的 16#2000 字节，扩容 CPU 是从 16#74000 开始的 16#C000 字节。

每段大小可以修改。但在一般情况下，保持原有设置，不做修改。对于非扩容目标设置和扩容目标设置，其大小均默认为 16#1000。

数据段最大数可以修改。对于非扩容目标设置，全局数据段最大数不得超过 2，对于扩容目标设置，全局数据段最大数不得超过 6。

最大程序组织单元说明了用户可以使用的最多的 POU 的个数。如果用户程序所用 POU 的个数超出了规定数，编译时会有错误提示。此时，用户可以更改该设置，将最大 POU 数的设置增大。在一般情况下，直接选择“确认”按钮，保持原有设置，不做修改。

目标设置

配置(C): HollySys CoDeSys SP for C16x

目标平台 | **存储配置** | 通用 | 网络功能 | 视图

	基地址	大小	区域
代码:	16#30000	16#7000	
全局:	16#39000	16#1000 每一段	
内存:	16#3D000 ☐ 自动	16#1000	
输入:	16#3E000 ☐ 自动	16#100	
输出:	16#3E100 ☐ 自动	16#100	
保留:	16#3E200 ☐ 自动	16#1000	

☒ 保留在段内

数据内存总数: 16#4000

最大POU数: 256

数据段最大数: 3

确认 取消

图 4-1-3 非扩容目标设置内存分配设置

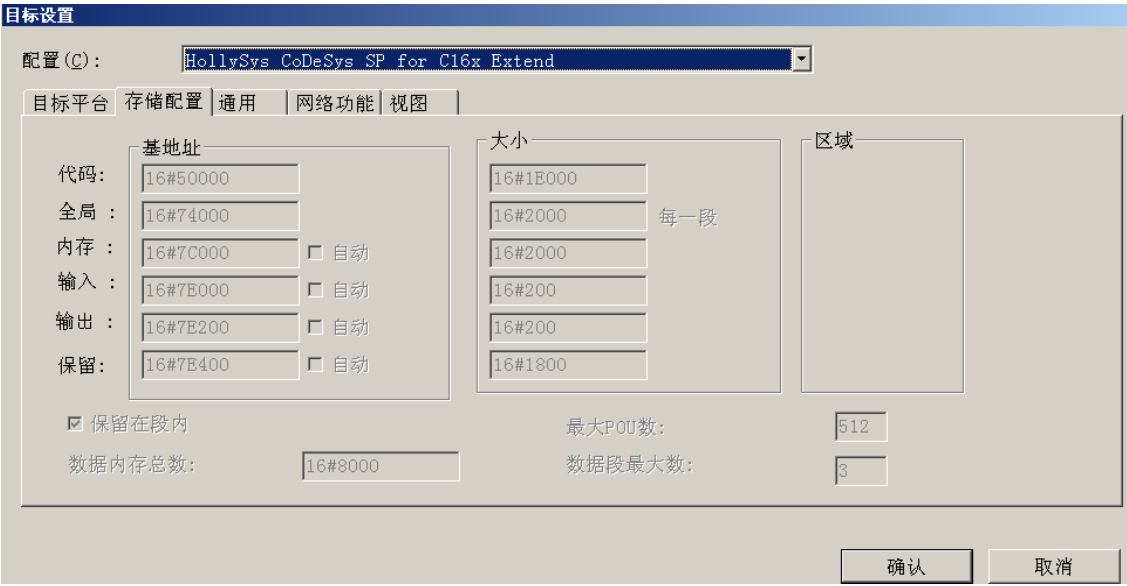


图 4-1-4 扩容目标设置内存分配设置

表 4-1-1 内存数据区分配

数据类型	说明	绝对地址		大小	
		非扩容	扩容	非扩容	扩容
代码	用户程序区。	16#30000	16#50000	16#7000 (28KB)	16#1E000 (120KB)
全局	用户未指定地址的变量定义。	16#39000	16#74000	16#3000 (12KB)	16#6000 (24KB)
内存	M 区。 用户指定到 M 区地址的变量 (AT%M...)： MB0～MB99：系统占用。 MB300～MB799：具有掉电保持功能。 注：用户组态定义时，所指定的地址为 M 区的相对地址，即偏移从 0 开始。	16#3D000	16#7C000	16#1000 (4KB)	16#2000 (8KB)
输入	I 区。 硬件配置中所占输入区。	16#3E000	16#7E000	16#100 (256B)	16#200 (512B)
输出	Q 区。 硬件配置中所占输出区。	16#3E100	16#7E200	16#100 (256B)	16#200 (512B)
保留	Retain 区。 用户定义的 Retain 型变量，不能指定地址。	16#3E200	16#7E400	16#1000 (4KB)	16#1800 (6KB)

➤ 通用

第三项为目标的通用设置。这里说明了 I/O 组态的一些信息，并说明了 CPU 所采用的字节存取模式，如图 4-1-5 所示。

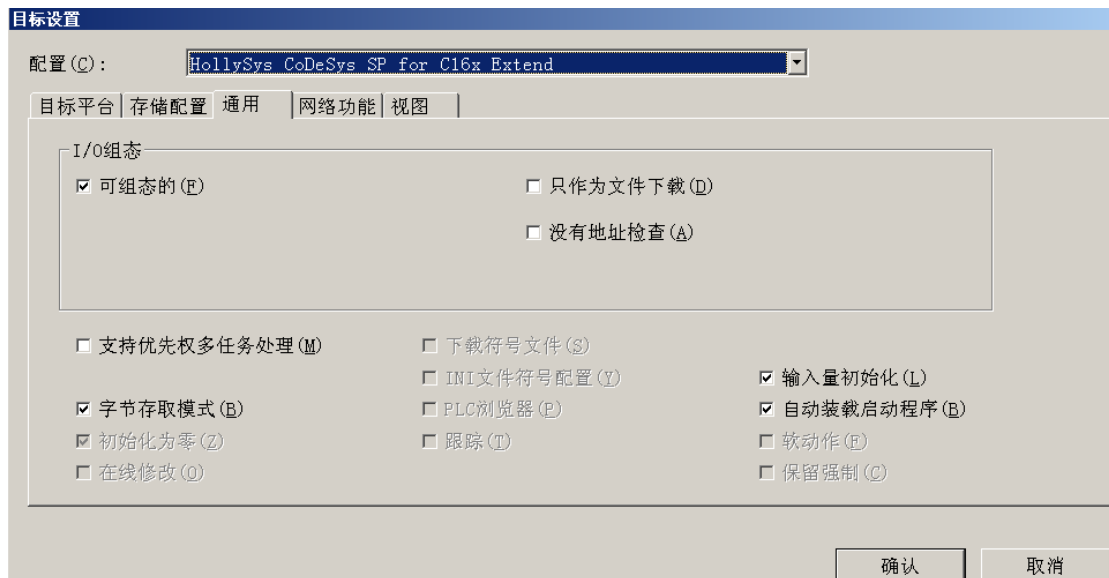


图 4-1-5 目标通用设置

➤ 网络功能：选择默认设置。

➤ 视图：选择默认设置。

此外，当用户改变了 CPU 模块的硬件配置（例如由非扩容 CPU 改为扩容 CPU）而程序不变时，只需将目标设置中的配置部分直接选成“Hollysys CoDeSys SP for C16x Extend”，再进行保存和编译即可。

4.2 创建主程序

每一个工程都必须在程序中建立一个主程序。主程序调用其它子程序。在新建工程时，系统会自动指定一个名字为“PLC_PRG”的程序作为用户程序的主程序，不得更改。否则，用户程序无法正常运行。

目标设置完成后，会自动弹出“创建 POU”对话框，如图 4-2-1 所示。POU 程序语言可选 IL、LD、FBD、SFC、ST 或 CFC 中的任意一种，这里选用梯形图 LD 语言。在“POU 类型”中选择“程序”，主程序名默认为“PLC_PRG”，点击“确认”按钮。



图 4-2-1 创建主程序“PLC_PRG”

4.3 硬件模块配置

PLC 系统通过硬件模块（包括 CPU 模块和扩展模块）采集和处理现场的数据。PLC 系统通过输入通道和输出通道与其他设备进行数据交换。输入通道采集现场的数据，输出通道控制生产过程中的电气设备。为了完成采集和控制任务，需要根据具体的工程，对 PLC 系统的硬件模块进行相应的配置。

4.3.1 配置 CPU 模块

在“资源”选项卡中选择“PLC 配置”，弹出配置界面。点击鼠标右键，选择 CPU 模块的型号，建立“PLC Configuration”结构树。例如选择 LM3107 模块，如图 4-3-1 所示。

CPU 模块的 I/O 具有固定的 I/O 地址。例如 CPU 模块 LM3107 共有 14 个 DI 通道和 10 个 DO 通道，其中：

输入部分以字为单位，从 %IW0 开始，每个输入通道占一个位，存储地址依次为：%IX0.0、%IX0.1、.....、%IX0.7、%IX1.1、%IX1.2、.....、%IX1.7，其中前 14 个地址为有效地址。

输出部分以字为单位，从 %QW0 开始，每个输出通道占一个位，存储地址依次为：%QX0.0、%QX0.1、.....、%QX0.7、%QX1.0、%QX1.1、.....、%QX1.7，其中前 10 个地址为有效地址。

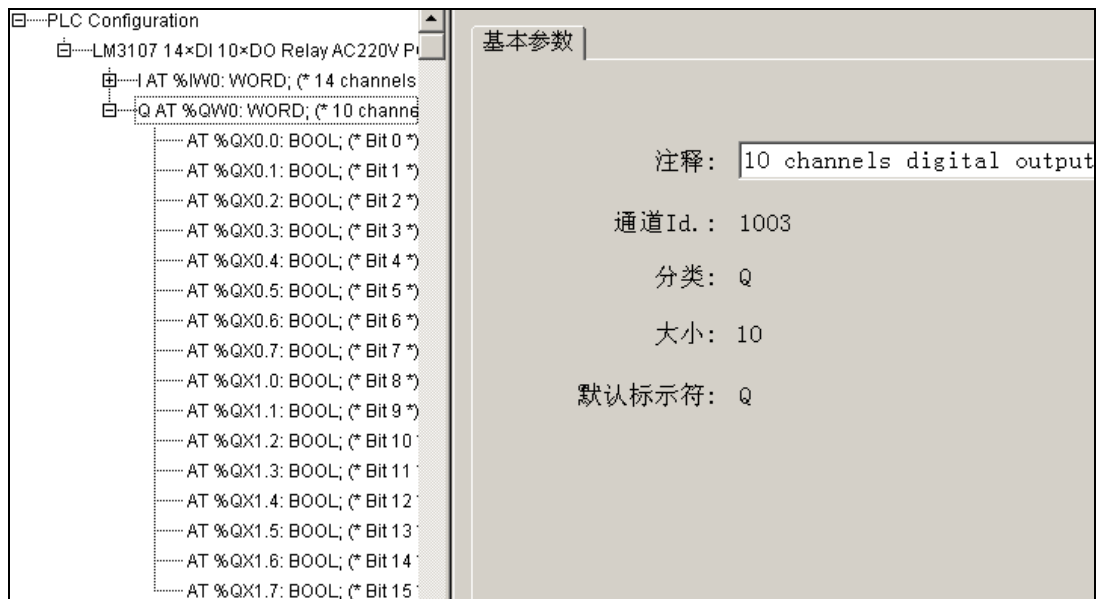


图 4-3-1 LM3107 基本参数设置

输入通道的滤波参数设置如图 4-3-2 所示，其默认值为“64”，如果需要修改此参数，可在“Value”中选择。

滤波参数对于高速输入通道是无效的。在用到高速输入通道的地方，无需设置相应通道的滤波参数。即便做了相应设置，对高速输入通道也是无影响的。

其他 CPU 模块的参数配置与此类似。根据实际控制要求，选择合适的 CPU 模块后，便可以添加所需的扩展模块。

基本参数 模块参数					
Index	Name	Value	Default	Min.	Max.
1	Input_Filter_CH0	64	64		
2	Input_Filter_CH1	64	64		
3	Input_Filter_CH2	64	64		
4	Input_Filter_CH3	64	64		
5	Input_Filter_CH4	64	64		
6	Input_Filter_CH5	64	64		
7	Input_Filter_CH6	64	64		
8	Input_Filter_CH7	64	64		
9	Input_Filter_CH8	64	64		
10	Input_Filter_CH9	64	64		
11	Input_Filter_CH10	64	64		
12	Input_Filter_CH11	64	64		
13	Input_Filter_CH12	64	64		
14	Input_Filter_CH13	64	64		

图 4-3-2 LM3107 模块参数设置

4.3.2 添加扩展模块

在选择 CPU 模块后，便要添加所需的扩展模块。在 CPU 模块上点击鼠标右键，出现模块操作对话框，选择其中的“Append Subelement”项，从弹出的扩展模块列表中选择所需的模块，如图 4-3-3 所示。例如，如果选择添加“LM3230”模块，则会在“PLC Configuration”下显示添加的模块，如图 4-3-4 所示。

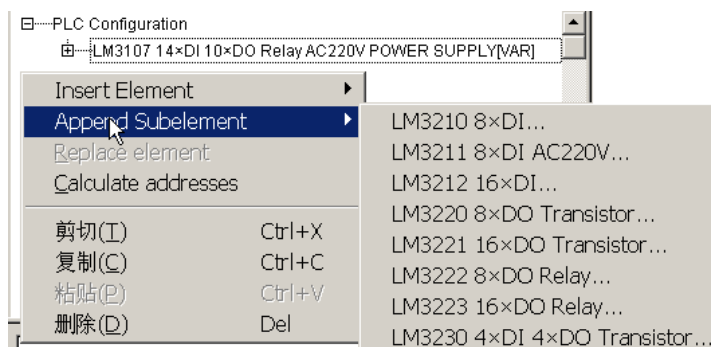


图 4-3-3 添加扩展模块（1）

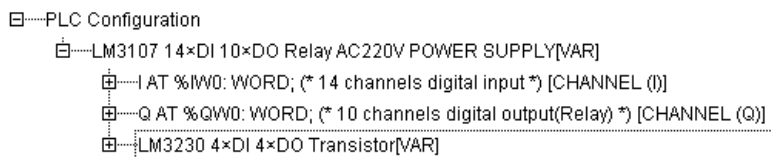


图 4-3-4 添加扩展模块（2）

4.3.3 插入扩展模块

当需要在某扩展模块前插入一个扩展模块时，应使光标处于该扩展模块位置处，点击鼠标右键，则会出现模块操作对话框，选择“Insert Element”项，并从弹出的扩展模块列表中选择所需的模块。例如选择“LM3210”模块，即可完成扩展模块的插入，如图 4-3-5 和图 4-3-6 所示。

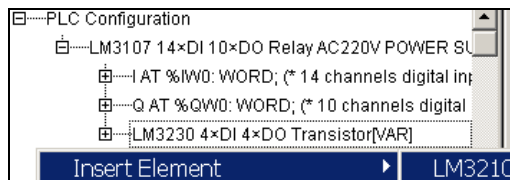


图 4-3-5 插入扩展模块（1）

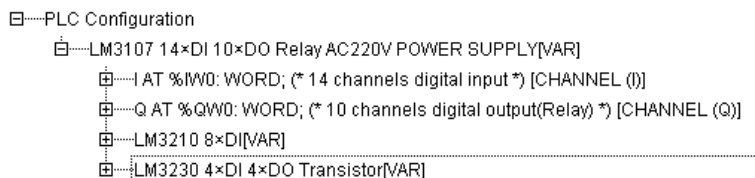


图 4-3-6 插入扩展模块（2）

4.4 扩展模块参数设置

CPU 模块和扩展模块的每一个 I/O 通道都对应一个实际的物理设备，其对应关系反映在随模块同时显示的“基本参数”中。“基本参数”包括节点号、输入地址、输出地址和诊断地址等信息，如图 4-4-1 所示。其中“节点号”表示模块硬件连接的先后次序，扩展模块根据连接次序，依次为“0”、“1”、“2”等，用户不可随意更改。“输入地址”和“输出地址”表示模块通道对应 I/O 存储区的起始地址。

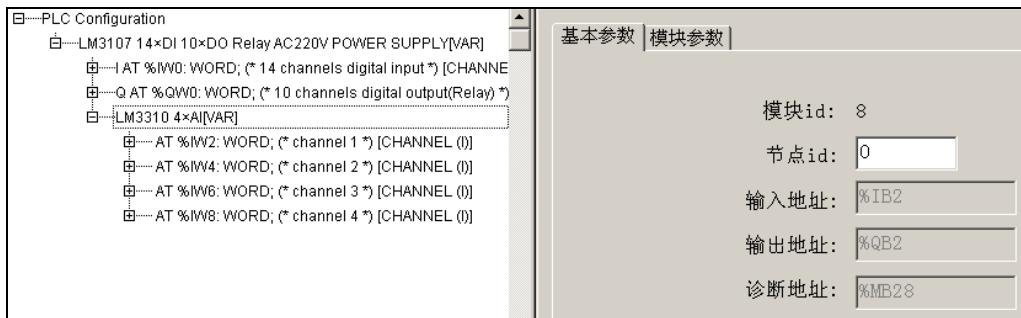


图 4-4-1 扩展模块的基本参数

扩展模块地址受其节点号、输入地址、输出地址和模块类型的影响。例如，如果四路模拟量输入模块 LM3310 为 CPU 模块 LM3107 后边第一个扩展模块，则通道地址分别为 %IW2、%IW4、%IW6 和 %IW8。双击模块或点击其前面的“+”号，可以看到模块的类型及各通道的具体 I/O 地址。

配置好模块后，如果需要，还可以对模块进行 I/O 变量定义，以便访问。双击字符“AT”，激活变量名输入框，键入变量名即可。“AT”后面的字符“%”表示地址，即“%”后面的字符为“AT”前面变量名的地址。

如果需要，还可以为某些通道定义总体的名字，例如 CPU 模块的“%IW0”默认定义为“T”，这样可以用“T.0”直接访问第一个输入点，而无需写成“%IX0.0”。另外，也可以为各通道分别定义变量名，如图 4-4-2 中，将“%IX0.0”定义为“start”，将“%IW2”定义为“temp1”。

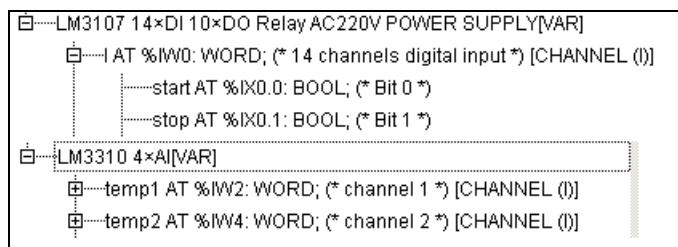


图 4-4-2 I/O 变量定义

当光标点击在“PLC Configuration”处时，可以看到在窗口的右侧有一“设置”选项卡，其具体选项包括“自动计算地址”、“检查重叠地址”和“在工程中保存组态文件”等，如图 4-4-3 所示，其中每个选项的具体含义为：

- 选中“自动计算地址”，表示“基本参数”中的节点号、输入地址、输出地址和诊断地址可以根据模块硬件连接的先后次序自动排序。
- 选中“检查重叠地址”，可以方便地检查出在编程中不小心定义的重叠地址，有利于程序被编译成功。
- 选中“在工程中保存组态文件”，可以方便地在工程中保存组态文件。

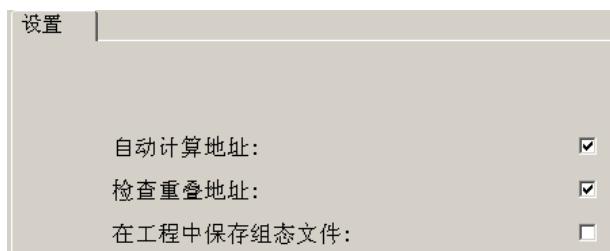


图 4-4-3“设置”选项卡

打开“模块参数”选项，该“模块参数”可以设置输入通道的滤波参数，如图 4-4-4 所示，其中每个字段的具体含义为：

- Index: 表示模块参数索引号。
- Name: 表示模块参数，即相应输入通道滤波参数名称。
- Value: 设置输入通道滤波参数，默认为 64，可以修改此值。
- Default: 输入通道滤波参数默认值为 64。

Index	Name	Value	Default	Min.	Max.
1	Input_Filter_CH0	64	64		
2	Input_Filter_CH1	64	64		
3	Input_Filter_CH2	64	64		
4	Input_Filter_CH3	64	64		
5	Input_Filter_CH4	64	64		
6	Input_Filter_CH5	64	64		
7	Input_Filter_CH6	64	64		
8	Input_Filter_CH7	64	64		
9	Input_Filter_CH8	64	64		
10	Input_Filter_CH9	64	64		
11	Input_Filter_CH10	64	64		
12	Input_Filter_CH11	64	64		
13	Input_Filter_CH12	64	64		
14	Input_Filter_CH13	64	64		

图 4-4-4 模块参数

4.4.1 数字量模块

以 LM3230 模块为例来介绍数字量模块的参数设置。按照前述方法，添加 LM3230 模块后，如图 4-4-5 所示。在基本参数设置中，节点 id 是系统根据扩展模块的扩展顺序自动产生，依次为“0”、“1”、“2”等，无须修改。输入地址、输出地址和诊断地址也是系统根据扩展模块的扩展顺序自动生成的，而且相应的地址均为灰体，不允许用户修改。

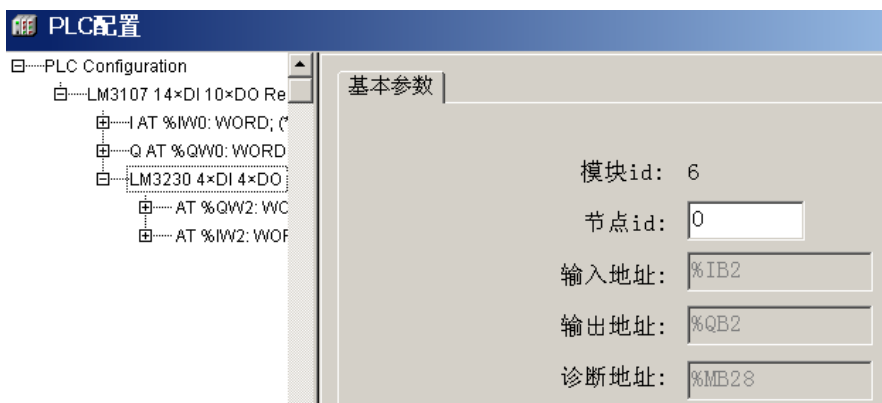


图 4-4-5 数字量输入模块基本参数

前面介绍了模块的基本参数设置，下面介绍通道的基本参数设置。

LM3230 数字量模块包含 4 路数字量输入处理通道和 4 路晶体管输出处理通道。

点击输出通道 `AT %QW2: WORD; (* 4 channels digital output(Transistor) *) [CHANNEL (Q)]`，PLC 配置

窗口的右侧会出现该模块输出通道的基本参数。同样可见输入通道的基本参数。

在输出通道的基本参数中，“注释”表示可以根据程序的需求加入通道注释，而且会显示在左侧的（* *）中间，（*4 channels digital output(Transistor)*）为该通道的默认注释。“通道 Id”、“分类”和“大小”等参数是自动生成的，不允许用户修改。

对于输入通道和输出通道每一位的设置，则需要打开扩展节点，选中每一位，如图 4-4-6 所示，在右侧“基本参数”栏，显示每一位的基本参数设置。与通道参数设置类似，同样也可以修改其默认注释。

关于数字量模块的具体技术规格，请参见《HOLLiAS-LEC G3 系列小型一体化 PLC 硬件手册》。

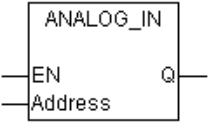


图 4-4-6 数字量输入模块输入/输出通道基本参数

4.4.2 模拟量输入模块

如果 PLC 配置中使用了模拟量输入扩展模块，则程序中需调用模拟量输入扩展功能块 ANALOG_IN。在设置模拟量输入扩展模块参数之前，先对模拟量输入扩展功能块 ANALOG_IN 及其应用做一简单介绍。

➤ 功能块示例

POU 语言	程 序
梯形图（LD）	

➤ 输入输出说明

输入数据存放地址	含义		
%IWxx	模拟量输入通道依次对应 PLC 配置中所规定的 IW 区地址		
输入	功能	数据类型	值
EN	使能	BOOL	0: 无效，不对模拟输入模块进行扫描 1: 使能，对模拟输入模块进行扫描
Address	该模块节点号	BYTE	0—7（与 PLC 配置里的节点号一致）
输出	功能	数据类型	值
Q	是否读到有效数据	BOOL	0: 未读到数据 1: 读到有效数据

✧ 举例（Analog_IN 功能块的用法）

变量定义与梯形图如图 4-4-7 所示，需要在功能块 ANALOG_IN 中的 Address 处输入值。功能块 ANALOG_IN 中 Address 处的输入值 0 应当与图 4-4-8 所示的模拟量输入模块配置表中光标所在处的节点 id 号 0 一致。

如果需要使用多个模拟量输入模块，则需要配置多个 Analog_IN 功能块，每个功能块对应的 Address 输入值应当与对应模块的节点 id 号保持一致。

在图 4-4-7 所示的梯形图中，当 EN 置位时，对模拟量输入模块进行扫描，4 个通道的扫描值分别存储在图 4-4-8 所示的%IW2、%IW4、%IW6、%IW8 中。当 EN 复位时，不对模拟量输入模块进行扫描。

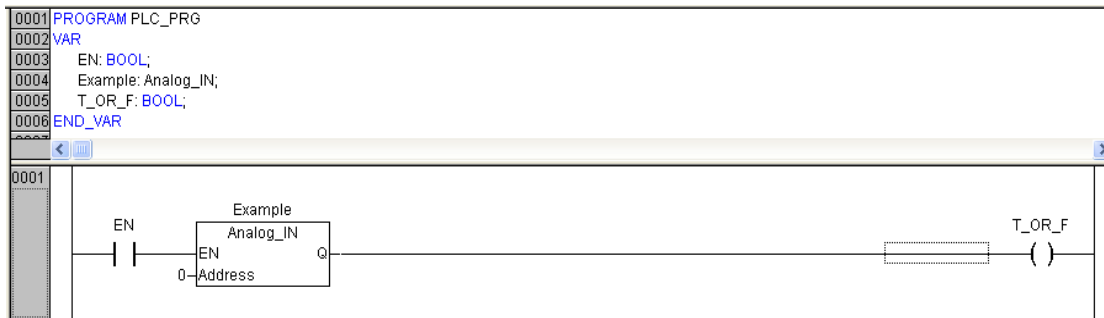


图 4-4-7 模拟量输入模块变量定义与梯形图

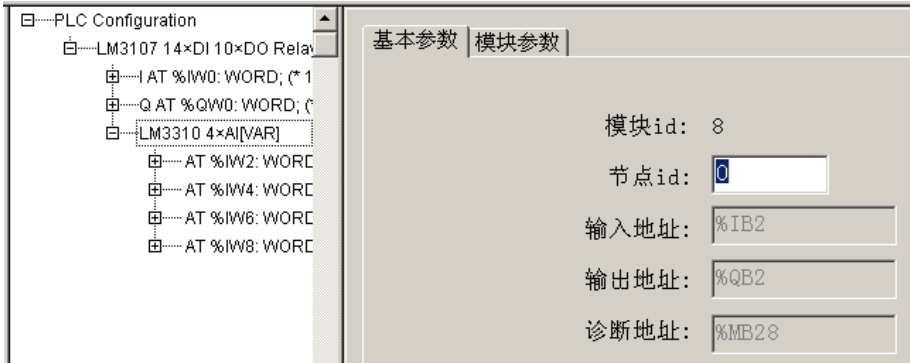


图 4-4-8 模拟量输入模块配置表

以上简单介绍了 ANALOG_IN 功能块在 LD 语言中的应用。对于模拟量输入模块参数的设置，以 LM3310 为例进行介绍。LM3310 为 4 通道模拟量输入模块，具体模块参数设置如图 4-4-9 所示。

基本参数		模块参数			
Index	Name	Value	Default	Min.	Max.
1	Filter_Factor	1	1		
2	Deadband	0	0	0	4080

图 4-4-9 LM3310 模块参数设置

- Filter_Factor 是滤波参数，取值可为 1、2、4、8、16、32。滤波参数与 RC 低通滤波的时间常数之间有对应关系，如表 4-4-1 所示。对于滤波参数，如果没有特殊要求，一般选择默认值。

表 4-4-1 滤波参数值与 RC 低通滤波的时间常数对应关系

滤波参数值	对应 RC 低通滤波的时间常数
1（默认值）	（不滤波）
2	80ms
4	160ms
8	320ms
16	640ms
32	1280ms

- Deadband 表示用户设定的死区值，数值范围为 0-4080。在启动滤波功能时，当模拟量的当前采集结果和上次滤波后的值之差超过设定死区值时，LM3310 直接输出当前采集结果；否则输出经过滤波后的转换结果。死区值为 0 表示禁用死区参数。

LM3310 通道参数设置如图 4-4-10 所示。通过选择“Value”中的 4-20mA、0-20mA 或 0-10V 等不同的输入信号类型，可以确定该通道模拟量输入信号的类型。表 4-4-2 给出了输入信号量程与输入数据范围的对应关系。关于各种模拟量模块的参数设置，详见手册后面的附录部分。关于模拟量输入模块的具体技术规格，请参见《HOLLiAS-LEC G3 系列小型一体化 PLC 硬件手册》。

基本参数

通道参数

Index	Name	Value	Default	Min.	Max.
1	XFactor	4-20mA	4-20mA		
2	Channel_Enable	Enable	Enable		

图 4-4-10 LM3310 通道参数设置

表 4-4-2 LM3310 输入信号量程与输入数据范围的对应关系

拨码开关设置	信号模式	量程范围	量程所对应的输入数据范围	
			十进制值	十六进制值
OFF	电压信号	0～10V	0～65535	0x00～0xFFFF
OFF	电流信号	0～20mA	0～65535	0x00～0xFFFF
ON	电流信号	4～20mA	0～65535	0x00～0xFFFF

4. 4. 3 模拟量输出模块

在设置扩展模块参数之前，同样也需要对模拟量输出扩展功能块 ANALOG_OUT 及其应用做一简单介绍。这里以 2 通道的模拟量输出模块 LM3320 为例进行介绍。

➤ 功能块示例

POU 语言	程 序
梯形图（LD）	ANALOG_OUT — EN — Address Q —

➤ 输入输出说明

数据输出存放地址	含义		
%QWxx	PLC 配置中所规定的 QW 区地址依次对应模拟量输出模块的通道		
输入	功能	数据类型	值
EN	使能	BOOL	0: 无效，不对模拟输出模块进行扫描 1: 使能，对模拟输出模块进行扫描
Address	该模块节点号	BYTE	0—7（与 PLC 配置里的节点号一致）
输出	功能	数据类型	值
Q	是否正确输出数据	BOOL	0: 未输出数据 1: 正确输出数据

◇ 举例（Analog_OUT 功能块的用法）

变量定义与梯形图如图 4-4-11 所示，需要在功能块 ANALOG_OUT 中的 Address 处输入值。功能块 ANALOG_OUT 中 Address 处的输入值 1 应当与图 4-4-12 所示的模拟量输出模块配置表中光标所在处的节点 id 号 1 一致。

如果要使用多个模拟量输出模块，则需要配置多个 Analog_OUT 功能块，每个功能块对应的 Address 输入值应当与对应模块的节点 id 号保持一致。

在图 4-4-11 所示的梯形图中，当 EN 置位时，对模拟量输出模块进行扫描，2 个通道的输出值分别存储在图 4-4-12 所示的%QW2、%QW4 中。当 EN 复位时，不对模拟量输出模块进行扫描。

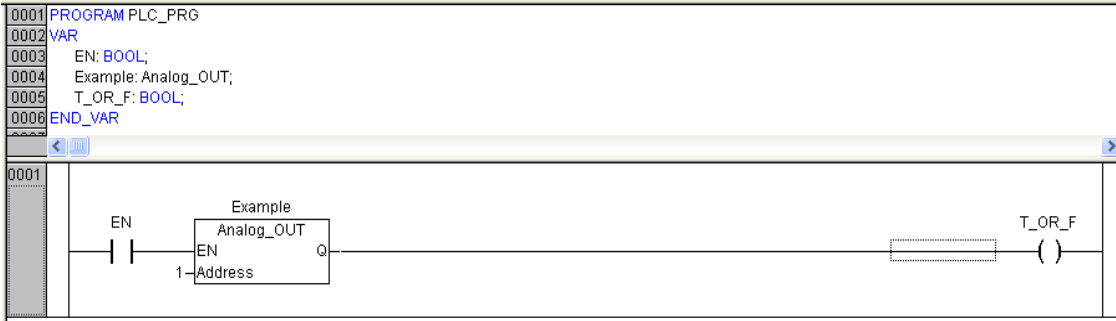


图 4-4-11 模拟量输出模块变量定义与梯形图

图 4-4-12 模拟量输出模块配置表

以上简单介绍了 ANALOG_OUT 功能块在 LD 语言中的应用。LM3320 模块通道参数设置如图 4-4-13 所示，通过选择“Value”中 0-20mA 和 0-10V 等不同的输出信号类型，可以确定该通道模拟量输出信号的类型。表 4-4-3 给出了 LM3320 输出信号量程与输出数据范围的对应关系。关于模拟量输出模块的具体技术规格，请参见《HOLLIAS-LEC G3 系列小型一体化 PLC 硬件手册》。

图 4-4-13 LM3320 通道参数设置

表 4-4-3 LM3320 输出信号量程与输出数据范围的对应关系

信号模式	量程范围	量程所对应的输入数据范围	
		十进制值	十六进制值
电压信号	0~10V	0~1638	0x00~0x666
电流信号	0~20mA	0~3277	0x00~0xCCD

4.5 保存工程

在主菜单中选择“文件”/“保存”菜单，或在工具栏中点击“”按钮，可以保存当前工程。

在“文件名”中填入新建工程的文件名，建议使用具有一定实际含义的字母或数字。

“保存类型”选择“*.pro”，工程文件将保存在默认目录\Hollysys\PowerPro\Projects 下，如图 4-5-1 所示。

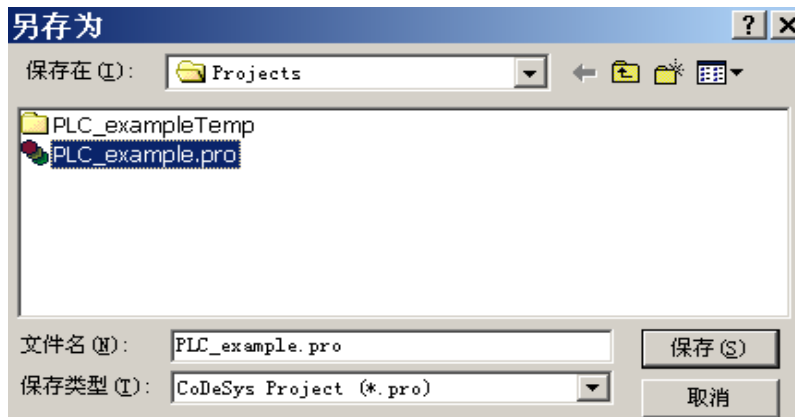


图 4-5-1 保存工程 (1)

工程文件保存后，主界面左上角的“（Untitled）*”变为保存的工程文件名，如图 4-5-2 所示。在创建工程的整个过程中，要养成随时存盘的习惯，以免由于误操作而造成数据的丢失。当工程内容改变却未保存时，主界面左上角工程名的后面会出现一个“*”号，保存后“*”号消失。

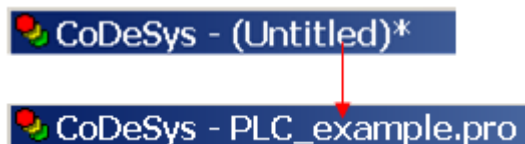


图 4-5-2 保存工程 (2)

可以使用“工程”/“选项”菜单，或者“资源”选项卡中的“工程选项”，来设置系统的某些属性。使用选择“工程”/“选项”会自动弹出设置对话框。在对话框的左边选定一个分类后，在右边出现相应的设置选项。所做的更改立即生效，同时被保存入初始化文件中，并在下一次启动系统时自动载入。



本章小结

本章介绍了工程创建的整个过程，包括目标设置、创建主程、PLC 模块配置序和扩展模块配置等。尤其对扩展模块的参数设置作了详细的说明。在使用软件的过程中，要注意模块参数和通道参数配置的正确性，因为这将直接影响程序能否正常下装及通讯是否正确。

第5章 IEC 编程基础

PowerPro 软件遵循国际电工技术委员会（International Electrotechnical Commission，缩写为 IEC）的 IEC61131-3 标准。本章主要介绍 LD、FBD、IL、ST、SFC 及 CFC 等 IEC 标准编程语言。

5.1 POU 的基本概念

POU 是程序组织单元(Program Organization Unit)的简称。POU 可以是函数(Function)、功能块(Function Block)或程序(Program)。POU 由声明部分和代码部分组成。POU 的编程语言可以是 LD、FBD、IL、ST、SFC 及 CFC 等，如表 5-1-1 所示。PowerPro 软件提供了上述 6 种功能强大的 IEC 标准编程语言，用户可以根据需要灵活地选择其中的一种或几种语言进行程序的编写。所有的编程语言在编程时不区分大小写。如果出现小写的操作指令和关键字，换行后会自动转化成大写格式。

表 5-1-1 编程语言

语言	功能说明
LD	梯形图（Ladder Diagram）语言。用图形化的方法实现各种运算。以节为计算顺序划分单位。
FBD	功能块图（Function Block Diagram）语言。用图形化的方法实现各种运算。以节为计算顺序划分单位。
IL	指令列表（Instruction List）语言。类似于汇编语言，运行速度快，但是可读性较差。
ST	结构化文本（Structured Text）语言。类似于 PASCAL 和 BASIC 等语言，易于编写各种循环嵌套计算、复杂数学计算和先进智能计算等高级应用程序。
SFC	顺序功能图（Sequential Function Chart）语言。非常适合顺序控制领域，例如控制设备的启停、连锁和保护等逻辑。
CFC	连续功能图（Continuous Function Chart Editor）语言。实际上是 FBD 的另外一种形式。在整个程序中，可以自定义运算块的计算顺序，易于实现流程运算。

5.1.1 POU 的类型

POU 分为程序(Program)、功能块(Function Block)和函数(Function)等三种类型。

- 程序(Program)是为了完成某项任务而编写的语句序列，是一组指令的集合。程序是唯一可执行的 POU，是逻辑执行的主体。程序可以通过任务组态来激活，也可以通过其它程序来调用。
- 功能块(Function Block)是预先编好的、实现某种运算的程序。功能块在执行时，输入量可以是一个或多个值，输出量可以是一个或多个执行结果。与函数不同，功能块本身没有返回值。
- 函数(Function)也是预先编好的、实现某种运算的程序。函数在执行时，会针对一系列特定的输入，产生一个输出结果，这个输出结果被赋给函数本身，称为返回值。函数只能被其它 POU 调用，函数本身不能单独执行。

5.1.2 POU 的调用

POU 的调用有两种方法。

- 被其它已经调用的 POU 来调用。
- 通过任务配置来调用，这种方法仅限于程序调用。当程序中没有进行任务配置时，系统会自动调用主程序 PLC_PRG。

POU 的调用要遵循以下原则，如图 5-1-1 所示。

- 程序可以调用函数、功能块和其它程序。
- 功能块可以调用函数和其它功能块。
- 函数可以调用函数。

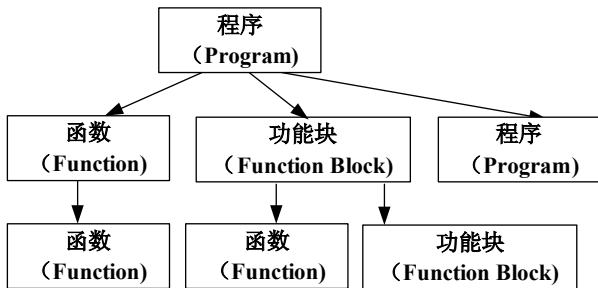


图 5-1-1 POU 的调用

5.1.3 POU 的组成

POU 的组成如图 5-1-2 所示。所有的 POU 都包含一个声明部分和一个代码部分。代码部分可以选用 IEC 标准编程语言的一种或几种来编写。不同性质的 POU，其标题名称也不相同。函数必须指定数据类型。

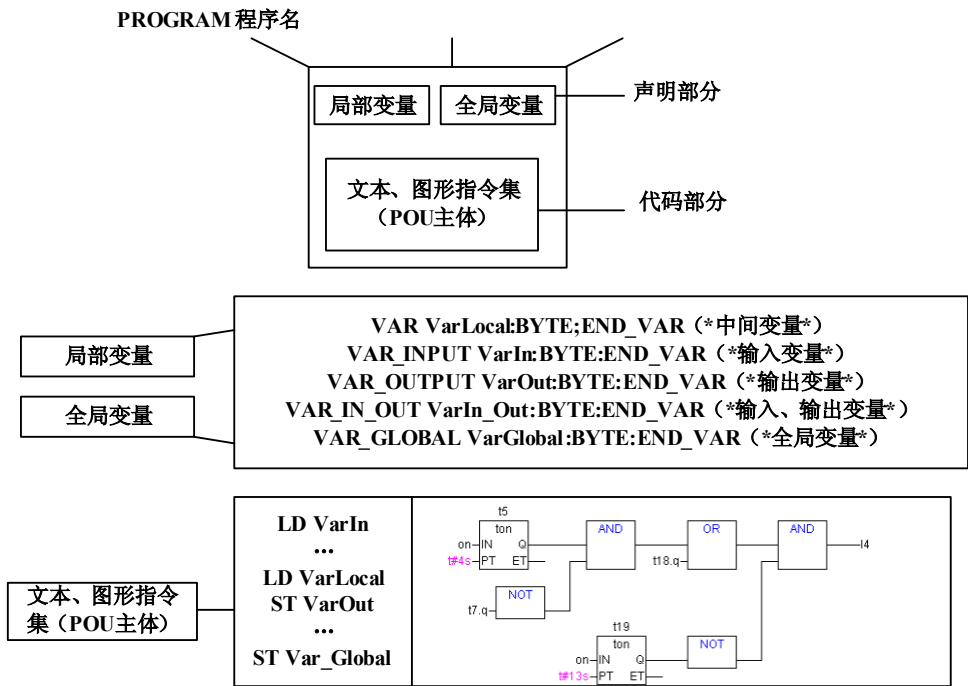


图 5-1-2 POU 的结构图

5.1.4 主程序 PLC_PRG

程序 PLC_PRG 被默认为主程序，是一个特殊的 POU。每个工程必须包含这个主程序才能正常运行。系统默认每个控制周期调用一次这个 POU，不需要进行额外的任务组态。所以，工程必须以 PLC_PRG 为主程序，通过它来实现对其它 POU 的调用。

5.2 梯形图 LD

LD 是梯形图（Ladder Diagram）的简称。LD 是一种图形化的编程语言。用 LD 可以方便地构造逻辑运算。LD 主要由触点、线圈、功能块和连接线等编程元件组成。LD 通过水平线和垂直线连接成平面网状图。一般称最左边的垂直线为“能量线”，其状态永远是真（TRUE）。各编程元件以一定的规则互相连接，最终连接到这条能量线上，形成一个个“节”、“段”或“网络”，完成特定的逻辑运算，如图 5-2-1 所示。

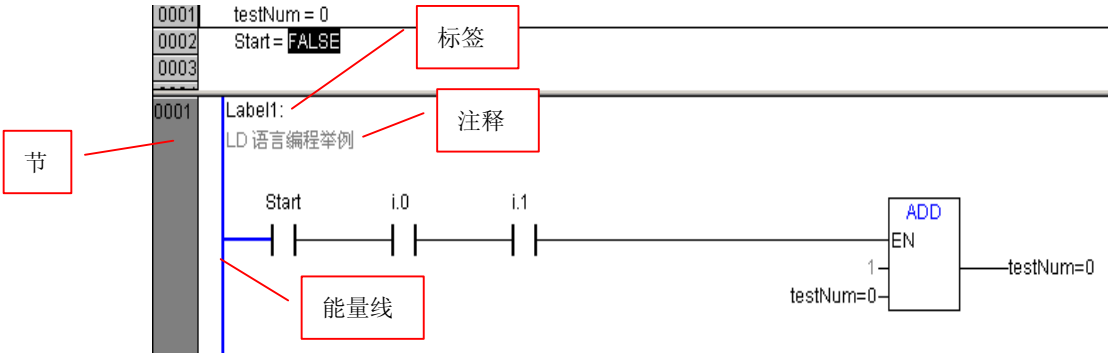


图 5-2-1 梯形图编辑器

在工作区域的代码部分点击右键，在右键快捷菜单中可以看到当前状态常用的命令，如图 5-2-2 所示。

- 前节、后节：在光标所在当前节的前面或后面增加空节。
- 串联、并联：在光标位置添加与之串联或并联的触点。
- 功能块：在光标位置插入功能块。
- 输出：在光标所在点添加线圈。
- 使能运算符：在光标位置添加带有使能端的 IEC 运算符、函数、功能块或子程序。
- 跳转：设置一个跳转。如果条件为真，则跳转。
- 返回：如果当前 POU 被其它 POU 调用，当返回条件为真时，返回到调用它的 POU。
- 注释：可以在每一节中加入注释，增加程序的可读性。

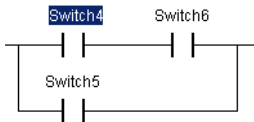
前节(N)	
后节(E)	Ctrl+T
串联(Q)	Ctrl+K
并联(P)	Ctrl+R
功能块(F)	Ctrl+B
输出(L)	Ctrl+L
使能运算符(E)	
插入块(I)	
跳转(J)	
返回(B)	
注释(C)	

图 5-2-2 右键快捷菜单

5.2.1 光标位置

下面介绍梯形图可能的光标位置，以便在使用“插入”菜单下的各项命令时，识别当前位置。

- 文本处（光标位置 1）：



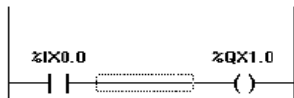
- 触点或功能块处（光标位置 2）：



- 线圈处（光标位置 3）：



- 线圈和触点之间的连接线处（光标位置 4）：



5.2.2 操作说明

- 添加节

“插入”/“前节”、“后节”命令在 LD 编辑器中添加一个新节。等同于右键菜单的“前节”、“后节”。

- 输入标签

每个网络都有一个标签，缺省为空。标签在每个网络的首行，鼠标直接点击即可输入标签。

- 插入节注释

每个节可有多行节注释。“插入”/“注释”命令可在当前节插入一个注释，默认文本为“Comment”。在“右侧串联”/“选项”的“最大注释”字段中，可以设定输入节注释使用的最大行数（缺省值是 7）。在“最小注释”字段中可设定输入节注释使用的最小行数（缺省值是 0），如图 5-2-3 所示。例如，如果输入数字 2，那么在每一个节起始的标签行之后会有两个空行。如果最小注释大于 0，系统会自动留出注释行，点击注释行可直接输入注释。与程序文本相比，注释文本显示为灰色。



图 5-2-3 注释设定

➤ 插入“串联”触点

快捷菜单：。在当前位置前串联插入一个触点。

如果标记位置是一个线圈（光标位置 3）或触点和线圈之间的连接线（光标位置 4），那么新的触点会被顺序连接到前一个触点。

触点标记文本缺省值为“???”。点击文本输入所要的变量或常量。此时，可以使用输入助手（快捷键 F2），直接从变量列表中选择输入，如图 5-2-4 所示。

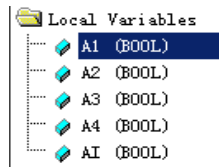


图 5-2-4 变量列表

➤ 插入“并联”触点

快捷菜单：。在光标的标记位置处并联插入一个触点。

如果标记位置是一个线圈（光标位置 3）或触点和线圈之间的连接线（光标位置 4），那么新的触点会并联到触点上。

触点标记文本缺省值为“???”。点击文本输入所要的变量或常量。此时，可以使用输入助手（快捷键 F2），直接从变量列表中选择输入，如图 5-2-4 所示。

➤ 添加“输出”

快捷菜单：。在光标的标记位置输出一个线圈。

如果标记位置是一段位于触点和线圈之间的连接线（光标位置 4），那么新线圈平行插入到线圈下面。如果标记位置是一个线圈（光标位置 3），那么新线圈平行插入到上面。

线圈标记文本缺省值为“???”。点击文本输入所要的变量或常量。此时，可以使用输入助手（快捷键 F2），直接从变量列表中选择输入，如图 5-2-4 所示。

线圈只能是并联的，从左向右传递逻辑值，并且把值保存到合适的逻辑变量中。可以预置状态 ON（对应逻辑变量 TRUE 值）或者 OFF（对应逻辑变量 FALSE 值）。

➤ “反向”操作

快捷菜单：。触点和线圈取非。

如果线圈取非，则取非后的值会被保存到对应的逻辑变量中。如果触点取非，只有当逻辑值是 FALSE 时，才能连通。

➤ “置位/复位”操作

快捷菜单：。线圈可以定义成置位或者复位状态。

用线圈符号（S）表示一个置位线圈，一旦设置为 TRUE 值，置位线圈将一直保持为 TRUE，直到被重新复位。

用线圈符号（R）表示一个复位线圈，一旦设置为 FALSE 值，复位线圈将一直保持为 FALSE，直到被重新置位。

➤ 插入功能块和使能运算符

LD 语言可以用触点来控制其它的 POU，也可以直接控制其它的 POU。这里的 POU 可以是使能运算符、函数、程序或者功能块。

当插入一个功能块时，会自动弹出帮助窗口，可以选择标准或自定义的程序和功能块。

当插入一个使能运算符时，会出现带有一个 EN 标志的使能输入端。使能输入端 EN 的输入为 BOOL 类型。当使能输入端 EN 为 TRUE 值时，运算才被执行，如图 5-2-5 所示。

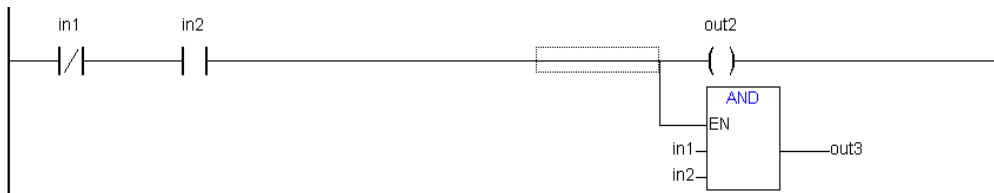


图 5-2-5 插入使能运算符

可以借助帮助来输入使能运算符的关键字。用鼠标激活运算符关键字（默认为“AND”），按下快捷键 F2，或者调用主菜单“编辑”/“提示输入”命令，在弹出的帮助窗口中选择合适的运算符，便添加了相应的使能运算符，如图 5-2-6 所示。

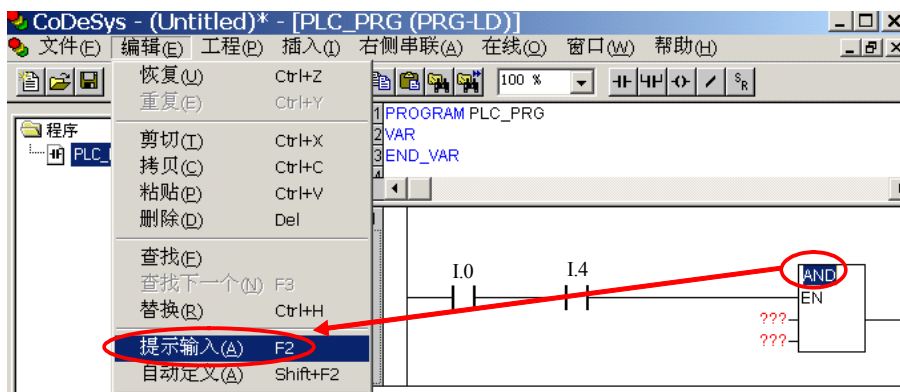


图 5-2-6 自动提示输入运算符

在程序中加入带使能端的功能块和函数，使得逻辑控制更加方便，如图 5-2-7 所示。

在应用中经常会混淆“功能块”与“使能运算符”的概念。其实二者是有严格区别的，“功能块”与“使能运算符”具有不同的调用形式。对于“功能块”，其自带使能端，无论是否使能，在程序运行时均会执行该功能块。对于“使能运算符”，只有在使能端 EN 有效时，才可以调用该使能运算符。

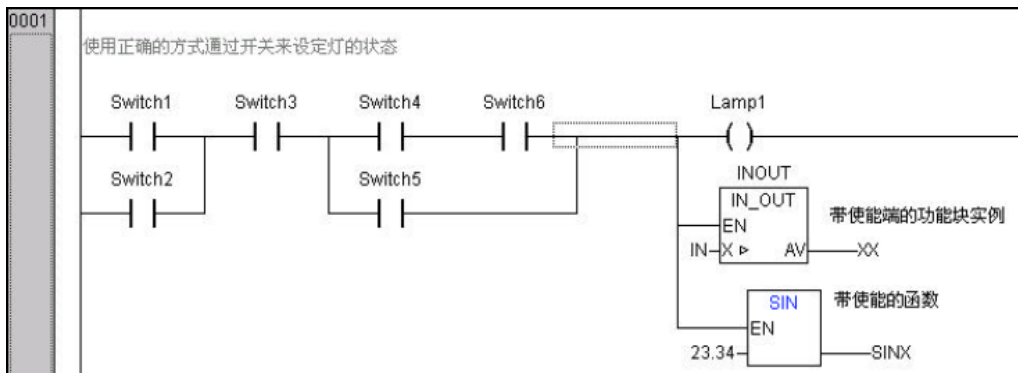


图 5-2-7 带使能端的功能块和函数

5.3 功能块图 FBD

FBD 是功能块图（Function Block Diagram）的简称。FBD 是一种图形化的编程语言，与 LD 的结构类似。FBD 由一系列“节”组成，每“节”由一系列方块组成。每“节”完成一段相对独立的运算，这些运算可以包括逻辑表达式、算术表达式、功能块、连线、输入、输出、跳转和返回等，如图 5-3-1 所示。

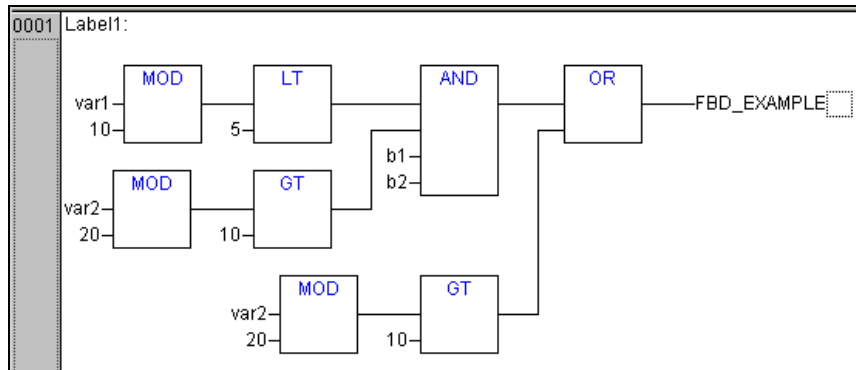
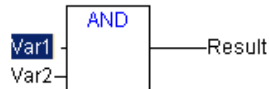


图 5-3-1 功能块图语言

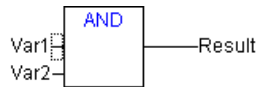
5.3.1 光标位置

通过虚线矩形框确认当前的光标位置。下面介绍可能的光标位置，便于在使用“插入”菜单下的各项命令时识别当前位置。

- 文本（光标位置 1）：



- 输入（光标位置 2）：



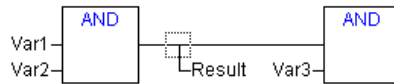
- 操作符、函数或功能块（光标位置 3）：



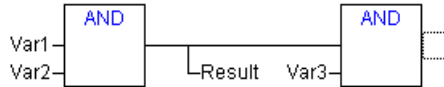
- 输出（光标位置 4，后面紧跟着赋值或跳转）：



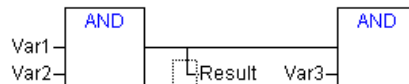
- 赋值、跳转或返回的线段交叉点（光标位置 5）：



- 节的末尾输出端（光标位置 6）：



- 赋值前面的交叉线（光标位置 7）：



5.3.2 操作说明

- 添加“输入”

快捷菜单：。在当前光标位置插入一个函数或功能块的输入端。

对于某些运算符，输入的数量是变化的，有时需要扩展运算符的输入。例如，ADD 可以是两个数相加，也可以是更多的数相加。选中输入（光标位置 2），插入的新输入成为功能块的第一个输入。如果要插入一个位于末端位置的输入，必须选中功能块本身（光标位置 3）。插入的输入缺省值为文本“???”。点击选中文本，改变成所需要的常量或变量。对此可以使用输入辅助。

在需要的时候，增加功能块的输入可以大大简化程序，如图 5-3-2 所示。

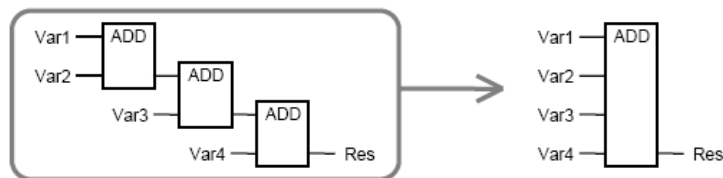


图 5-3-2 扩展输入端

如何快速切换输入端？位于运算符右边的分支与函数的第一个输入端相连，但此时需要改为与第二个输入端相连。可以选中第一个输入执行命令“编辑”/“剪切”。然后选中第二个输入执行命令“编辑”/“粘贴”。这样，此分支连接到第二个输入端。

- 添加“输出”

快捷菜单：。在当前光标位置插入一个功能块的输出端。

对于某些功能块，输出的数量可能是变化的。此命令可以扩展功能块的输出。

➤ 插入“运算符”

快捷菜单: 。在当前位置插入一个运算符。

如果输入被选中（光标位置 2），运算符被插入到输入的前面。此运算符的第一个输入连接到选中输入的左边分支，新运算符的输出连接到选中的输入。

如果输出被选中（光标位置 4），运算符被插入到输出的后面。此运算符的第一个输入连接到选中的输出，新运算符的输出连接到原来连接的分支上。

如果运算符、函数或功能块被选中（光标位置 3），旧的元素被新的运算符代替。分支的连接与没有被替换之前的情形相同。如果旧元素有比新运算符更多的分支，那么多余的分支将被删除。

如果一个跳转或返回被选中，那么运算符会插入到跳转或返回之前。运算符的第一个输入与选中的元素的左边的分支相连，运算符的输出连接到选中元素的右边的分支。

如果“节”的最后一个光标位置被选中（光标位置 6），那么运算符会被插入到最后一个元素之后，运算符的第一个输入连接到选中位置的左边分支。

被插入的运算符缺省总是 AND。选中关键字，可以把 AND 转换成其它的运算符。也可以借助“提示输入”从运算符类型列表中选择所要的运算符。新运算符的输入端将被自动连接到前面分支上。所有没有连接的输入端都标以“???”，可以将其删除或改变成所要的常量或变量。

➤ 添加“赋值”

快捷菜单: 。插入一个赋值符号，将运算符的结果赋值输出。

根据当前选中位置的不同，插入的位置也有所不同。具体地说，赋值符号插入到输入（光标位置 2）的前面，在输出（光标位置 4）的后面，在交叉线（光标位置 5）的前面，在节的末尾（光标位置 6）的后面。

为了给一个已存在的赋值插入一个附加赋值，使用“输出”命令。

➤ 设置“跳转”

快捷菜单: 。设置一个跳转。如果条件为真，则跳转到指定位置。

根据选中的位置，插入到输入的前面（光标位置 2）、输出的后面（光标位置 4）、交叉线的前面（光标位置 5）或节末尾（光标位置 6）的后面。

对于插入的跳转，可以被赋给它的标签代替。

➤ 插入“返回”

快捷菜单: 。插入一个返回。

在当前 POU 被其它 POU 调用，且返回条件为真时，返回到调用它的 POU。

根据选中的位置，插入到选中的输入的前面（光标位置 2）、选中的输出的后面（光标位置 4）、选中的交叉线前面（光标位置 5）或“节”的末尾（光标位置 6）。

➤ “反向”操作

快捷菜单: 。反向输入、输出、跳转或返回指令。

反向的图标是一个连接的小圆。如果光标选中输入（光标位置 2），那么输入被反向。如果选中输出（光标位置 4），那么输出被反向。如果选中跳转或返回，那么跳转或返回的输入端被反向。再次执行反向命令，则取消反向。

➤ “置位/复位”操作

快捷菜单：。定义输出为置位输出或复位输出。

置位输出显示为 S，复位输出显示为 R，如图 5-3-3 所示。如果输出 TRUE 值，则置位端设为 TRUE 并一直保持此值。如果输出 FALSE，则复位端设为 FALSE 并一直保持此值。此命令多次交替执行，输出在置位、复位和正常值之间交替。

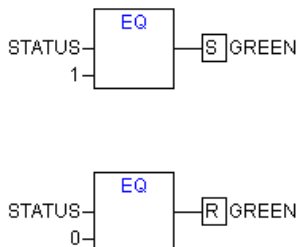


图 5-3-3 置位与复位

FBD 语言的简单应用示例如图 5-3-4 所示。本程序可以产生“1s 断 2s 通”的脉冲信号。

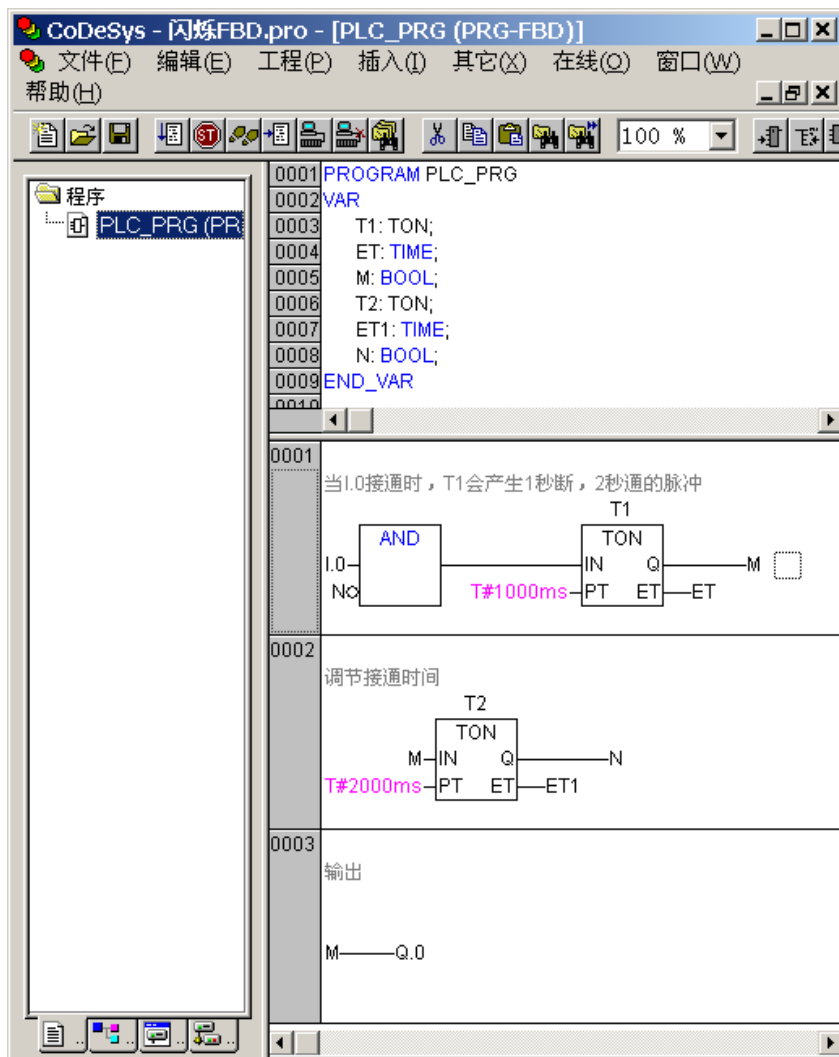


图 5-3-4 FBD 语言示例

5.4 指令列表 IL

指令列表（IL）是一种汇编语言风格的编程语言，程序不易阅读，但执行速度最快。IL 语言包含一系列的指令，每条指令占据一行，包含一个运算符和一个或多个用逗号隔开的操作数。操作数之间用逗号分隔。每行开始可以有标签，标签后要有冒号。每行结束可以有注释，注释用“（* *）”括起来。每行指令之间可以插入空行。IL 编辑器是一种文本编辑器，具有常见 Windows 文本编辑器的功能，点击菜单栏或鼠标右键可以进行编辑。

5.4.1 操作说明

打开“插入”菜单，选择所要插入的内容，如图 5-4-1 所示。

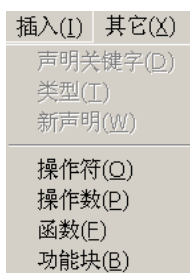


图 5-4-1 “插入”菜单

➤ “插入”/“运算符”

在编程区任何位置点击此命令，从弹出窗口中选择所需的运算符，例如“ABS”。

➤ “插入”/“操作数”

在编程区任何位置点击此命令，从弹出窗口中选择所需的操作数，点击“确认”。

➤ “插入”/“函数”

在编程区任何位置点击此命令，从弹出窗口中选择所需的函数，点击“确认”。

➤ “插入”/“功能块”

在编程区任何位置点击此命令，从弹出窗口中选择所需的功能块，点击“确认”。

5.4.2 程序举例

◇ 举例

下面是一个用 IL 语言实现的简单运算程序。

声明：

VAR

A: REAL;

B: REAL;

C: REAL;

END_VAR

程序:

```
LD      10    (*将数字 10 赋予当前值*)
ADD     A     (*当前值与变量 A 加运算后结果存入当前值*)
GE      B     (*当前值与变量 B 进行大于等于比较*)
JMPCL Next1  (*上一表达式结果为真，则跳转至标志 Next1 处*)
LD      A     (*将变量 A 的值赋予当前值*)
ADD     B     (*当前值与变量 B 加运算后结果存入当前值*)
ST      C     (*将当前值赋予变量 C*)
JMP Next2    (*无条件跳转至标志 Next2 处*)
Next1:   (*标志*)
LD      A     (*将变量 A 的值赋予当前值*)
SUB     B     (*当前值与变量 B 减运算后结果存入当前值*)
ST      C     (*将当前值赋予变量 C*)
Next2:   (*标志*)
```

IL 语言支持两种修饰符 C 和 N。C 表示条件执行，只有当前一个表达式的值为真（TRUE）时，指令才被执行。N 与 JMP、CAL 和 RET 连用时表示条件非执行，只有当前一个表达式的值为假（FALSE）时，指令才被执行；其它情况下，表示操作数取负。表 5-4-1 列出了 IL 的所有运算符以及它们的修饰符和相应意义。

表5-4-1 IL语言运算符列表

运算符	修饰符	意 义
LD	N	将操作数赋予当前值
ST	N	将当前值赋予操作数
S		如果当前结果是 TRUE，把布尔型操作数置为 TRUE
R		如果当前结果是 TRUE，把布尔型操作数置为 FALSE.
AND	N、(	位逻辑运算符 AND
OR	N、(	位逻辑运算符 OR
XOR	N、(	位逻辑运算符 XOR
ADD	(	加
SUB	(	减
MUL	(	乘
DIV	(	除
GT	(	>大于判断
GE	(	>=大于等于判断
EQ	(	=等于判断
NE	(	<>不等于判断
LE	(	<=小于等于判断
LT	(	<小于判断
JMP	C、N	转移到标签
CAL	C、N	调用其它 POU
RET	C、N	退出 POU 并且返回到调用处
)		计算延迟操作

◇ 举例

以下是利用修饰符进行 IL 编程的例子：

```
LD    TRUE    (*将 TRUE 赋予当前值*)
ANDN  BOOL1    (*BOOL1 变量取反后与当前值进行与运算*)
JMP   mark    (*如果结果是真，那么跳转到标签“mark”处*)
LDN   BOOL2    (*将 BOOL2 取反后赋予当前值*)
ST    ERG     (*将当前值存入 ERG*)
```

Mark:

```
LD    BOOL2    (*将 BOOL2 赋予当前值 *)
ST    ERG     (*将当前值存入 ERG*)
```

如果在运算符之后插入括号，那么括号里的值可以看成是一个操作数。例如：

```
LD    2
MUL   2
ADD   3
ST    ERG
```

运行后 ERG 的值是 7。但是，如果加入一对圆括号：

```
LD    2
MUL   (2
ADD   3
)
```

ST ERG

运行后 ERG 的值是 10，MUL 运算符只有到“)”才执行，等同于执行 MUL 5。

◇ 举例

IL 语言简单应用示例如图 5-4-2 所示。本程序产生“1s 断 2s 通”的脉冲信号

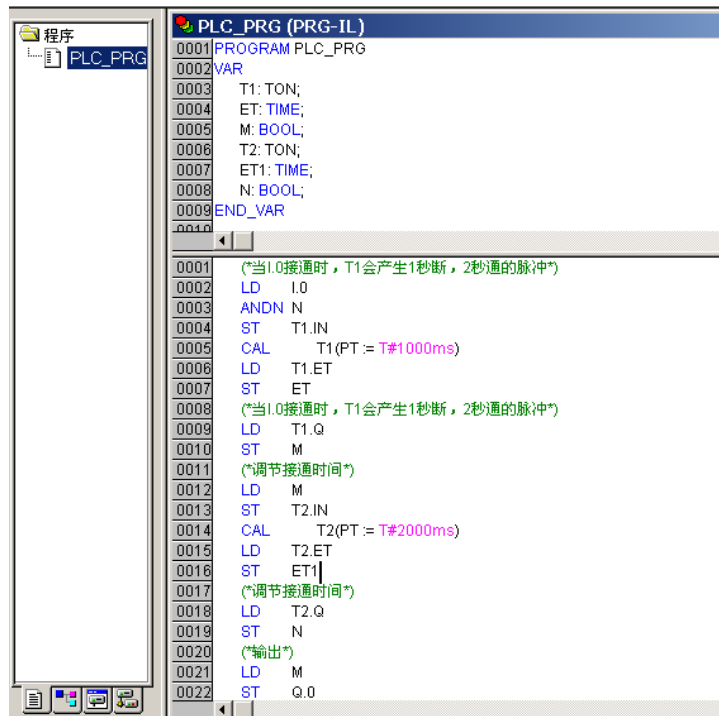


图 5-4-2 IL 语言示例

5.5 结构化文本 ST

结构化文本(ST)可以执行选择语句(IF...THEN...ELSE)和循环语句(WHILE...DO), 类似于 PASCAL 和 BASIC 等高级语言。

5.5.1 ST 表达式

ST 语言中的表达式由运算符和操作数组成。操作数可以是常量、变量、函数调用或另一个表达式。表达式的计算通过执行具有不同优先级的运算符完成。有最高优先级的运算符先被执行, 然后依次执行下一个优先级的运算符, 直到所有的运算符被处理完。有相同优先级的运算符按从左到右的顺序执行。ST 语言的运算符如表 5-5-1 所示。

表5-5-1 ST语言运算符

运算	符号	优先级
放入圆括号	(表达式)	最高优先级
函数调用	函数名 (参数列表)	
求幂	EXPT	
求负	-	
求补	NOT	
乘积	*	
除	/	
取模	MOD	
加	+	
减	-	
比较	<、>、<=、>=	
相等	=	
不等	<>	
逻辑与	AND	
逻辑异或	XOR	
逻辑或	OR	最弱优先级

5.5.2 ST 指令

ST 语言的指令如表 5-5-2 所示。

表5-5-2 ST语言指令

指令类型	例子
赋值	A:=B; CV := CV + 1; C:=SIN(X);
调用功能块并且赋初值	TP(IN:= %IX0.5, PT:=t#30);
使用功能块输出	A:=TP.Q;
返回	RETURN;
IF	D:=B*B; IF D<0.0 THEN C:=A; ELSE IF D=0.0 THEN C:=B; ELSE C:=D; END_IF;
CASE	CASE INT1 OF 1: BOOL1 := TRUE; 2: BOOL2 := TRUE; ELSE BOOL1 := FALSE; BOOL2 := FALSE; END_CASE;
FOR循环	J:=101; FOR I:=1 TO 100 BY 2 DO IF ARR[I] = 70 THEN J:=I; EXIT; END_IF; END_FOR;
WHILE循环	J:=1; WHILE J<= 100 AND ARR[J] <> 70 DO J:=J+2; END_WHILE
REPEAT循环	J:=-1; REPEAT J:=J+2; UNTIL J= 101 OR ARR[J] = 70 END_REPEAT;
退出程序	EXIT;
空指令	;

➤ 赋值

执行赋值操作时，等号左边是操作数（变量或地址），右边是被赋予的表达式的值。

例如：Var1 := Var2 * 10;

➤ 调用功能块

通过写入功能块实例的名字，并且在随后的圆括号中赋给参数值来调用功能块。例如：
变量声明：

TPInst:TP;

VarBOOL1: BOOL;

VarBOOL2: BOOL;

程序：

TPInst(IN:= VarBOOL1,PT:= T#5s); (*参数 IN 和 PT 设定时钟脉冲的触发信号和高电平的长度*)

VarBOOL2:=TPInst.Q; (*输出脉冲值 Q 赋给变量 VarBOOL2*)

➤ 返回指令

返回指令可以根据条件退出 POU。

➤ IF 指令

使用 IF 指令可以检查条件，根据条件执行相应的指令。

语法：

IF <逻辑表达式> THEN

<IF 指令>

{ELSIF <逻辑表达式 1> THEN

<ELSE IF 指令 1>

ELSIF <逻辑表达式 n> THEN

<ELSE IF 指令 n>

ELSE

<ELSE 指令>}

END_IF;

其中{}的部分可选。

如果<逻辑表达式>返回 TRUE，那么只有<IF 指令>被执行，其它的指令不被执行。同样，从<逻辑表达式 1>开始，相继执行逻辑表达式，直到其中一个表达式返回 TRUE 为止，返回 TRUE 的逻辑表达式对应的指令被执行。

如果没有逻辑表达式生成 TRUE，那么只有<ELSE 指令>被执行。例如：

IF temp<17

THEN heating_on := TRUE;

ELSE heating_on := FALSE;

END_IF;

这里，如果温度低于 17 度，打开加热器，反之则保持关闭状态。

➤ CASE 指令

使用 CASE 指令，可以在结构中用一个相同的条件变量表示几个条件指令。

语法：

```
CASE <Var1> OF  
<Value1>: <指令 1>  
<Value2>: <指令 2>  
<Value3, Value4, Value5>: <指令 3>  
<Value6 .. Value10>: <指令 4>  
...  
<Value n>: <指令 n>  
ELSE <ELSE 指令>  
END_CASE;
```

CASE 指令根据下面的模型来执行：

如果变量<Var1>有值<值 i>，那么<指令 i>被执行。

如果变量<Var1>没有任何指定的值，那么<ELSE 指令>被执行。

如果变量的几个值都需要执行相同的指令，那么可以把几个值相继写在一起，并且用逗号分开。这样，就会有相同的执行指令。

如果对于变量的一个范围需要执行相同的指令，可以写入初值和终值，中间用两个点分开。这样，条件就会有相同的执行。例如：

```
CASE INT1 OF  
1, 5: BOOL1 := TRUE;  
      BOOL3 := FALSE;  
2:   BOOL2 := FALSE;  
      BOOL3 := TRUE;  
10..20: BOOL1 := TRUE;  
        BOOL3:= TRUE;  
ELSE  
      BOOL1 := NOT BOOL1;  
      BOOL2 := BOOL1 OR BOOL2;  
END_CASE;
```

➤ FOR 循环

使用 FOR 循环，可以编写循环过程。

语法：

```
INT_Var :INT;  
FOR <INT_Var> := <INIT_VALUE> TO <END_VALUE>  
{BY <Step Size>} DO  
<Instructions>  
END_FOR;
```

其中{}的部分可选。

只要计数<INT_Var>不大于<END_VAULE>，指令就会被执行。指令执行之前，首先检查这个条件，如果<INIT_VALUE>大于<END_VALUE>，指令就永远不会被执行。

当指令被执行时，<INT_Var>总是增加步长<Step Size>。步长可以是任意的整数值。如果不写步长，缺省值是 1。当<INT_Var>大于<END_VALUE>时，循环结束。例如：

```
FOR Counter:=1 TO 5 BY 1 DO
```

```
Var1:=Var1*2;
```

```
END_FOR;
```

```
Erg:=Var1;
```

假定 Var1 的缺省值是 1，那么循环结束后，Var1 的值为 32。

注意，<END_VALUE>一定不能等于计数变量<INT_VAR>的极限值。如果计数变量的类型是 SINT（范围从-128-127），<END_VALUE>是 127，则会进入死循环。

➤ WHILE 循环

WHILE 循环用起来很象 FOR 循环，不同的是，结束条件可以是任意的逻辑表达式。指定一个条件，当条件满足的时候，循环被执行。

语法：

```
WHILE <逻辑表达式>
```

```
<指令>
```

```
END_WHILE;
```

只要<逻辑表达式>的值返回 TRUE，<指令>就会被重复执行。如果在第一次计算时，<逻辑表达式>的值已经是 FALSE，那么指令永远不会被执行。如果<逻辑表达式>的值永远不会是 FALSE，那么<指令>被无休止的执行，产生一个相对时间延迟。例如：

```
WHILE counter<>0 DO
```

```
Var1 := Var1*2;
```

```
Counter := Counter-1;
```

```
END_WHILE
```

在一定意义上，WHILE 和 REPEAT 循环比 FOR 循环功能更强大。因为不需要在执行循环之前计算循环次数。因此，在有些情况下，用这两种循环就可以了。然而，如果清楚知道循环次数，那么 FOR 循环更好。

➤ REPEAT 循环

REPEAT 循环不同于 WHILE 循环，因为只有在指令执行以后才检查中断条件。无论结束条件怎样，循环至少执行一次。

语法：

```
REPEAT
```

```
<指令>
```

```
UNTIL <逻辑表达式>
```

```
END_REPEAT;
```

直到<逻辑表达式>的值返回 TRUE，<指令>才停止执行。如果在第一次计算时，<逻辑表达式>产生 TRUE，那么<指令>只被执行一次，如果<逻辑表达式>不会产生 TRUE，那么<指令>将无休止的循环，导致相对时延。例如：

```
REPEAT
```

```
Var1 := Var1*2;
```

```
Counter := Counter-1;
```

```
UNTIL
```

```
Counter=0
```

```
END_REPEAT;
```


➤ EXIT 指令

如果 EXIT 指令出现在 FOR、WHILE、REPEAT 循环中，那么不管中断条件如何，EXIT 出现时循环终止。

✧ 举例

下面是一个用 ST 语言实现简单运算的小程序：

变量声明：

PROGRAM PLC

VAR

A:BOOL;

B:BOOL;

C:INT;

END_VAR

程序：

IF A=TRUE THEN

C:=20;

ELSE IF B=TRUE THEN

C:=30;

ELSE C:=50;

END_IF

➤ 简洁、易读的语言 ST

结构化文本这个名字就已经表明它是用于结构化编程的。也就是说，ST 语言为特定的经常使用的结构化编程提供了预先确定的结构。

✧ 举例

比较下面两段分别用 IL 和 ST 语言实现 2 的乘幂的循环的程序代码。

用 IL 语言：

Loop:

LD Counter

NE 0

NOT

JMPC END_LOOP

LD Var1

MUL 2

ST Var1

LD Counter

SUB 1

ST Counter

JMP Loop

End_LOOP:

LD Var1

ST ERG

用 ST 语言：

WHILE counter<>0 DO

Var1:=Var1*2;

Counter:=counter-1;

END_WHILE

Erg:=Var1;

可以看出，用 ST 语言实现的循环不仅简洁，而且易于阅读。

5.6 顺序功能图 SFC

SFC 是顺序功能图（Sequential Function Chart）的简称，是一种图形化的编程语言，用来描述程序中不同动作的时间顺序。SFC 由一系列的步和转移组成，如图 5-6-1 所示。步定义动作，转移控制顺序。

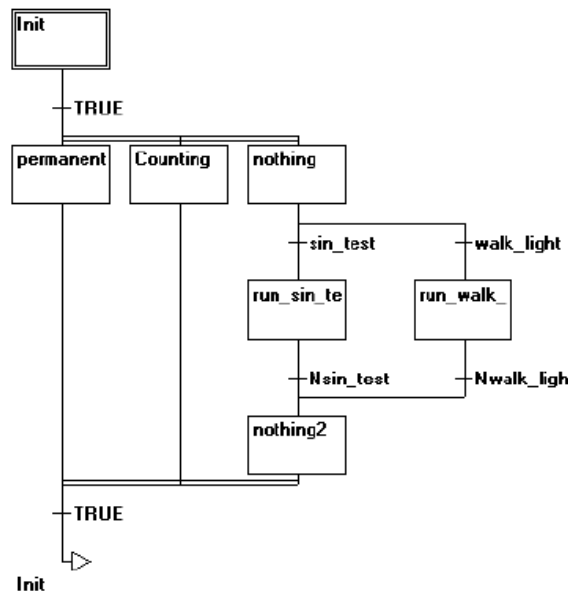


图 5-6-1 顺序功能图语言

5.6.1 基本概念

➤ 单步

SFC 语言包含一系列的步，这些步通过有向的连接彼此联系。步分为简化步和 IEC 步。

简化步包含一个动作和一个显示步动作的标志。如果步包含动作，则在步的右上角出现一个小三角形。

IEC 步包含一个或多个动作和逻辑变量。新插入的步是否为 IEC 步，取决于是否选择了“其它”/“使用 IEC 步（I）”命令。必须添加特殊的 SFC 库 Iecsfc.lib 才能够使用 IEC 步。

➤ 动作

动作是使用其它语言实现的一系列指令，可以用 IL 或 ST 语言实现的指令语句，也可以是用 LD、FBD 或 SFC 实现的网络。

对于简化步，动作总是和步直接相关。用鼠标双击动作所属的步，进行编辑。

对于 IEC 步，在对象组织器中选中 SFC 程序，点击右键，选择“添加动作”命令创建新动作。可以赋给 IEC 步多个动作，同时这些动作也可以被多个步重复使用。

IEC 步的动作显示在步的右边框中。左边字段包含可能有时间常量的限定符，右边字段包含动作名即逻辑变量名。

IEC 的动作分散在步中。在它们所属的 POU 中，这些动作可以被重复使用。使用“其它”/“关联动作”命令添加动作，使用“其它”/“清除动作或转移”命令删除已添加的动作。

包含两个动作的 IEC 步举例，如图 5-6-2 所示。

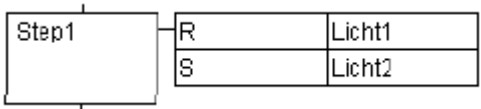


图 5-6-2 顺序功能图语言举例

➤ 入口动作和出口动作

“添加入口动作”和“添加出口动作”命令可以在步中加入入口动作和出口动作。

入口动作只有当步在活动状态时立即执行一次。出口动作只在步不活动之前执行一次。有入口动作的步在左下角有“E”标志进入，有出口动作的步右下角有“X”标志退出。可以用任意语言实现入口和出口动作，鼠标双击步的相应角，即可编辑入口或者出口动作。

入口和出口动作步的举例，如图 5-6-3 所示。

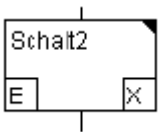


图 5-6-3 入口和出口动作步的举例

➤ 转换/转换条件

步之间的切换就是转换。只有当步的转换条件为真时，步的转换才进行。即前步的动作执行完后，如果有出口动作则执行一次出口动作，后步如果有入口动作则执行一次后步的入口动作，然后按照控制周期执行该活动步的所有动作。

转换条件可以是逻辑变量、逻辑地址、逻辑常量或者是由其它语言编程实现的逻辑。

➤ 活动步

调用 POU 之后，初始化步（双边的步）的动作首先执行。动作被执行的步称为活动步。每次循环执行活动步的动作。在线模式下，活动步以蓝色显示。为了更容易地跟随过程，在线模式下，所有的活动动作象活动步一样以蓝色显示。每次循环以后做一次检查，查看哪个动作是活动的。在一个控制周期中，执行活动步的所有动作。之后，如果下步的转换条件是 TRUE，下步将变成活动步，在下个周期执行。

➤ 限定符

把 IEC 步和动作关联，需要使用限定符。IEC 步的使用如图 5-6-4 所示，限定符对应的含义如表 5-6-1 所示。

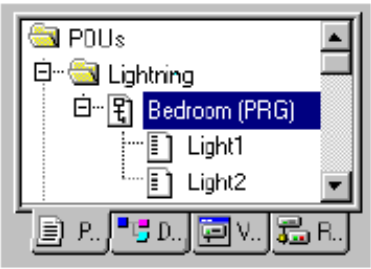


图 5-6-4 使用 IEC 步

表5-6-1 SFC限定符

限定符	名称	说明	备注
N	非存储	普通型 ：动作随步活动，当步开始激活时，动作开始执行；当步退出时，动作停止执行。	
S	设置存储	置位型 ：当步开始激活时，动作开始执行；当步退出时，动作还将继续执行。一直等到有 R 限定符给它复位之后，动作才将停止执行。在动作没有被 R 停止之前，任何限定符也不能使得动作停止（包括 P 限定符）。	
R	重置	复位型 ：当步开始激活时，动作停止执行。	
L	时限	限时结束型 ：动作在一定时间被激活。最长时间是步活动时间。当步开始激活时，动作开始执行；同时开始计时，当时间到达设定值后，动作将停止执行。如果时间还没有到达设定值时，步的转移条件已经满足，此时，动作也将停止执行。	要加时间常数 (设定值)
D	延时	延时开始型 ：当步开始激活时，开始计时，当时间到达设定值后，动作将开始执行；当步退出时，动作停止执行。如果时间还没有到达设定值时，步的转移条件已经满足，在这种情况下，动作将不可能被执行。	要加时间常数 (设定值)
P	脉冲	脉冲型 ：当步开始激活时，动作开始执行，并且动作只被执行一次，之后动作将停止执行。	
SD	存储和延时	延时绝对开始置位型 ：当步开始激活时，开始计时，当时间到达设定值后，动作将开始执行；当步退出时，动作还将执行，一直等到有 R 限定符给它复位之后，动作才将停止执行。在动作没有被 R 停止之前，任何限定符也不能使得动作停止（包括 P 限定符）。如果时间还没有到达设定值时，步的转移条件已经满足，在这种情况下，计时还在进行，一直到时间达到设定值后，动作将被执行，此时不管当前步是不是原来的步了。	要加时间常数 (设定值)
DS	延时和存储	延时开始置位型 ：当步开始激活时，开始计时，当时间到达设定值后，动作将开始执行；当步退出时，动作还将执行，一直等到有 R 限定符给它复位之后，动作才将停止执行。在动作没有被 R 停止之前，任何限定符也不能使得动作停止（包括 P 限定符）。如果时间还没有到达设定值时，步的转移条件已经满足，在这种情况下，动作将不可能被执行。	要加时间常数 (设定值)
SL	存储和时限	绝对限时结束型 ：在一定时间内动作是活动的。当步开始激活时，动作开始执行；同时开始计时，当时间到达设定值后，动作将停止执行。如果时间还没有到达设定值时，步的转移条件已经满足，此时，动作还将继续执行，直到当时间到达设定值后动作才会停止执行。限制符 L、D、SD、DS 和 SL 需要 TIME 常量格式的时间值。	要加时间常数 (设定值)

➤ SFC 中的隐含变量

在 SFC 中可以隐含声明的变量。每步的标志都存储了步的状态。

对于简化步，步标志为<步名>。对于 IEC 步，步标志为<步名>.x。

当步是活动状态的时候，步标志的值为 TRUE。当步是非活动状态的时候，步标志的值为 FALSE。

对于 IEC 步，可以使用变量<动作名>.x 询问动作是否是活动的。对于 IEC 步，可以使用隐含变量<步名>.t 用来询问步的活动时间。

➤ SFC 标志

在 SFC 中，步的活动状态时间取决于它的属性状态时间，有时会设置一些特殊标志。同样，可以设置变量来控制顺序功能图中的程序流。要使用这些标志，必须在全局或局部变量中作为输入或者输出变量声明这些标志。

SFCEnableLimit: BOOL 型变量，变量值是 TRUE 时，步的超时将会注册在 SFCWError 中，忽略其它的超时。

SFCInit: BOOL 型变量，变量值是 TRUE 时，顺序功能图返回到初始化步，同时其它的 SFC 标志重置。只要变量有 TRUE 值，初始化步保持活跃，但不被执行。只有当 SFCInit 被重新设置为 FALSE，功能块才恢复正常执行。

SFCQuitError: BOOL 型的变量，变量值是 TRUE 时，SFC 图停止执行。在变量 SFCError 中，重置一个可能的超时。当变量重新获得 FALSE 时，在活动步中所有以前的时间重置。

SFCPause: BOOL 型变量，变量值是 TRUE 时，SFC 图停止执行。

SFCError: 当超时发生在 SFC 图中时，设置这个逻辑变量。如果一个超时发生而变量 SFCError 不重置，则不会注册其它超时。

SFCTrans: BOOL 型变量。当转换执行时，变量值为 TRUE。

SFCErrorStep: STRING 型变量，存储一个导致超时发生的步名。

SFCErrorPOU: STRING 型变量，包含一个发生了超时的块名。

SFCCurrentStep: STRING 型变量，步名存储在活动变量中，不受时间监视。一旦在同时顺序下，步存储在右外侧的分支里。

5.6.2 操作说明

➤ 选中元素

被选中的元素周围用点线矩形圈住。点击鼠标左键，或用键盘的上下键可以选中元素。按住 SHIFT 键，点击鼠标左键可以选中一组元素。注意，删除步时，必须同时选中此步前转移或后转移，一起删除。

➤ “插入”/“前步转移”：，快捷键<Ctrl>+<T>

执行此命令，在选中元素前面插入步和转移。

➤ “插入”/“后步转移”：，快捷键<Ctrl>+<E>

执行此命令，在选中元素后面插入步和转移。

- “插入”/“右选择分支”：，快捷键<Ctrl>+<A>

选中元素必须以转移开始和结束，执行此命令，新插入的分支作为选中元素的右分支，新分支包括一个转移。

- “插入”/“左选择分支”：

选中元素必须以转移开始和结束，新插入的分支作为选中元素的左分支，新分支包括一个转移。

- “插入”/“右并行分支”：，快捷键<Ctrl>+<L>

选中元素必须以步开始和结束，执行此命令，新插入的分支作为选中元素的右分支，新分支包括一个步。如果此并行分支需从其它元素跳转过来，那么此并行分支还要加标签。

- “插入”/“左并行分支”：

选中元素必须以步开始和结束，执行此命令，新插入的分支作为选中元素的左分支，新分支包括一个步。如果此并行分支需从其它元素跳转过来，那么此并行分支还要加标签。

- “插入”/“跳转”：，快捷键<Ctrl>+<U>

选中元素所在的分支必须是选择分支，执行此命令，在此分支末尾插入跳转。新跳转旁边有文本字符串 Step，可以改为要跳转到的步的名字或并行分支的跳转标签。

- “插入”/“转移跳转”：

选中元素所在的分支必须是并行分支，执行此命令，在此分支末尾插入转移和跳转。新跳转旁边有文本字符串 Step，可以改为要跳转到的步的名字或并行分支的跳转标签。

- “插入”/“添加入口动作”

入口动作只在步被激活时执行一次，入口动作可以选择一种语言编写。带入口动作的步在左下角有个“E”。

- “插入”/“添加出口动作”

出口动作只在步被解除激活前执行一次，出口动作可以选择一种语言编写。带出口动作的步在右下角有个“X”。

➤ “其它”/“右侧并行粘贴”

选中元素必须是以步开始和结束，执行此命令，剪贴板的内容作为选中元素的右分支。剪贴板的内容也必须以步开始和结束。

➤ “其它”/“并行分支增加标签”

选中元素必须是并行分支的前转移，执行此命令，此并行分支的上方出现标签。标签缺省的是“Parallel”加“序列号”，可以编辑，如图 5-6-5 所示，改为“abc”，删除标签名，就可以删除跳转标签。

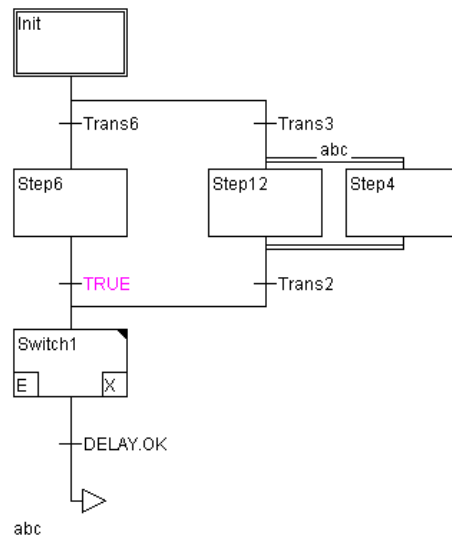


图 5-6-5 并行分支增加标签

➤ “其它”/“后面粘贴”

选中元素，执行此命令，剪贴板的内容粘贴到选中元素的第一个步或转移后面。注意，普通的粘贴（“编辑”/“粘贴”）会粘贴到选中元素的前面。

➤ “其它”/“放大动作/转移”：快捷键<Alt>+<Enter>

执行此命令，弹出编辑器，查看动作或转移条件。如果动作或转移条件是空的，执行此命令，可以在弹出窗口中选择一种语言，进入相应编辑器，编写内容。

注意，编辑器中的转移条件优先于直接在转移标记中编写的条件。如图 5-6-6 所示，如果 $i > 100$ ，那么转移条件是 FALSE，尽管转移标记中写的是 TRUE。

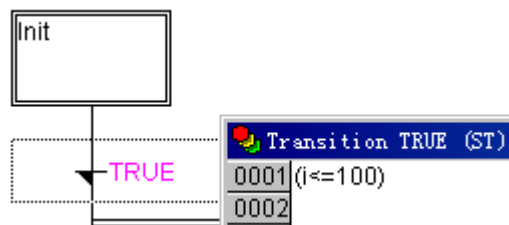


图 5-6-6 转移条件有优先级

➤ “其它”/“清除动作/转移”

执行此命令，清除动作或转移。

➤ 选择分支

在 SFC 中，两个以上的分支可定义为选择分支。每个选择分支必须开始和结束于转换。选择分支包含并行分支和其它选择分支。选择分支开始于水平行（选择开始）结束于水平行（选择结束）或者结束于跳转。

如果选择起始行之前的步是活动的，那么每个选择分支的第一个转换从左到右执行。从左边开始，转换条件是 TRUE 则第一个转换被打开，随后的步被激活。

➤ 并行分支

在 SFC 中，两个以上的分支可以定义成并行分支。每个并行分支开始和结束于一个步。并行分支开始于一个双行（并行开始）结束于一个双行（并行结束）或者是一个跳转。

如果前步的并行开始行是活动的，并且这一步的转换条件是 TRUE，那么所有并行分支的第一步都成为活动的。这些分支彼此是平行处理的。当前面的步都是活动的并且这一步之前的转换条件是 TRUE，则并行结束行之后的步被激活。

➤ 跳转

跳转实际上是连接到有跳转符号的步的操作。当不允许产生彼此交叉的连接时就需要跳转。

图 5-6-7 所示是一个 SFC 语言典型功能的举例。

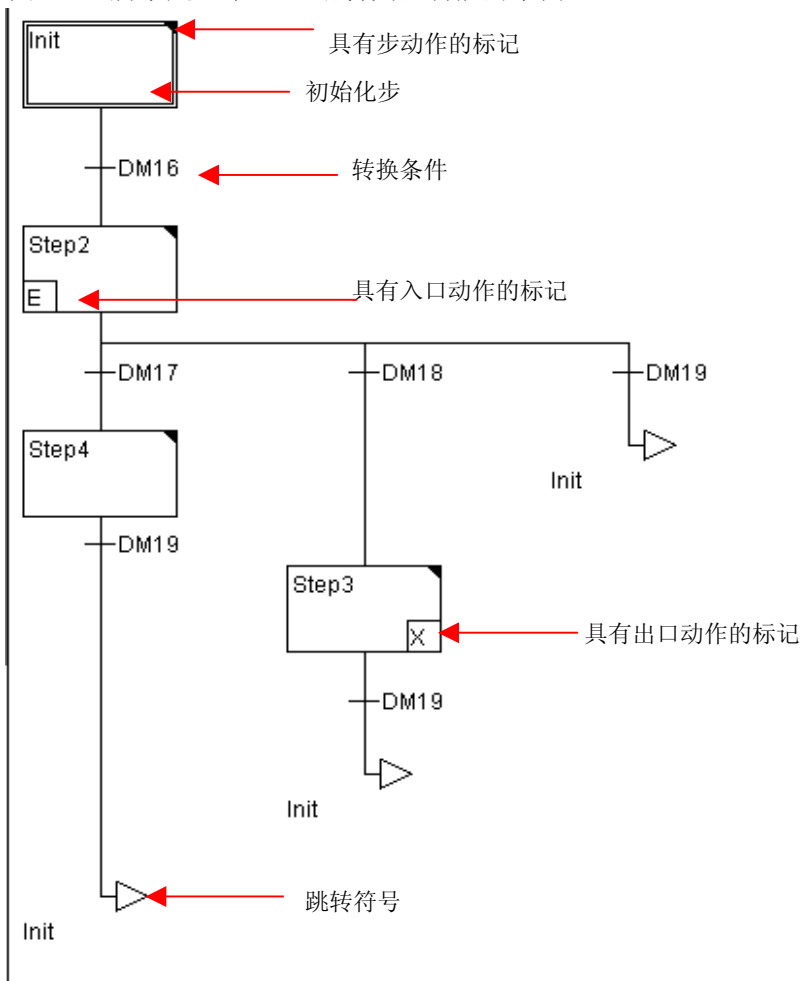


图 5-6-7 SFC 功能图解说

5.7 连续功能图 CFC

5.7.1 CFC 编辑器

CFC 是连续功能块图（Continuous Function Chart）的简称。CFC 是一种图形化的编程语言。CFC 基于 FBD 语言，但没有网络限制，摆放元素更加灵活。

CFC 编辑器是一种图形编辑器，如图 5-7-1 所示，通过菜单栏或鼠标右键可以进行编辑。元素可以摆放在编程区任意位置。用鼠标拖拽在元素之间连线，当元素移动位置时，编辑器会自动调整连线长度。当元素之间空间不够时，连线会变为红色，一旦空间够用，红线会变为普通连线。

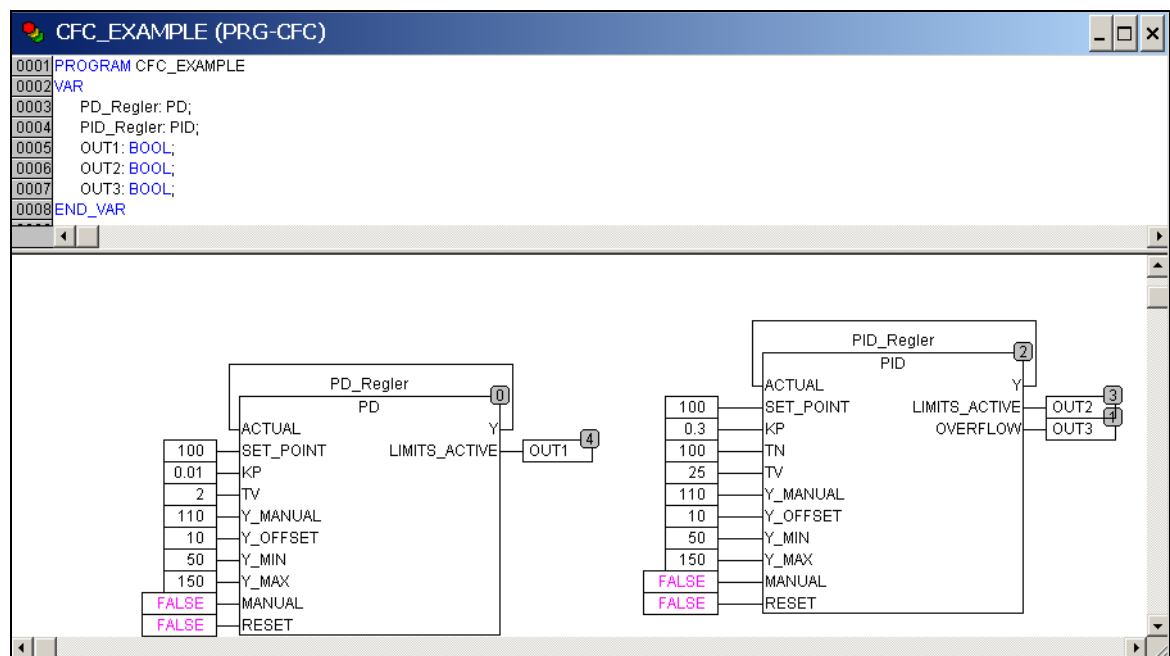


图 5-7-1 CFC 编辑器

5.7.2 操作说明

CFC 的元素包括块、输入、输出、跳转、标记、返回和注释等。其中块分为操作符、函数、功能块和程序四种形式。

➤ 光标位置

名称	图形元素	选中元素	文本域	输入引脚和输出引脚
光标位置	编程工具元素	光标处于各元素的中继线	光标处于蓝色区域及注释的光标所在位置上	光标处于元素的输入引脚和输出引脚上
块				
输入				
输出				
跳转				
标记				—
返回			—	
注释				—

➤ 选中元素

在元素中继线处点击鼠标左键，可以选中元素。

如果想同时选中几个元素，按住<Shift>键并选中单个元素。也可以用鼠标左键在编辑器中画矩形区域选中其中几个元素。“其它”/“全选”选中所有元素。

➤ 移动元素

当光标在位置 a 时，或按住<Shift>键同时选中几个元素时，按住鼠标左键可以在编辑器中移动元素，到合适的位置后释放左键。如果释放位置处已有其它元素或超出编辑区，被移动元素会跳回原位置，移动失败。

➤ 连线

一个元素的输入引脚只能连一个输出引脚（本元素的输出引脚或其它元素的输出引脚），而一个元素的输出引脚可以连几个输入引脚（本元素的输入引脚或其它元素的输入引脚）。如图 5-7-2 所示，有三种方式在 E1（a）和 E2（ADD）之间连线。在连线时，编辑器会检查双方的数据类型是否匹配。如果不匹配，光标会变为“禁止”样式，连线失败。

把鼠标放在 E1 的输出引脚上，按下左键，拖拽到 E2 的输入引脚上，释放左键。

把鼠标放在 E2 的输入引脚上，按下左键，拖拽到 E1 的输出引脚上，释放左键。

按下左键不放，移动 E1 或 E2，使它们的输入引脚和输出引脚接触。

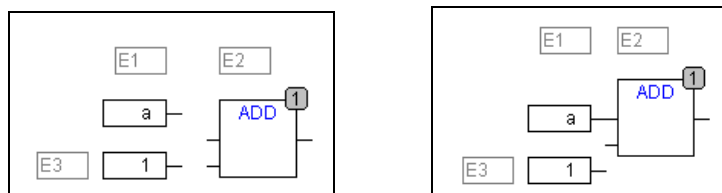


图 5-7-2 连线

➤ 删除连线

如图 5-7-2 所示，有三种方式删除 E1 (a) 和 E2 (ADD) 之间的连线。选中 E1 的输出引脚，按下<Delete>键或“编辑”“删除”，如果 E1 的输出引脚有几条线，则会同时删除。选中 E2 的输入引脚，按下<Delete>键或“编辑”“删除”。选中 E2 的输入引脚，用鼠标左键拖拽连线到编辑器空白位置，释放。

➤ “插入”/“块”：，快捷键<Ctrl>+

执行此命令，可以插入操作符、函数、功能块和程序。新插入的块随鼠标移动，移动到合适的位置点击鼠标左键，插入成功。起初插入的是 AND，可以把 AND 改为其他名称。或者选中 AND，按功能键 F2，从帮助窗口中选择名称。

➤ “插入”/“输入”：，快捷键<Ctrl>+<I>

执行此命令，可以插入输入。新插入的输入随鼠标移动，移动到合适的位置点击鼠标左键，插入成功。文本域“???”要输入变量或常量。也可按功能键 F2，从帮助窗口中选择。

➤ “插入”/“输出”：

执行此命令，可以插入输出。新插入的输出随鼠标移动，移动到合适的位置点击鼠标左键，插入成功。文本域“???”要输入变量。也可以按功能键 F2，从帮助窗口中选择。

➤ “插入”/“跳转”：，快捷键<Ctrl>+<J>

执行此命令，可以插入跳转。新插入的跳转随鼠标移动，移动到合适的位置点击鼠标左键，插入成功。文本域“???”要输入要跳转到的跳转标记。

➤ “插入”/“标记”：，快捷键<Ctrl>+<L>

执行此命令，可以插入标记。新插入的标记随鼠标移动，移动到合适的位置点击鼠标左键，插入成功。文本域“???”要输入跳转标记。在线模式时，编辑器自动在 POU 末尾自动插入标记“RETURN”。

➤ “插入”/“返回”：，快捷键<Ctrl>+<R>

执行此命令，可以插入返回。新插入的返回随鼠标移动，移动到合适的位置点击鼠标左键，插入成功。注意，此处的“返回”和在线模式时的 POU 末尾的标记“RETURN”不同，标记“RETURN”是为了 POU 结束本周期执行之前，自动执行下周期。

➤ “插入”/“注释”：，快捷键<Ctrl>+<K>

执行此命令，可以插入注释，用<Ctrl>+<Enter>换行。新插入的注释随鼠标移动，移动到合适的位置点击鼠标左键，插入成功。

➤ “其它”/“顺序”/“显示”

执行此命令，切换执行顺序的显示与否。缺省的是显示（命令前有“√”）。

➤ “其它”/“顺序”/“拓扑”

执行此命令，执行顺序根据拓扑结构排列，也就是所有元素从上到下，从左到右排列。注意，执行顺序显示在元素的右上脚。

➤ “其它”/“顺序”/“根据数据流排列”

执行此命令，执行顺序根据数据流排列，而不是根据元素所在的位置排列。

CFC 应用举例如图 5-7-3 所示。

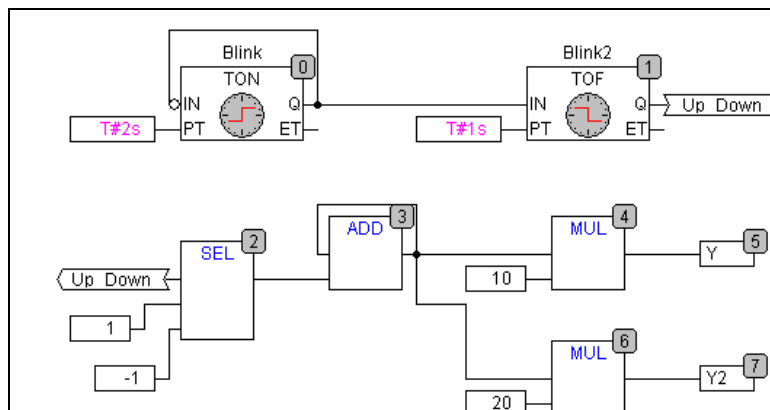


图 5-7-3 连续功能图语言示例

5.8 IEC 运算符

运算符是数字运算中所要执行的动作的特定符号。每个运算符实现一个特定的功能，例如 ADD 代表相加，MUL 代表相乘。

PowerPro 软件支持所有标准的 IEC 运算符。与系统提供的库函数相比，这些运算符在整个工程中隐含识别。运算符作为关键字可在所有编辑器中使用，以蓝色文本显示。

IEC 运算符可分为以下几类：

- 算术运算符：ADD、MUL、SUB、DIV、MOD、INDEXOF、SIZEOF
- 位串运算符：AND、OR、XOR、NOT
- 移位运算符：SHL、SHR、ROL、ROR
- 选择运算符：SEL、MAX、MIN、LIMIT、MUX
- 比较运算符：GT、LT、LE、GE、EQ、NE
- 地址运算符：ADR
- 调用运算符：CAL
- 类型转换运算符：BOOL_TO、TO_BOOL、整型之间的转换等
- 数值函数：ABS、SQRT、LN、LOG、EXP 等



本章小结

本章主要介绍 POU 的一些基本概念，还对 IEC 标准编程语言 LD、FBD、IL、ST、SFC 及 CFC 等进行了介绍。

第6章 变量

本章主要介绍变量的声明和变量的数据类型。变量是编程的基本概念。

6.1 操作数

在可编程控制器中,使 CPU 完成某种操作的命令称为指令,指令的集合称为指令系统。用于指挥 CPU 执行一定操作和完成一定功能的若干指令的组合称为功能块。指令系统是可编程控制器硬件和软件的桥梁,是可编程控制器程序设计的基础。

与计算机的操作指令类似,可编程控制器指令的基本形式也是由操作码和操作数组成。操作码表示 CPU 所要执行的操作类型和所要完成的操作功能。操作数表示 CPU 所要操作的对象和目的。常量、变量、地址和可能的函数调用都可以作为操作数。

6.1.1 常量

控制运算的主要操作对象是数据。把数值不变的数据称为常量。

➤ 布尔常量

布尔常量只有两个:逻辑值 TRUE 和 FALSE (也可表示为 1 和 0), TRUE 等价于 1, FALSE 等价于 0。

➤ 时钟常量

时钟常量一般用来操作时钟,由“T#”(或“t#”)加上“时钟值”构成,时钟值的单位包括天(d)、小时(h)、分(m)、秒(s)和毫秒(ms)。注意,它们的正确顺序为 d、h、m、s、ms。例如 T#12h38m16s 表示 12 小时 38 分 16 秒。

◇ 举例

下面是正确的时钟常量:

T#18ms (*18 毫秒的一个时钟常量*)

T#100s12ms (*100 秒 12 毫秒的一个时钟常量,高单位允许超限*)

t#12h34m15s (*12 小时 34 分 15 毫秒的一个常量*)

下面是错误的时钟常量:

t#5m68s (*低单位不允许超限*)

15ms (*没有 T#*)

t#4ms13d (*顺序错误*)

➤ 日期常量

日期常量由“D#”(“d#”、“DATE#”或“date#”)加上“日期值”构成。

◇ 举例

DATE#2005-05-06 (*日期常量 2005 年 5 月 6 日*)

d#1980-09-22 (*日期常量 1980 年 9 月 22 日*)

➤ 时间常量

时间常量用于存储时间，由“TOD#”（“tod#”、“TIME_OF_DAY#”或“time_of_day#”）加上“时间值”构成。时间值的格式为：小时:分钟:秒（可以用实数形式输入秒）。

◇ 举例

TOD#00:00:00 (*时间常量为 0 点 0 分 0 秒*)

TIME_OF_DAY#15:36:30.123 (*时间常量为 15 点 36 分 30.123 秒*)

➤ 日期时间常量

日期常量和时间常量合并起来称为日期时间常量，由“DT#”（“dt#”、“DATE_AND_TIME#”或“date_and_time#”）加上“日期时间值”构成。

◇ 举例

DT#1980-09-22-15:45:18

(*时间日期常量为 1980 年 9 月 22 日 15 点 45 分 18 秒*)

date_and_time#2001-03-09-00:00:00

(*时间日期常量为 2001 年 3 月 9 日 0 点 0 分 0 秒*)

➤ 数字常量

数字常量的数值可以是二进制、十进制、八进制和十六进制。如果整数值不是十进制值，可以用“进制”加符号“#”放在整数值前面来表示。十进制的 10 至 15 在十六进制中表示为 A 至 F。可以在数字中使用下划线连字符。数字常量的数据类型可以是 BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT、UDINT。默认情况下，不允许“较大”的数据类型作为“较小”的数据类型使用。例如，DWORD 类型的常量不能简单地当作 INT 类型使用，必须使用数据类型转换指令进行转换之后才可以使用。

◇ 举例

14 (*十进制数 14*)

2#1001_0011 (*二进制数 1001_0011*)

8#67 (*八进制数 67*)

16#AE (*十六进制数 AE*)

➤ 实数常量

实数常量用十进制小数和指数来表示，遵循标准的科学计数法格式。实数常量的数据类型是 REAL。

◇ 举例

7.4 (*实数 7.4*)

1.64e+009 (*实数 1.64e+009*)

➤ 字符串常量

字符串常量在两个单引号之间，可以包含空格和特殊字符。

◇ 举例

'Abby and Craig' (*字符串 Abby and Craig *)

':-)' (*字符串:-)*)

在字符串中，当在字符中出现以\$开始的复合字符时，解释为表 6-6-1 的形式。

表 6-6-1 特殊字符的显示

字符串	代表含义
\$\$	\$字符
\$'	单引号
\$L 或 \$l	输入行
\$N 或 \$n	新行
\$P 或 \$p	输入页
\$R 或 \$r	换行
\$T 或 \$t	Tab 键

6.1.2 变量

控制运算的主要操作对象是数据，这些数据的值一般都是实时变化的。把这些变化的数据叫做变量。或者说，变量就是计算机中数据存储单元的标识。

应该先了解一下变量的具体分类。下面就是从不同的角度对变量进行分类的情况。

按照数据类型不同，变量可以分为标准类型和用户自定义类型。其中标准类型包括布尔型（BOOL）、整型（INT）、实型（REAL、LREAL）、字符串型（STRING）以及时间型（TIME）等。

按照有效范围（使用范围）的不同，变量可以分为全局变量和局部变量。局部变量只在整个工程的一部分程序中有效，其它程序不能引用。全局变量则可以被整个工程的任意程序引用，在整个工程中均有效。

按照属性的不同，变量分为中间变量、输入型变量、输出型变量、输入输出型变量等。

按照能否掉电保护，变量分为保留型变量和非保留型变量。

6.2 变量的声明

在编程过程中，所作的操作都是针对变量（含常量）进行的。以一个加法为例， $AM3=AM1+AM2$ 。在这个等式中，AM1、AM2、AM3 都被称为变量，变量 AM3 的当前值根据变量 AM1 与 AM2 之和的变化而变化。

在控制算法组态中所用到的变量的名称和数据类型必须是事先明确的，这就需要首先对所用到的变量进行定义，这个定义过程叫做变量声明。

6.2.1 声明编辑器

在程序中使用的所有变量都需要进行声明，定义其数据类型、初始值或硬件地址等等。

全局变量在“资源”全局变量编辑器中定义。

局部变量在所对应的 POU 编辑器的声明部分定义。

枚举和结构变量在“数据类型”选项卡中定义。

这些可进行变量声明的窗口统称为声明编辑器。变量声明的语法格式基于 IEC61131-3 标准。图 6-2-1 是一个正确声明变量的例子。

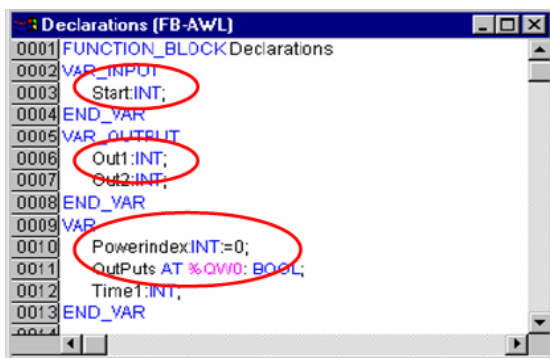


图 6-2-1 声明变量

➤ 关键字

关键字不能用做变量名称。在所有的编辑器中关键字都用蓝色大写字母书写。在声明编辑器中，使用“插入”/“声明关键字”命令打开关键字列表“Declarations”，如图 6-2-2 所示。

ARRAY	FUNCTION	TYPE
AT	FUNCTION_BLOCK	VAR
CONSTANT	OF	VAR_ACCESS
END_FUNCTION	PERSISTENT	VAR_CONFIG
END_FUNCTION_BLOCK	PROGRAM	VAR_EXTERNAL
END_PROGRAM	READ_ONLY	VAR_GLOBAL
END_STRUCT	READ_WRITE	VAR_IN_OUT
END_TYPE	RETAIN	VAR_INPUT
END_VAR	STRUCT	VAR_OUTPUT

图 6-2-2 声明关键字

➤ 句法颜色

如图 6-2-3 所示，所有的编辑器中不同的文本带有不同的颜色，不但可以避免错误，更有助于快速地发现错误。例如，注释没有被括上，从字体颜色上就会立即得到提示。

关键字输入带有自动提示，可以避免错误的输入。

蓝色：关键字。

绿色：编辑器中的注释。

粉色：特殊的常数（例如 TRUE/FALSE、T#3s、%IX0.0）。

红色：输入错误（例如无效时间常数、小写的关键字）。

黑色：变量、常数、标点符号。

```

0001 PROGRAM ST_EXAMPLE
0002 VAR
0003   xVal:INT:= 0;
0004   yVal:INT:= -250;
0005   bottom:INT:= -250;
0006   stripes:ARRAY[0..5] OF INT := 0, 90, 180, 270, 360, 450;
0007   stelle:INT;
0008   i:INT;
0009   rot:INT;
0010   y:BOOL;
0011   wait:INT:= 0;
0012   inv:BOOL := TRUE;
0013   inv2:BOOL := FALSE (*注释*)
0014   scale:INT := 1000;
0015   run:BOOL:=FALSE;
0016   run_string:STRING(20):='Start';
0017   onset:INT := 2;
0018 END VAR

```

图 6-2-3 句法颜色

➤ 注释

注释用特殊的符号（* *）括起来。注释可放在文本编辑器的任何位置，包括所有的声明、语言和用户定义的数据类型。在 LD 编辑器中，注释可以被插入到任何一个网络中。选择一个网络，执行“插入”/“注释”即可。

6.2.2 声明格式

声明变量的语法格式：

<Identifier> {AT<Address>} : <Type> {:= <initialization>};

其中在{ }中的部分是可选的。

➤ 变量标识符（Identifier）

变量标识符也就是通常所说的变量名。变量在 POU 的局部变量表或全局变量表中进行声明时，应当注意以下几点：

变量名中不能包含空格和特殊字符，不能多次声明，不能和关键字使用相同的名字。

变量名不区分大小写。例如 VAR1、Var1 和 var1 表示相同的变量。

变量名识别下划线。例如 A_BCD 和 AB_CD 被认为是两个不同的变量名。

变量名中不能有连续的下划线。例如 A__B 是错误的变量名。

➤ 数据类型（Type）

数据类型包括标准类型和用户自定义类型。使用“插入”/“类型”命令，可以打开变量的可选类型列表。系统支持的变量类型可分成以下几类：

标准类型（Standard Types）：布尔型、整型等。

用户自定义类型（User defined Types）：结构体、枚举类型等。

标准功能块（Standard Function Blocks）：实例声明。

用户自定义函数（User defined Functions）：实例声明。

声明数据类型可使用快捷方式，通过在行末输入<Ctrl>+<Enter>激活快捷方式。当一整行标识符同时声明变量时，声明类型由此行的最后一个标识符决定，可使用的标识符有：

B 或 BOOL 标识 BOOL

I 或 INT 标识 INT

R 或 REAL 标识 REAL

S 或 STRING 标识 STRING

如果不符合上面的条件，那么默认类型为布尔型（BOOL），最后一个标识符不确定整行的类型。每一个常数根据声明的类型都会转换成一个初始值或一个字符串。一个地址（例如%MD12）根据 AT 属性被扩展。

◇ 举例

快捷方式

A

A B I 2

STR S 2;A string

X %MD12 R 5;Real Number

B !

完整声明

A:BOOL;

A,B:INT:=2;

STR:STRING(2); (*A string*)

X AT %MD12:REAL := 5.0;(* Real Number *)

B: BOOL;

6.2.3 AT 声明

AT 声明是对直接地址（中间区、输入区和输出区）的映射，直接把一个变量连接到确定的硬件地址。

◇ 举例

```
impulse AT %IX3.2: BOOL;  
counter AT %QX0.0: BOOL;  
download AT %MX2.2: BOOL;
```

➤ 开关类输入通道声明

例：P_110_1 AT%IX1.0 : BOOL;

P_110_1	AT %	I	X	1	.0	:BOOL;
---------	------	---	---	---	----	--------

其中：

第 1 项：P_110_1——变量名

第 2 项：AT %——关键字

第 3 项：I——输入

第 4 项：X——位寻址

第 5 项：1——硬件配置中该模块分配的字节号

第 6 项：.0——该字节的位号，与实际通道 1 对应

第 7 项：:BOOL;——声明变量类型为布尔型

➤ 开关类输出通道声明

例：P_110_2 AT%QX2.7 : BOOL;

P_110_2	AT %	Q	X	2	.7	:BOOL;
---------	------	---	---	---	----	--------

其中：

第 1 项：P_110_2——变量名

第 2 项：AT %——关键字

第 3 项：Q——输出

第 4 项：X——位寻址

第 5 项：2——硬件配置中该模块分配的字节号

第 6 项：.7——该字节的位号，与实际通道 8 对应

第 7 项：:BOOL;——声明变量类型为布尔型

➤ 中间存储区声明

例：RUN AT%MX2.6 : BOOL;

RUN	AT %	M	X	2	.6	:BOOL;
-----	------	---	---	---	----	--------

其中：

第 1 项：RUN——变量名

第 2 项：AT %——关键字

第 3 项：M——中间存储区

第 4 项：X——位寻址

第 5 项：2——中间存储区的字节号

第 6 项：.6——该字节的位号

第 7 项：:BOOL;——声明变量类型为布尔型

6.2.4 初始化

在声明变量时可以同时进行变量的初始化，用符号“:=”赋值。对于基本的变量类型，初始化值都是常数。如果没有赋值，所有变量的初始缺省值是 0。

➤ 变量的初始化

var1:INT:=12; (定义整型数 var1，初始值 12)

➤ 数组的初始化

arr1:ARRAY [1,2,3,4] OF INT := 1,3(7);

arr2:ARRAY [1..5] OF INT := 2(1),3(2);

X1:ARRAY[1..11] OF REAL :=0,1,2,3,4,5,6,7,8,9,10;

Y1:ARRAY[1..11] OF REAL :=0,10,20,30,40,50,60,70,80,90,100;

6.2.5 自动声明变量

系统支持变量自动定义功能。当程序中出现一个新变量时，系统会自动弹出对话框，要求进行变量定义，如图 6-2-4 所示。其中“名字”和“类型”是完成声明变量所必需的选项。“类别”默认为中间变量，“初始值”默认为 0。则在变量声明部分会显示如下的变量声明：

VAR_GLOBAL

DI7 AT%IX0.6:BOOL:=TURE; (*第七通道开关量输入*)

END_VAR

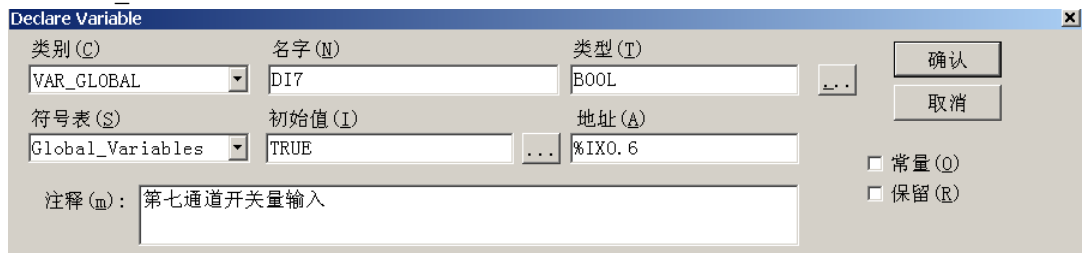


图 6-2-4 自动声明变量

自动声明变量对话框的各项含义如下所述。

类别：类型选择。

名字：声明变量的名称。

类型：数据类型选择。

符号表：只有在“类别”选择“VAR_GLOBAL”时，符号表才会出现“Global_Variables”。则相应的程序声明会显示在“资源”选项卡下“全局变量”文件夹中的 Global_Variables 对应的变量声明区，如图 6-2-5 所示。

初始值：变量的初始值。

地址：定义变量的地址。

注释：变量的含义。

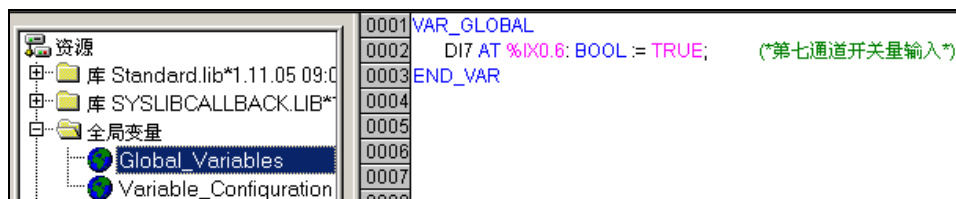


图 6-2-5 声明变量的代码部分

新建变量时，系统可以自动定义。但当变量被删除时，定义语句不会自动删除，继续保留在编辑器中。可使用“工程”/“查看”/“未使用变量”命令查找到这些无用的变量声明。

6.2.6 表格声明变量

在“工程”/“选项”/“Editor”对话框里选中“声明为表”项，或在程序编辑区选中右键菜单项“定义为表格”，声明编辑器会显示成表格的形式，如图 6-2-6 所示。

	VAR	VAR_INPUT	VAR_OUTPUT	VAR_IN_OUT	CONSTANT	RETAIN	INFO
	名称	地址	类型	初始值	注释		
0001	AO1		Analog_OUT				
0002	DP		DP_Slave				
0003	xunhuan		BOOL				
0004	TP1		TP				
0005	TP2		TP				
0006	TON_1		TON				
0007	TON_2		TON				
0008	TON_3		TON				
0009	TON_4		TON				
0010	TON_5		TON				

图 6-2-6 表格声明变量

➤ 字段

每一个变量表格都有下列字段：

名称：输入变量名称。

地址：如果必要，输入变量的地址（AT 声明）。

类型：输入变量类型。当定义一个功能块时，输入功能块的类型名。

初始值：输入变量的可能初始化值（在“:=”赋值符之后）。

注释：输入注释。

在表格形式的声明编辑器中，在最初设置时，“名称”字段缺省为“Name”，“类型”字段缺省为“BOOL”。根据需要可以自行更改这些设置，变量名和类型是完成声明变量所必需的选项。

去除右键命令“表格声明”前的“√”，可切换至常规形式。

表格声明变量以简洁的填表方式代替了常规的语句声明方式，非常适合初学者。

➤ 插入新声明

使用“插入”/“新声明”命令，或右键点击空白处选择“新声明”，可添加一个新的变量到声明表中。新变量会插入到当前光标的下一行，如图 6-2-7 所示。



图 6-2-7 插入新声明

6.3 变量的数据类型

PowerPro 软件支持的变量数据类型包括标准类型和用户自定义类型。

6.3.1 标准数据类型

➤ 布尔型数据类型

布尔型变量可被赋予“TRUE”真或“FALSE”假。这个值为逻辑量，占用 1 位存储空间。

➤ 整型数据类型

整型变量可以是 BYTE、WORD、DWORD、SINT、USINT、INT、UINT、DINT 和 UDINT。它们覆盖的数据范围和占用存储空间的大小均不相同，如表 6-3-1 所示。

注意，当较长的数据类型转换为较短的数据类型时，会丢失高位信息。

表 6-3-1 数据类型范围和存储空间

类型	类型名称	数据下限	数据上限	存储空间
BOOL	布尔型	0	1	8bit
BYTE	字节型	0	255	8 Bit
WORD	字型	0	65535	16 Bit
DWORD	双字型	0	4294967295	32Bit
SINT	短整型	-128	127	8 Bit
USINT	无符号短整型	0	255	8 Bit
INT	整型	-32768	32767	16 Bit
UINT	无符号整型	0	65535	16 Bit
DINT	长整型	-2147483648	2147483647	32 Bit
UDINT	无符号长整型	0	4294967295	32 Bit

➤ 实型数据类型

REAL 和 LREAL 是浮点数，用于显示有理数。可以显示十进制数据，包括小数部分。也可以被描述成指数形式。REAL 是 32 位浮点数，LREAL 是 64 位浮点数。（本软件不支持 LREAL 类型）

◇ 举例

R1:REAL:=1.64e+009

➤ 字符串型数据

STRING 型变量的声明部分在圆括号里指定了字符的数量。如果不说明大小，缺省的大小是 80 个。

◇ 举例

35 个字符的字符串声明：

str1: STRING(35) := 'This is a string';

➤ 时间型数据类型

时间型变量分为 DATE、TIME、TOD、DT 几种，用于输入时间数据。

DATE 日期型变量：用于输入日期值。声明格式为：“d#”、“D#”、“DATE#”。

◇ 举例

RQ1:=DATE#1996-05-06;

RQ2:=d#2002-1-1;

TIME 时间型变量：系统允许声明时间值，通常被用在标准库中定义操作时间。声明格式为“t#”、“T#”、“TIME#”、“time#”。在“#”后面可以跟实际的时间声明，在这个声明中，可以包括日期、时、分、秒和毫秒。时间的声明必须按照以下顺序：日期-时-分-秒-毫秒。在定义时间型变量的时候要注意，低位不可以超过数据上限，而高位则可以超过数据上限。

◇ 举例

正确的定义：

TIME1 := T#14ms;

TIME1 := T#100S12ms; (*允许最高位超过其数据上限*)

TIME1 := t#13d12h34m15s;

错误的定义：

TIME1 := t#5m68s; (*低位超限*)

TIME1 := 15ms; (*缺少提示符 T#*)

TIME1 := t#4ms13d; (*输入顺序有误*)

TOD 时刻型变量：使用这种类型的变量可以存储时刻。声明格式为“TOD#”、“tod#”、“TIME_OF_DAY#”、“time_of_day#”。在“#”后跟时刻，格式为时：分：秒。系统允许秒为实数，即秒可以带小数部分。

◇ 举例

TIME_OF_DAY#15:36:30.123;

tod#00:00:00;

DT 日期+时刻型变量：日期型变量和时刻型变量可以组合成日期+时刻型变量，被称为“DATE_AND_TIME”型变量。声明格式为“DT#”、“dt#”、“DATE_AND_TIME#”、“date_and_time#”。在“#”后跟日期+时刻，格式为：年-月-日-时：分：秒。

◇ 举例

DATE_AND_TIME#1996-05-06-15:36:30;

dt#1972-03-29-00:00:00;

6.3.2 自定义数据类型

➤ 数组

可以根据基本数据类型来定义一维、二维和三维数组。数组可以定义在 POU 的变量声明部分，也可以定义在全局变量列表中。数组的标识符为 ARRAY。

数组定义的语法格式：

<数组名> : ARRAY [<L1>..<>U1>, <L2>..<>U2>, <L3>..<>U3>] OF <基本数据类型>;

其中 L1、L2 和 L3 表示字段范围的最小值，U1、U2 和 U3 表示字段范围的最大值。字段范围必须是整数。

在数组定义的同时，给数组中的元素赋值称为初始化数组。在数组定义时，可以初始化数组中所有元素，也可以不进行初始化。

◇ 举例：定义数组

Card_game: ARRAY [1..13, 1..4] OF INT; (*定义一个整型的二维数组 Card_game*)

◇ 举例：数组的完全初始化

Arr1:ARRAY [1..5] OF BYTE:= 1,2,3,4,5;

Arr2:ARRAY [1..2,3..4] OF INT := 1,3(7); (*即 1,7,7,7 的缩写形式*)

Arr3:ARRAY [1..2,2..3,3..4] OF INT := 2(0),4(4),2,3; (*即 0,0,4,4,4,4,2,3 的缩写形式*)

◇ 举例：结构数组的初始化

TYPE STRUCT1:

STRUCT

p1:int;

p2:int;

p3:dword;

END_STRUCT

END_TYPE

A1[1..3] OF STRUCT1:=(p1:=1,p2:=10,p3:= 3),(p1:=2,p2:=0,p3:=2),(p1:=4,p2:=5,p3:=1);

◇ 举例：数组的部分初始化

Arr1:ARRAY [1..10] OF BYTE:= 1,2;

对于那些没有预先赋值的元素，按照基本数据类型的缺省初始值进行初始化。在此例中，元素[3]到[10]被初始化为 0。

➤ 指针

程序运行时，变量地址和功能块地址保存在指针中。

指针定义的语法格式：

<指针名> : POINTER TO <数据类型/功能块>;

指针可以指向任意的数据类型、功能块和自定义类型。地址运算符 ADR 用于把变量或功能块的地址赋给指针。在指针后面增加取内容运算符“^”，可以获取指针所指的内容。

◇ 举例

```
pt:POINTER TO INT;      (*定义一个整型数据的指针 pt*)
Var_int1:INT := 5;       (*定义整型变量 Var_int1，使其等于 5*)
Var_int2:INT;            (*定义整型变量 Var_int2*)
pt := ADR(Var_int1);     (*取出 Var_int1 变量的地址，将地址值赋给 pt*)
Var_int2:= pt^;          (*将指针 pt 所指地址的值赋给 Var_int2，即 Var_int2=5*)
```

➤ 枚举

枚举是一种用户自定义的数据类型，由一些字符常量所组成。这些常量被称为枚举值。只要枚举值声明在 POU 内，则在整个工程范围内，这些枚举值都可以被识别。

最好将枚举以对象的形式创建在对象组织器的数据类型选项卡中。通过在数据类型上添加对象来创建一个新的枚举值。

枚举以关键字 TYPE 开始，以关键字 END_TYPE 结束。

枚举定义的语法格式：

```
TYPE <标识符> : (<Enum_0>, <Enum_1>, ..., <Enum_n>);
END_TYPE
```

由相应的类型标识符所定义的变量可以接收枚举值，并且可以被枚举值初始化。这些值是兼容的，即可以象使用 INT 类型的值一样使用这些枚举值。可以给这些变量分配数字。如果枚举值没有初始化，则从 0 开始计数。当初始化时，保证初始化值是递增的。程序运行时，可以查看数字的有效性。

◇ 举例

```
TYPE TRAFFIC_SIGNAL : (Red, Yellow:=1, Green:=10);
END_TYPE                (*每种颜色的初始值：Red=0, Yellow=1, Green=10*)
TRAFFIC_SIGNAL1 : TRAFFIC_SIGNAL;
TRAFFIC_SIGNAL1:=0;      (*交通信号灯的的值是 Red*)
FOR i:= Red TO Green DO
    i := i + 1;
END_FOR;
```

注意，在一个枚举型变量中，或在同一程序的所有枚举型变量中，不能两次使用相同的枚举值，如下例所示。

◇ 举例

```
TRAFFIC_SIGNAL: (red, yellow, green);
COLOR : (blue, white, red);
错误：red 不能在枚举变量 TRAFFIC_SIGNAL 和 COLOR 中同时使用。
```


➤ 结构

在对象组织器的数据类型选项卡中，以对象的形式来创建结构变量。结构变量以关键字 TYPE 和 STRUCT 开始，以关键字 END_STRUCT 和 END_TYPE 结束。

定义结构变量的语法格式：

TYPE <结构名>:

STRUCT

<变量声明 1>

<变量声明 2>

...

<变量声明 n>

END_STRUCT

END_TYPE

结构是一种可以在整个工程中被识别的数据类型，而且可以象引用标准数据类型一样引用结构。唯一的限制是，结构变量不能指定地址，即不允许进行 AT 声明。

✧ 举例：定义名为 Polygonline 的结构

TYPE Polygonline:

STRUCT

Start:ARRAY [1..2] OF INT;

Point1:ARRAY [1..2] OF INT;

Point2:ARRAY [1..2] OF INT;

Point3:ARRAY [1..2] OF INT;

Point4:ARRAY [1..2] OF INT;

End:ARRAY [1..2] OF INT;

END_STRUCT

END_TYPE

✧ 举例：初始化结构

P1:Polygonline:=(Start:=3,3,Point1=5,2,Point2:=7,3,Point3:=8,5,Point4:=5,7,End:=3,5);

访问结构成员的语法：

<结构名>.<结构成员名>

✧ 举例

如果结构名为 Week，其中的一个成员名为 Monday，则可以用下面的形式访问该成员：

Week.Monday

6.4 全局变量的声明

在对象组织器中打开“资源”选项卡，会看到“全局变量”文件夹，如图 6-4-1 所示，其中包括全局变量（VAR_GLOBAL）和变量配置（VAR_CONFIG）。在这里定义的全局变量可以在整个工程中使用。

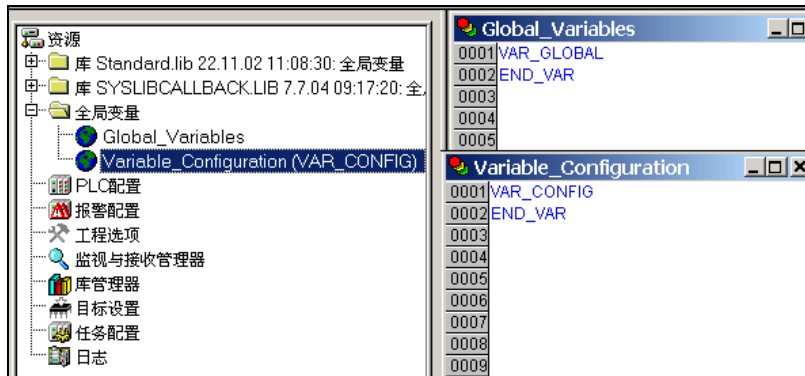


图 6-4-1 全局变量窗口

6.4.1 在全局变量中声明

全局变量在关键字 VAR_GLOBAL 和 END_VAR 之间定义。语法：

VAR_GLOBAL

(*变量声明*)

END_VAR

模块通道的 I/O 变量也可以在这里定义。

◇ 举例

VAR_GLOBAL

Start:BOOL;

Reset:BOOL;

Count:INT;

END_VAR

6.4.2 在变量配置中声明

在功能块中，如果把变量的声明放在关键字 VAR 和 END_VAR 之间，则可以指定不完整定义的输入和输出地址，其中不完整的定义地址用*号标志。

◇ 举例

FUNCTION_BLOCK locio

VAR

loci AT %I*: BOOL := TRUE;

loco AT %Q*: BOOL;

END_VAR

这里定义了两个局部 I/O 变量 loci 输入 (%I*) 和 loco 输出 (%Q*)。

在程序 PROGRAM PLC_PRG 中，功能块实例声明如下：

VAR

Hugo: locio;

Otto: locio;

END_VAR

则要在变量配置窗口中对功能块实例中的变量进行定义，格式如下：

VAR_CONFIG

PLC_PRG. Hugo.loci AT %IX0.0 : BOOL;

```

PLC_PRG. Hugo.loco AT %QX0.0 : BOOL;
PLC_PRG. Otto.loci AT %IX1.0 : BOOL;
PLC_PRG.Otto.loco AT %QX0.3 : BOOL;
END_VAR

```

其中功能块实例中的变量名的格式如下：程序名.功能块名.变量名。

6.4.3 在 PLC 配置中的声明

在 PLC 配置中也可以声明全局变量。如图 6-4-2 所示，定义了两个全局变量 In 和 Out。

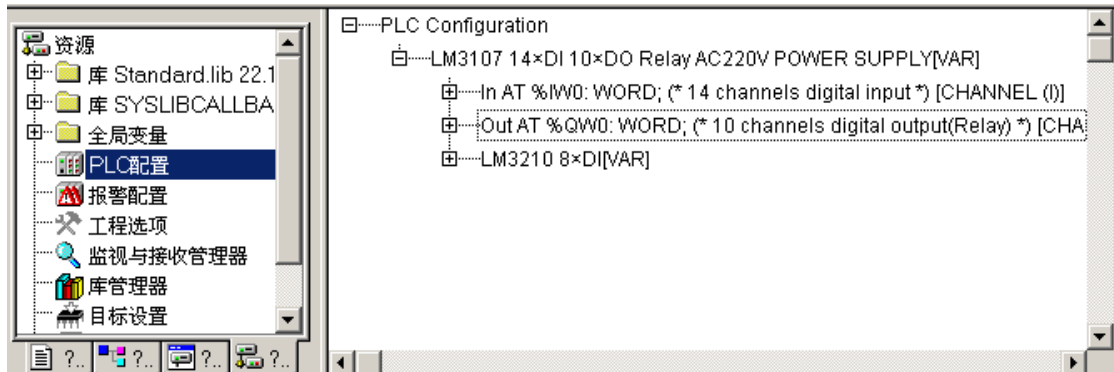


图 6-4-2 在 PLC 配置中声明全局变量

对于全局变量而言，可以指定该变量定义在 I 区、Q 区、M 区、保留区，若无特殊指定，则默认定义在全局变量区。但应注意：全局变量不要与局部变量使用同一变量名，否则会导致变量名重名而引起混淆。

6.5 局部变量的声明

中间变量、输入变量、输出变量和输入/输出变量等都属于局部变量，在各个程序所对应的声明编辑器中定义，而且只能在本程序中使用。不同程序中的局部变量可以同名。

6.5.1 声明中间变量

在关键字 VAR 和 END_VAR 之间声明中间变量。这些变量只能在程序的内部调用，没有外部的联系，不能在本程序之外被调用。

✧ 举例

```

VAR
    Loc1:INT;      (*局部变量*)
END_VAR

```

6.5.2 声明输入变量

在关键字 VAR_INPUT 和 END_VAR 之间声明输入变量。调用输入变量时，变量的初始值随着变量的调用一起给出。

✧ 举例

```

VAR_INPUT
    in1:INT;      (*输入变量*)
END_VAR

```

6.5.3 声明输出变量

在关键字 VAR_OUTPUT 和 END_VAR 之间声明输出变量。输出变量调用程序的返回值，并可以被进一步调用。

✧ 举例

```
VAR_OUTPUT
    out1:INT;      (*输出变量*)
END_VAR
```

6.5.4 声明输入/输出变量

在关键字 VAR_IN_OUT 和 END_VAR 之间声明输入/输出变量。对于中间变量而言，传送的变量值是变化的，所以中间变量的输入值不能是一个常数。

✧ 举例

```
VAR_IN_OUT
    inout1:INT;    (*输入/输出变量*)
END_VAR
```

6.5.5 声明保留型变量

保留型变量以关键字 RETAIN 标识。保留型变量在掉电之后仍能保存原有的值不变。当程序再次运行后，保存的值自动对系统进行安全恢复。例如操作计时器，它在掉电之后仍能保持计时值，而所有其它类型的变量都被初始化，被赋予新的初始值或标准初始值。多用于自定义功能块。

✧ 举例

```
VAR RETAIN
    rem1:INT;      (*保留型变量*)
END_VAR
```

6.5.6 声明常数

常数以关键字 CONSTANT 标识。句法：

```
VAR CONSTANT
    <Identifier>:<Type>:=<initialization>;
END_VAR
```

✧ 举例

```
VAR CONSTANT
    con1:INT:=12;  (*常数*)
END_VAR
```



本章小结

本章介绍了变量的相关概念，并按照变量的分类介绍了变量的声明。

第7章 库

工程创建过程中，经常会引用一些标准函数或通用的功能块算法，如与运算、或运算、加减运算、定时器等。在 PowerPro 软件中，把用来实现这些常用功能的函数和功能块集合起来建立专门的库，本章介绍库的概念、分类及制作。

7.1 库的概念

编写 PLC 程序的过程中，经常会引用一些 G3 指令或者功能块，如字符串处理指令、触发器功能块、计数器功能块、PID 控制功能块等等。在 PowerPro 软件中，把用来实现这些常用功能的 G3 指令和功能块集合起来进行分类，然后建立专门的库。

库是 G3 指令与功能块的集合，所有的库都包含“库名.lib”文件（包含 G3 指令和功能块的输入输出代码），调用 G3 指令和功能块只需载入相应的“库名.lib”文件。

最常用的库主要有标准库（Standard.lib）、应用库（Util.lib、Util_no_Real.lib）、系统库（SysLibC16x.lib、SyslibCallBack.lib）等。标准库和系统库在建立工程时就自动绑定到工程，可以直接调用，其它库则需要添加到工程后才可以调用。系统提供的所有库文件都以库文件*.lib 的形式存在于\Hollysys\PowerPro\Library 目录下。

7.1.1 库管理器

库管理器用来管理库函数和功能块，包含了系统提供的所有标准函数和功能块。选择“窗口”/“库管理器”选项打开库管理器，如图 7-1-1 所示。

库管理器可以显示所有和当前工程连接的库，可以像使用用户自定义的程序、功能块、函数、数据类型和变量一样使用函数库里的程序、功能块、函数、数据类型和变量。

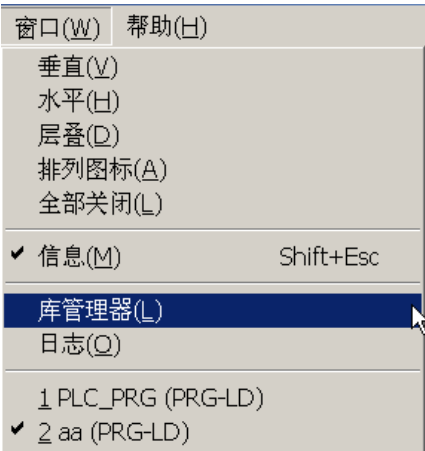


图 7-1-1 打开库管理器

标准库管理器窗口分成 4 个区域，如图 7-1-2 所示。绑定到工程的库列表位于左上区域，左下区域显示所选择库的程序、数据类型或全局变量。选择一个函数或功能块后，其变量声明出现在库管理器的右上区域，右下区域是函数或功能块的图形显示。

学会查看函数库是非常重要的。在函数库中，给出了一些关于该函数非常重要的信息，例如，该函数中有几个输入变量及中间变量，它们各自的数据类型是什么，有哪些中间变量是必须赋予初始值的，以及变量的注释等等。

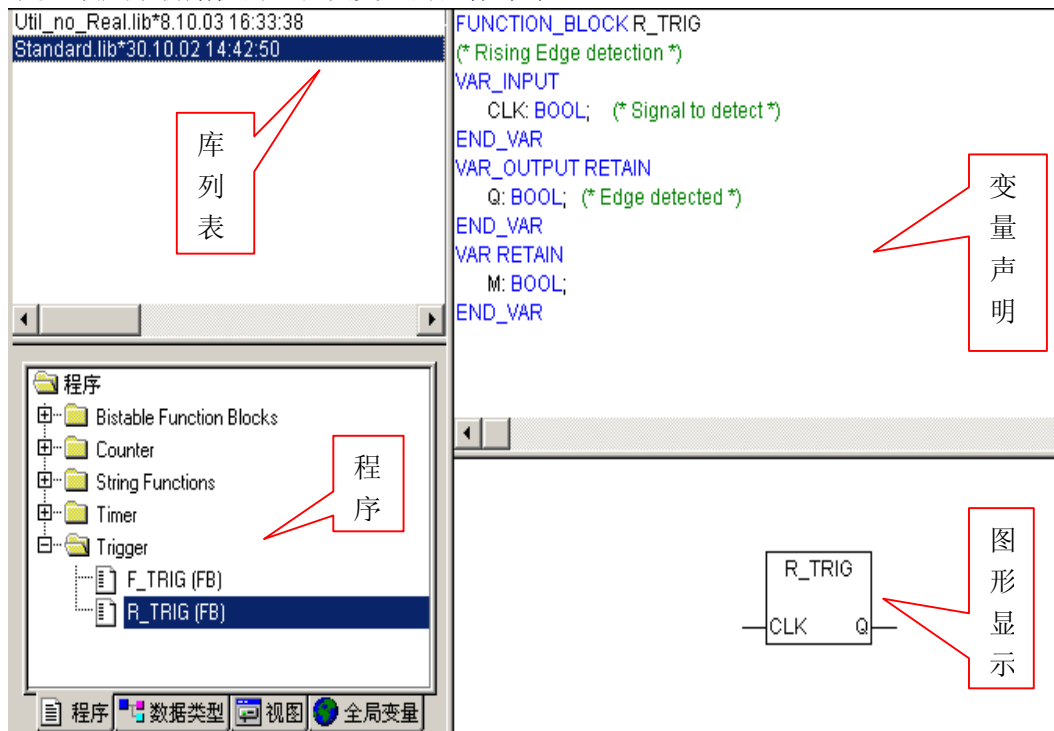


图 7-1-2 库管理器窗口

7.1.2 添加库

当库列表区的库文件已不能满足目前的编程需要时，则需要添加库。注意，由于 Util.lib 和 Util_no_Real.lib 的数据类型有冲突，会产生编译错误，不允许同时添加。使用库时，需要保证相应的库文件存在于如下目录：PowerPro 安装目录\Library\。添加库的过程如下所述。使用“插入”/“添加库”命令，或在库管理器窗口的库列表位置选择鼠标右键菜单命令“添加库”，如图 7-1-3 所示。



图 7-1-3 添加库

选择所需要的库文件，点击“打开”。不论哪种库，只需要打开对应的*.lib 文件即可，如图 7-1-4 所示。

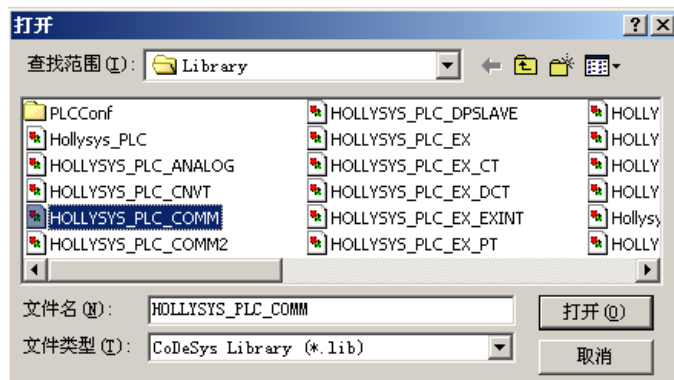


图 7-1-4 打开库

上面所选择的库被添加到库列表中，该库所包含的功能块被显示在如图 7-1-5 所示的鼠标位置。注意，凡是添加到库管理器的库都会占用用户程序空间，所以建议用户只添加所使用的库。

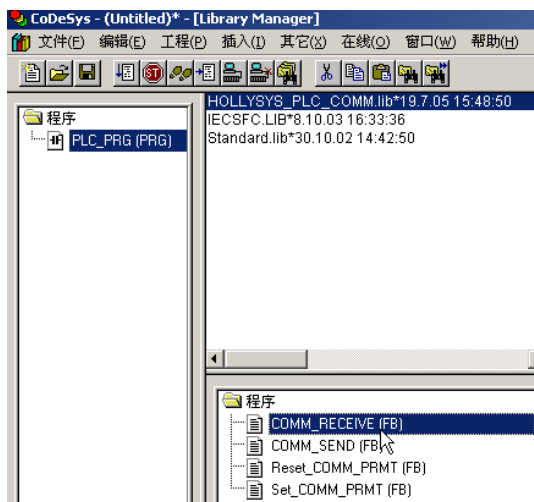


图 7-1-5 显示库

7.1.3 删除库

使用“编辑”/“删除”命令，或选择鼠标右键菜单“删除”，可以从工程和库管理器中删除已添加的库。

7.2 库的分类

库中指令和功能块的执行代码存在于不同的位置。为了用户使用方便，按照与功能块类似的分类方法，库被分为三类。

➤ 第一类：内部库

内部库所包含 G3 指令与内部功能块的执行代码存在于“库名.lib”之中，可以使用 PowerPro 软件打开库文件对 G3 指令或功能块的执行程序进行修改，用户也可以自己制作内部库。当程序下装到 PLC 之中，占用用户程序空间较大。

➤ 第二类：外部扩展库

外部扩展库所包含的 G3 指令与外部扩展功能块的执行代码存在于“库名.hex”文件之中，用户无法使用 PowerPro 软件打开此类库文件，修改此类功能块的执行代码。使用时应保证 hex 文件的文件名与 lib 文件的文件名一致，且存在于同一目录下。当程序下装到 PLC 之中，占用用户程序空间较大。

➤ 第三类：外部 G3 库

外部 G3 库所包含的 G3 指令与外部 G3 功能块的执行代码已经存在于 PLC 底层系统之中，用户无法修改此类库所包含的执行代码。当程序下装到 PLC 之中，占用用户程序空间很少。

➤ 在使用时应当注意下列问题：

- ✓ 使用 G3 指令或者功能块必须载入相关的库文件（库名.lib）。
- ✓ 标准库 Standard.lib 在工程建立时自动载入，无需用户再次载入。
- ✓ 只要载入内部库和外部扩展库，即使不调用其中的功能块，也会占用用户程序空间，建议只载入需要的库。

➤ PowerPro 软件常用的库包括：

- ✓ 标准库 Standard.lib;
- ✓ 应用库 Util.lib 和 Util_no_real.lib;
- ✓ 系统库 SysLibCallBack.lib 和 SysLibC16x.lib;
- ✓ IEC 动作库 Iecsfc.lib;

关于指令与库的详细内容，请参见《HOLLiAS-LEC G3 系列小型一体化 PLC 指令与功能块手册》。

以下对常用的库进行简单介绍。

7.2.1 标准库

标准库 Standard.lib 在工程建立时自动载入，无需用户再次载入，主要包含如图 7-2-1 所示的外部指令与外部扩展功能块。该库可实现的功能如表 7-2-1 所示。

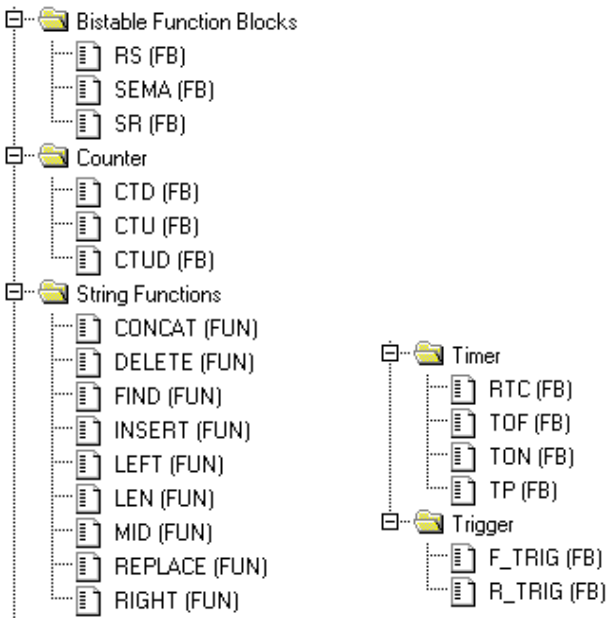


图 7-2-1 标准库 Standard.lib

表 7-2-1 标准库 Standard.lib 功能

双稳态功能（Bistable Function Blocks）	计数器功能（Counter）
RS（双稳态，复位优先）	CTD（减计数）
SEMA（软信号灯）	CTU（加计数）
SR（双稳态，置位优先）	CTUD（加减计数）
字符串功能（String Function）（指令）	定时器功能（Timer）
CONCAT（字符串合并）	RTC（获取时间）
DELETE（字符串删除）	TOF（延时关）
FIND（字符串位置查找）	TON（延时开）
INSERT（字符串插入）	TP（定宽脉冲）
LEFT（获取最左边开始的字符串）	触发器功能（Trigger）
LEN（字符串长）	F_TRIG（下降沿检测）
MID（获取字符串中的一部分）	R_TRIG（上升沿检测）
REPLACE（替换）	
RIGHT（获取最右边开始的字符串）	

7.2.2 应用库

应用库包括 Util.lib 和 Util_no_Real.lib。

应用库 Util.lib 属于内部库，使用时需要用户载入，主要包含如图 7-2-2 所示的外部指令与外部扩展功能块。该库可实现的功能如表 7-2-2 所示。

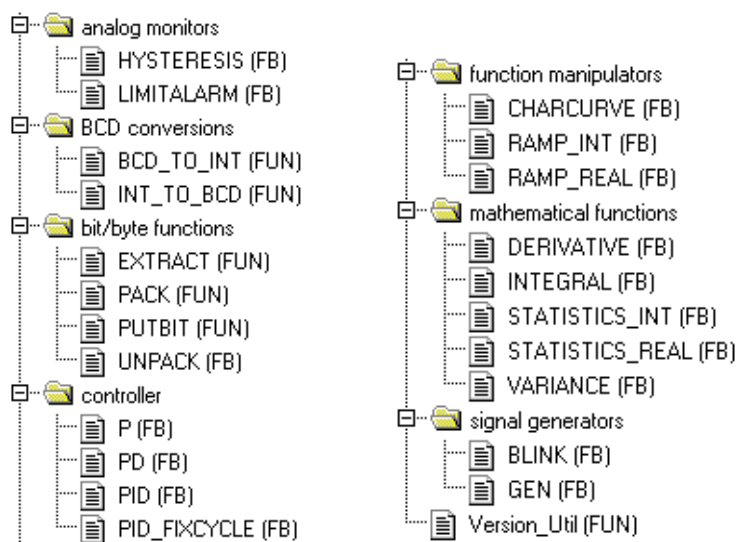


图 7-2-2 应用库 Util.lib

表 7-2-2 应用库 Util.lib 功能

模拟量监视 (analog monitors)	BCD 码转换 (BCD conversions)
HYSTERESIS (滞后)	BCD_TO_INT (BCD 码转整型)
LIMITALARM (上下限报警)	INT_TO_BCD (整型转 BCD 码)
位转换功能 (bit/byte functions)	高等数学运算 (mathematical functions)
EXTRACT (位提取)	DERIVATIVE (微分)
PACK (位整合)	INTEGRAL (积分)
PUTBIT (位赋值)	STATISTICS_INT (整型统计)
UNPACK (位拆分)	STATISTICS_REAL (实型统计)
	VARIANCE (平方偏差)
控制器 (controller)	函数操纵器 (function manipulators)
P (比例控制器)	CHARCURVE (特征曲线)
PD (比例微分控制器)	RAMP_INT (整型限速)
PID (比例积分微分控制器)	RAMP_REAL (实型限速)
信号发生器 (signal generators)	库版本查看
BLINK (脉冲信号发生器)	Version_Util (库版本查看)
GEN (典型周期信号发生器)	

应用库 Util_no_Real.lib 与应用库 Util.lib 的内容基本相同，只是没有与 Real 类型变量有关的函数，如图 7-2-3 所示。编译下载后占用的用户程序空间少。

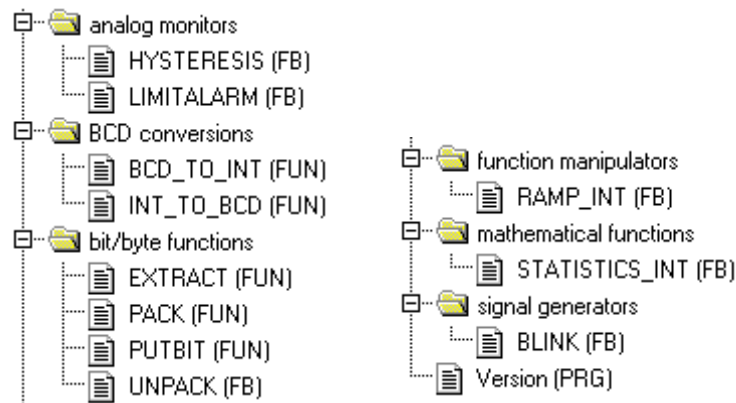


图 7-2-3 应用库 Util_no_Real.lib

7.2.3 系统库

系统库包括 SysLibC16x.lib 和 SysLibCallBack.lib。
系统库 SysLibC16x.lib 包含 PowerPro 系统的运行信息，可实现版本（Version）功能，如图 7-2-4 所示的。SyslibGetVersion2300 获取 CPU 下为软件版本信息。



图 7-2-4 系统库 SysLibC16x.lib

系统库 SysLibCallBack.lib 包含 PowerPro 系统的事件调用（SysCallRegister）和解除事件调用（SysCallUnregister），如图 7-2-5 所示。



图 7-2-5 系统库 SysLibCallBack

7.2.4 IEC 动作库

IEC 动作库 Iecsfclib 包含 PowerPro 系统的 SFC 动作控制（SFCActionControl，IEC61131-3），如图 7-2-6 所示。

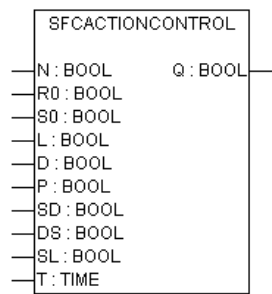


图 7-2-6 IEC 动作库 Iecsfclib

对于库的具体应用请参见《HOLLiAS-LEC G3小型一体化PLC指令与功能块手册》。

下面列出目前所包含的库文件：

- Hollysys_PLC_ANALOG.lib
- Hollysys_PLC_CNVT.lib
- Hollysys_PLC_COMM.lib
- Hollysys_PLC_COMM2.lib
- Hollysys_PLC_DPSlave.lib
- Hollysys_PLC_EtherNet.lib
- Hollysys_PLC_EX.lib
- Hollysys_PLC_EX_CT.lib
- Hollysys_PLC_EX_DCT.lib
- Hollysys_PLC_Ex_DCT32.lib
- Hollysys_PLC_EX_EXINT.lib
- Hollysys_PLC_EX_PT.lib
- Hollysys_PLC_EX_PTRun.lib
- Hollysys_PLC_Ex_TIMER.lib
- Hollysys_PLC_External.lib
- Hollysys_PLC_HDRTC.lib
- Hollysys_PLC_HDRTCALM.lib
- Hollysys_PLC_IO.lib
- Hollysys_PLC_Math.lib
- Hollysys_PLC_Modbus_CRC.lib
- Hollysys_PLC_PTCYC.lib
- Hollysys_PLC_PTOftrl.lib
- Hollysys_PLC_Util.lib
- Iecsfc.lib
- Standard.LIB
- SysLibC16x.lib
- SysLibCallback.lib
- SysLibVisu.lib
- Util.lib
- Util_no_Real.lib

7.3 库的制作

在工程实施中，如果有一些算法或逻辑是大量重复的，而且可以应用于多个工程中，则可以将这部分逻辑制作成库，以方便于程序编写、工程维护管理及部门的技术资源共享。还可以加上口令，以保护库的具体实现算法和逻辑不被查看和修改。以下说明如何制作库。

首先新建一个工程，“目标设置”中“配置”选“None”，如图 7-3-1 所示。



图 7-3-1 自定义库（1）

确认后，弹出“创建 POU”窗口，如图 7-3-2 所示。“POU 类型”选择“功能块”。根据需要填写“新 POU 名”，一般来说命名要尽量采用能反映其实际用途的字符。在图 7-3-2 中，“POU 语言”选择了 ST，“新 POU 名”为“Generate_CRC”，即 CRC 校验。

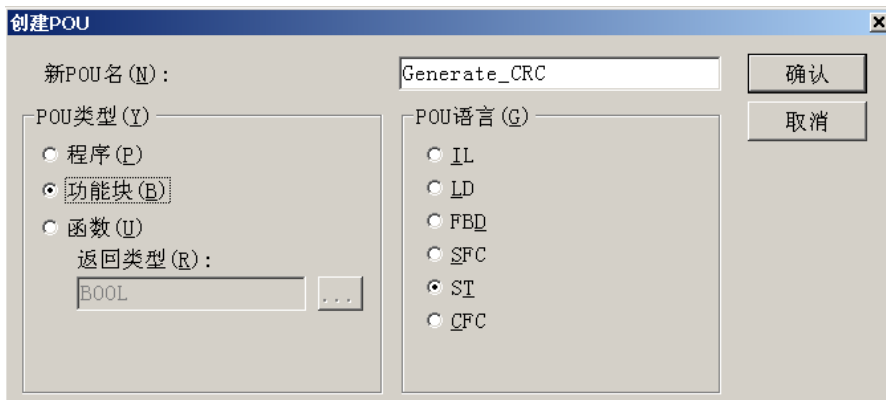


图 7-3-2 自定义库 (2)

设置完毕后自动生成一个功能块结构体，可以先保存为库文件，再往里面添加所需要的内容。保存时要注意要存在PowerPro\Library目录下，文件名根据需要定义，保存类型要选择内部库（Internal Library），如图7-3-3所示，使算法逻辑在库文件中实现。还有一种是外部库（External Library），其算法逻辑在模块中实现，库文件中只定义使用的变量。

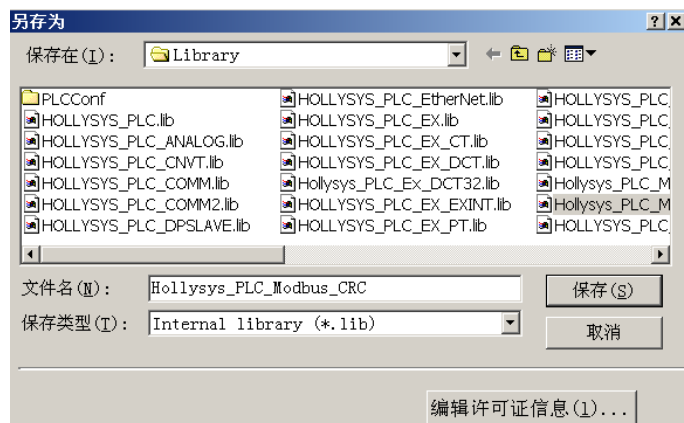


图 7-3-3 自定义库 (3)

◇ 举例

本例是专为产生 CRC16 校验而制作的一个 ST 语言的自定义库，如图 7-3-4 所示。编译通过后，就可以在其它的工程中使用此库。使用前注意在库管理器中先要加入。

如果想要在其它的计算机上使用此库，则只需要将此库文件复制到其它计算机的软件安装目录 PowerPro\Library 下，再添加到工程中即可。

一个库中可以包括多个功能块或函数。如果想要添加功能块或函数，只需打开库文件，在程序中添加新的功能块即可。

注意，库中的功能块不允许递归调用，即不允许自己调用自己。

```
0001 FUNCTION_BLOCK Generate_CRC
0002 VAR_INPUT
0003   pData:POINTER TO BYTE;
0004   byteCounter:WORD;
0005 END_VAR
0006 VAR_OUTPUT
0007   CRC_Code:WORD;
0008   FINISH:BOOL:=FALSE;
0009 END_VAR
0010 VAR
0011   Reg16:WORD;
0012   j:BYTE;
0013   i:WORD;
0014   mval:WORD;
0015   temp_byte:BYTE;
0016
0017   Reg16 := 16#FFFF;
0018   mval := 16#A001; (*0xA001=1010000000000001*)
0019
0020 FOR i:=0 TO byteCounter-1 BY 1 DO
0021   temp_byte:=pData^;
0022   pData:=pData+1;
0023   Reg16:=(Reg16 XOR temp_byte);
0024   FOR j:=0 TO 7 BY 1 DO
0025     flg := 0;
0026     flg:=(Reg16 AND 16#0001); (*get b0 value*)
0027     Reg16 := SHR(Reg16,1); (*RIGHT shift 1 bit, set b15=0*)
0028     IF (flg = 1) THEN
0029       Reg16:=Reg16 XOR mval;
0030     END_IF;
0031   END_FOR;
0032 END_FOR;
```

图 7-3-4 自定义库（4）

对于自定义库，可以加口令保护。这与对程序加密的原理相同，只需在“资源”选项卡中，选中“工程选项”，打开“Passwords”，设定相应的口令，即可实现加密，如图7-3-5所示。

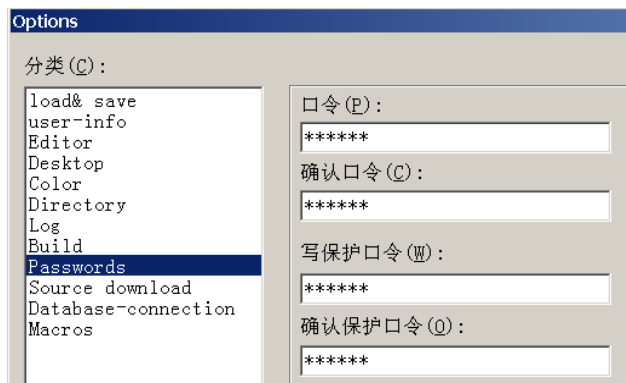


图 7-3-5 自定义库加密



本章小结

本章主要介绍了库的管理、分类和制作。在实际工程中，标准库（Standard.lib）一般不能满足工程需求，因此需要添加相应的库。此外，当有特殊需求时，可以自定义库。

第8章 管理 POU

本章主要介绍与 POU 的管理相关的几个常用功能选项，包括 POU 的创建和调用等，并介绍在实际应用中如何为工程配置任务。

8.1 管理 POU 菜单

在 PowerPro 软件主界面对象组织器的“程序”选项卡中，在程序列表中选择相应的程序，点击鼠标右键，弹出管理 POU 菜单，如图 8-1-1 所示。

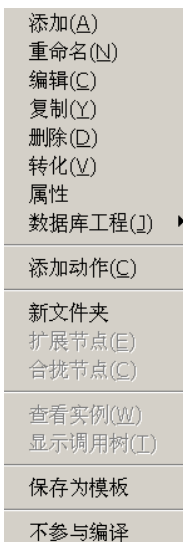


图 8-1-1 管理 POU 菜单

8.1.1 建立文件夹

对于较大的工程，为了便于管理和查阅，程序、数据类型和全局变量可以统一地组织在文件夹中，并且可以设置多级文件夹（前面带有加号）。通过拖拉可以在对象组织器的范围内移动程序，点中程序并按住鼠标左键拖到指定的位置即可，如图 8-1-2 所示。文件夹使工程的组织构架更加清楚，对程序没有任何影响。“新建组”命令在对象组织器中插入一个新文件夹。如果某个文件夹被选中，新建文件夹“New Folder”将创建在它的下层。如果某个文件被选中，新建文件夹“New Folder”将在它的同层创建。

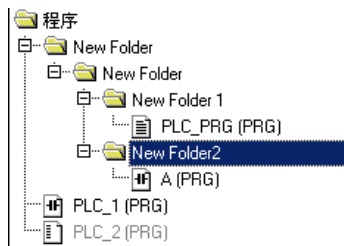


图 8-1-2 文件夹

8.2 创建 POU

8.2.1 创建程序

在对象组织器中选“程序”选项卡，在程序列表中点击右键，选择“添加”，如果列表中无 POU，则新 POU 名默认为“PLC_PRG”。在弹出的“创建 POU”对话框中，选择“POU 类型”为“程序”，如图 8-2-1 所示。

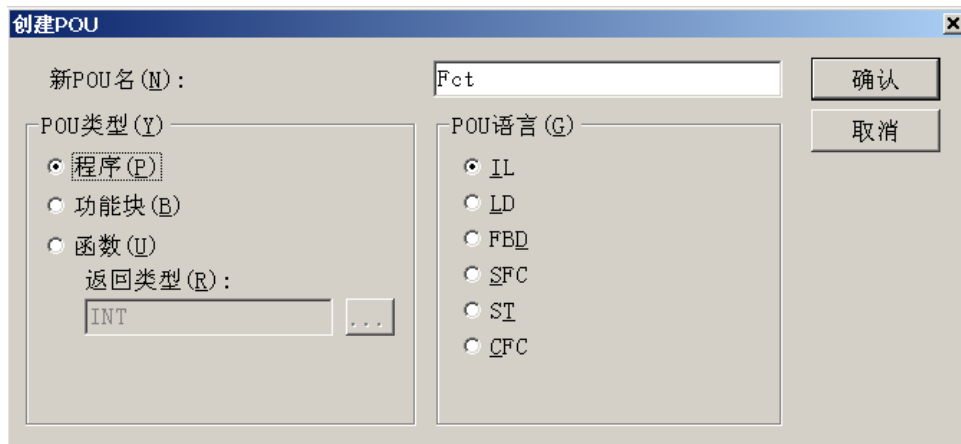


图 8-2-1 创建程序

8.2.2 创建函数

函数的外形结构与功能块类似，区别是函数只有一个输出端，以下说明如何创建函数。

在对象组织器中选“程序”，选择“添加”。在弹出的“创建 POU”对话框中选择“POU 类型”为“函数”，“返回类型”、“POU 语言”和“新 POU 名”可根据需要选择和输入，如图 8-2-2 所示。如果新 POU 名为“Fct”，则“Fct”即为函数名。通常，命名要尽量采用能反映其实际用途的字符，便于识别。

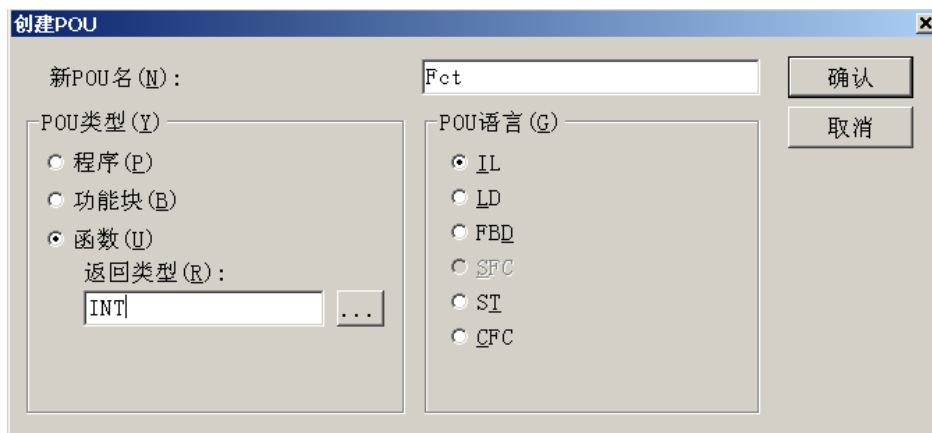


图 8-2-2 自定义函数（1）

设置好后，自动生成函数结构体，可以在里面添加所需要的内容，如图 8-2-3 所示。

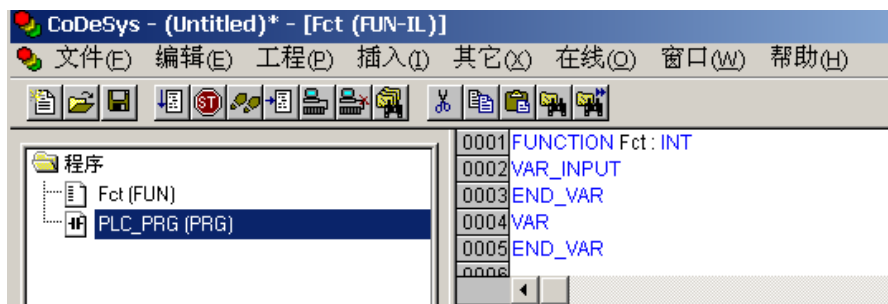


图 8-2-3 自定义函数（2）

下面以制作一个数学计算式为例来介绍函数的编写，如图 8-2-4 所示。从 VAR_INPUT 到第一个 END_VAR 之间用于定义输入变量，从 VAR 到第二个 END_VAR 之间用于定义中间变量，下方窗口进行函数算法编写。

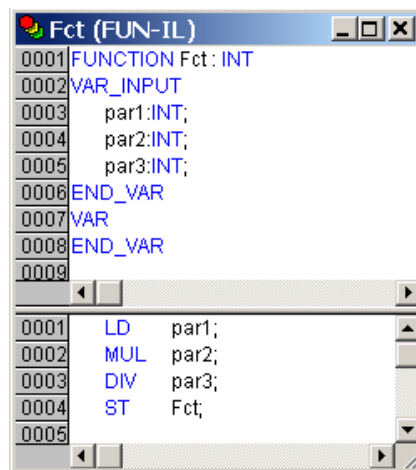


图 8-2-4 自定义函数（3）

编写完毕保存。编译通过后，可以在其它程序中直接调用，如图 8-2-5 所示。

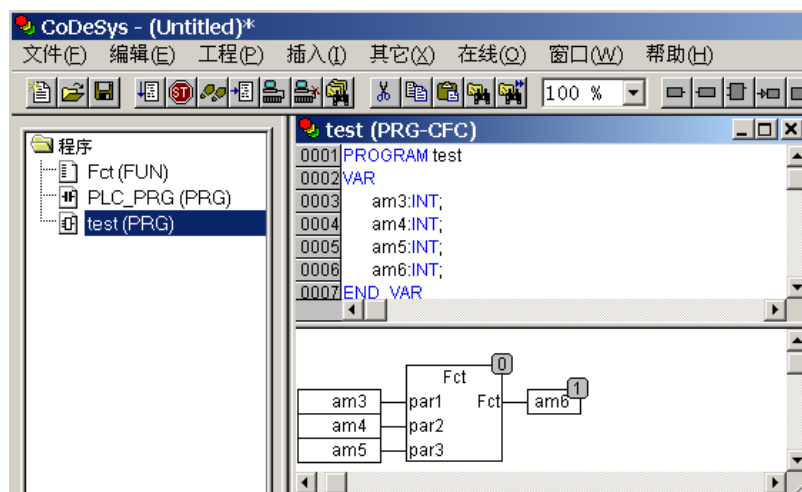


图 8-2-5 自定义函数（4）

8.2.3 创建动作

动作在顺序功能图 SFC 中是一个非常重要的概念，IEC 标准只承认 SFC 中的动作。

动作代表一个进一步的功能实现。这种功能也可以由别的语言以创建一般功能块的形式来实现。动作隶属于一个功能块或程序，可以像调用功能块一样，来调用功能块的动作。调用格式为：<程序名>.<动作名>或<实例名>.<动作名>。

“添加动作”命令可在当前选定的程序或功能块下，创建一个动作。在弹出的对话框中填入动作名，并选择动作实现的语言，如图 8-2-6 所示。

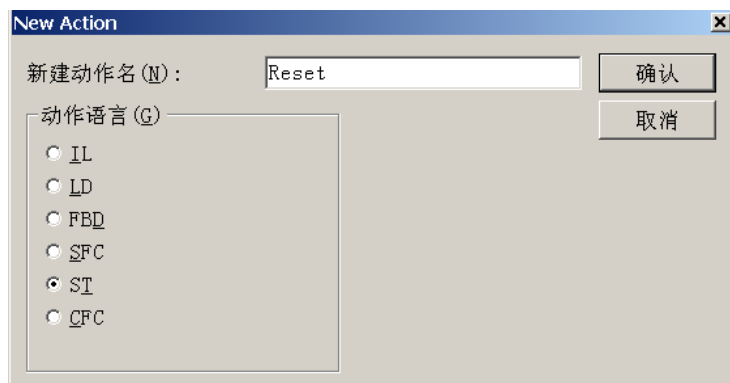


图 8-2-6 添加动作

新动作放置在对象组织器中当前程序的下层。创建成功后，在当前程序前面会出现一个加号，点击加号，可展开程序，出现新添加的动作块，如图 8-2-7 所示。

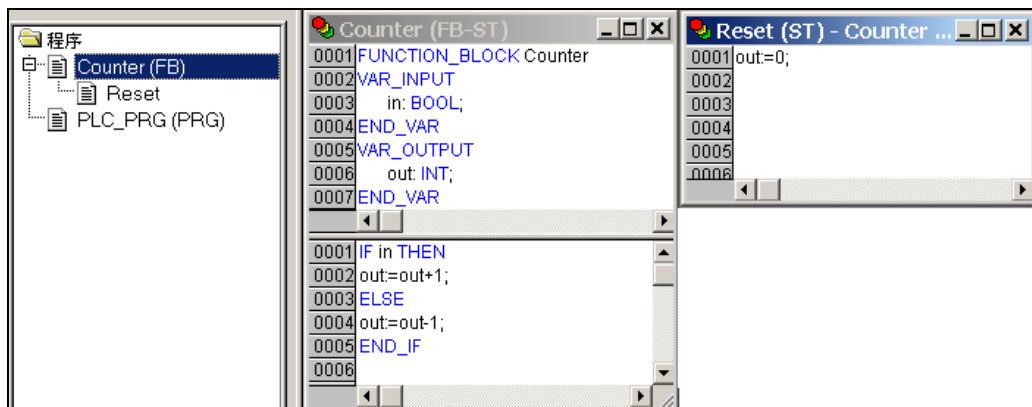


图 8-2-7 动作举例

在图 8-2-7 中，功能块 Counter 根据输入变量“in”的值，增加或者减少输出变量“out”的值。功能块中的动作 Reset，设置输出变量的值为 0。

◇ 举例：动作调用

声明部分：

```
PROGRAM PLC_PRG
```

```
VAR
```

```
    Inst : Counter;
```

```
    ERG:INT;
```

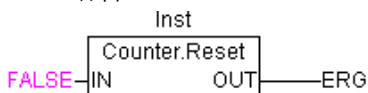
```
END_VAR
```

ST 语言：

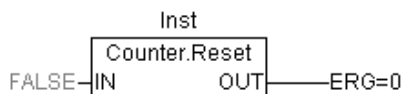
```
Inst.Reset (In := FALSE);
```

```
ERG := Inst.out;
```

FBD 语言：



在线模式下：



由于调用动作 Reset 设置输出变量“out”的值为“0”，所以，无论输入变量“in”为何值，输出的结果 ERG 都为“0”。

8.3 调用 POU

POU 的调用有两种方法。

- 被其它已经调用的 POU 来调用。
- 通过任务配置来调用，这种方法仅限于程序调用。当程序中没有进行任何任务配置时，系统会自动调用主程序 PLC_PRG。

POU 的调用要遵循以下原则。

- 程序可以调用函数、功能块和其它程序。
- 功能块可以调用函数和其它功能块。
- 函数可以调用函数。

8.3.1 调用程序

程序是唯一可执行的 POU。程序可以调用功能块和函数。程序的声明以关键字 PROGRAM 开始，如图 8-3-1 所示。

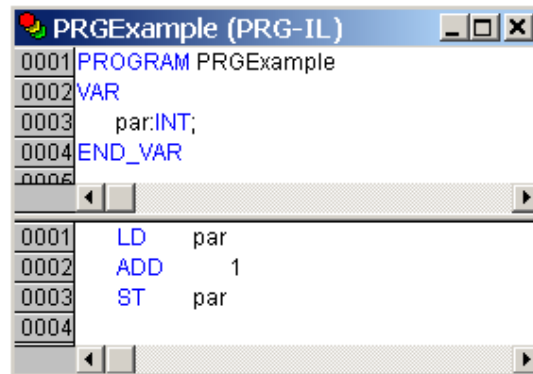


图 8-3-1 程序举例

程序可以被其它程序调用，但不允许在函数中调用程序。

可以采用不同语言调用上述程序“PRGExample”。如果调用使程序的输出值发生变化，程序会保持这个结果，直到下一次被调用。

IL 语言调用：

CAL PRGExample

LD PRGexample.PAR

ST ERG

ST 语言调用：

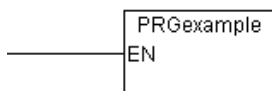
PRGExample;

Erg := PRGexample.PAR;

FBD 语言调用：



LD 语言调用：



8.3.2 调用功能块

➤ 功能块

一个功能块是一个 POU，本身没有返回值，输出一个或多个值。功能块声明以关键字 `FUNCTION_BLOCK` 开始。在图 8-3-2 中，用 IL 编写了一个功能块 FUB，有两个输入变量和两个输出变量。其中一个输出是两个输入的乘积，另一个输出是相等比较结果。

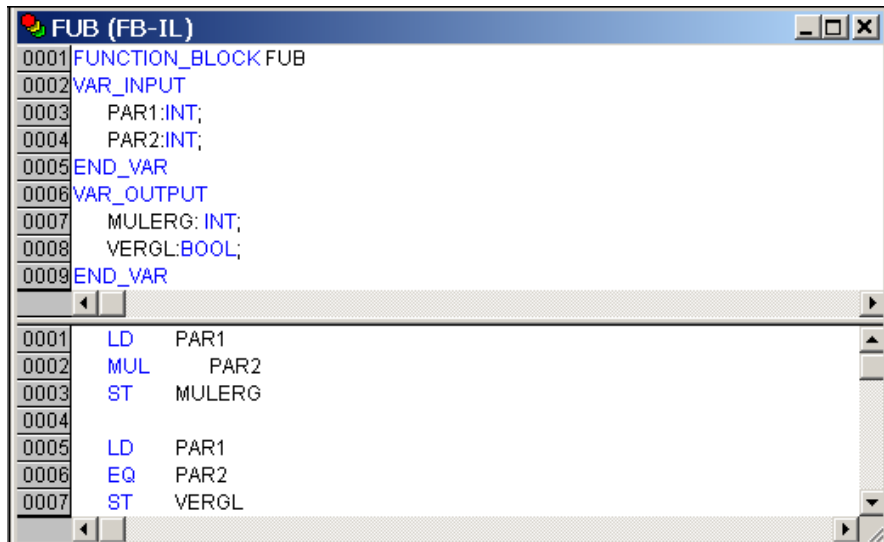


图 8-3-2 功能块举例

➤ 功能块实例声明

要想调用功能块，必须对功能块进行实例声明。例如定义一个名称为 `INSTANZ` 的 FUB 功能块，实例声明如下：

`INSTANZ: FUB;`

实例针对功能块而言，每个功能块实例就是一个独立的、可完成特定逻辑功能的对象。不同的程序、不同的任务都可以定义和调用功能块的应用实例，每个调用实例都占用独立的内存，保留独立的逻辑状态。通过定义实例实现对功能块的调用。

从外部只能改变功能块输入和输出参数。不允许对功能块的内部变量直接赋值。功能块实例名可以作为函数或功能块的输入。

➤ 调用功能块

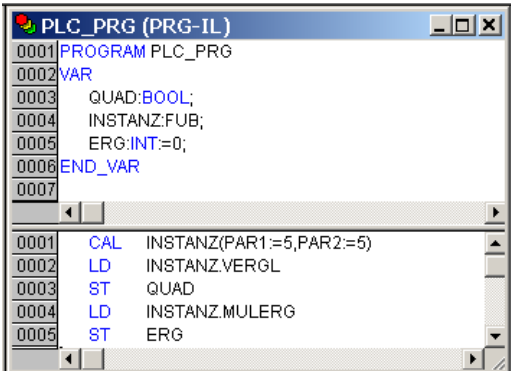
功能块的调用，只改变功能块实例中的值，结果只有当相同实例被调用时才起作用。通过输入“实例名.变量名”，调用功能块中的变量。

在文本语言 IL 和 ST 中，可以在功能块实例名后加圆括号，设置输入参数的初始值。和声明变量时初始化一样，赋值使用符号“:=”。

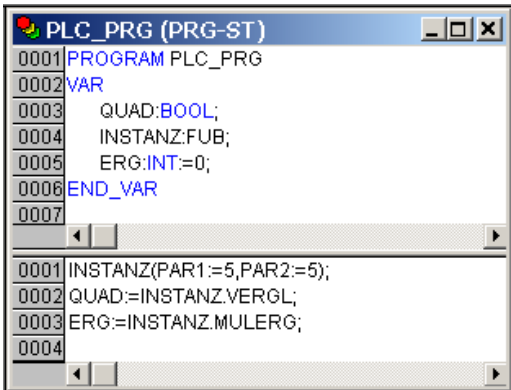
对于 SFC 语言，功能块调用只能发生在单步运行方式下。

功能块执行以后，所有的值都保留到下次执行之前保持不变。由于功能块中存在中间变量，使表面上相同的输入参数实际上可能输出不同的输出值。

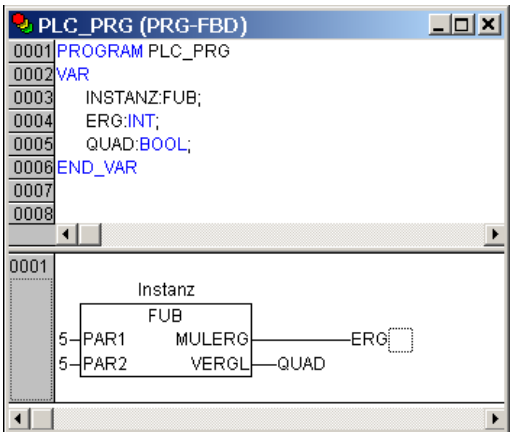
采用不同语言调用上述功能块 FUB 如图 8-3-3 所示。乘积的结果保存在变量 ERG 中，比较的结果保存在 QUAD 中。



IL 语言调用



ST 语言调用（声明部分与 IL 相同）



FBD 语言调用（声明部分与 IL 相同）

图 8-3-3 功能块调用

➤ “功能块”与“使能运算符”区别解析

介绍了功能块之后，下面介绍使能运算符，以便进一步区分功能块与使能运算符的区别。在前面提到过二者的大体区别是，功能块与使能运算符的调用方式不同。对于功能块，其自备使能端，不论是否使能，在程序运行时，均会执行该功能块。然而对于使能运算符，只有在使能端 EN 有效时，才可以调用该使能运算符。但是对于具体的应用则没有介绍，这里用 LD 语言编写一个简单程序进行介绍，如图 8-3-4 所示。

从图中可以看到，通电延时计时器 T1 的输入端 IN 相当于使能运算符的使能端，即所谓的自备使能端。程序开始运行时，当延时 1s 后，Q.0 便置 1，PLC 中的对应通道 Q.0 灯变亮。然而对于使能运算符 ADD，在添加使能运算符时就产生其使能端 EN，只有使能端有效时，方可调用此使能运算符。即只有当 I.0 为 1 时，ADD 使能运算符才会运行。

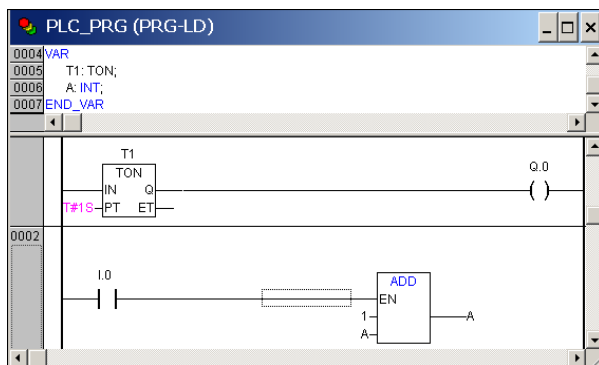


图 8-3-4 功能块与使能运算符

8.3.3 调用函数

一个函数是一个 POU。函数执行时，会对一系列特定的输入产生唯一数据类型的输出结果。相对于功能块而言，函数只有一个输出结果，没有任何内部条件。也就是说，只要给定相同的输入参数，调用函数必定得到相同的运算结果。平时所使用的各种数学运算，例如 SIN(X)等就是典型的函数类型。当定义函数时，函数必须接收一个数据类型作为返回值（返回数据类型）。函数的计算结果赋给函数本身，即函数的输出变量就是函数名本身。函数定义以关键字 FUNCTION 开始。

在图 8-3-5 中，用 IL 语言编写了函数 Fct，有三个输入变量，返回的结果是前两个数的乘积除以第三个数。调用时，函数相当于表达式里的一个运算符，可以赋初值。

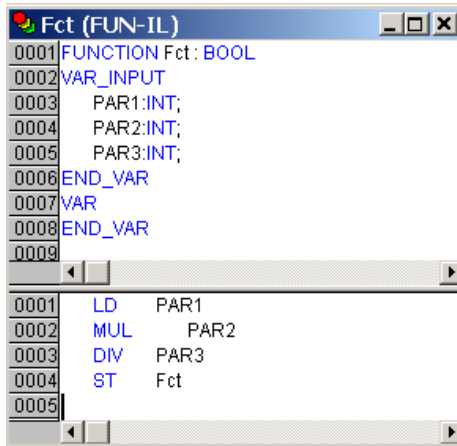


图 8-3-5 函数举例

◇ 举例：调用上述函数 Fct

IL 语言调用：

LD 7

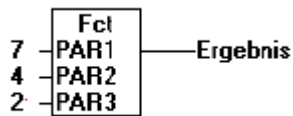
Fct 4,2

ST Result

ST 语言调用：

Result := Fct(7, 4, 2);

FBD 语言调用：



在 SFC 语言中，函数调用只能用单步或转化来实现。

◇ 举例

下面再列举几个函数调用的例子。

例 1：定义函数 CheckBounds，用这个函数来检查工程变量范围的溢出情况，如图 8-3-6 所示。

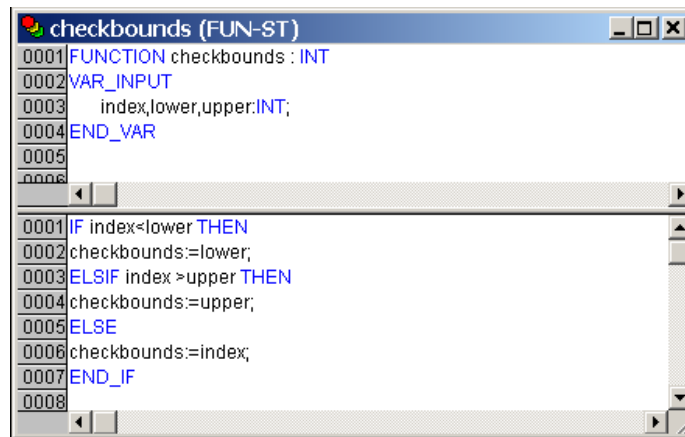


图 8-3-6 函数调用举例（1）

例 2：如果在工程中定义了名为 CheckDivByte、CheckDivWord、CheckDivDWord 和 CheckDivReal 的变量，图 8-3-7 所示为实现 CheckDivReal 函数的例子。

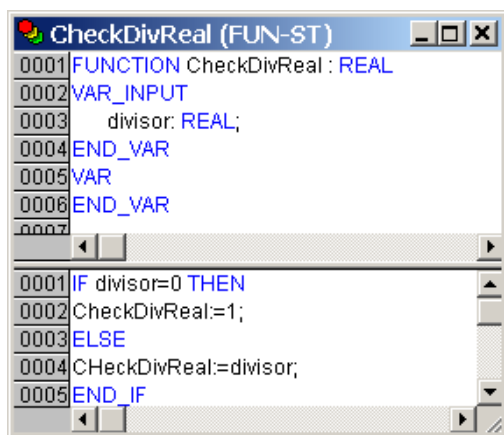


图 8-3-7 函数调用举例（2）

运算符 DIV 使用函数 CheckDivReal 的输出作为除数，如图 8-3-8 所示。避免被 0 除，除数 d 由原值 0 强制成 1，除的结果是 799。

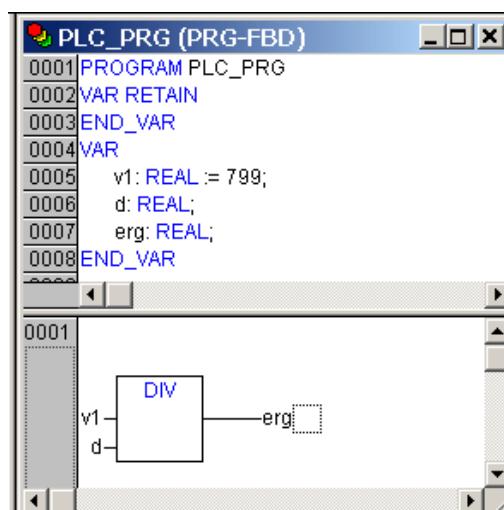


图 8-3-8 函数调用举例（3）

8.4 任务配置

对于一个工程，可以根据需要配置多个任务，调用不同的程序。在通常情况下，建议只定义一个任务，并通过它调用主程序，其它程序则通过主程序来间接调用。这种调用方法只有程序可以使用，功能块和函数不能这样调用。

严格地说，如果没有使用任务配置，在单任务环境中，系统默认 PLC_PRG 为主程序，自动且唯一调用它，通过它实现对其它程序的调用。如果使用任务配置，程序的调用依赖于任务分配。在一般情况下，PLC 控制单任务环境即可满足要求，不需要进行任务配置。

8.4.1 配置任务

在“资源”选项卡中双击“任务配置”，右侧弹出任务配置窗口，鼠标右键点击窗口中的“任务配置”项，选择“Append Task”项，如图 8-4-1 所示。

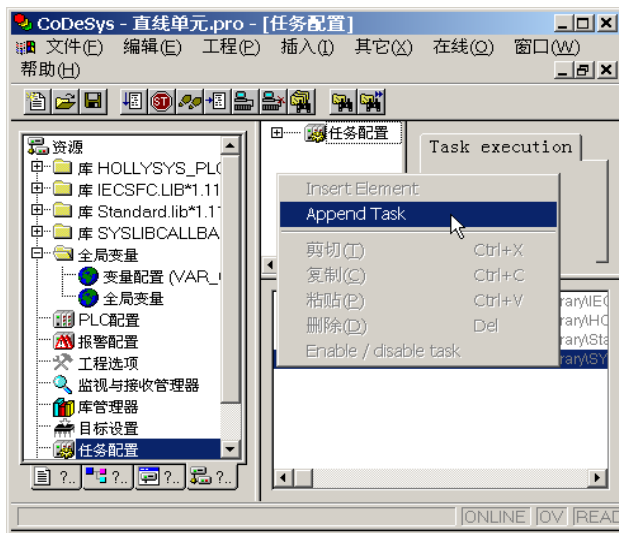


图 8-4-1 任务配置

在弹出的“Taskattributes”窗口中可以分别填入“任务名”、优先级“Priority”、“时间间隔”等信息，如图 8-4-2 所示。

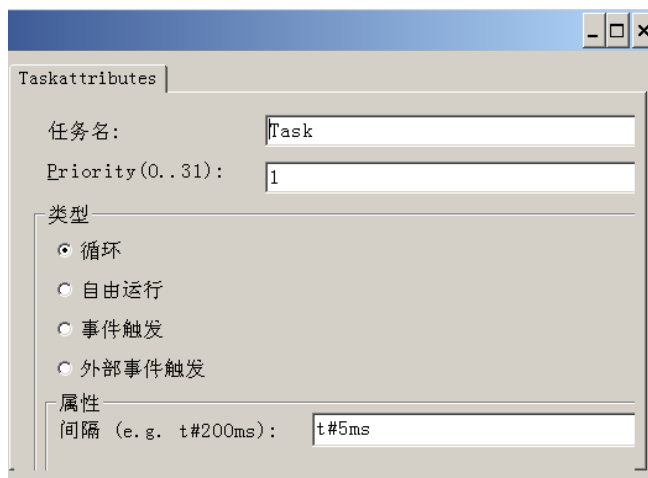


图 8-4-2 任务属性

➤ 任务名

任务的名字，以字母加数字的方式任意填写。

➤ Priority

任务的优先级。对于小型 PLC 不区分优先级，先配置的任务先调用。所以采用默认值，不需进行任何修改。

➤ 类型

循环：周期地处理任务，周期定义在“属性”的“间隔”中。如果选择“循环”类型，则会发现在新建任务“NewTask”的左侧出现“🔄”图标。

自由运行：只要启动程序，就处理任务，运行一次后自动重新启动程序，如此循环。如果选择“自由运行”类型，则会发现在新建任务“NewTask”的左侧出现“🏁”图标。

事件触发：只要变量有上升沿，就启动任务，变量定义在“属性”的“事件”中。如果选择“事件触发”类型，则会发现在新建任务“NewTask”的左侧出现“🔌”图标。当类型“事件触发”时有效，输入全局变量，使用按钮或 F2 键可以得到所有全局变量列表。PLC 不支持此项功能。

外部事件触发：PLC 不支持此项功能。

➤ 属性

间隔：当“类型”中选择“循环”时，要填入间隔时间。任务包含的程序运行的周期根据控制速度的需要填写。填写时注意在前面加“t#”（固定格式），单位可选 ms 毫秒、s 秒、m 分钟、h 小时、d 天等。

8.4.2 系统事件

系统事件用于在工程中调用 POU，而不是用于在任务中调用 POU。当相应事件触发时，调用相应的 POU，例如 T2、T3、T4 定时器溢出和快速外部中断脉冲到达等都会产生中断，调用相应的事件。

在“资源”选项卡中双击“任务配置”，右侧弹出任务配置窗口，点击窗口中“任务配置”下的“System events”，右边窗口显示所有可用的系统事件，如图 8-4-3 所示。在右边窗口中选择需要使用的系统事件，即在系统事件的前面方框里打勾，然后在该系统事件后面的“called POU”处，填入当事件触发时需要调用的程序。系统事件的含义见表 8-4-1 所示。

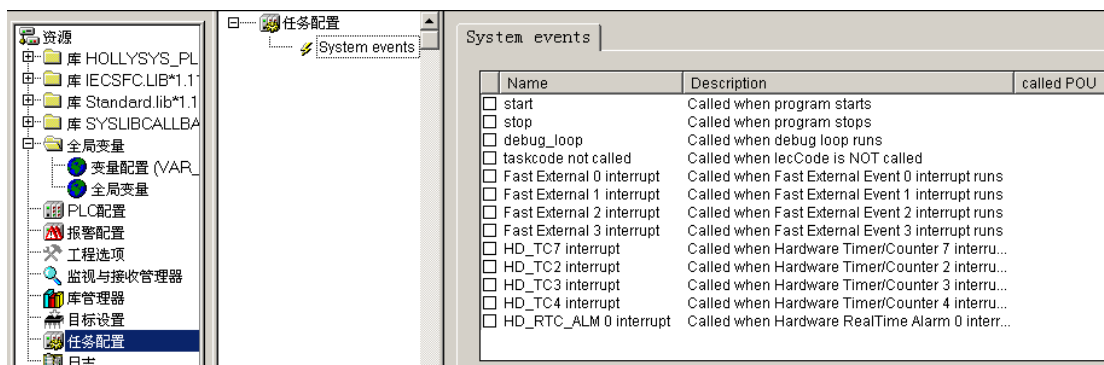


图 8-4-3 System events 对话框

表 8-4-1 系统事件

Name	Description	Called POU
Start	当程序开始时调用	当事件触发时需要调用的程序，可在此处定义其 POU 的名称。
Stop	当程序停止时调用	
Debug_loop	当调试循环运行时被调用	
Taskcode not called	不调用任务代码	
Fast External 0 interrupt	快速外部中断 0 产生时调用	
Fast External 1 interrupt	快速外部中断 1 产生时调用	
Fast External 2 interrupt	快速外部中断 2 产生时调用	
Fast External 3 interrupt	快速外部中断 3 产生时调用	
HD_TC7 interrupt	定时器 T7 中断产生时调用	
HD_TC2 interrupt	高速计数器 T2 产生中断时调用	
HD_TC3 interrupt	高速计数器 T3 产生中断时调用	
HD_TC4 interrupt	高速计数器 T4 产生中断时调用	
HD_RTC_ALM0 interrupt	实时时钟报警 0 产生中断时调用	

◇ 举例

✓ 要求

不受 PLC 扫描周期的影响，实现下列功能：

每当 I0.6 到达一个上升沿，PLC 立刻响应该脉冲，并产生一个中断，%MW100 中的值增加 10。

每当 I0.7 到达一个上升沿，PLC 立刻响应该脉冲，并产生一个中断，%MW102 中的值增加 15。

✓ 程序分析

按上述要求，硬件需要使用 LM3106 CPU 模块，软件需要使用以下模块：

Fast_ExINT_LM3106（快速外部中断）

程序分为如下 3 个部分：

主程序——定义 LM3106 CPU 模块的快速外部中断模式。

INT3PRO——I0.6 脉冲到达执行的中断程序，%MW100 中的值增加 10。

INT2PRO——I0.7 脉冲到达执行的中断程序，%MW102 中的值增加 15。

✓ 程序编制

主程序：

首先要将程序所用到的库 Hollysys_PLC_Ex_ExINT.lib 添加到库管理器。按照要求，I0.6 和 I0.7 每到达一个上升沿，就产生一个中断，I1.0 不使用，则 Fast_ExINT_LM3106 功能块的 Mode=16#50，主程序的变量定义和梯形图如图 8-4-4 所示。

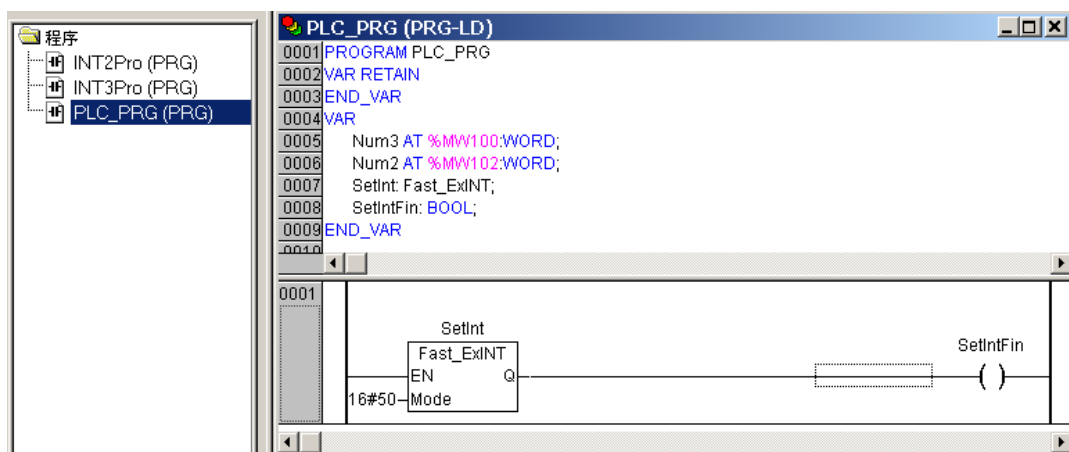


图 8-4-4 主程序的变量定义和梯形图

由于使用了快速外部中断 3 (I0.6) 和快速外部中断 2 (I0.7)，所以在如图 8-4-5 所示的位置前打勾。

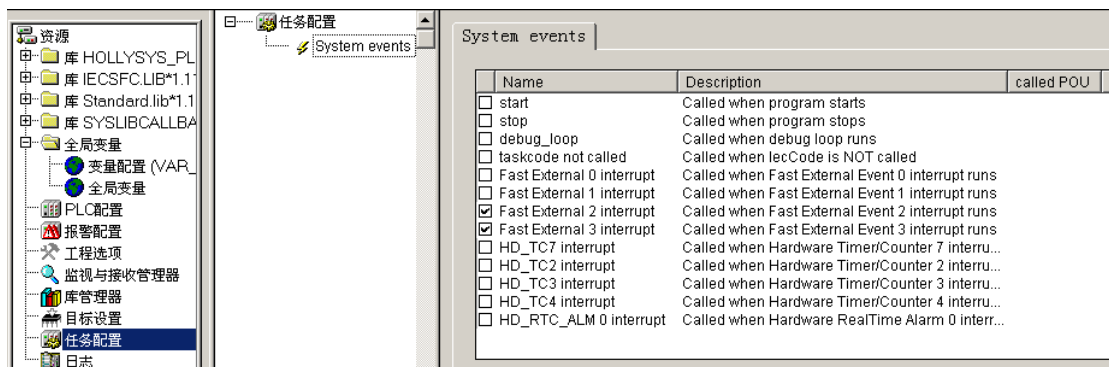


图 8-4-5 快速外部中断选择

在 Fast External 2 interrupt 和 Fast External 3 interrupt 后面分别创建一个子程序 INT2Pro 和 INT3Pro，分别点击 Create POU，则 2 个中断程序被创建成功，如图 8-4-6 所示。

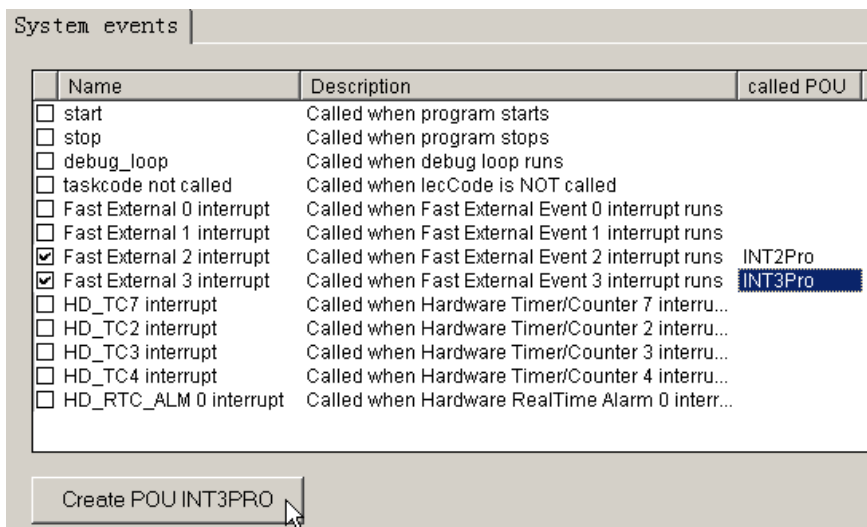


图 8-4-6 创建中断程序

所创建的中断程序默认为 ST 语言，可以选择转换为 LD 语言。转换前需要编译通过，如图 8-4-7 所示。



图 8-4-7 将 ST 转换为 LD

INT3Pro 中断程序梯形图如图 8-4-8 所示。

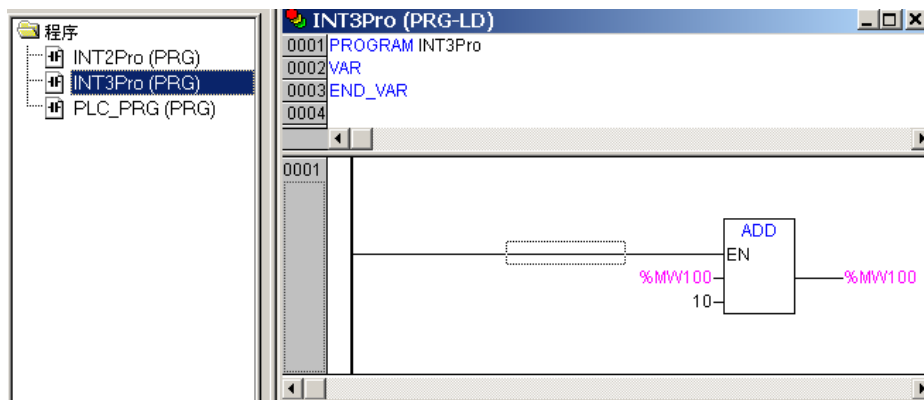


图 8-4-8 INT3Pro 中断程序梯形图

INT2Pro 中断程序梯形图如图 8-4-9 所示。

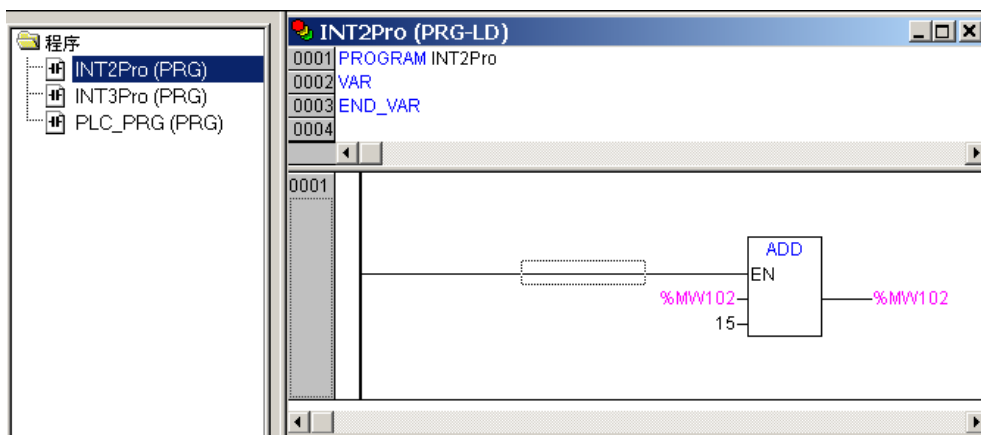


图 8-4-9 INT2Pro 中断程序梯形图

然后就可以在 INT2Pro 和 INT3Pro 中编写梯形图程序。当然也可以不进行语言的转化，使用 ST 直接编写。当系统事件触发时，调用相应的程序。

注意，系统事件不支持仿真模式，只有在程序编译通过且登录运行后才会响应该事件。当一次进行多个任务配置时，要求先进行全部编译之后，再保存文件。

8.4.3 任务调用程序

鼠标右键点击“Task”项，在弹出的菜单上选择“Append Program Call”，弹出“程序调用”对话框，如图 8-4-10 所示。

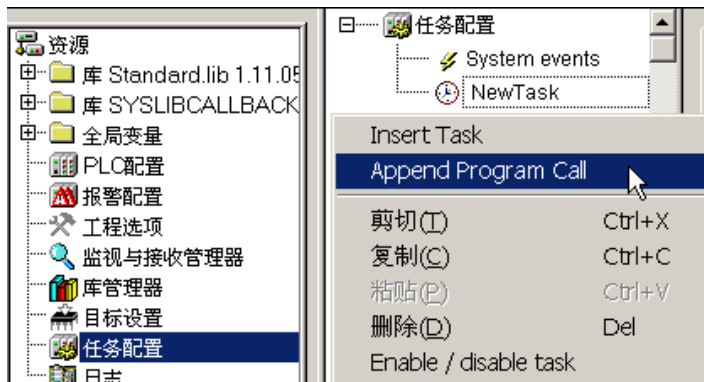


图 8-4-10 添加程序 (1)

鼠标点击如图 8-4-11 中的省略号，选择需要调用的程序，点击“确认”按钮返回，程序即被任务所调用。



图 8-4-11 添加程序 (2)

图 8-4-12 所示的例子是一个名称为“task”、优先级为“1”、时间周期为“5ms”的任务，通过这个任务调用 reset 程序。

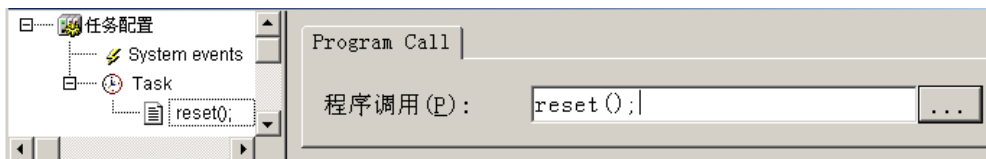


图 8-4-12 任务配置示例



本章小结

本章主要介绍了 POU 的管理、创建和调用，并介绍了任务配置。

第9章 管理工程

9.1 管理工程菜单

PowerPro 软件以工程文件的形式来保存用户程序，所有的信息都集成存放在以*.pro 为后缀的工程文件中。PowerPro 软件的默认安装目录结构如图 9-1-1 所示，其中“Library”和“Projects”分别用来存放库文件和工程文件。系统提供了很多针对工程的操作，帮助用户更好地管理工程。

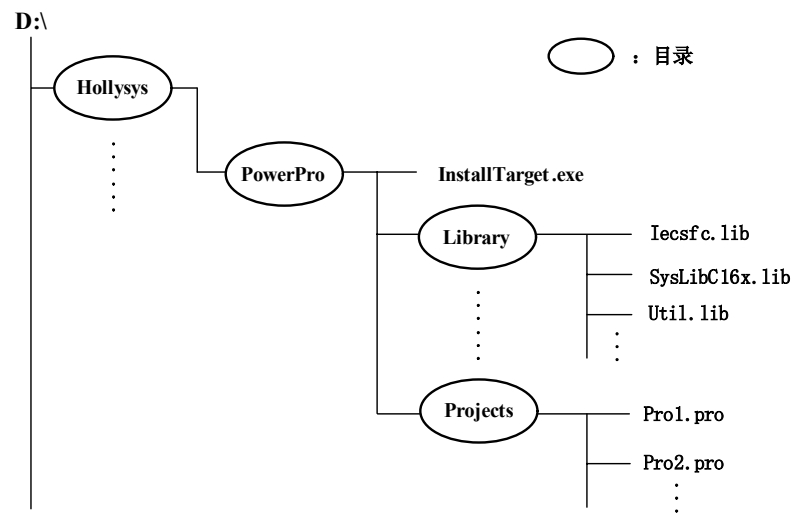


图 9-1-1 软件目录结构

打开主菜单中的“工程”菜单，如图 9-1-2 所示。

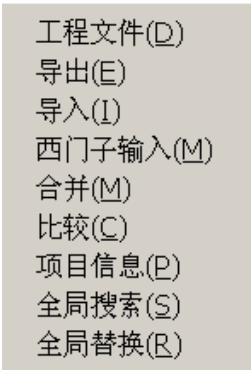


图 9-1-2 管理工程菜单

下面介绍几项常用的工程管理命令。

9.1.1 打印工程文件

使用“工程”/“工程文件”命令可以打印整个工程的文档，或者选择其中的一部分打印。工程文件由工程信息、文档内容、程序和资源等元素构成，如图 9-1-3 所示，其中资源包括全局变量、PLC 配置、报警配置、工程选项、监视与接收管理器、任务配置和参数管理等内容。

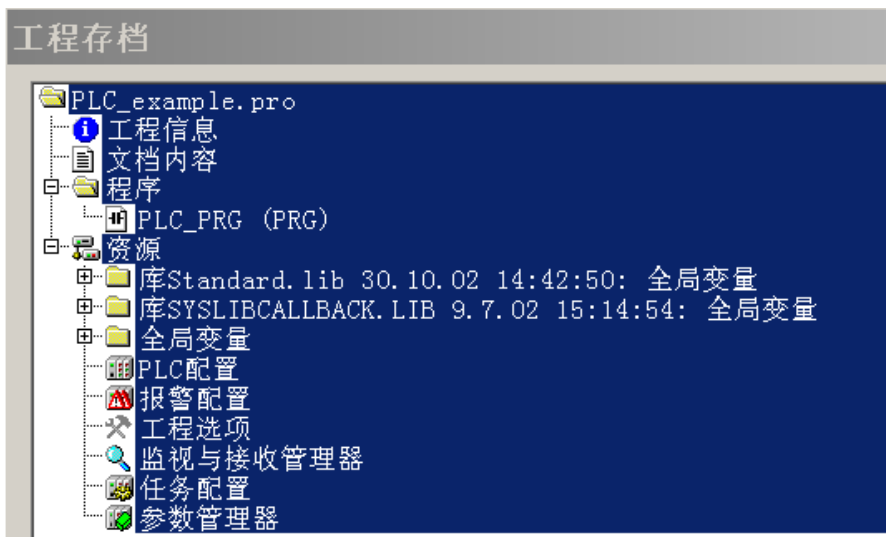


图 9-1-3 打印工程文件

用蓝色高亮可以选择需要打印的区域。如果要选定整个工程，直接在第一行选中工程文件夹即可。如果只想选定一个单独的对象，点击相应的对象。对象的前面带有加号的为多重对象，点击加号可扩展。选好后弹出打印对话框，可设置打印属性。可以按照一定的格式打印一个工程文件。选择“文件”/“打印机设置”，弹出如图 9-1-4 所示的对话框。



图 9-1-4 Documentation Setup 对话框

➤ 文件

在文件字段，输入带“.dfr”扩展名的文件名，页面布局会保存在此文件中。保存设置的缺省文件是 DEFAULT.DFR。

➤ 浏览

如果要保存在一个已存在的布局里，可点击“浏览”按钮浏览目录找到所需的文件。

➤ 编辑

如果点击“编辑”按钮，设置页的对话框则会出现。选择“插入”/“占位符”，可分别插入页、程序组织单元名、文件名、日期、内容。选择好插入的内容及位置后，按住鼠标左键，将其拉伸为矩形后，松开鼠标左键，即可将其放在可打印的页面上和文本区。

➤ 为每一个工程开始新页

如果选中“为每一个工程开始一个新页”，则打印时另起新页。否则不另起新页。

➤ 为每一个子工程开始新页

如果选中“为每一个子工程开始一个新页”，则打印时另起新页。否则不另起新页。

➤ 打印机设置

使用“打印机设置”按钮，打开打印机设置。点击“属性”，同样可以弹出“打印”对话框。

图 9-1-5 所示为一个自定义的页面布局，分别在相应的位置插入页码（Page）、程序组织单元名（POUName）、文件名（FileName）、日期（Date）和内容（Content）等占位符，占位符相当于通过将矩形置于布局中进行占位。在打印输出时，相应的内容会映射在相应位置处。当布局设置好后，点击“确认”。

如果模板被更改，则 PowerPro 软件会提示是否关闭以前保存的信息。

另外，在“工程”/“选项”/“Desktop”下，如果选中“显示打印边界”，则会在程序的相应范围内出现红色虚线，以示打印界限，否则，不会显示该虚线。

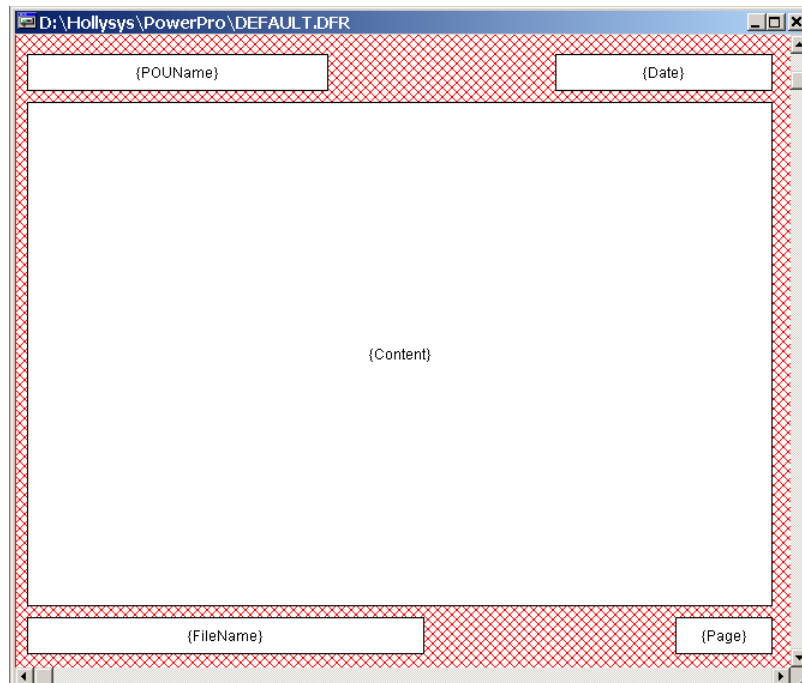


图 9-1-5 页面布局设置

9.1.2 导入导出工程

“工程”/“导入”、“导出”命令可以导入或导出工程对象，以便在不同的工程文件中交换程序。在导出文件时，对话框底部的“每个对象对应一个文件”选项，可以选择对象导出到同一个文件，还是导出到不同的文件中，如图 9-1-6 所示，

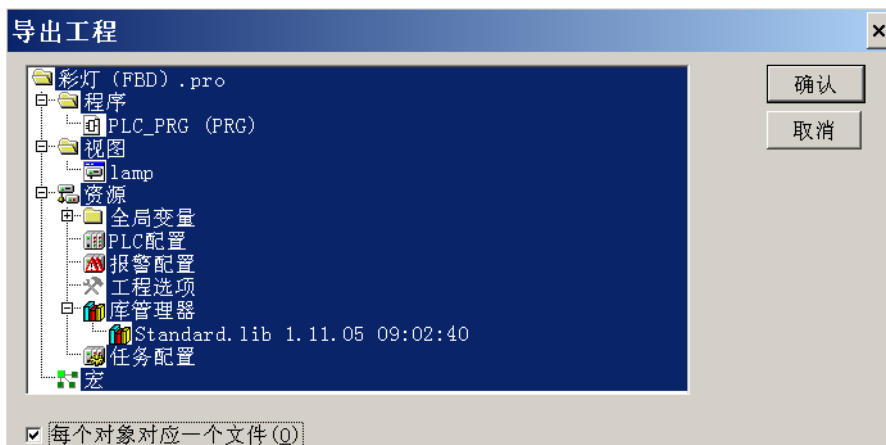


图 9-1-6 导出工程 (1)

设定完成后，弹出导出工程对话框，如图 9-1-7 所示。在“目录”下指定导出对象的保存路径，即在指定目录下生成以*.exp 为扩展名的若干导出文件，同时弹出消息窗口列表，显示相关信息。

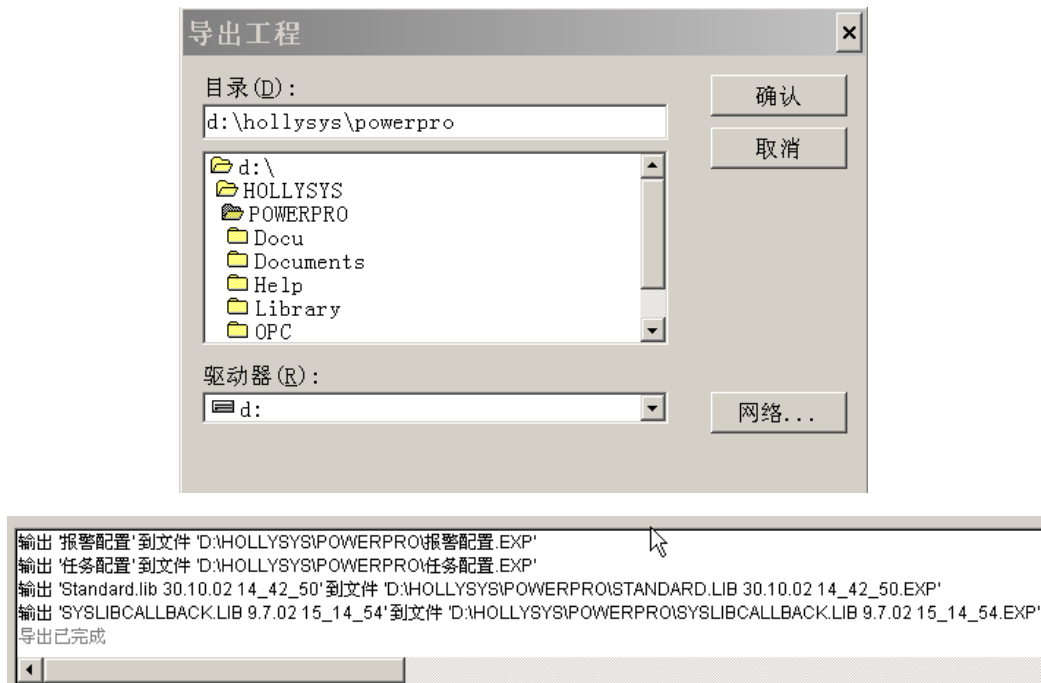


图 9-1-7 导出工程 (2)

导入工程对象时，选定导入的文件*.exp，对象即被导入到当前工程的相应窗口中去。如果在该窗口中，已经有一个同名的对象存在，则会出现对话框，询问是否替换它：“对象已经存在，要替换它吗？”，如图 9-1-8 所示。

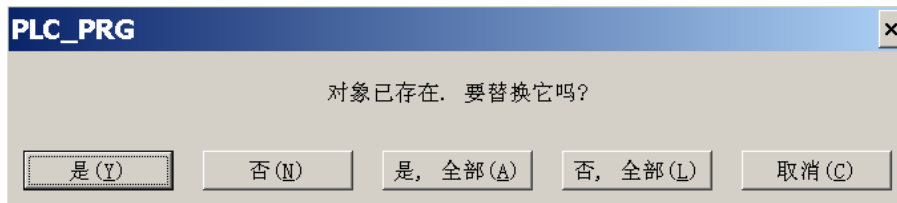


图 9-1-8 覆盖提示

导入的对象和导出的结果都存放在*.exp 文件中。通过*.exp 文件，可以在不同的工程之间搭起一座沟通的桥梁，可以方便地交换工程中的所有对象，如图 9-1-9 所示。

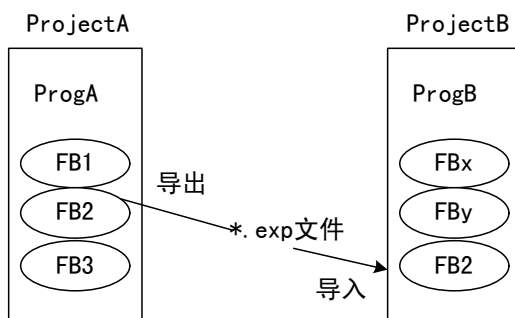


图 9-1-9 导出、导入示意图

9.1.3 合并工程

“工程”/“合并”命令可以把其它工程中的对象合并到当前工程中。打开合并工程对话框，选定一个工程，弹出该工程的所有对象列表，选择需要合并的对象，如图 9-1-10 所示。

对于库或资源的合并，也会弹出对话框提示是否覆盖。

对于程序的合并，会将新程序添加到原程序列表中。如果合并的程序与当前工程中的程序同名，则会弹出对话框询问是否覆盖。

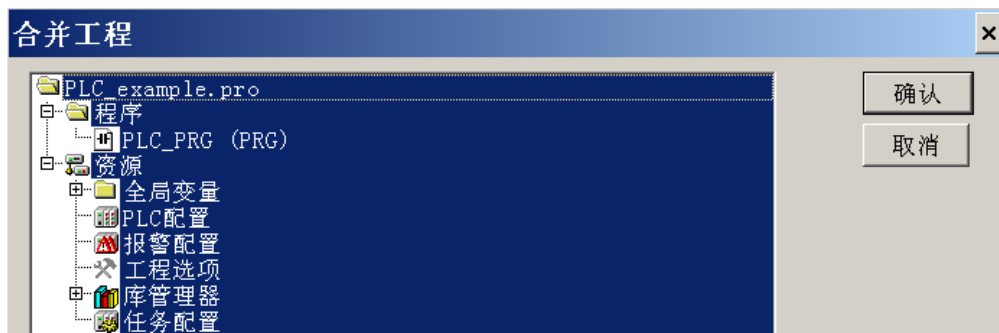


图 9-1-10 合并工程

9.1.4 比较工程

“工程”/“比较”命令可比较当前工程与另一个工程的所有对象的内容。如果想知道是否在当前工程中做了修改，那么可以比较当前打开的工程和它的前一个版本。当执行此命令后，弹出工程比较对话框，如图 9-1-11 所示。

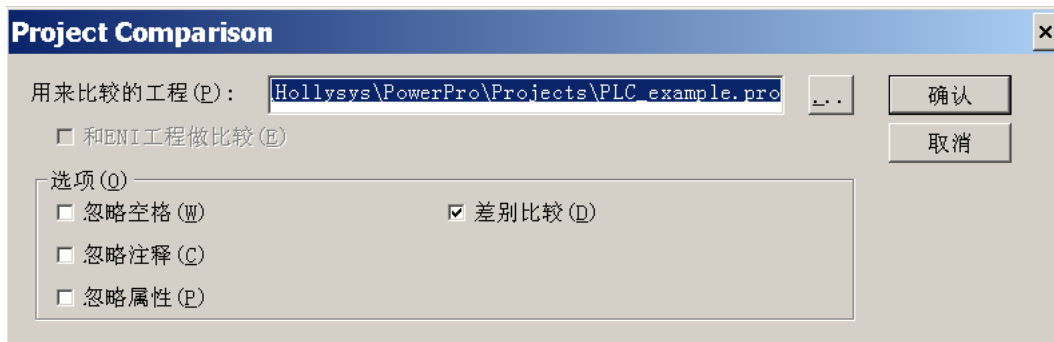


图 9-1-11 比较工程（1）

选定与当前工程比较的目标工程后，弹出比较结果列表，如图 9-1-12 所示。不同颜色的对象文字代表不同的比较结果，共有五种可能的颜色：

- 黑色：对象内容相同。
- 红色：对象内容不一致。
- 蓝色：当前工程中新增加的对象。
- 绿色：当前工程中没有的对象。
- 灰色：两个工程中内容不一致的对象，鼠标双击对象可查看详细内容。

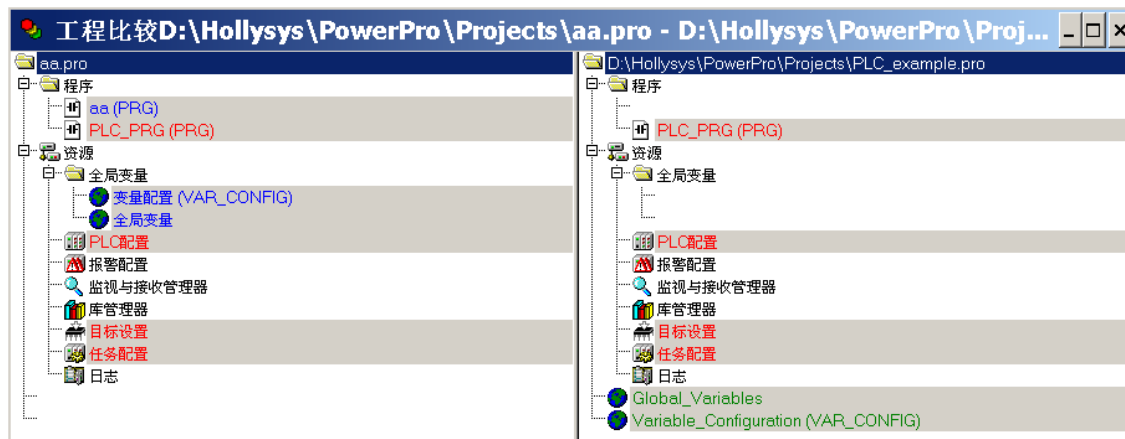


图 9-1-12 比较工程（2）

在具体的程序编写过程中，还可以利用“比较”命令，对修改前后的程序代码进行比较，如图 9-1-13 所示。在比较模式下，不允许编辑工程对象，所有操作功能禁用。只有关闭了比较列表窗口，才能对工程进行编辑。

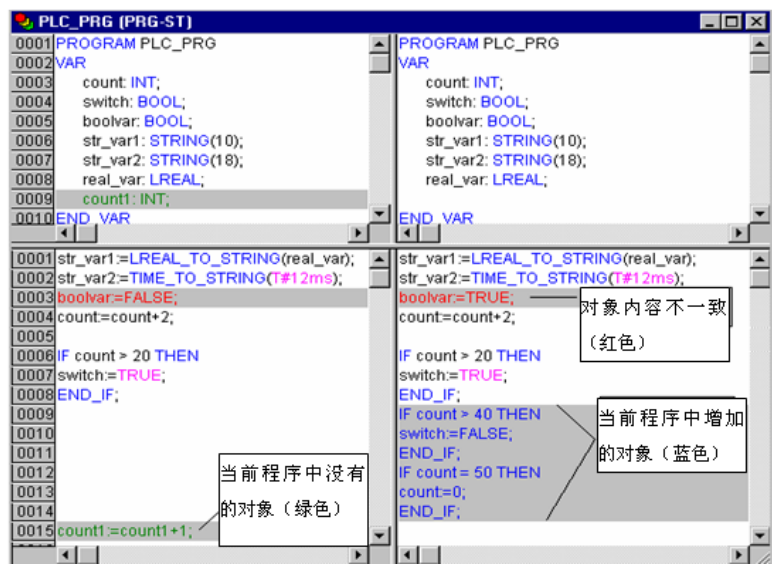


图 9-1-13 比较程序

9.1.5 用户口令

“工程”/“选项”/“Passwords”选项可以设定单一用户口令，保护 CPU 免受未授权的访问，防止文件被随意地打开和更改，如图 9-1-14 所示。其中“口令”字段为工程设置浏览权限，“写保护口令”字段中为工程设置更改权限。

口令(P):

确认口令(C):

写保护口令(W):

确认保护口令(Q):

图 9-1-14 设定口令选项

输入口令时，对应每一个字符出现一个星号（*），在“确认口令”字段再次输入同样的内容。设定成功后，下次打开工程时，就需要通过口令验证。如果出现如图 9-1-15 所示的对话框，说明两次输入口令不一致，需要重新输入。

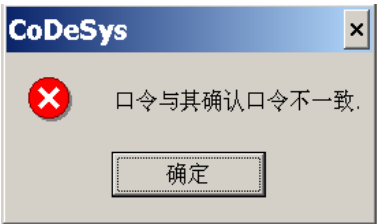


图 9-1-15 口令错误提示

对于一个写保护的工程，在没提供口令的情况下是不能被更改的。如果打开一个文件时系统提示输入写保护口令，而只是简单的点击“取消”按钮，那么工程中所有的编辑设定功能都禁用，只可以使用编译或仿真等查看功能。

如果存在多个用户，需要定义用户组，为每个用户组分配不同的访问权限。系统共提供了 8 个用户组，它们对工程对象拥有不同的访问权限。每个用户组的成员都通过口令来确认身份。“工程”/“用户组命令”用来设定用户组的口令。用户组从 0 到 7，0 组拥有管理员权限，只有 0 组的成员可以决定其它用户组和对象的口令和访问权，如图 9-1-16 所示。

建立一个新工程时，所有的口令初始状态都是空的。当一个口令被设置为 0 组，用户以 0 组成员的身份进入工程时，就可以对其它级别的组设定口令。如图 9-1-16 所示，在左边的组合框“用户组”中选定组，在右边输入相应的口令。

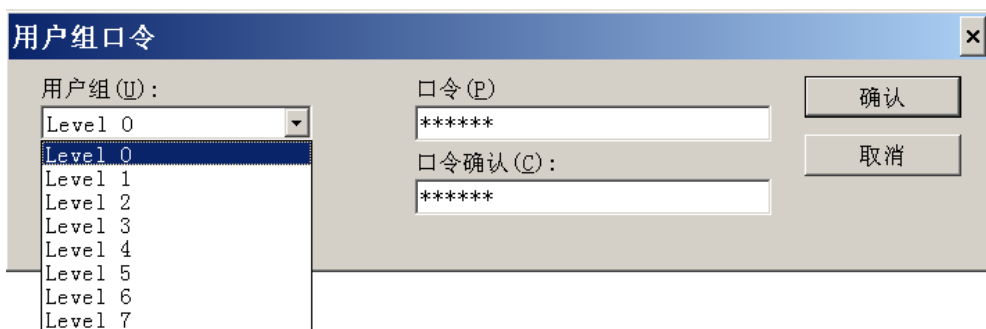


图 9-1-16 用户组命令

使用“工程”/“对象”/“属性”命令可以为用户组设置不同的访问权限，如图 9-1-17 所示。访问权限有三种可用的设置：

- 无访问权限：对象不允许被此用户组的成员打开。
- 只读权限：对象能够被此用户组的成员读入，但不能被更改。
- 完全权限：对象能够被此用户组的成员打开和更改。

访问权限的设置仅对选中的对象有效，每个 POU、硬件配置或全局变量都是对象成员。如果忘记了口令请及时与制造商联系，口令随工程一起保存。

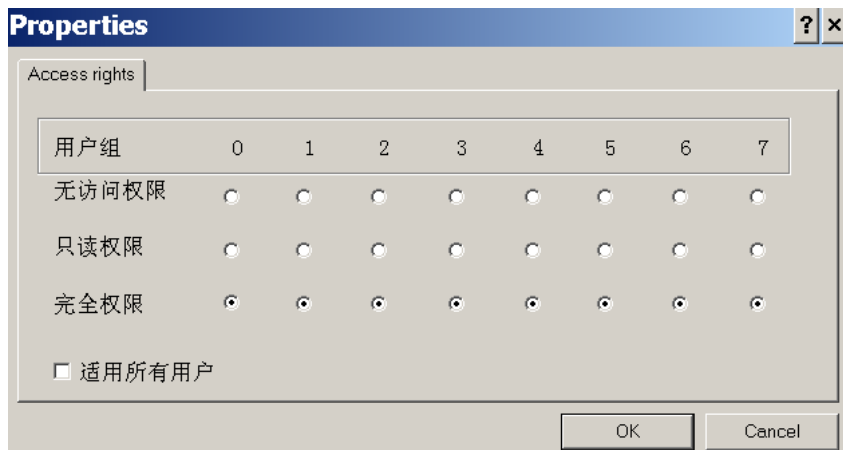


图 9-1-17 用户操作权限设定

下面是一个简单的用户组命令的应用例子。

打开“工程”/“对象”/“属性”，设置用户组权限，如图 9-1-18 所示。
设置用户组 0 和 1 的访问权限为完全权限。
设置用户组 2 和 3 的访问权限为只读权限。
设置用户组 4、5、6 和 7 的用户权限为无访问权限。
设定好用户组权限后，分别对各用户组设置相应口令。

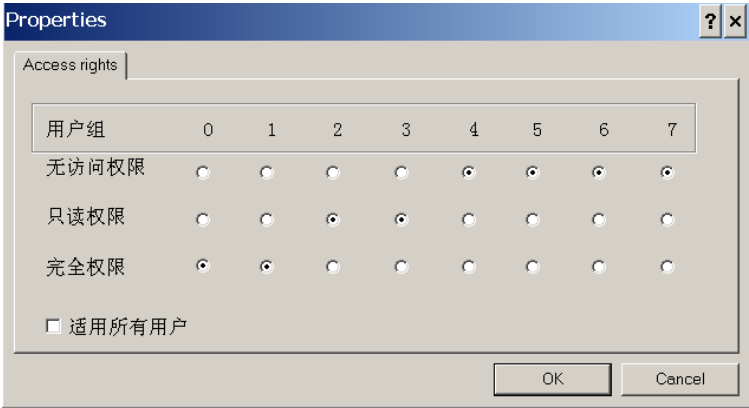


图 9-1-18 设定用户组权限属性

这里简单设定用户组口令，如图 9-1-19 所示。

先对“Level 0”进行口令设置，在“口令”空栏处填入“0”，在“口令确认”空栏处填入“0”，点击“确认”，则“Level0”口令设置完毕。

同样，分别对 Level1、Level2 和 Level3 设置口令。用户组与口令的对应关系设置如表 9-1-1 所示。

由于用户组 4、5、6 和 7 的用户权限为无访问权限，所以无需设置其口令。

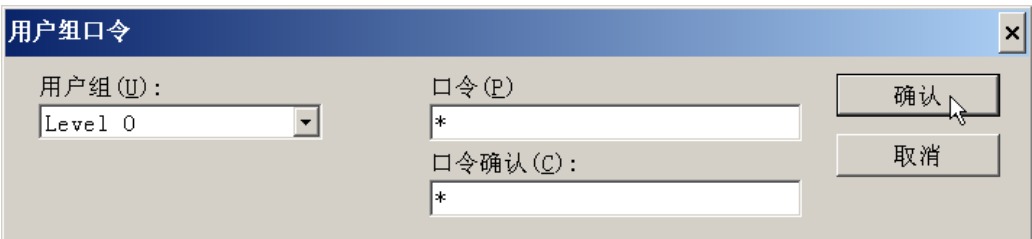


图 9-1-19 设定用户组权限口令

表 9-1-1 用户组与口令

用户组	口令
Level 0	0
Level 1	1
Level 2	2
Level 3	3

分别设置好不同用户组的口令后，在下次打开程序时，会提示要求输入用户组口令。对于不同用户组，因其设置的口令不同，可以不同的身份进入程序，而且享有不同的操作权限。现以“Level0”的用户组进入，输入口令如图 9-1-20 所示。



图 9-1-20 输入用户组权限口令

如果输入的口令与用户组 0 所设置的口令不同时，会出现如图 9-1-21 所示的提示。



图 9-1-21 输入用户组权限口令错误

如果输入的口令与用户组“Level0”所设置的口令一致，则会以用户组“Level0”的身份进入。对于用户组 Level4、Level5、Level6、Level7 和 Level8 的成员，由于无访问权限，则会弹出如图 9-1-22 所示的对话框。

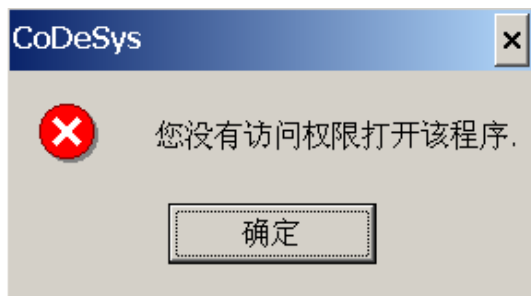


图 9-1-22 用户组无访问权限

9.2 工程选项设置

“工程选项”有两种打开方式。一种是点击“工具”下拉菜单中的“选项”，另一种是双击“资源”选项卡中的“工程选项”，如图 9-2-1 所示。

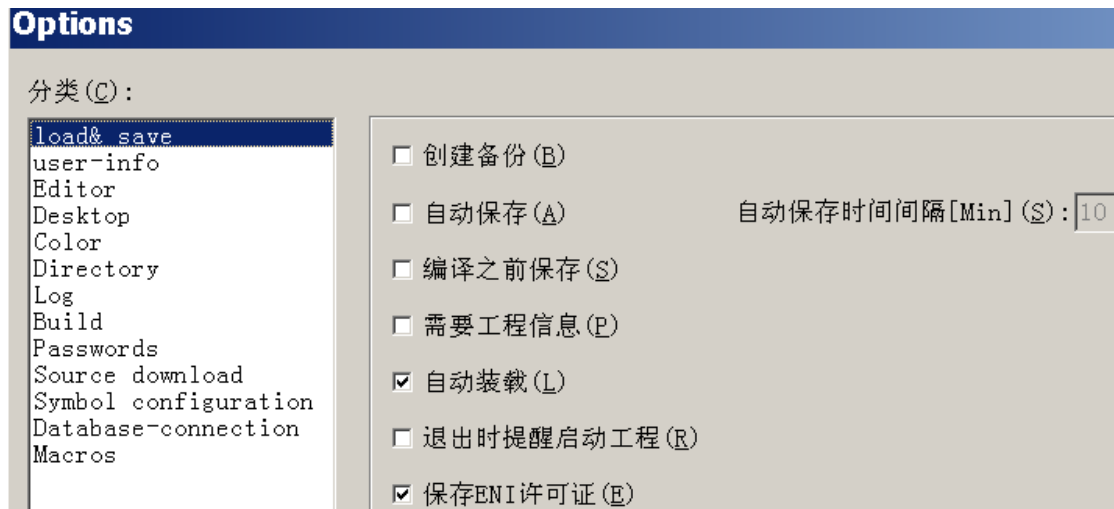


图 9-2-1 工程选项设置

打开“工程选项”设置对话框，该选项被分为几大类。每个选项对应右边的不同设置。这些选项包括：

- 下装与保存 Load&Save
- 用户信息 User Infomation
- 编辑器 Editor
- 窗口 Desktop
- 颜色 Color
- 目录 Directories
- 日志 Log
- 编译 Build
- 口令 Passwords
- 源代码下装 Source download
- 符号配置 Symbol configuration
- 数据库连接 Database-connection
- 宏 Macro

在上述选项中，PLC 不支持“源代码下装 Source download”、“符号配置 Symbol configuration”、“数据库连接 Database-connection”和“宏 Macro”，这里不作介绍。

下面分别对“下装与保存 Load&Save”、“用户信息 User Infomation”、“编辑器 Editor”、“窗口 Desktop”、“颜色 Color”、“目录 Directories”、“日志 Log”、“编译 build”和“口令 passwords”等进行介绍，其余均采用默认值。

9.2.1 下装与保存

在“工程选项”下，选择“Load&Save”选项，在窗口右侧显示其设置选项，如图 9-2-1 所示。下装与保存“Load&Save”的具体选项设置可以分为如下几类情况。

- 选中“创建备份”，系统会在保存工程的时候以相同路径建立一个扩展名为“.bak”的备份文件。通过这种方式可以保存最后一次保存之前的工程文件。
- 选中“自动保存”，那么工作时，工程会经常不断按照设定的“自动保存时间间隔”，以扩展名“.asd”保存临时文件。该文件会在程序正常退出时被删除，系统默认的保存路径为 D:\Hollysys\PowerPro\Projects。如果系统非正常退出（例如电源故障），文件就会被保留下来。当再次打开程序时，出现如图 9-2-2 所示的消息框，可以选择是打开自动保存文件还是打开原始文件。

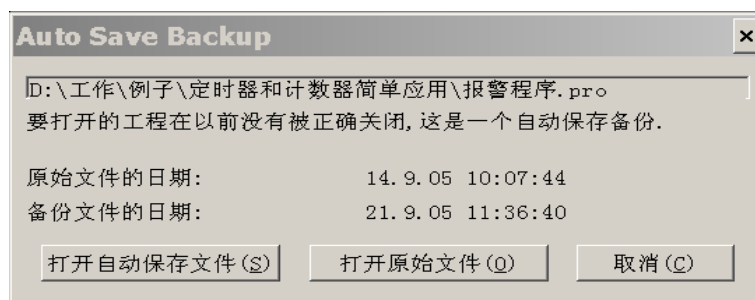


图 9-2-2 自动恢复提示

- 选中“编译前保存”，每次执行编译命令前，自动保存“*.asd”临时文件。
- 选中“需要工程信息”，那么当保存一个新工程或用新名字另存成一个工程时，工程信息对话框会自动被调用，相当于自动执行“工程”/“项目信息”命令。项目信息包含工程的标题、目录、作者和版本等信息，作为工程文件的一部分，可打印。
- 选中“自动装载”，软件在运行时会自动载入上一次最后打开的工程。
- 选中“退出时提醒启动工程”，如果工程已经改变和下载，但是自从上次下载启动工程之后没有创建新的启动工程，那么在用户退出前将会弹出如图 9-2-3 所示的对话框：“自上次下载后，没有创建启动工程，是否退出？”。这里对启动工程加以定义。所谓“启动工程”，即保存在 PLC 的 flash 中，上电后运行的用户程序。
- 选中“保存 ENI 许可证”，则会保存 ENI 许可证。

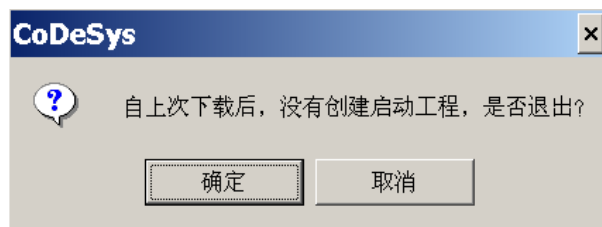


图 9-2-3 “退出时提醒启动工程”对话框

9.2.2 用户信息

在“工程选项”下，选择“User Infomation”选项，在窗口右侧显示其设置选项。用户信息的具体内容设置如图 9-2-4 所示。在相应的用户名、姓名缩写和公司等空格处填入相应的用户信息内容。



图 9-2-4 用户信息

9.2.3 编辑器

在“工程选项”下，选择“Editor”选项，在窗口右侧显示其设置选项。编辑器的设定包括自动声明、自动格式化、表元素、声明为表、标记、位值等选项，如图 9-2-5 所示。

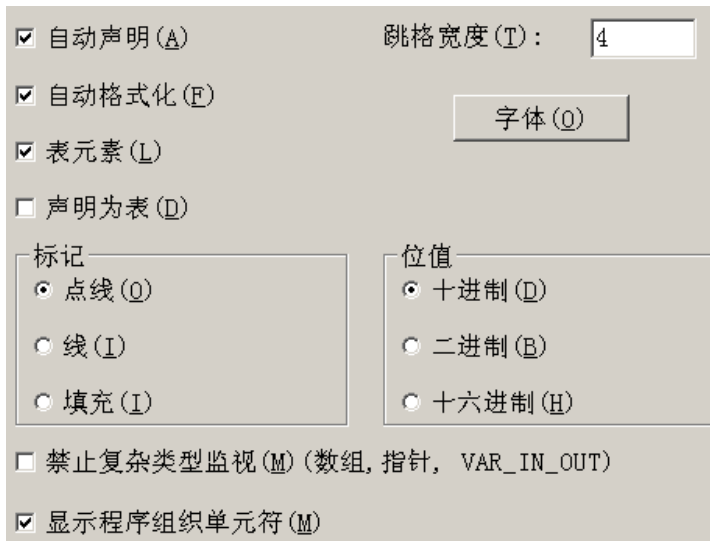


图 9-2-5 编辑器选项

- 选中“自动声明”，那么在用户编辑的程序中一旦出现了没有定义的新变量，系统会自动弹出对此变量定义的窗口，提示定义变量。
- 选中“自动格式化”，系统在编辑器中将会执行自动格式，即当编辑完成一行后，这一行会自动进行格式转化，将小写的操作数用大写表示，插入空格使各列均匀分开。

- 选中“声明为表”，声明编辑器将以表格的形式出现，如图 9-2-6 所示。用户可以采用填表的方式定义变量。这张表是卡片索引形式的对话框，共有六张变量卡和一张信息卡。变量卡分别是 VAR 本地、VAR_INPUT 输入、VAR_OUTPUT 输出、VAR_IN_OUTPUT 输入/输出、CONSTANT 常数和 RETAIN 保留型变量等。对于每一个变量，需要设置变量的名称、地址、类型、初始值和注释等字段。INFO 信息卡自动显示 POU 类型及名称。

	VAR	VAR_INPUT	VAR_OUTPUT	VAR_IN_OUT	CONSTANT	RETAIN	INFO
	名称	地址	类型	初始值	注释		
0001	T1		TON				
0002	ET		TIME				

图 9-2-6 表格声明

- “跳格宽度”设定在编辑器中空格的宽度，缺省的设置是 4 个字符宽。字符的宽度取决于选择的字体。
- “字体”按钮设定工作区域中所有窗口的字体。大字体放大了打印输出的大小，对每一种编辑器都适用。
- “标记”设定在图形编辑器中当前选择是由虚线矩形表示，还是由实线矩形表示，或是由填充矩形表示。
- “位值”设定在线模式下变量（类型 BYTE、WORD 和 DWORD 等）显示的格式是十进制[D]数，还是二进制[B]数，或是十六进制[H]数。

◇ 举例

十进制[D]	二进制[B]	十六进制[H]
a=53	a=2#0000 0000 0011 0101	a=16#0035
b=57	b=2#0000 0000 0011 1001	b=16#0039

- 选中“禁止复杂类型监视（数组，指针，VAR_IN_OUT）”选项，则在线模式下，复杂的数据类型不会出现在监视窗口，例如数组和指针类型等。
- 选中“显示程序组织单元符”选项，则在程序编辑过程中，如果调用相应的功能块，会自动添加“PowerPro/Library”路径下的 bmp 文件，丰富功能块内容，增加软件的友好性。

9.2.4 窗口

在“工程选项”下，选择“Desktop”选项，在窗口右侧显示其设置选项，如图 9-2-7 所示。

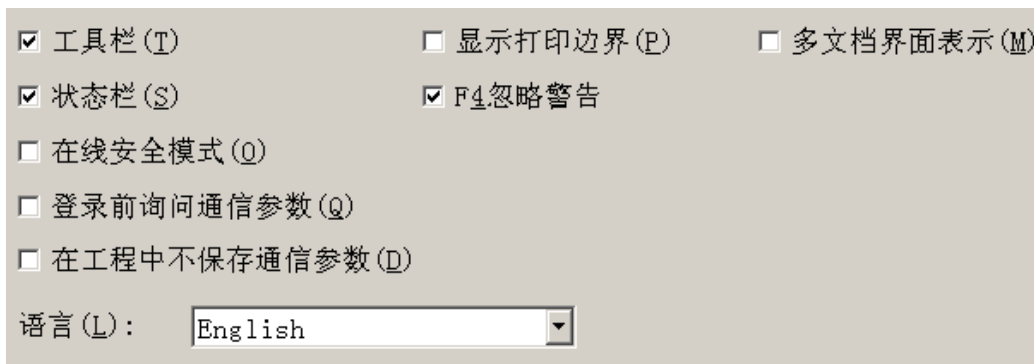


图 9-2-7 窗口选项

- 选中“工具栏”，在主窗口中出现工具栏，显示菜单的常用命令快捷按钮。
- 选中“状态栏”，在主窗口的最下部出现状态栏。
- 选中“在线安全模式”，当执行在线命令时，例如“运行”、“停止”、“复位”或“断点”等在线命令，会弹出确认对话框，再次确认操作。这样可以进一步提高安全性，避免误操作。
- 选中“登录前询问通信参数”，则会在登录时提示通讯参数对话框，方便校准通讯参数，例如参数是否设置及其设置的正确性。这就无须再打开“在线”菜单来设置“通讯参数”。否则，则不具备此项功能。
- 在“语言”选项中，只能选择菜单和对话框的显示语言，默认为 English。
- 选中“显示打印边界”，在每一个编辑窗口，将每页打印纸可打印区域用红色虚线框出。这个区域的大小取决于打印机属性（纸张、布局）和“Content”区域的大小（文件/打印机设置），如图 9-2-8 所示。
- 选中“F4 忽略警告”，当程序编译后，在消息窗口使用快捷键 F4 快速跳转时，会略过出现警告（warning）的行，只跳到出现错误（error）的行。

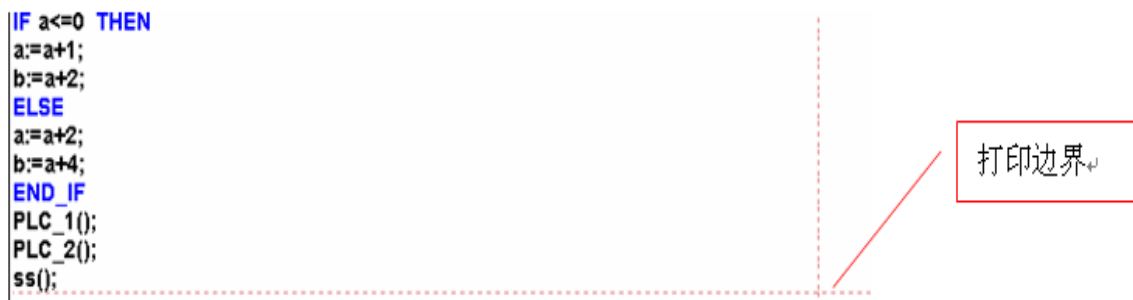


图 9-2-8 可打印区域显示

9.2.5 颜色

在“工程选项”下，选择“Colors”选项，在窗口右侧显示其设置选项，如图 9-2-9 所示。

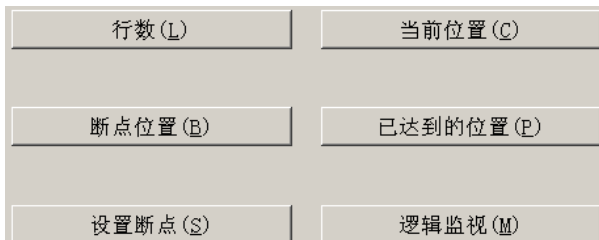


图 9-2-9 颜色选项

根据用户的需求和习惯，可对系统的某些特定显示颜色进行重新设置。采用颜色标识，主要针对在线调试的情况，可以直观方便地观察程序当前运行的情况。点击对应选项按钮，打开颜色对话框，按照特定需求设定相应的颜色。一般情况采用默认值。

- “行数”：缺省浅灰色，程序编辑器中网络号或行号的背景颜色。
- “当前位置”：缺省红色，在线运行时，遇到断点停止的网络号或行号的背景颜色。
- “断点位置”：缺省深灰色，可设置断点的网络号或行号的背景颜色。
- “已到达的位置”：缺省绿色，显示流控制时，已执行的网络号或行号的背景颜色。
- “设置断点”：缺省浅蓝色，设置了断点的网络号或行号的背景颜色。
- “逻辑监视”：缺省深蓝色，在线模式下，数字逻辑 TRUE 的颜色。

9.2.6 目录

在“工程选项”下，选择“Directories”选项，在窗口右侧显示其设置选项，如图 9-2-10 所示。在“工程”栏内可以设置工程的库、编译文件、配置文件和文件视图等所在的目录。在“目标”栏内显示“配置文件”所在目录，对于“库”默认为无路径，此项是系统自动生成的，不可修改。PowerPro 软件安装时自动生成的文件目录会显示在“公用”栏内的库、编译文件、上传文件、配置文件和文件视图等的目录中。

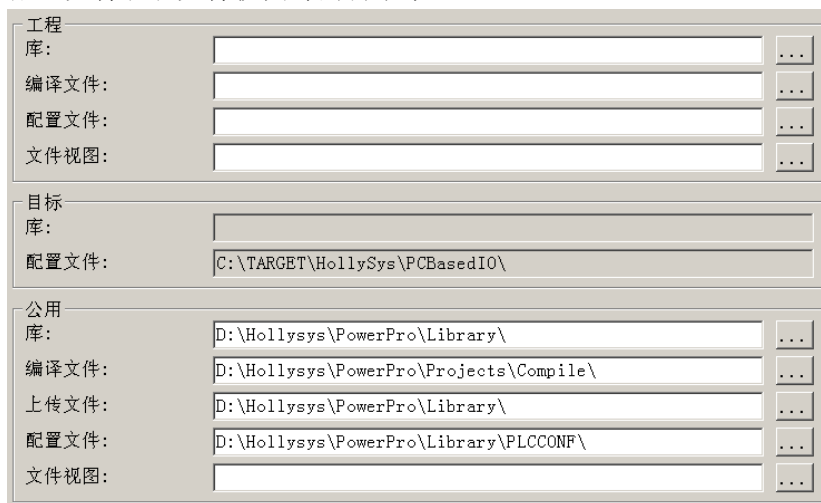


图 9-2-10 目录

9.2.7 日志

所谓“日志”表示工程按一定的年月日顺序记录用户所作的一系列动作。其中记录内容包括登录、运行、初始化、输入值、退出、清除缓存区和无法登录等。在“工程选项”下，选择“Log”选项，在窗口右侧显示其设置选项，如图 9-2-11 所示。



图 9-2-11 日志选项

- 选中“工程日志目录”，可以修改工程日志的保存路径，其默认的路径为 D:\Hollysys\PowerPro。
- “最大工程日志”设定日志窗口中最多显示的在线会话的个数。
- 选中“激活日志”，在工程中启用日志记录功能，显示日志列表，如图 9-2-12 所示。在日志中分别记录了登录、运行、初始化、输入值、退出和清除缓存区等一系列用户动作、状态变化或内部动作。

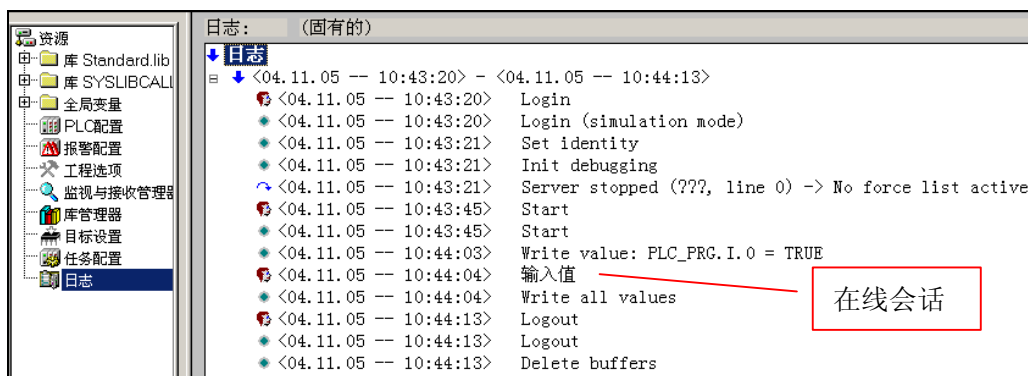


图 9-2-12 日志列表

- 在“选择”栏中，选定每个在线会话所记录的内容，包括用户动作、状态变化、内部动作和例外等。
- “日志”窗口在“离线”和“在线”两种模式下均可使用。图 9-2-12 所示为一个在离线仿真模式下产生的日志列表。

9.2.8 编译

在“工程选项”下，选择“Build”选项，在窗口右侧显示其设置选项，如图 9-2-13 所示。

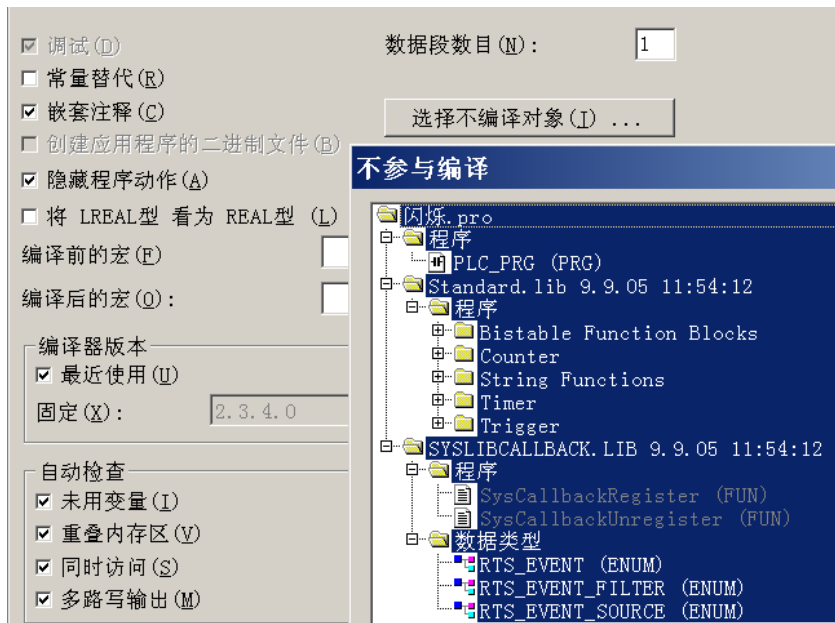


图 9-2-13 编译

- “调试”：当此项被选中时，附加的调试代码会增加代码段长度。当此项未被选中时，会加快工程的速度，减少代码段长度。另外，只有选中该项，登录后才可以运行“断点”和“单步”等调试功能。
- “常量代替”：每个常量的值是直接下装的，在“在线模式”下，常量的值为绿色，强制值、输入值和监视常量不再有效。如果此选项无效，则该常量值会通过变量访问下装到存储区。
- “嵌套注释”：允许使用嵌套的注释语句。

◇ 举例：嵌套注释

```
(*  
a:=inst.out; (*检查*)  
b:=b+1;  
*)
```

- “数据段数目”：用于设置编译的数据段的个数。
- “编辑器版本”：如果选中“最近使用”，会给出最近使用的版本号。
- “自动检查”：

选中“未用变量”，在编译时会自动检查未使用的变量。

选中“重叠内存区”，在编译时会自动检查重叠内存区。

选中“同时访问”，在编译时会自动检查同时访问。

选中“多路写输出”，在编译时会自动检查多路写输出。

“自动检查”与“工程”菜单栏里的“查看”具有同样的功能。唯一不同的是，“工程/查看”只有在编译后，可以通过“查看”选择要查看的内容，例如未用变量等，而且只能一一查看。而自动检查则可以选择多项，在编译时一起实现自动检查。

图 9-2-14 所示为当上述四项均选中，在编译时，消息窗口自动生成的检查情况。

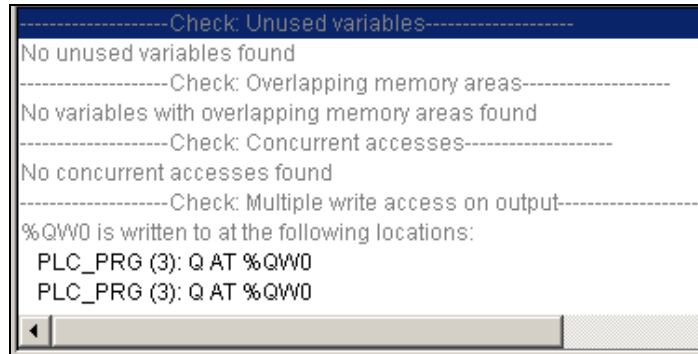


图 9-2-14 编译消息

9.2.9 口令

如果程序需要加密，则需要对该程序设置相应的口令。

在“工程选项”下，选择“Passwords”选项，在窗口右侧显示其设置选项，即可设置相应的口令。口令代码完全根据个人需求自行设置。如果保密性强，建议多设置几位口令代码。在图 9-2-15 所示的例子中，设置了三为代码的口令。下面介绍其使用方法。

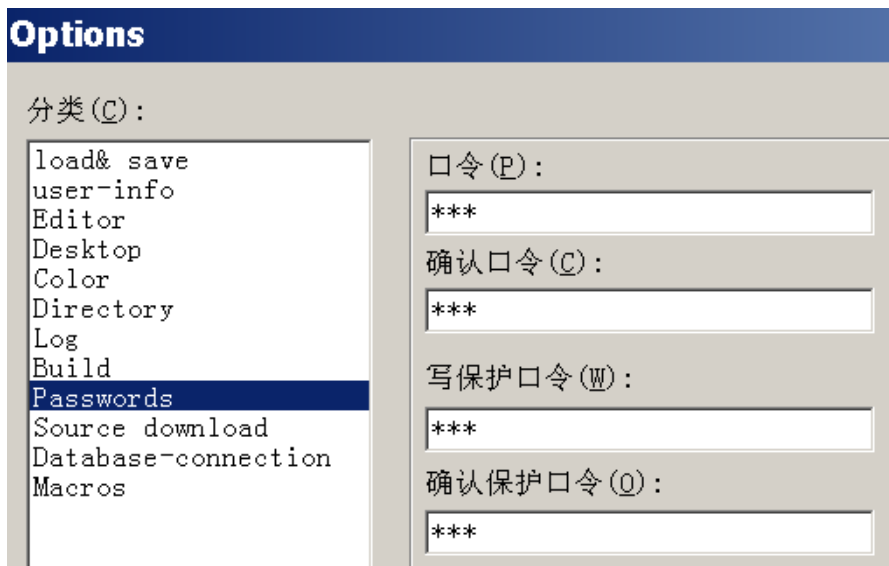


图 9-2-15 口令设置

在对程序进行口令设置且保存后，在下次打开该程序时，系统会提示要求输入口令。输入相应的口令，点击“确认”，如图 9-2-16 所示。



图 9-2-16 输入口令

点击“确认”后，窗口会弹出如图 9-2-17 所示的对话框，要求输入写保护口令。如果所输入的口令正确，则程序被打开，而且可以进行编辑。

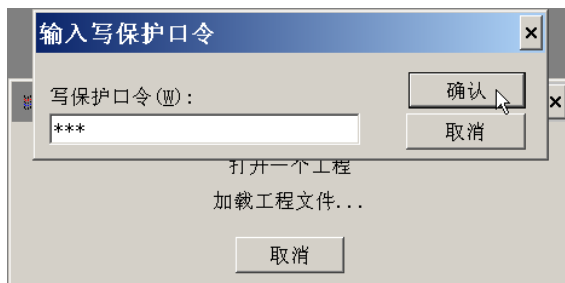


图 9-2-17 输入写保护口令

对于已经加密的程序，如果现在不需要加密，则只需打开“工程选项”\“Passwords”，将以前设置的所有密码删除并存盘。当再次打开该程序时，密码保护功能失效。



本章小结

本章主要介绍了“工程”菜单下各选项的使用。在通常情况下，并不需要修改各选项，选择其默认值即可。如果有特殊要求，则需要修改相应的选项。

第10章 编译与调试

当程序编写完毕后，要对程序进行编译。当编译通过后，才能将程序下装到 PLC 中。本章主要对 PowerPro 软件的编译与下装过程进行介绍。

10.1 编译

PowerPro 软件的“工程”菜单提供了“编译”和“全部编译”两种编译命令，用于检查程序有无语法错误，如图 10-1-1 所示。

- 编译：仅对程序的变化部分进行编译，并更新到原有的目标文件中。
- 全部编译：与“编译”命令不同，可以完全重新编译整个工程。
- 清空：清除上次编译和下载的信息。
- 载入在线修改信息：PLC 不支持此项功能。

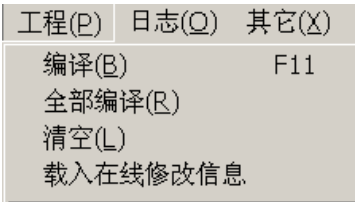


图 10-1-1 工程菜单

没有语法错误的程序才能生成可执行的目标文件。编译的结果会显示在消息窗口中，如图 10-1-2 所示。

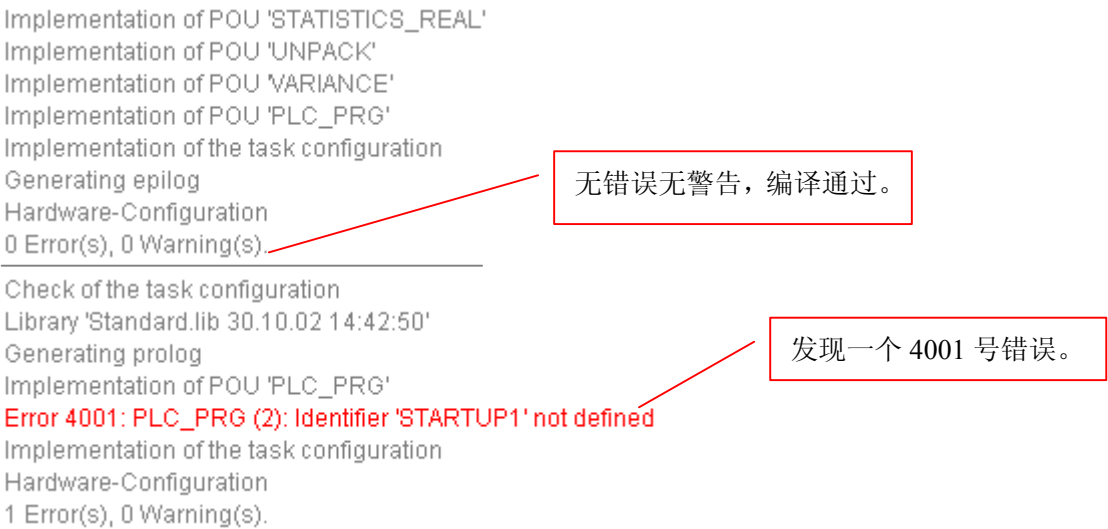


图 10-1-2 编译信息显示

10.2 下装

10.2.1 设备安装与连接

➤ 设备安装

首先根据实际工程的需要，选择合适的 CPU 模块和扩展模块。然后根据现场情况确定模块的安装方式，并初步确定 PLC 的工作方式。最后规划并制定合理的接线方案，将现场的传感器或执行器连接到 PLC 模块的接线端子上。

➤ 连接电缆

根据所选 CPU 模块的型号和类型，连接电源线，如图 10-2-1 所示。电源线接好之后，先不要接通电源。在检查所有电缆连接无误后，再接通系统电源，并确认 CPU 模块面板上的 RUN 指示灯点亮，并显示正常，以保证 PLC 可靠运行。注意，当电源线连接好之后，应该把端子盖扣好，以免造成不必要的人身伤害或设备损坏。

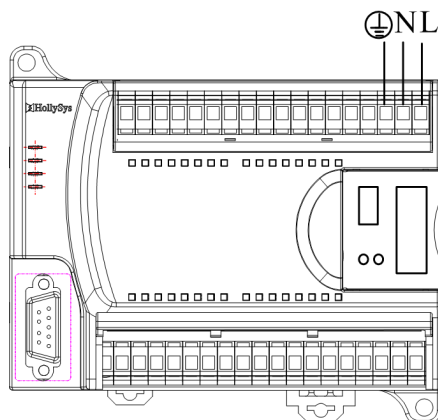


图 10-2-1 连接电源线

➤ 建立 PC 通讯

通过配套的编程电缆，将 CPU 模块连接到个人计算机（PC）的 RS232 串行通信接口，建立数据传递通道，如图 10-2-2 所示。由于 CPU 模块的 RS232 串行通讯接口是非隔离的，所以编程电缆的连接应该在 PLC 上电之前进行。

注意，CPU 模块 LM3108 和 LM3109 有 2 个串口，通过左边的 Port1 串口将程序下装到 PLC 中。

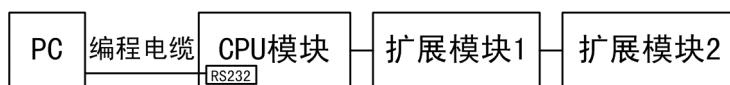


图 10-2-2 连接编程电缆

10.2.2 建立通信连接

将编译生成的目标文件下装到 CPU 模块中去，需要配置并选定通讯线路，建立本地计算机与目标模块之间的通信连接。实现步骤如下所述。点击“在线”菜单中的“通讯参数”，弹出“Communication Parameters”通讯参数对话框，如图 10-2-3 所示。

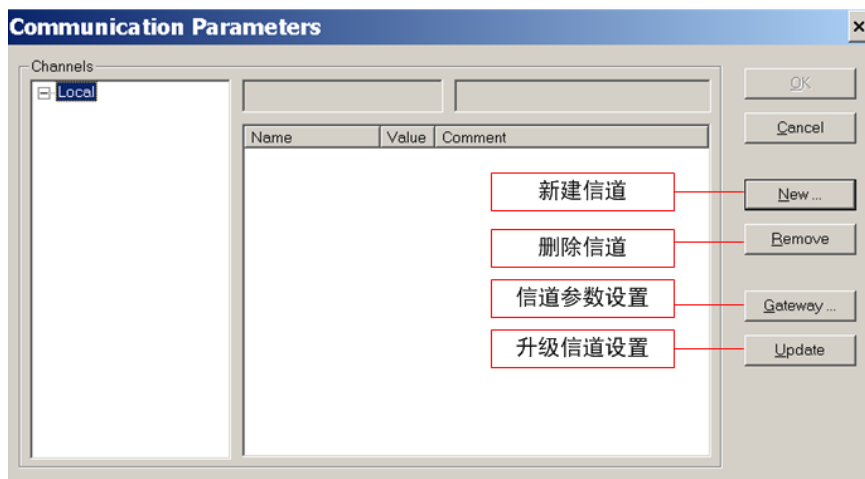


图 10-2-3 通讯参数对话框

点击“Gateway”进入信道参数设置，确认“Connection”通道参数设置为“Local”，点击“OK”按钮，如图 10-2-4 所示。

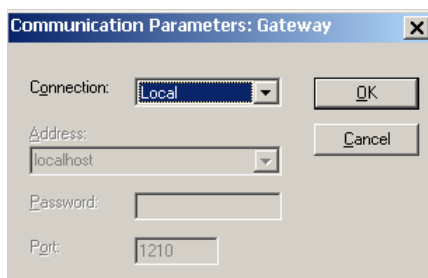


图 10-2-4 通道参数设置

确认后返回“Communication Parameters”通讯参数对话框，点击“New”按钮添加新信道，出现如图 10-2-5 所示的对话框。其中，信道名称默认为“Local_”，通信协议使用缺省的 RS232 协议，点击“OK”按钮返回“Communication Parameters”通讯参数对话框。

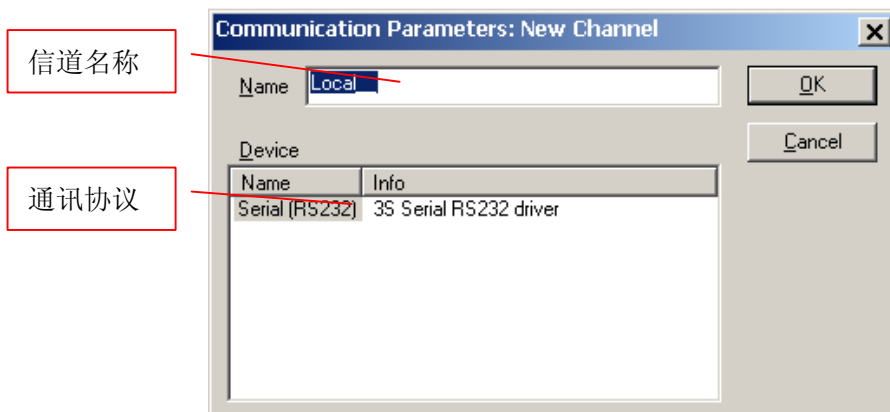


图 10-2-5 添加新信道

改变通讯速率。连续双击“Baudrate”中“Value”的对应值，使其变为“38400”，如图 10-2-6 所示，点击“OK”按钮确认。这样就建立了本地计算机与 PLC 的 CPU 模块之间的通信连接。

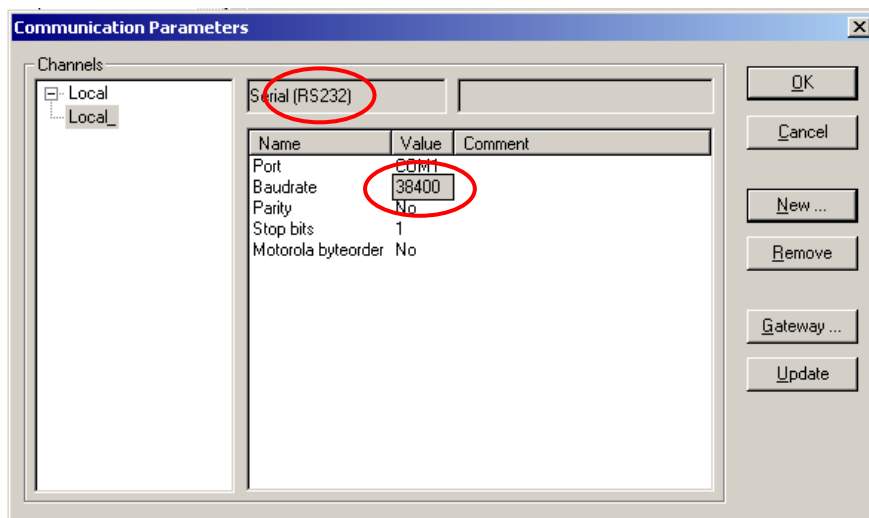


图 10-2-6 通讯速率设置

10.2.3 程序下装

➤ 下装程序并创建启动工程

编译通过的目标文件在进行下装时，会将全部的目标文件下装到模块中。同时将模块复位，所有变量返回到初始状态。在“在线”菜单中选定“登录”，建立本地计算机与 CPU 模块的连接，并出现系统提示下装信息，如图 10-2-7 所示。

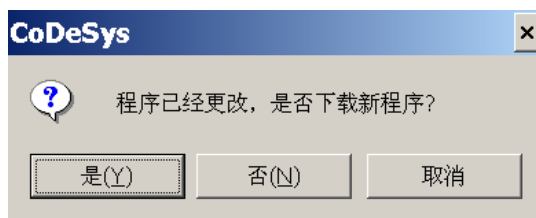


图 10-2-7 下装提示信息

如果程序已经更改，选择“是”按钮，将新程序下装到 CPU 模块中去。当出现如图 10-2-8 所示的创建启动工程提示信息时，点击“是”按钮。确保 PLC 断电后再上电运行此下装工程。下装结束。



图 10-2-8 创建启动工程提示信息

➤ 不下装而进入在线监控状态

如果希望不进行下装而直接进入在线监控状态，可以在系统提示下装信息时选择“否”，此时对工程的修改不会被下装到模块中，但可以监控模块中原有程序的运行情况。注意，如果在线监控之后仍然需要下装程序，可以选择“在线”下拉菜单中的“下载”来实现下装。

➤ 创建启动工程

程序调试完成后,选择“在线”/“创建启动工程”命令,同样弹出创建启动工程提示信息,如图 10-2-8 所示。

点击“是”按钮,停止当前程序的运行,在模块中创建启动工程。

点击“否”按钮,不停止当前程序的运行,同时在模块中创建启动工程。

点击“取消”按钮,撤消该命令。

如果已经创建了启动工程,系统会自动生成*.BIN 文件和*.HEX 文件。当再次创建启动工程时,系统会提示该文件已被创建,如图 10-2-9 所示。



图 10-2-9 提示信息

“创建启动工程”与“登录”的区别在于,“创建启动工程”命令使工程永久地保存在 CPU 模块中,“登录”命令仅将工程下装到模块中。当模块断电后,在“登录”命令操作下的工程不会被保存下来,而“创建启动工程”命令将工程复制到模块中,当模块重新上电后,模块会自动调出工程。

注意,采用串口参数设置和恢复模块进行串口参数设置的程序,在下载用户程序后,不要直接在 PowerPro 软件下运行该用户程序,应当选择“在线/退出”,退出登录状态,将 RUN/STOP 选择开关拨至 STOP 后,再拨至 RUN,使用户程序运行。否则,将会导致串口设备的复位,又恢复到默认的通讯参数。

10.3 显示参考数据

PowerPro 软件的“工程”菜单提供显示一些参考数据的命令,这些命令只有在编译通过后才有效。

10.3.1 查看实例

“工程”/“查看实例”命令可以打开和显示“程序”选项卡中当前功能块的实例列表,如图 10-3-1 所示。双击功能块也可以打开该列表。该命令只有在线模式下才有效,即工程编译成功通过,“在线”/“登录”下装到模块中。



图 10-3-1 实例列表

10.3.2 查看调用树

“工程”/“查看调用树”命令可以在一个新窗口中显示当前对象调用程序、函数和功能块的树型结构，直观地指出当前 POU 与工程中其它 POU 的先后调用关系，如图 10-3-2 所示。工程必须通过编译，该命令才有效。

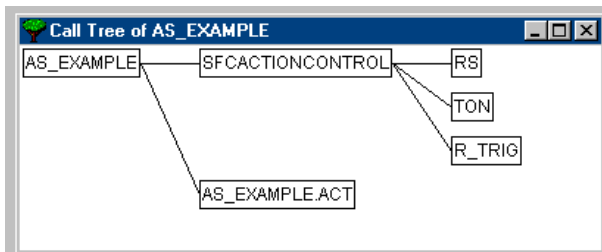


图 10-3-2 查看调用树

10.3.3 查看交叉引用列表

“工程”/“查看交叉引用列表”命令可以显示并查看所有应用程序点，如图 10-3-3 所示。所谓“应用程序点”是指某一个变量、地址或程序在全部工程中的位置。工程必须通过编译，该命令才有效。

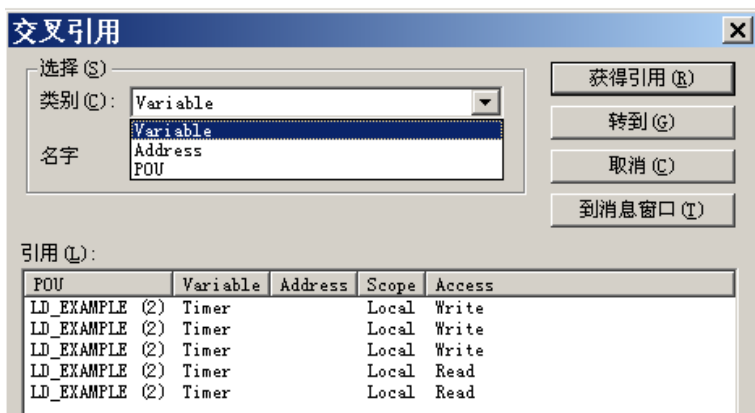


图 10-3-3 查看交叉引用列表

首先需要选择变量（Variable）、地址（Address）或程序（POU）的类别，然后输入名称，最后点击“获得引用”按钮，在“引用”栏中就得到相应的应用程序点列表，列出该点的信息。例如，是只读还是可写，是局部变量还是全局变量，变量是否被连到硬件地址上等信息。

选中交叉参考列表中的一行，按“转到”按钮，或者直接双击此行，程序会在编辑器中显示出相应的点。通过这种方式，可以任意跳到所需的应用程序点处，而无须进行费时的搜索。还可以使用“到消息窗口”按钮，使交叉参考列表直接显示在消息窗口中。鼠标双击跳到相应的程序位置，搜索起来更加方便。

10.3.4 查看

“查看”菜单如图 10-3-4 所示。

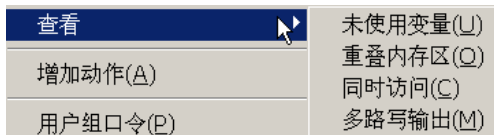


图 10-3-4 查看菜单

➤ “查看”/“未使用变量”

编写算法时，常常会删除某个变量，或重新命名某个变量，此时原变量的声明不会被自动删除，仍保留在声明编辑窗口中。这样，工程里就可能存在只有声明却从未被使用过的变量，占用内存空间，也不利于变量的管理。

“工程”/“查看”/“未使用变量”命令用来检查工程中是否存在只有声明却没有被使用的变量。工程必须通过编译，该命令才有效。检查结束后，消息窗口出现所有未使用变量的列表，如图 10-3-5 所示。

```
-----Check: Unused variables-----
PLC_PRG (3): a
PLC配置 (0): I
PLC配置 (0): StartUp
PLC配置 (0): Q
```

图 10-3-5 查看未使用变量

➤ “查看”/“重叠内存区”

选择此项，工程自动查看重叠内存区。如果没有重叠内存区，则会在消息窗口显示如下提示：No variables with overlapping memory area found（没有重叠内存区）。

➤ “查看”/“同时访问”

选择此项，工程自动查看有无同时访问，如果没有同时访问，则会在消息窗口显示如下提示：No concurrent accesses found（没有同时访问情况）。

➤ “查看”/“多路写输出”

选择此项，工程自动查看多路写输出情况，如果没有检查多路写输出情况，则会在消息窗口显示如下提示：No outputs found which are written to at more than one location（没有多路写输出）。

10.4 调试

系统所支持的调试命令都在“在线”菜单下，并且在调试状态下可以使用，如图 10-4-1 所示。在调试状态下，系统用不同的默认颜色来代表不同的状态和操作，例如逻辑真（蓝）、逻辑假（黑）、断点（浅蓝）、流控制（绿色）等等，这些颜色可以在“工程”/“选项”/“Color”中设定。了解各种颜色的含义，有利于程序的调试和监视。

在线(O)	窗口(W)	帮助(H)
登录(I)		Alt+F8
退出(X)		Ctrl+F8
下载(D)		
运行(R)		F5
停止(P)		Shift+F8
复位(E)		
冷复位(I)		
清空用户程序(Q)		
断点(B)		F9
断点对话框(L)		
跳出(S)		F10
跳进(N)		F8
单循环(Y)		Ctrl+F5
输入值(W)		Ctrl+F7
强制值(C)		F7
解除强制(A)		Shift+F7
输入/强制对话框(G)		Ctrl+Shift+F7
查看调用栈(K)		
显示流控制(E)		
仿真模式(M)		
通讯参数(U)		
源代码下装(Q)		
创建启动工程(C)		
写文件到PLC(W)		
从PLC中读取文件(R)		

图 10-4-1 在线菜单

10.4.1 进入调试状态

执行“在线”/“登录”命令进入调试状态。将程序下装到 PLC 的 CPU 硬件模块中，称为在线调试状态。如果没有连接硬件模块，在本地计算机模拟运行用户程序，称为仿真模式。“在线”/“仿真模式”被选中（出现“√”），登录时便会进入仿真模式。

在程序中，如果有与 RTS 上硬件相关的功能块时，是不可以使用仿真模式的。例如，系统事件是不可以使用仿真模式的。与 RTS 上硬件相关的功能块包括定制库中的外部扩展功能块和外部功能块，均不可使用仿真模式。

另外，设定自由口参数后，如果想要恢复原编程系统的下装与调试功能，需要将 RUN/STOP 开关拨到 STOP 位置，才可以进行编程系统登录。

10.4.2 退出调试状态

执行“在线”/“退出”命令退出调试状态，进入编程状态。

10.4.3 运行程序

执行“在线”/“运行”命令，启动下装到模块中的程序，或者启动处于仿真模式下的程序。一般需要运行程序的情况有以下几种：

- “在线”/“登录”之后。
- 用户程序使用“停止”命令终止之后。
- 用户程序设置了断点之后。
- 执行一个“单循环”之后。

10.4.4 停止程序

执行“在线”/“停止”命令，暂停程序在模块或在仿真模式下的运行，保存当前变量值。此时程序仍处于调试状态下，使用“在线”/“运行”命令可重新启动程序，从上次停止的地方开始继续运行。

10.4.5 复位

如果已经明确地定义了变量的初始值，“复位”命令会把当前变量的值设置为初始值，保留型变量的值保持当前值。按“F5”键，程序会重新按照复位后的初始值运行。

- “冷复位”命令重置所有的变量为初始值，包括保留型变量。只有常量在“冷复位”后保持原来的值不变。按“F5”键，程序会重新按照冷复位后的初始值运行。
- “清空用户程序”命令重置所有的变量为初始值，包括保留型变量和常量，并删除模块中的用户程序，模块返回到初始状态。

以上命令都会出现对话框，需要进一步确认复位操作，如图 10-4-2 所示。

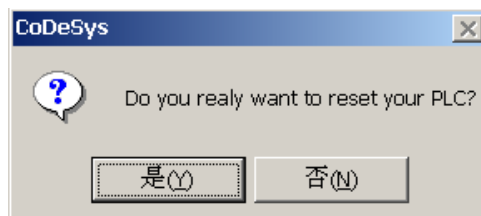


图 10-4-2 复位确认对话框

10.4.6 断点

断点是指可以在程序中设置进程停止的地方。通过设置断点，可以在程序的具体地点观察当前的变量值，便于分段调试程序。

➤ 设置断点

执行“在线”/“断点”命令，可在当前位置设置或删除一个断点。断点的设置位置取决于活动窗口中程序的语言类型。黑色背景的行号（网络号）表示该行可以设置断点，而灰色背景的行号（网络号）表示该行不可设置断点，如图 10-4-3 所示。

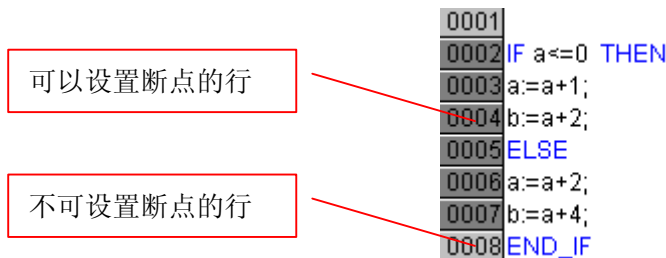


图 10-4-3 断点设置（1）

在 IL 和 ST 中，断点设置在当前光标所在的行。在 FBD 和 LD 中，断点设置在当前被选的网络中。在 SFC 中，断点设置在当前选定的步上。

可以直接用鼠标点击黑色背景的行号（网络号）设置或删除断点。如果该行被设置为断点，对应的行号（网络号）会呈现浅蓝色背景。

当程序运行到达所设置的断点处终止时，相应的行号（网络号）以红色背景来显示。使用“在线”/“运行”、“跳进”或“跳出”命令，可以继续运行程序。

例如，“登录”后在 0004 行设置一个断点，如图 10-4-4 所示。

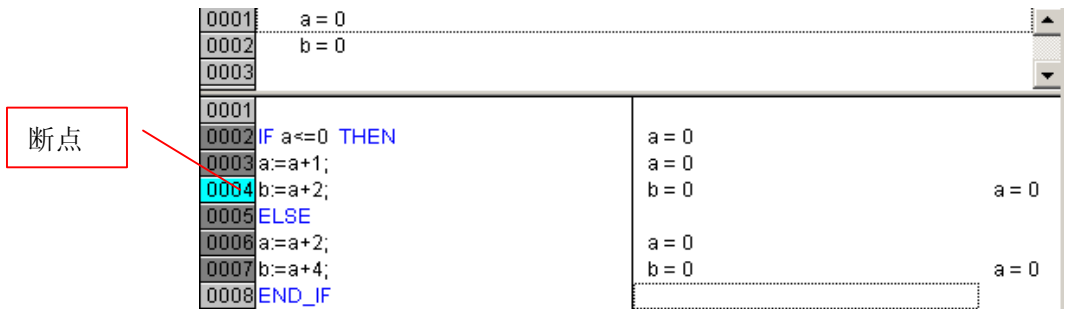


图 10-4-4 断点设置（2）

运行程序，执行到 0004 行断点处时，程序中断，变量 b 保持初始值不变，而上一行的变量 a 被重新赋值。如图 10-4-5 所示。

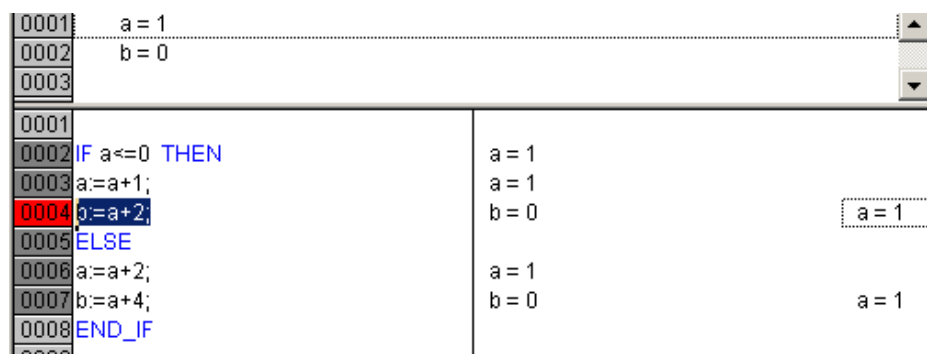


图 10-4-5 断点设置（3）

➤ 编辑断点

执行“在线”/“断点对话框”命令，弹出如图 10-4-6 所示的断点对话框，可以显示和编辑整个工程的断点。

设置断点：在“程序组织单元”项中选择一个需要设置断点的程序，在“位置”项中选择设置断点的行号（网络号），点击“添加”，则断点被添加到列表中。

删除断点：选中断点，点击“删除”，则删除该断点。

查看断点：选中断点，点击“转到”，则立即跳转到编辑器中断点设置的位置。

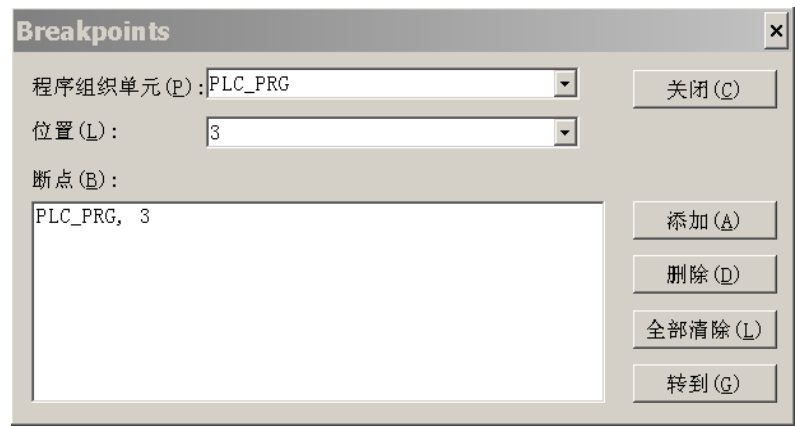


图10-4-6 设置断点对话框

当“在线”退出之后，所设置的断点被删除。注意，断点的个数应当限制在 100 以内，如果超出范围，则会出现如图 10-4-7 所示的提示。

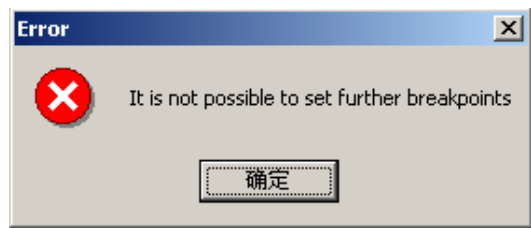


图10-4-7 断点设置（2）

10.4.7 单步

利用单步可以逐步检查程序逻辑的正确性。在对应的活动窗口中，对于不同的编程语言，单步的含义有所不同。

- 在 IL 中，执行程序直到下一个 CAL、LD 或者 JMP 命令。
- 在 ST 中，执行下一个指令。
- 在 LD 和 FBD 中，执行下一个节。
- 在 SFC 中，继续执行动作，直到下一步。

执行“在线”/“跳出”或“跳进”命令实现单步执行。当遇到断点时，程序执行当前语句后停止。当遇到功能块或函数时，“跳出”命令会跳过功能块或函数执行下一条语句，而“跳进”命令则跳入功能块或函数的内部单步执行。

10.4.8 单循环

执行“在线”/“单循环”命令使程序运行完一个周期后就停止运行。从本质上来说，“单循环”命令等同于“运行”命令，都可以使用户程序在线运行。只不过“单循环”命令让程序运行完一个周期后就自动停止，而“运行”命令让程序循环运行，直到执行“停止”命令为止。

10.4.9 变量输入值

- 设定新值

在调试状态下，用鼠标直接双击声明编辑器中的变量，弹出写变量对话框，如图 10-4-8 所示。在“新值”中填入变量的新值，点击“确认”按钮，当前变量的后面即出现淡蓝色文本标示的新值。对于布尔型变量，双击后直接在当前值的后面用淡蓝色文本标出 TRUE/FALSE 的状态切换，不使用写变量对话框输入。

- 激活新值

执行“在线”/“输入值”命令，或使用快捷键“Ctrl+F7”，变量的新值才能被激活，即输入到模块中。允许对在多个变量写入新值后执行“输入值”命令，将多个新值同时输入模块。

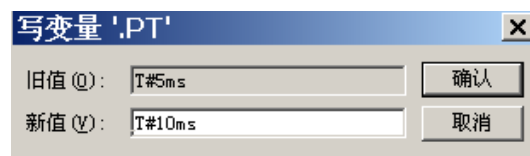


图10-4-8 写变量

10.4.10 变量强制值

- 强制值

强制写变量的方法与“输入值”命令一样，先输入新值，然后执行“在线”/“强制值”命令，或者使用快捷键“F7”写入强制值。

被强制的变量，在程序的每个循环之后都被写入强制值，直到执行“解除强制”命令后为止。对于“输入值”命令，变量只被写一次，而且允许变量被其它程序赋值。

当开关量的输入为物理点输入时，例如 I0.0，必须使用强制值，但在仿真模式下可以使用输入值。允许多个变量写入新值后执行“强制值”命令。

变量被强制时，其值用红色文本表示，如图 10-4-9 所示。

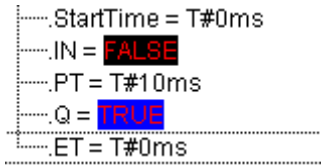
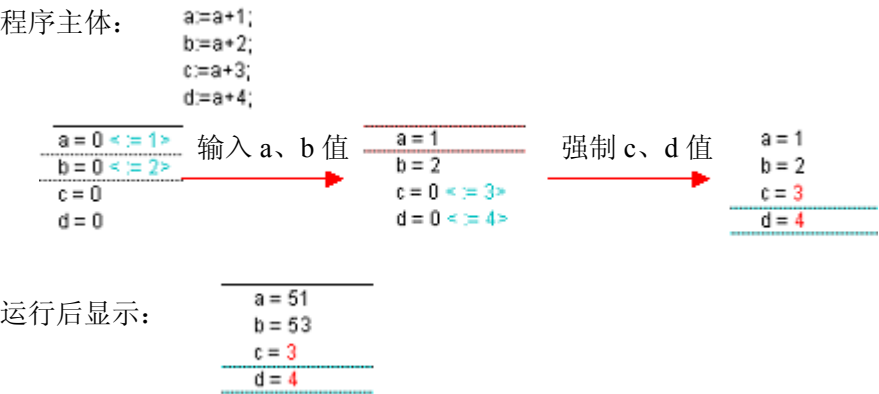


图10-4-9 强制值

通过下面这个例子，可以清楚地看出“输入值”和“强制值”的差别。



“输入值”命令只改变了变量 a、b 的当前值，程序一旦运行起来，变量 a、b 将会按照程序的设定自行累加，每执行一次，程序重新赋值一次。而“强制值”命令使变量 c、d 在每次程序执行时都被赋予强制值，所以一直保持 3、4 不变。

➤ 解除强制

执行“在线”/“解除强制”命令，终止对变量的强制赋值命令。强制赋值被解除后，变量值恢复黑色。

➤ 输入/强制对话框

执行“在线”/“输入/强制对话框”命令，则会弹出如图 10-4-10 所示的对话框，其中包括监视列表选项卡和强制列表选项卡等两个选项卡。

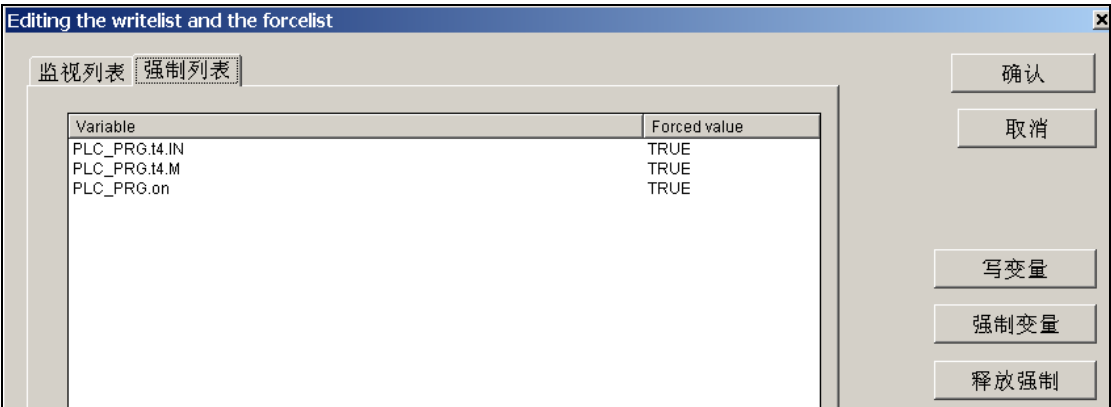


图10-4-10 输入/强制对话框

10.4.11 查看调用栈

执行“在线”/“查看调用栈”命令，则会在某个断点处，程序停止执行，显示调用堆栈里的当前程序的列表，如图 10-4-11 所示。

列表中第一个程序是系统默认的程序，或者是在“任务配置”中设定的第一个被调用的程序。列表中最后一个程序是当前被执行的程序。

选定了一个程序后，点击“转移到”按钮，则立即跳到该程序的执行窗口，显示正在被处理的行或段。

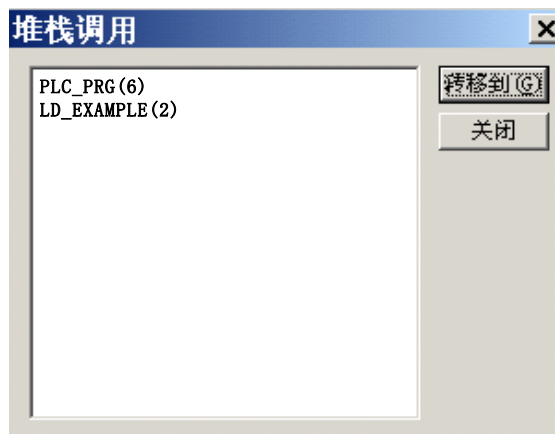


图10-4-11 堆栈调用

10.4.12 显示流控制

如果选择了流控制，“在线”/“显示流控制”菜单命令前会有一个“√”号出现。此后，在当前循环中，被执行的每一行或每一个网络都会被做上标记，运行的行和行号（或网络号）以绿色显示。在“单循环”模式下，用该命令可非常直观地看到程序的当前运行流程。

图 10-4-12 所示为一个简单的小程序，定义初值为 0 的整型变量 a。当 $a \leq 0$ 时累加 1，当 $a > 0$ 时累加 2。

0001	PROGRAM PLC_PRG
0002	VAR
0003	a: INT:=0;
0004	END_VAR
0005	
0002	IF a<=0 THEN
0003	a:=a+1;
0004	ELSE
0005	a:=a+2;
0006	END_IF

图10-4-12 程序示例（1）

“登录”后选中“显示流控制”命令，然后选择“单循环”方式。由于 a 的初值为 0，所以在第一次运行时，a 满足条件 $a \leq 0$ ，累加 1，当前 a 值变为 1。“显示流控制”命令将本次循环执行的行（2 行和 3 行）用绿色标示，如图 10-4-13 所示。

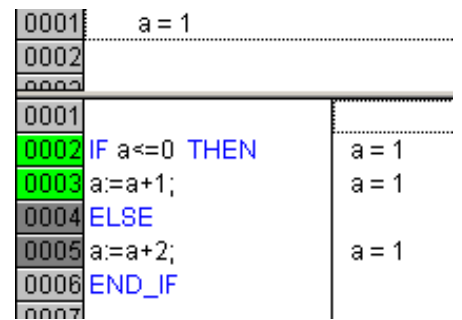


图10-4-13 程序示例（2）

再次选择“单循环”，此时由于 a 的值已经变为 1，满足程序中“ELSE”的条件，累加 2，当前值变为 3。“显示流控制”命令将本次循环执行的行（2 行和 5 行）用绿色标示，如图 10-4-14 所示。

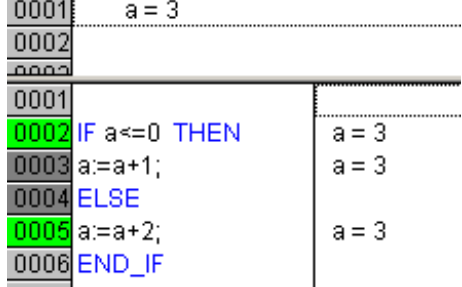


图10-4-14 程序示例（3）

10. 4. 13 监视与接收管理器

在对象组织器的“资源”选项卡中，可以打开监视与接收管理器的窗口，如图 10-4-15 所示。在调试过程中，可以在监视与接收管理器的窗口中集中监控工程中各程序的变量。监视与接收管理器可以将数值预置入某个变量，然后把它们作为一组发送到模块。同样，当前模块的值可以读入和存储在监视与接收管理器中。

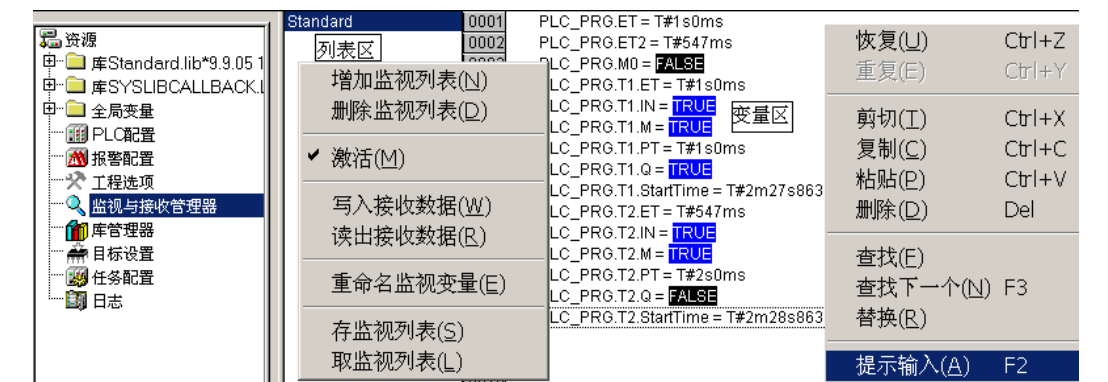


图10-4-15 监视与接收管理器

➤ 创建监视列表

在监视与接收管理器的列表区（左区）点击鼠标右键，从弹出的菜单中选择“增加监视列表”，并为列表输入合适的名称，点击“确认”。也可通过执行“插入”/“增加监视列表”命令创建监视列表。

➤ 修改监视列表名称

执行“重命名监视变量”，可以改变监视列表的名称。

➤ 选择监控变量

选择“输入提示”，或者利用编辑菜单，列出工程中已定义了的所有变量，用鼠标选择即可将变量加入到监视列表中。

➤ 保存监视列表

执行“存监视列表”，可以保存监视列表，扩展名为“*.wtc”。

➤ 导入监视列表

执行“取监视列表”，可以重新载入已经保存的监视列表。

➤ 激活监视列表

使“激活”之前出现符号“√”，激活监视列表，才可以监控列表中的变量。创建监视列表的各项功能仅在未激活前有效。

➤ 修改变量值

执行“写入接收数据”，可以将变量的值改写为设置的变量值。

➤ 读取变量值

执行“读出接收数据”，可以读取变量的当前值。



本章小结

本章介绍了程序的编译、下装和调试。这是编程必不可少的操作部分，应当重视。

第11章 视图

视图（Visualization）是 PowerPro 软件的一个组件，能够以图形的方式显示工程变量及其变化规律，用于实现控制过程的可视化。因此视图是 PLC 的人机界面（Human Machine Interface, HMI）。

PowerPro 软件的编程系统带有一个集成的视图编辑器。在开发控制系统应用程序的过程中，PowerPro 软件允许用户开发视图对象来观察和操作 PLC 的数据，而无需使用其它的开发工具。

11.1 创建视图文件

启动 PowerPro 软件，创建工程 project1.pro。在对象组织器中，点击“视图”选项卡，如图 11-1-1 所示。

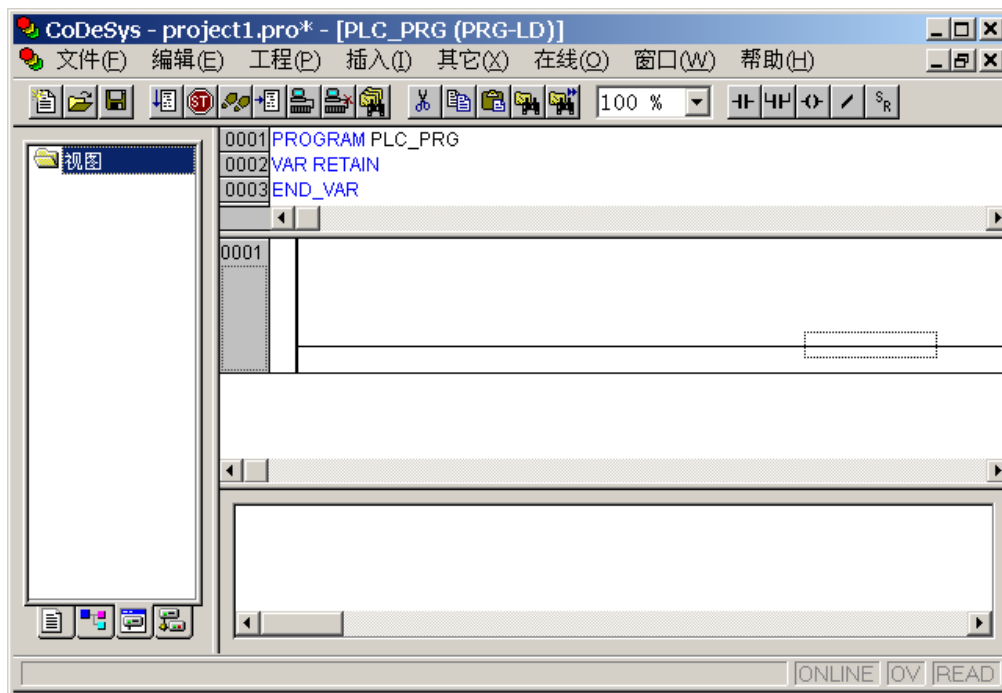


图 11-1-1 视图选项卡

在“视图”选项卡中点击鼠标右键，选择“添加”按钮，如图 11-1-2 所示。

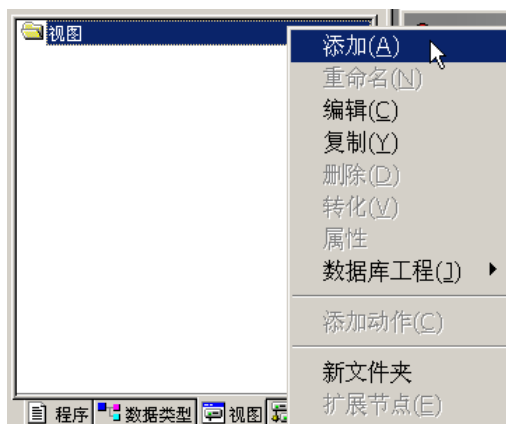


图 11-1-2 添加视图

弹出视图名称对话框，如图 11-1-3 所示。



图 11-1-3 视图名称

在“可视名”中输入视图的名称，例如 visu1，点击“确认”按钮，则在视图选项卡中创建了一个名为 visu1 的视图，右侧的工作区域为视图编辑区域，如图 11-1-4 所示。

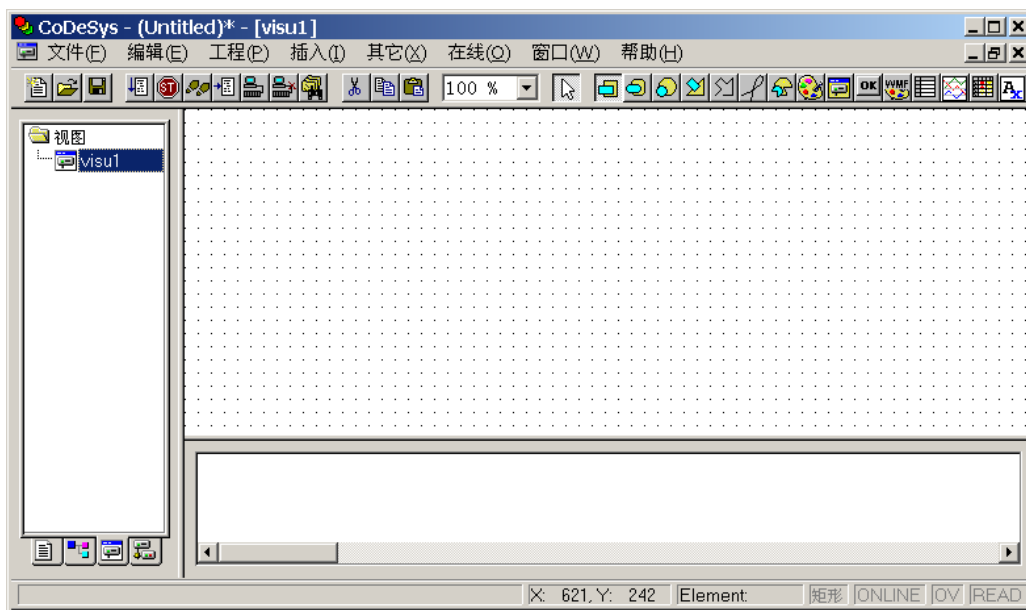


图 11-1-4 创建视图 visu1

11.2 视图编辑工具

编辑视图的工具位于视图编辑区域顶端的标题栏内，包括下拉菜单和快捷工具按钮。

点击标题栏的“插入”菜单，出现编辑视图的“插入”下拉菜单，如图 11-2-1 所示，可以选择不同的插入对象来添加所需的视图。这些视图对象是视图文件的基本组成单元。

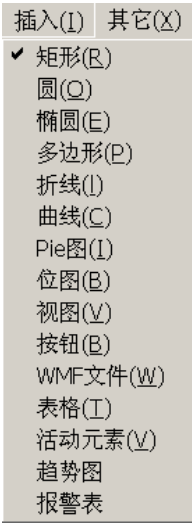


图 11-2-1 视图“插入”菜单

点击标题栏的“其它”菜单，出现编辑视图的“其它”下拉菜单，如图 11-2-2 所示，可以对视图对象的属性进行配置。在视图编辑区域点击鼠标右键也会弹出此菜单。

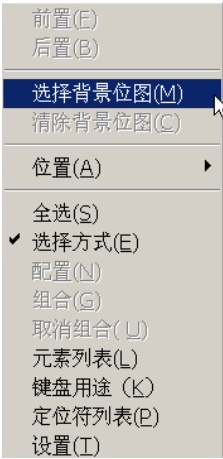


图 11-2-2 视图“其它”菜单

编辑视图的快捷工具按钮位于视图编辑区域顶端标题栏的工具栏中，如图 11-2-3 所示，与“插入”菜单完全相同。编辑视图的工具共有 15 种，其功能如表 11-2-1 所示。



图 11-2-3 视图快捷工具按钮菜单

表 11-2-1 编辑视图工具

名称	快捷工具	功能描述
矩形		在当前视图中插入矩形。
圆角矩形		在当前视图中插入圆角矩形。
椭圆		在当前视图中插入圆或椭圆。
多边形		在当前视图中插入多边形。
折线		在当前视图中插入直线或折线。
曲线		在当前视图中插入贝塞尔曲线。
Pie 图		在当前视图中插入饼图（圆或椭圆的楔形图）。
位图		在当前视图中插入位图文件。
视图		在当前视图中插入已经创建的其它视图。
按钮		在当前视图中插入按钮。
WHF 文件		在当前视图中插入 WMF 文件（Windows Metafile，即 Windows 图元文件格式）。
表		在当前视图中插入表格。
趋势图		在当前视图中插入趋势图。
报警表		在当前视图中插入报警表格。
ActiveX 控件		在当前视图中插入 ActiveX 控件。

11.3 视图编辑方法

11.3.1 绘制视图

在编辑视图时，将鼠标移到视图编辑工具按钮处，会看到相关工具的自动提示。

- 绘制矩形、圆角矩形、圆或椭圆等规则视图

在需要的工具按钮位置点击鼠标左键，选中所需的视图，然后将鼠标移动到视图编辑区域中需要绘制所选视图的位置。按下鼠标左键并拖动到所需视图的尺寸，抬起鼠标左键即可完成所选视图的绘制。

- 绘制多边形、折线等不规则视图

在需要的工具按钮位置点击鼠标左键，选中所需的视图，然后将鼠标移动到视图编辑区域中需要绘制所选视图的位置。在不规则视图的各个顶点处依次点击鼠标左键，可以画出不规则视图的各条边，在最后的顶点处双击鼠标左键，即可完成不规则视图的绘制。

- 绘制曲线

一条曲线由初始点、中间点和结束点等三个点来确定。依次点击鼠标左键三次即可完成一条曲线的绘制。点击鼠标左键确定曲线的初始点、中间点和结束点，以确定曲线的长度和弧度。鼠标左键第三次点击后，可以通过移动鼠标来改变曲线结束点的位置。双击鼠标左键结束曲线绘制过程。

- 复制视图

使用“编辑”/“复制”命令，或者<Ctrl>+<C>复合键，可以复制所选择的一个或多个视图对象。复制视图的另外一种方法是，选择要复制的视图对象，按下<Ctrl>键的同时点击该视图对象，则会在原来的视图对象上产生复制的视图对象。

- 视图中的状态栏

在视图中，鼠标指针的 X、Y 位置显示在状态栏上。状态栏上的位置值总是相对于视图的左上角。当鼠标位于视图对象上或者正在编辑视图对象时，状态栏上显示该视图对象的顺序号码。如果选择了要插入的视图对象，状态栏也会显示该视图对象的名称。

11.3.2 布置视图

在绘制视图的过程中，需要对视图进行修改和布置。

- 选择

用鼠标左键点击视图对象可以将其选中。

可以按下<Tab>键，选择视图对象中的第一个视图对象，再次按下<Tab>键则选择下一个视图对象。如果同时按下<Tab>和<Shift>键，可以按照视图对象中相反的顺序选择。

在选择一个视图对象后按下<Shift>键，同时点击相应的视图对象，可以选择多个视图对象。也可以按住鼠标左键不放，在要选择的视图对象上拉一个窗口，同样可选择多个对象。

➤ 全选

标题栏“其它”菜单中的“全选”菜单可以将当前视图中的所有视图对象全部选中。

➤ 选择方式

如果标题栏“其它”菜单中的“选择方式”菜单前有“√”，或者快捷工具栏中表示鼠标状态的快捷工具按钮被按下时，表示此时处于“选择”状态，可以选择视图对象。否则处于绘图状态。

➤ 选择方式和插入方式的切换

视图对象插入后，自动回到选择方式。如果要用同样方式再插入对象，可以再次选择相应的菜单命令。

切换的另外一种方法是，在选择方式下，同时按下<Ctrl>键和鼠标右键，可以在选择方式和插入方式之间快速转化。

➤ 拖动

点击鼠标左键选择视图对象。在所选的视图对象上按下鼠标左键，或者按下方向键，可以拖动一个或多个视图对象。

➤ 修改

用鼠标点击对象或者按下<Tab>键，可以选择一个视图对象。在所选择视图对象的周围

有一些小的黑色矩形。通过点击这些小的黑色矩形，按住鼠标左键，可以改变视图对象的大小，控制视图对象的轮廓。

选择一个视图对象后，同时显示旋转点。旋转点是一个中间带有白色十字的黑色圆圈，可按一定的角度使对象绕这点旋转。可以按下鼠标左键拖动旋转点。

对于多边形，使用同样的方法拖动每个顶角。鼠标拖动时，按下<Ctrl>键，可以插入一个顶角。按下<Shift>+<Ctrl>键，可以删除一个顶角。

在视图编辑区，点击鼠标右键，会弹出如图 11-3-1 所示的菜单。

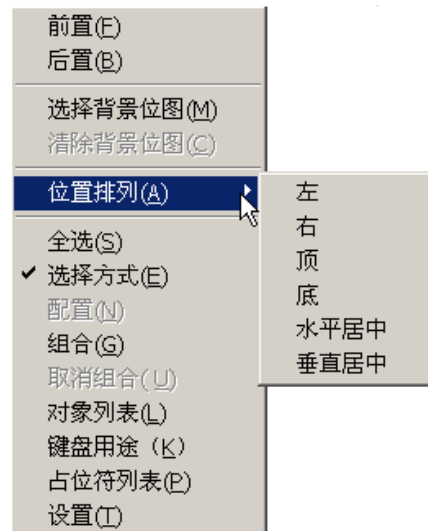


图 11-3-1 视图右键菜单

下面将依次介绍菜单中的选项。

➤ 前置

将选择的视图对象置于最上一层。选择一个视图对象，点击标题栏“其它”菜单中的“前置”菜单，可以将该视图对象放到其它视图对象的前面。

当视图对象以不同对象号显示时，视图对象实际上相当于以不同的图层来显示。将几个视图置于同一位置处时，会发现视图中对象号最大的置于最上层。如需使某一视图置于最上层时，可用此命令使该视图前置，且对象号变为当前最大值。

➤ 后置

将选择的视图对象置于最下一层。选择一个视图对象，点击标题栏“其它”菜单中的“后置”菜单，可以将该视图对象放到其它视图对象的后面。

与“前置”同理，可以使某一视图对象置于最底层，且将视图中对象号变为 0。

➤ 选择背景位图

从相应的路径选择合适的背景位图，选择“打开”，在视图中即可呈现所选择的背景位图。

➤ 清除背景位图

在背景位图设置好后，如果背景位图不满足需求，选择“清除背景位图”可对其进行清除。

➤ 位置排列

选择多个视图对象，点击标题栏“其它”菜单中的“位置”菜单，可以对该多个视图对象的位置进行排列。“位置”菜单有以下几个选项：

“左”表示被选中的视图对象以最左侧视图对象的左边沿为基准对齐排列，即左对齐。

“右”表示被选中的视图对象以最右侧视图对象的右边沿为基准对齐排列，即右对齐。

“顶”表示被选中的视图对象以最顶侧视图对象的顶边沿为基准对齐排列，即顶对齐。

“底”表示被选中的视图对象以最底侧视图对象的底边沿为基准对齐排列，即底对齐。

“水平居中”表示每个视图对象以所有视图对象的平均水平中心为基准对齐排列。

“垂直居中”表示每个视图对象以所有视图对象的平均垂直中心为基准对齐排列。

➤ 配置

对于不同的视图对象，其配置中的分类选项不同。对于“配置”，一定要在视图对象被选中的情况下，才显示该对象的一些配置，否则，该项为灰体。

➤ 组合

选择多个视图对象，点击标题栏“其它”菜单中的“组合”菜单，可以将多个视图对象组合成一个视图对象。组合后视图对象的行为与一个视图对象的行为相同。

➤ 取消组合

选择一个组合的视图对象，点击标题栏“其它”菜单中的“取消组合”菜单，可以将该组合的视图对象分解为多个单独的视图对象。

另外，视图编辑区右键菜单中的内容除了以上介绍的选项外，还包括对象列表、使用键盘、占位符列表和视图设置，这四个选项将在下面介绍。

11.3.3 对象列表

点击标题栏“其它”菜单中的“对象列表”菜单，打开视图对象列表对话框，如图 11-3-2 所示，其中包括视图对象的号码、类型和位置等信息。点击该对话框右侧的工具按钮，可以对其进行编辑。在绘图区点击鼠标右键，选中“对象列表”，也会弹出此对象列表。

- “确认”按钮：当对对象列表内容操作完毕后，点击“确认”按钮，关闭对话框。
- “最前”按钮：把选择的视图对象放在最上层，此时对象号最大。
- “最后”按钮：把选择的对象放在最底层，此时对象号最小。
- “向前一步”按钮：把选择的视图对象向上移动一层，此时对象号增大一号。
- “向后一步”按钮：把选择的视图对象向下移动一层，此时对象号减小一号。
- “删除”按钮：删除选择的对象。
- “取消”按钮：取消上一次操作
- “重做”按钮：恢复上一次操作。
- “编辑”按钮：对视图对象进行编辑操作。

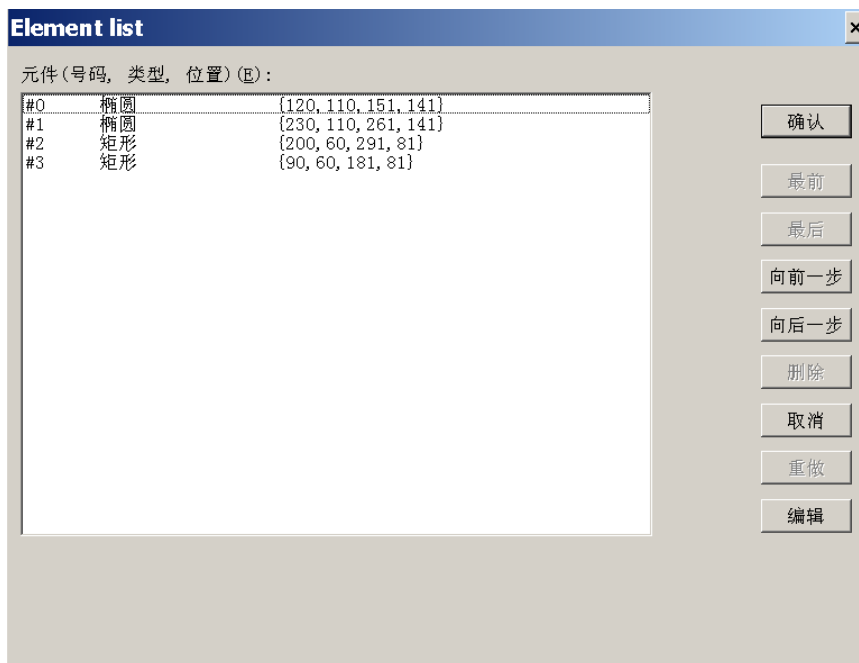


图 11-3-2 视图对象列表对话框

11.3.4 使用键盘

可以通过选择“Shift”、“Ctrl”及“Key”下拉菜单里的可选按键来实现动作的运行。

通过“Expression”表达式来描述要实现的动作或显示运行动作后的结果。使用键盘对于视图的每个物体可独自设置，因此，在不同视图中相同的按键设置可以起到不同作用。

◇ 举例

下面的表 11-3-1 和表 11-3-2 为两个视图 VIS_1 和 VIS_2 的不同的按键设置。

表 11-3-1 VIS_1 键盘设置

Shift	Ctrl	Action	Key	Expression
x		Toggle	A	PLC_PRG.automatic
	x	Zoom	Z	VIS_2

表 11-3-2 VIS_2 键盘设置

Shift	Ctrl	Action	Key	Expression
x		Exec	E	INTERN LANGUAGE DEUTSCH
	x	Zoom	Z	VIS_1

如果在“在线模式”下，且当前视图为 VIS_1，按住“Shift+A”复合键，则 PLC_PRG 中的 automatic 变量会触发“toggle”，另外，如果按住“Ctrl+Z”复合键，则会自动跳到视图 VIS_2。

对于“Action”的各选项可参考表 11-3-3。

表 11-3-3 键盘动作

Action	含义
Toggle	切换变量
Tap true	键控变量置为真
Tap false	键控变量置为假
Zoom	缩放视图
Exec	执行程序
Text	文本显示的文本输入变量

可以通过选择按键配合“shift”或“ctrl”组成复合键，触发相应的动作。

11.3.5 占位符列表

占位符列表如图 11-3-3 所示。

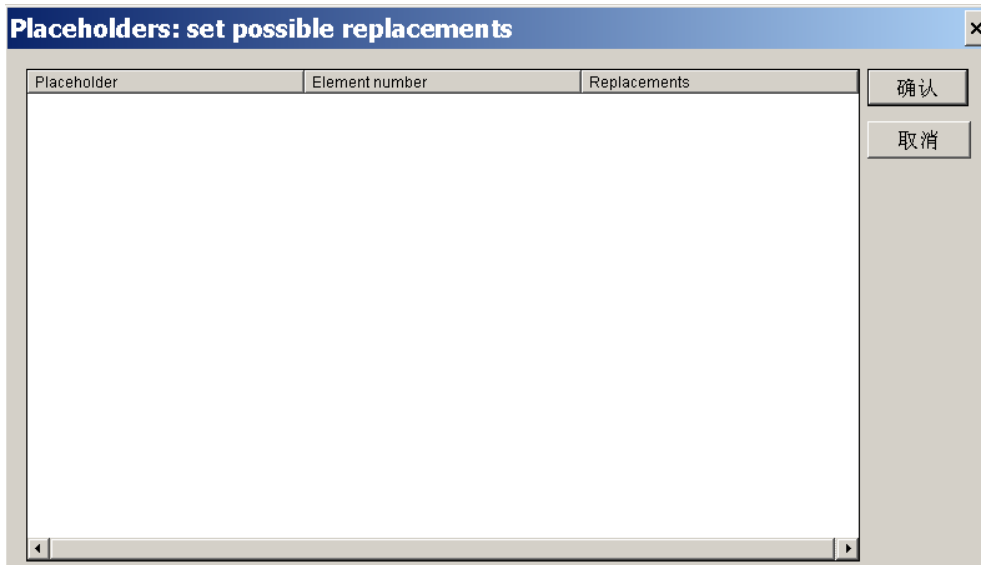


图 11-3-3 占位符列表

- Placeholder: 列出用于配置所有视图对象的占位符。
- Element number: 显示包含占位符的对象号。
- Replacements: 可以输入一些字符串。例如文本、变量或表达式等。

11.3.6 视图设置

视图设置如图 11-3-4 所示。



图 11-3-4 视图设置

- **Display:** 填入相应扩大或缩小比例改变屏幕大小，可以选择是否在视图中显示对象号，如图 11-3-4 所示。

- **Frame:** 框架设置，如图 11-3-5 所示。

选择“自动滚动”，在画某个对象或移动某个对象时达框架边界时，视图窗口会跟随该对象自动移动。

选择“包含背景视图”，背景视图将会符合窗口框架大小，否则只考虑对象。

选择“在线方式最佳匹配”，视图以最佳效果进行运行模拟。

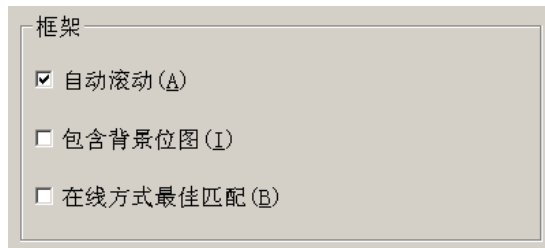


图 11-3-5 框架设置

- **Grid:** 网格设置，如图 11-3-6 所示。

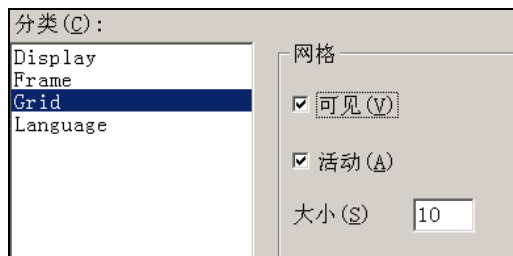


图 11-3-6 网格设置

选择网格“可见”，则会在窗口中呈现网格，可以通过改变网格大小调整网格间距，最小值为 10。

选择“活动”，则会以网格为单位进行移动，否则可实现在窗口任意空间移动。

- **Language:** PLC 不支持语言设置。

11.4 视图属性配置

11.4.1 属性配置方法

在编辑视图的过程中，选中某一个视图对象并点击鼠标右键，或点击标题栏的“其它”菜单，则会弹出“其它”下拉菜单，并且激活了其中的“配置”菜单，如图 11-4-1 所示。对于“配置”菜单，一定要在视图对象被选中的情况下才被激活，否则该项为灰体。

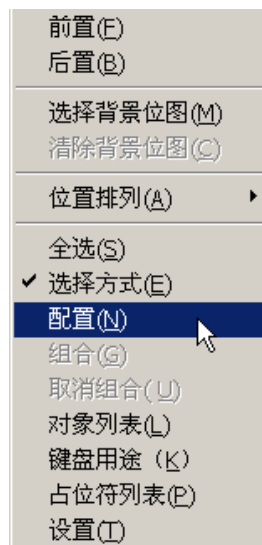


图 11-4-1 视图“其它”/“配置”菜单

点击此“配置”菜单，弹出视图对象属性配置对话框，如图 11-4-2 所示，则可以对所选中的视图对象的属性进行配置。用鼠标左键双击视图对象则可以快速打开此对话框。

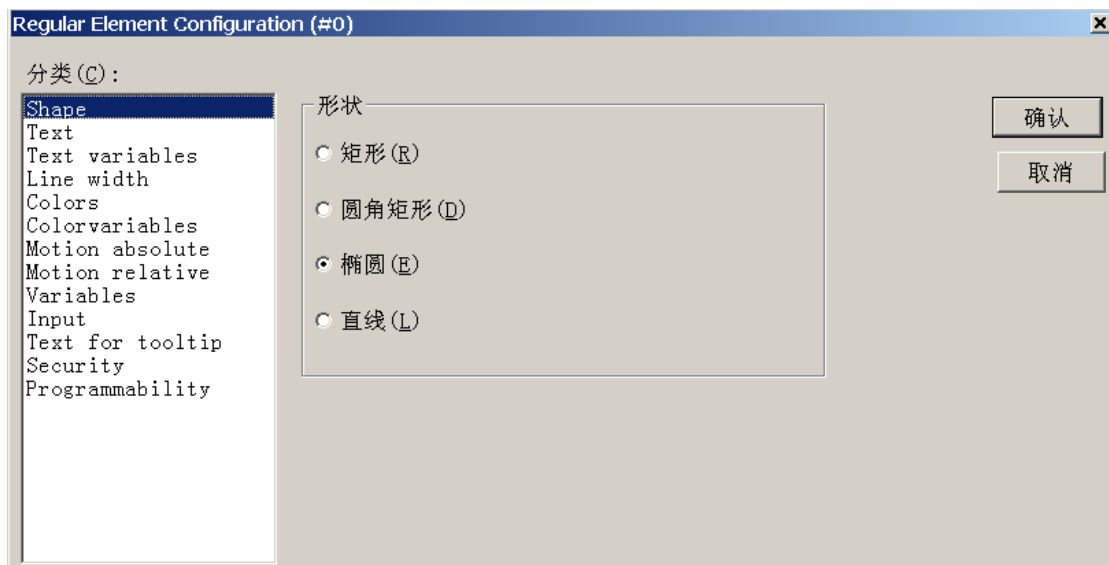


图 11-4-2 视图对象属性配置对话框

11.4.2 视图对象的属性

对于不同的视图对象，其属性配置中的分类选项不同，属性配置对话框右侧的对应参数内容也不相同。可以通过设置不同的参数变量来定义该视图对象，从而改变视图对象的属性。表 11-4-1 给出了视图属性与视图对象的对应关系。其中符号“√”表示有对应关系。

表 11-4-1 属性分类与视图对象的对应关系

视图属性		视图对象														
		矩形	圆角矩形	椭圆	多边形	折线	曲线	饼图	位图	视图	按钮	图元文件	表格	趋势图	报警表	控件
属性分类	属性名称															
Shape	形状	√	√	√	√	√	√									
Text	文本	√	√	√	√	√	√	√	√	√	√	√				
Text variables	文本变量	√	√	√	√	√	√	√	√	√	√	√				
Line width	线宽	√	√	√	√	√	√	√	√	√		√				
Colors	颜色	√	√	√	√	√	√	√						√		
Color variables	颜色变量	√	√	√	√	√	√	√	√	√		√				
Motion absolute	绝对移动	√	√	√	√	√	√	√	√	√		√				
Motion relative	相对移动	√	√	√					√	√		√				
Variables	变量	√	√	√	√	√	√	√	√	√	√	√				
Input	输入	√	√	√	√	√	√	√	√	√	√	√				
Text for tooltip	提示文本	√	√	√	√	√	√	√	√	√	√	√	√	√	√	
Security	安全	√	√	√	√	√	√	√	√	√	√	√	√	√	√	
Programmability	编程	√	√	√	√	√	√	√	√	√	√	√				
Angle	角度							√								
Bitmap	位图属性								√		√					
Visualization	视图属性									√						
Group	组框架属性											√				
Table	表格属性												√			
Columns	列												√		√	
Rows	行												√			
Selection	选择												√			
Trend	趋势图属性													√		
Alarm table	报警表属性														√	
Settings for sorting	排序设置														√	
Settings for alarm table	报警表设置														√	
Control	控件类型															√
Method calls	方法调用															√
Display	显示															√

11.5 视图静态属性

所谓视图的静态属性是指描述图形对象基本形状的参数，主要包括形状、文本、线条宽度、颜色等。

11.5.1 形状

形状 Shape 属性用来定义视图对象的形状。对于规则视图，形状 Shape 属性可以选择矩形、圆角矩形、椭圆或直线等。对于不规则视图，形状 Shape 属性可以选择多边形、折线或曲线等，如图 11-5-1 所示。形状 Shape 属性的改变只在所确定的范围内进行。

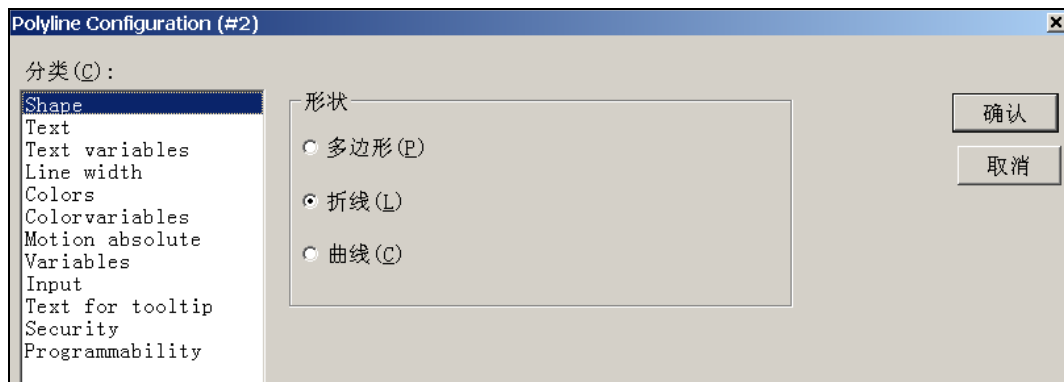


图 11-5-1 不规则视图对象属性配置对话框

11.5.2 文本

在视图对象中可以添加文本，用文本 Text 属性来设置，如图 11-5-2 所示。

➤ 内容

在“内容”文本框中输入文本，按<Ctrl>+<Enter>组合键换行。

➤ 水平

在“水平”选项中设置文本在视图对象中的左、中、右位置。

➤ 垂直

在“垂直”选项中设置文本在视图对象中的上、中、下位置。

点击“字体”或“标准字体”按钮，可以设置文本的字体。



图 11-5-2 文本设置

在标准 C 库中，常用的占位符有%s（字符格式）、%f（浮点型格式）、%d（整型格式）和%x（16 进制整型）等，其含义如表 11-5-1 所示。

表 11-5-1 占位符与含义

占位符	含义
%a	星期缩写
%A	星期全写
%b	月缩写
%B	月全称
%c	日期时间
%d	一月中的天数（01-31）
%H	24 小时格式（00-23）
%I	12 小时格式（01-12）
%j	一年中的天数（001-366）
%m	月（01-12）
%M	分钟（00-59）
%p	12 小时 A.M/P.M 格式
%S	秒（00-59）
%U	一年的第几个星期(00-53)，周日为星期第一天
%w	星期（0-6，周日是 0）
%W	一年的第几个星期(00-53)，周一为星期第一天
%x	日期
%X	时间
%y	不含世纪的年（00-99）
%Y	含世纪的年
%z	时区名，如：中国标准时间
%Z	
%%	百分号

✧ 举例

如果在文本内容中填入%2.5f mm，则在程序运行时会显示 32.8889 mm。

✧ 举例

在视图中设置如图 11-5-3 所示的格式。程序运行结果如图 11-5-4 所示。显示当前时间格式：中国标准时间-年-月-日 时：分：秒

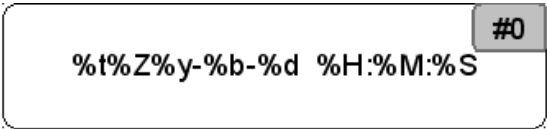


图 11-5-3 时间占位符应用

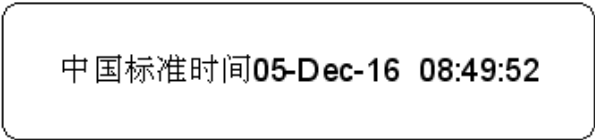


图 11-5-4 程序运行结果

11.5.3 线宽

线宽 Line width 属性用来定义视图对象线条的宽度。图 11-5-5 所示为线宽设置对话框，共有 5 种线宽可以直接选择。选中“其它”选项可以输入线宽。在“线宽变量”中可以输入控制线宽的工程变量，使线宽产生动画效果。输入工程变量时可以使用 F2 功能键。在线模式下，动态线宽属性会覆盖静态线宽属性。



图 11-5-5 线宽设置

11.5.4 颜色

颜色 Colors 属性用来定义视图对象的颜色和报警的颜色。图 11-5-6 所示为视图对象颜色设置对话框，可以分别设置视图对象的内部颜色和边框颜色。选中“无填充色”或“无边框色”可以创建透明的图形对象。视图对象的报警颜色可以用填充颜色或边框颜色来显示。在线模式下，动态颜色属性会覆盖静态颜色属性。

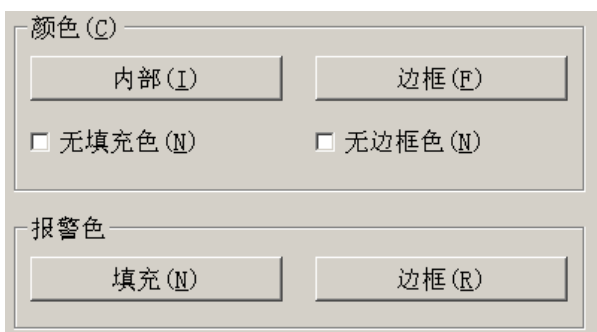


图 11-5-6 视图颜色设置

11.5.5 工具提示文本

工具提示文本 Text for tooltip 属性用来定义视图对象的工具提示文本。图 11-5-7 所示为视图对象工具提示文本设置对话框，可以在“内容”文本框中输入表示视图对象的文本。离线或在线模式下，当鼠标移动到该图形对象上时，会显示所输入的文本。在“内容”文本框中输入文本，可以按<Ctrl>+<Enter>组合键换行。

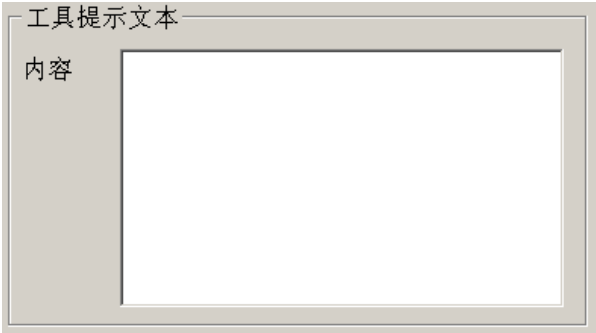


图 11-5-7 工具提示文本

11.5.6 安全属性

安全 Security 属性用来定义视图对象的访问权限。图 11-5-8 所示为视图对象安全设置对话框，可以定义用户对于视图对象的访问权限。用户组有 8 组，访问权限分为“无访问权限”、“只读权限”和“完全控制权限”等 3 种。在线模式下，具有不同的访问权限的用户组可以实现的操作不同。

- 具有“无访问权限”的用户组，不具有访问视图对象的权限，视图对象不可见。
- 具有“只读权限”的用户组，视图对象可见，但不可操作，即无输入操作权限。
- 具有“完全控制权限”的用户组，可以对视图对象进行完全控制。

如果需要将此安全权限的配置用于其他视图对象中，选中“适用所有视图对象”即可。

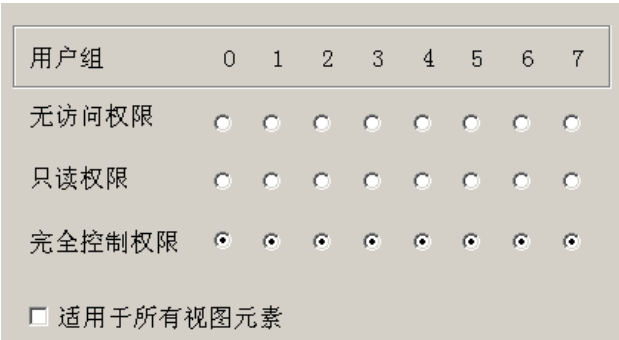


图 11-5-8 操作权限设置

11.5.7 位图属性

在视图可以插入位图。位图 Bitmap 属性用来设置插入到视图中位图的参数。图 11-5-9 所示为位图设置对话框。

➤ 位图

点击“位图”文本框后的按钮可以选择所需要的位图文件。

选中“背景透明”，可以创建透明的图形对象。

➤ 框架

在“框架”选项中可以设置位图框架的属性。

“各项异性”表示在改变位图的尺寸时，位图的高度和宽度可按任意比例进行拉伸。

“各项同性”表示在改变位图的尺寸时，位图的高度和宽度比例保持不变。

“固定”表示在改变位图的尺寸时，位图本身的高度和宽度大小固定不变。只是可根据需求，改变位图的显示大小。

在选中“固定”选项前提下，再选中“绘图”表示显示位图对象的框架，否则，不显示位图对象的框架；如果再选中“剪切”表示当位图对象的框架小于位图时，只显示框架内的位图，而不显示框架外的位图。如果想显示整个位图大小，可拉伸框架的大小。

“颜色”和“报警色”可以分别设置位图框架的颜色和报警色。

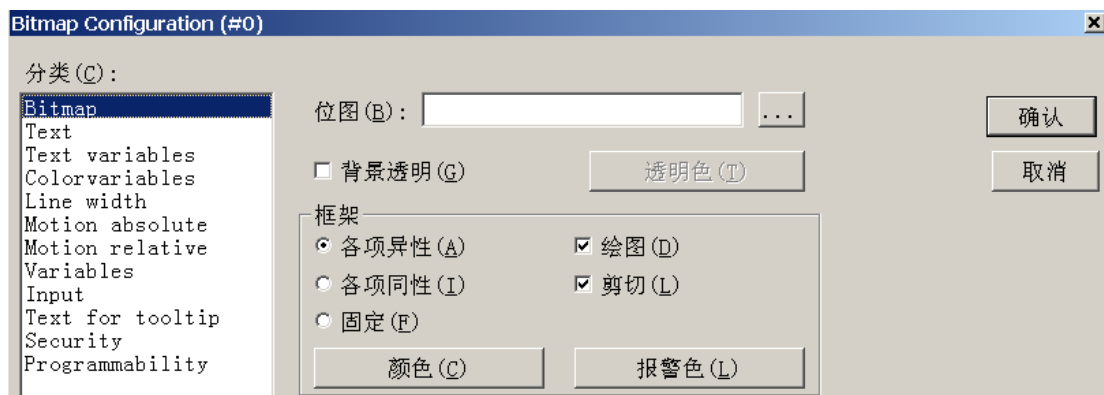


图 11-5-9 位图设置

11.5.8 视图属性

在视图可以插入其它视图。视图 Visualization 属性用来设置插入到视图中其它视图的参数。图 11-5-10 所示为视图设置对话框。

➤ 视图

点击“视图”文本框后的按钮可以选择所需要的视图文件。

点击“占位符”按钮可以设置视图的替换占位符。

➤ 框架

在“框架”选项中可以设置视图框架的属性。

“各项异性”表示在改变视图的尺寸时，视图的高度和宽度可按任意比例进行拉伸。

“各项同性”表示在改变视图的尺寸时，视图的高度和宽度比例保持不变。

“固定”表示在改变视图的尺寸时，视图的本身的高度和宽度大小是固定不变。只是可根据需求，改变视图的显示大小。

在选中“固定”选项前提下，再选中“绘图”表示显示视图对象的框架，否则，不显示视图对象的框架；如果再选中“剪切”表示当视图对象的框架小于视图时，只显示框架内的视图，而不显示框架外的视图。如果想显示整个视图大小，可拉伸框架的大小。

“颜色”和“报警色”可以分别设置视图框架的颜色和报警色。

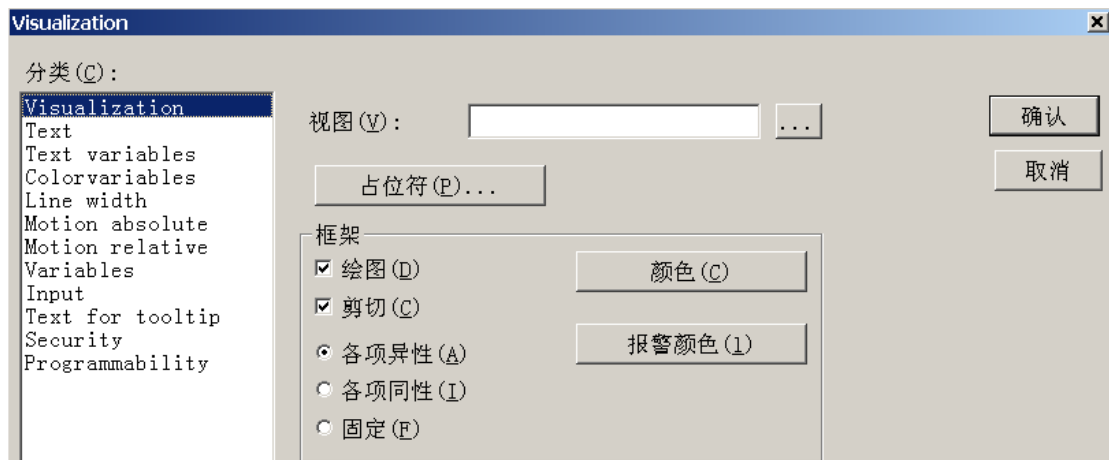


图 11-5-10 视图设置

11.5.9 组框架属性

在视图可以插入 WMF 图元文件格式的图形文件。WMF 图元文件的组框架 Group 属性用来设置插入到视图中 WMF 图元文件的参数。图 11-5-11 所示为 WMF 图元文件组框架 Group 设置对话框。

在“框架”选项中可以设置视图框架的属性。

选中“绘图”表示显示视图对象的框架。

选中“各向同性的”表示在改变视图的尺寸时，视图的高度和宽度比例保持不变。

选中“剪切”表示当视图对象的框架小于视图时，只显示框架内的视图，而不显示框架外的视图。

“颜色”和“报警颜色”可以分别设置视图框架的颜色和报警色。

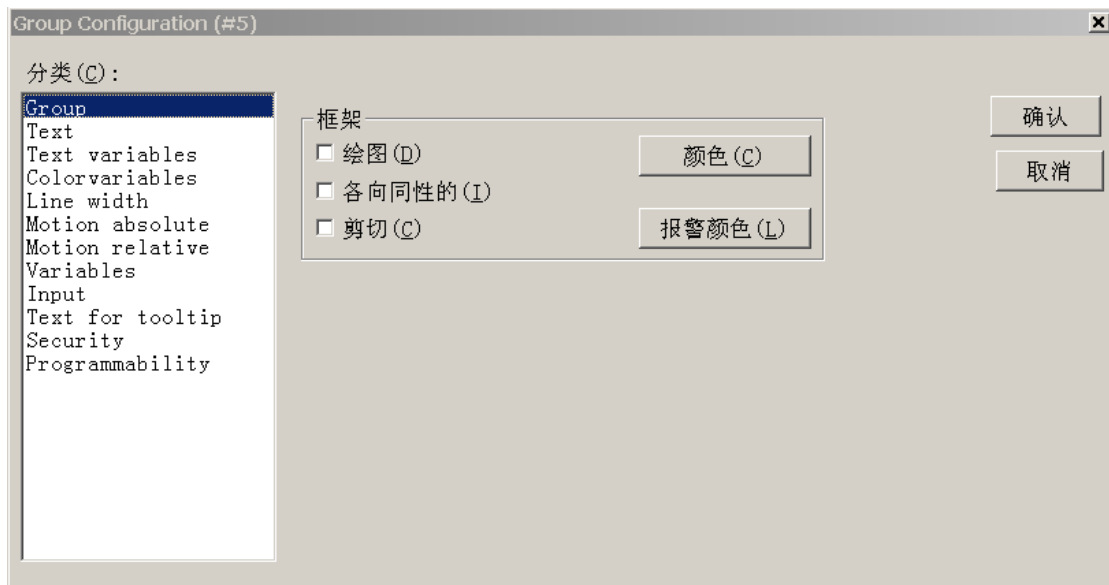


图 11-5-11 图元文件组框架属性设置

11.5.10 角度

角度 Angle 属性用来定义饼图对象的角度。双击饼图对象，弹出饼图对象属性配置对话框，如图 11-5-12 所示。点击角度 Angle 属性，在“开始角度”和“结束角度”文本框中分别输入饼图对象的开始角度和结束角度，则将会以顺时针方向画出所需要的饼图。选中“只显弧段”，则饼图对象不显示夹角，只显示弧段。

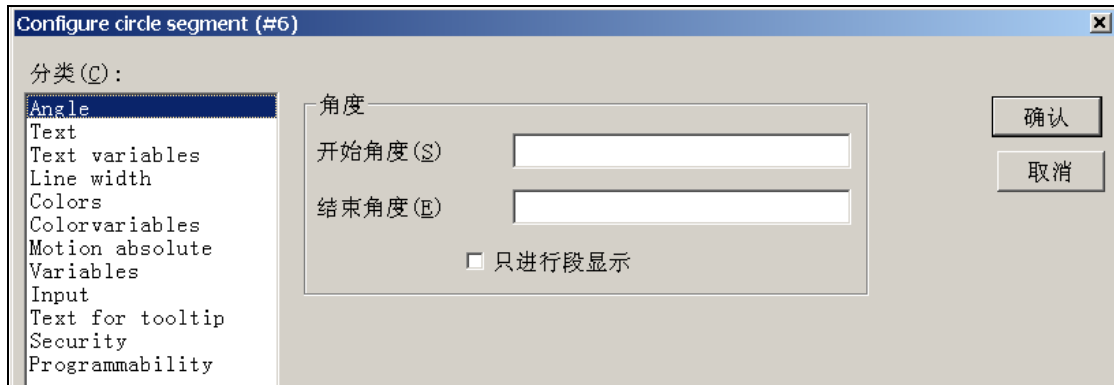


图 11-5-12 饼图对象属性配置对话框

◇ 举例

声明变量

```
PROGRAM PLC_PRG
```

```
VAR
```

```
    angle_start: REAL := 0;
```

```
    angle_end: REAL := 90;
```

```
END_VAR
```

视图运行结果如图 11-5-13 所示。



图 11-5-13 饼图示例

11.6 视图编程

11.6.1 编程属性

视图对象的属性不仅可以进行静态参数的设置或常规工程变量的定义，而且可以进行结构变量的定义，这些结构变量专门用于视图对象的编程。为此，在库 SysLibVisu.lib 中定义了结构体 VisualObjectType，其成员分量可以用来定义绝大多数图形对象的属性。为了避免图形对象属性的多重定义，常规工程变量的值将覆盖结构变量的值，而且二者都覆盖静态属性的定义。

为了使用结构变量来配置图形对象的属性，可以打开图形对象属性配置对话框，选择编程属性 Programmability，如图 11-6-1 所示。选中“对象名称”选择框，在其后的文本框中输入结构变量的名称，则会自动声明 VisualObjectType 结构体类型的结构变量，该结构体 VisualObjectType 在库 SysLibVisu.lib 中已经被定义。这样声明的结构变量是全局变量，其声明是不可见的隐含声明，前提是必须在库管理器中添加 SysLibVisu.lib 库。

重新编译工程后，新定义的结构变量将出现在工程中。

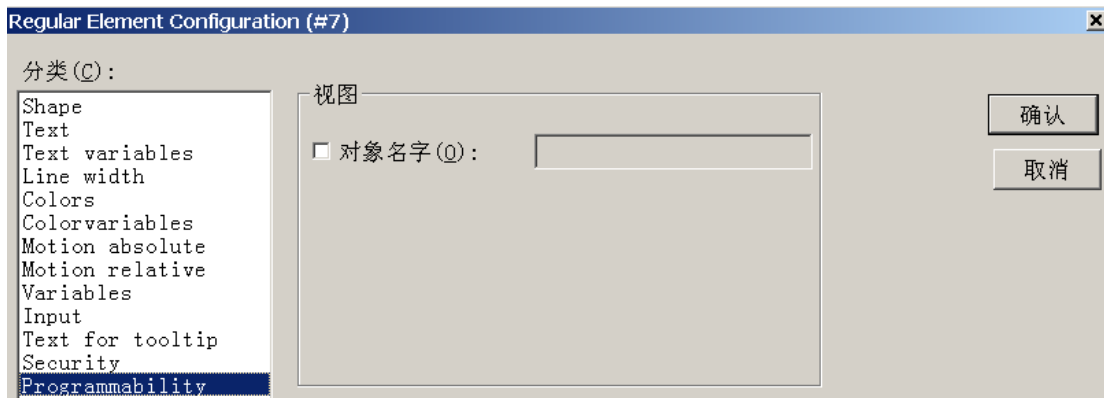


图 11-6-1 编程属性的设置

11.6.2 视图库

视图库 SysLibVisu.lib 中结构体 VisualObjectType 的定义如下所示。

TYPE VisualObjectType

STRUCT

```
(* Absolut movement *)
nXOffset:INT;
nYOffset:INT;
nScale:INT;
nAngle:INT;
(* Variables *)
bInvisible:BOOL;
stTextDisplay:STRING;
bToggleColor:BOOL;
bInputDisabled: BOOL;
stTooltipDisplay:STRING;
(* Text and font *)
dwTextFlags:DWORD;
dwTextColor:DWORD;
nFontHeight:INT;
dwFontFlags:DWORD;
stFontName:STRING;
(* Line *)
nLineWidth:INT;
(* Color *)
dwFillColor: DWORD;
dwFillColorAlarm: DWORD;
dwFrameColor: DWORD;
dwFrameColorAlarm: DWORD;
dwFillFlags: DWORD;
dwFrameFlags: DWORD;
(* Relative movement *)
nLeft:INT;
nTop:INT;
nRight:INT;
nBottom:INT;
```

END_STRUCT

END_TYPE

结构体 VisualObjectType 中的成员分量的数据类型及功能如表 11-6-1 所示。成员分量的前缀字母表示数据类型，均为小写字母，其含义分别为：

n 表示 INT，dw 表示 DWORD，b 表示 BOOL，st 表示 STRING。

表 11-6-1 成员分量的数据类型及功能

成员分量及数据类型	功能	示例（结构变量为vis1）	对应的图形对象属性
nXOffset : INT;	X方向位移	vis1.nXOffset:=val1; （沿 X 方向移动到 X=val1 处）	Motionabsolute: X-Offset
nYOffset : INT;	Y方向位移	vis1.nYOffset:=val2; （沿 Y 方向移动到 Y=val2 处）	Motionabsolute: Y-Offset
nScale : INT;	改变大小	vis1.nScale:=plc_prg.scale_var; （取值 plc_prg.scale_var）	Motion absolute: Scale
nAngle : INT;	旋转角度	vis1.anglevar:=15; （顺时针旋转 15 度）	Motion absolute: angle
bInvisible : BOOL;	可见/不可见	vis1.visible:=TRUE; （图形对象可见）	Color: No color inside+ No frame color Colorvariables: FillColor+Framecolor
stTextDisplay : STRING;	显示文本	vis1.TextDisplay:='ON / OFF'; （显示文本ON / OFF）	Text: Content
bToggleColor : BOOL;	颜色在 TRUE 与 FALSE 之间切换	vis1.bToggleColor:=alarm_var; （当变量 alarm_var 为 TRUE 时，分别显示 dwFillColorAlarm 和 dwFrameColorAlarm 定义的颜色）	Input: Toggle variable Variables: Change color
bInputDisabled : BOOL;	取值 FALSE，忽略配置 Input	vis1.bInputDisabled:=FALSE; （此图形对象不能输入值）	Variables: Input Disable
stTooltipDisplay : STRING;	工具提示文本	vis1.stTooltipDisplay:='Switch for'; （显示工具提示文本Switch for）	Text for Tooltip: Content:
dwTextFlags : DWORD;	文本位置：： 1 左对齐 2 右对齐 4 水平居中 8 顶对齐 10 底对齐 20 垂直居中 （取值可以相加）	vis1.dwTextFlags:=24; （文本居中，4+20）	Text: Horizontal and Vertical options Textvariables: Textflags
dwTextColor : DWORD;	文本颜色（颜色的定义见表后示例）	vis1.dwTextColor:=16#00FF0000; （蓝色）	Textvariables: Textcolor
nFontHeight : INT;	字体高度，取值范围 10-96 像素	vis1.nFontHeight:=16; （字体高度为 16 像素）	Textvariables: Font heighth
dwFontFlags : DWORD;	字体： 1 斜体字 2 粗体字 4 下划线 8 取消 （取值可以相加）	vis1.dwFontFlags:=10; （显示粗体字并取消，2+8）	Textvariables: Fontflags
stFontName : STRING;	改变字体	vis1.stFontName:='Arial'; （使用 Arial 字体）	Textvariables: Fontname
nLineWidth : INT;	框架的线宽（像素）	vis1.nLWidth:=3; （框架线宽为 3 像素）	Line width

(续前表)

dwFillColor : DWORD;	填充颜色 (颜色的定义见表后示例)	vis1.dwFillColor:=16#00FF0000; (填充蓝色)	Color: Color Inside Colorvariables: Fill color
dwFillColorAlarm : DWORD;	当 bToggleColor 为 TRUE 时, 填充颜色 (颜色的定义见表后示例)	vis1.dwFillColorAlarm:=16#0080800; (当 bToggleColor 为 TRUE 时, 填充灰色)	Color: Alarm color Inside Colorvariables: Fill color alarm
dwFrameColor : DWORD;	框架颜色 (颜色的定义见表后示例)	vis1.dwFrameColor:=16#00FF0000; (框架为蓝色)	Color: Color Frame Colorvariables: Frame color
dwFrameColorAlarm : DWORD;	当 bFrameColor 为 TRUE 时, 填充颜色 (颜色的定义见表后示例)	vis1.dwFrameColorAlarm:=16#008080080; (当 bFrameColor 为 TRUE 时, 填充灰色)	Color: Alarm color Frame Colorvariables: Frame color alarm
dwFillFlags: DWORD;	显示颜色: 0 显示颜色 >0 忽略颜色设置	vis1.dwFillFlags:=1; (显示颜色)	Color: No color inside + No frame color Colorvariables: Fillflags
dwFrameFlags : DWORD;	显示框架: 0 全显示 1 虚线(---) 2 点() 3 点划线(_._._) 4 双点划线(_._._) 8 不显示	vis1.FrameFlags:=1; (框架显示虚线)	Colorvariables: Frameflags

颜色取值定义: vis1.dwFillColor := 16#00FF00FF;

颜色取值以八位 16 进制数 16#00FF00FF 表示, 包括蓝、绿和红等三种基本颜色。每种颜色以两位 (0-255) 表示, 上述三种颜色依次占据后六位。在符号 16#后面的两位 00 必须保留, 以满足 DWORD 数据类型的语法规则。对于每种颜色, 可以取值 0-255, 如下所示:

16#	00	FF	00	FF
关键字	保留位	蓝色	绿色	红色

◇ 举例 (颜色取值示例)

```
PROGRAM PLC_PRG
```

```
VAR
```

```
  n:INT:=0;
```

```
  bMod:BOOL:=TRUE;
```

```
END_VAR
```

```
(* Blinking element *)
```

```
n:=n+1;
```

```
bMod:=(n MOD 20) > 10;
```

```
IF bMod THEN
```

```
  blinker.nFillColor := 16#00808080; (* 灰色 *)
```

```
ELSE
```

```
  blinker.nFillColor := 16#00FF0000; (* 蓝色 *)
```

```
END_IF
```

11.7 视图动态属性

所谓视图的动态属性是指描述图形对象动画效果的参数，即用工程变量来控制图形对象产生动画效果。

11.7.1 文本变量

文本变量 Text variables 用来定义文本的动态属性。图 11-7-1 所示为文本变量设置对话框，在文本框中输入相应的变量，即可实现动画效果。输入变量时可以使用 F2 功能键。

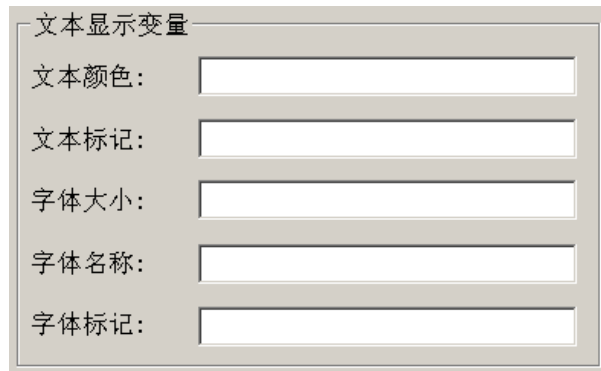


图 11-7-1 展示了“文本显示变量”对话框。该对话框包含以下设置项：

文本显示变量	
文本颜色:	
文本标记:	
字体大小:	
字体名称:	
字体标记:	

图 11-7-1 文本变量设置

11.7.2 颜色变量

颜色变量 Color variables 用来定义颜色的动态属性。图 11-7-2 所示为颜色变量设置对话框，在文本框中输入相应的变量，即可实现动画效果。输入变量时可以使用 F2 功能键。

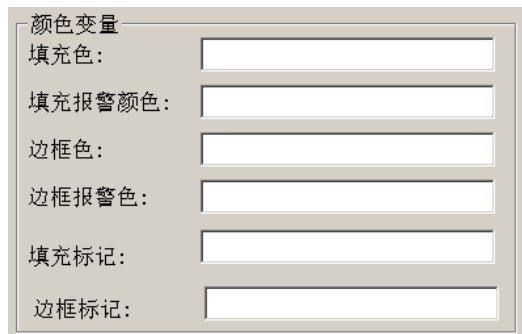


图 11-7-2 展示了“颜色变量”对话框。该对话框包含以下设置项：

颜色变量	
填充色:	
填充报警颜色:	
边框色:	
边框报警色:	
填充标记:	
边框标记:	

图 11-7-2 颜色变量设置

11.7.3 绝对运动

绝对运动 Motionabsolute 用来定义绝对运动的动态属性。图 11-7-3 所示为绝对运动设置对话框，在文本框中输入相应的变量，即可实现动画效果。输入变量时可以使用 F2 功能键。

- X 位移：设置变量的水平移动一定距离的变量。
- Y 位移：设置变量的上下垂直移动一定距离的变量。
- 比例：设置比例系数，改变视图大小。如果要将新视图对象变为原视图对象的 1 倍，则比例取值为 1000。如果要将新视图对象变为原视图对象的 2 倍，则比例取值为 2000。
- 角度：视图对象的旋转角度。正值表示顺时针旋转，负值表示逆时针旋转。对于矩形，相当于视图对象以基点为原点相对平移一定角度，而不进行旋转。对于除矩形外的视图对象，例如多边形和椭圆等，都是以基点为原点进行整体旋转，也就是说，多边形等视图对象的每一点都在旋转。

图 11-7-3 绝对运动设置

11.7.4 相对运动

相对运动 Motion relative 用来定义相对运动的动态属性。图 11-7-4 所示为相对运动设置对话框，在文本框中输入相应的变量，即可实现动画效果。输入变量时可以使用 F2 功能键。对于坐标轴，X 方向“右”为正，Y 方向“上”为正，即变量取值为正时，视图对象的边缘向正向移动一定距离，否则，向负向移动。

- 左边缘：视图对象左边缘移动一定距离。
- 上边缘：视图对象上边缘移动一定距离。
- 右边缘：视图对象右边缘移动一定距离。
- 下边缘：视图对象下边缘移动一定距离。

图 11-7-4 相对运动设置

11.7.5 变量

变量 Variables 用来定义图形对象的其它动态属性。图 11-7-5 所示为变量设置对话框，在文本框中输入相应的变量，即可实现动画效果。输入变量时可以使用 F2 功能键。

- 在“不可视变量”中输入逻辑变量，其值决定图形对象的可视属性。如果变量值为 FALSE，则图形对象是可见的。如果变量值为 TRUE，则图形对象是不可见的。
- 在“改变颜色变量”中输入逻辑变量，其值决定图形对象的颜色。如果变量值为 FALSE，则图形对象显示缺省颜色。如果变量值为 TRUE，则图形对象显示报警颜色。
- 在“文本显示”字段可输入显示值。如果已经在“文本”的“内容”中输入了“%s”，则“在线模式”时，“文本显示”的值会代替“%s”显示在文本区域。

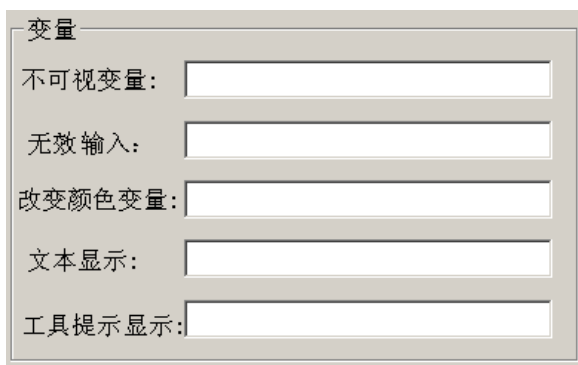


图 11-7-5 变量设置

11.7.6 输入

输入 Input 用来定义图形对象的输入动态属性。图 11-7-6 所示为输入设置对话框，在文本框中输入相应的变量，即可实现动画效果。输入变量时可以使用 F2 功能键。

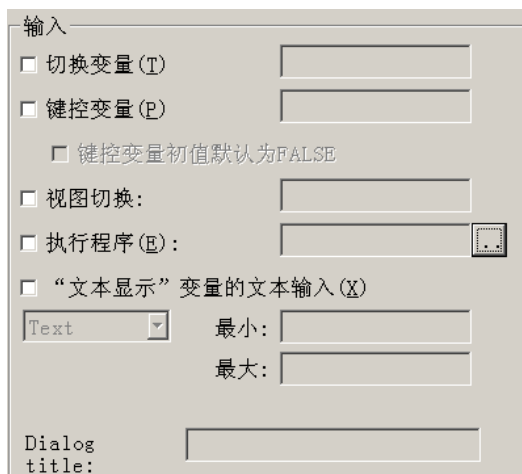


图 11-7-6 输入设置

- 切换变量：选择“切换变量”字段，在线模式时，在对象上每次点击都会改变位于输入字段的变量值。鼠标每次点击，逻辑变量的值都会从 TRUE 变成 FALSE，或者相反。
- 键控变量：选项“键控变量”允许在线模式下改变逻辑变量的值。这个逻辑变量位于输入字段。把鼠标光标放在对象上，按下鼠标键不要松开，例如变量值从 TRUE 改为 FALSE。当释放鼠标时，变量的值返回到初始值 TRUE。
- 视图切换：选择字段“视图切换”，允许在随后字段里输入同一工程的视图对象名。在线模式时，在视图窗口中，使用鼠标点击改变到已输入的对象。如果输入 STRING 类型的程序变量，例如是 PLC_PRG.xxx，而不是一个视图对象，那么这个变量可以用来定义视图对象名，例如 visu1。鼠标点击时，系统将显示这个视图对象，例如 xxx := visu1。
- 执行程序：选择“执行程序”，在输入字段中输入任意的可执行程序。在线模式下，鼠标点击对象，执行该可执行程序。
- “文本显示”变量的文本输入：选择““文本显示”变量的文本输入”，在线模式下，通过视图对象分配给变量一个值。点击对象产生一个编辑框架，给变量键入新值，按下<Enter>键接受这个值。

11.8 表格

在编辑视图时，可以在视图中插入表格，如图 11-8-1 所示。

	G.arr1[0]	G.arr1[1]	G.arr1[2]	G.arr1[3]	G.arr1[4]	G.arr1[5]
0						
1						
2						
3						
4						
5						

图 11-8-1 插入表格

双击表格对象，弹出表格对象属性配置对话框，如图 11-8-2 所示，可以设置表格 Table、列 Columns、行 Rows、选择 Selection、工具提示文本 Text for tooltip 和安全 Security 等属性。输入变量时可以使用 F2 功能键。

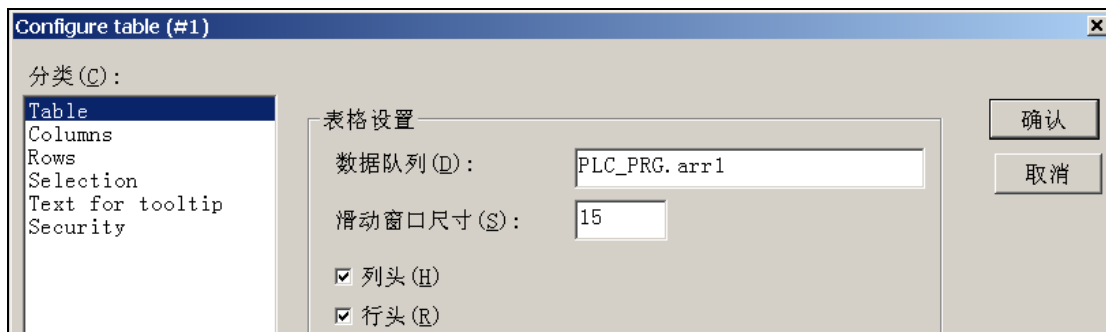


图 11-8-2 表格配置对话框

11.9 趋势图

在编辑视图时，可以在视图中插入趋势图，如图 11-9-1 所示。

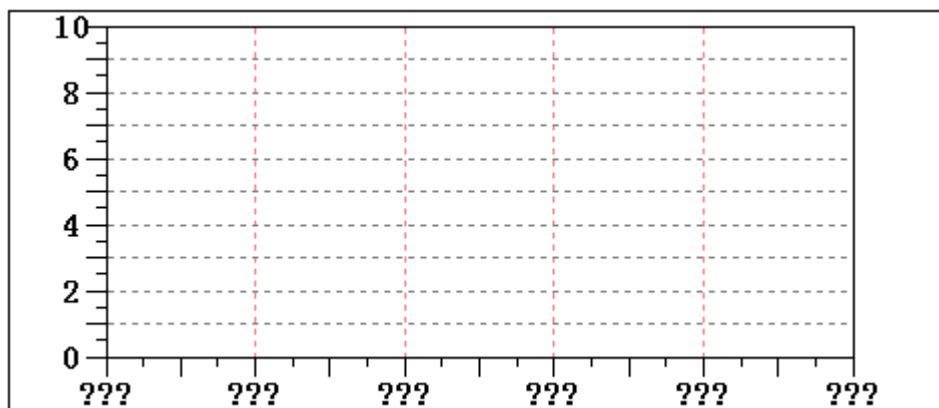


图 11-9-1 插入趋势图

双击趋势图对象，弹出趋势图对象属性配置对话框，如图 11-9-2 所示，可以设置趋势图 Trend、颜色 Colors、工具提示文本 Text for tooltip 和安全 Security 等属性。输入变量时可以使用 F2 功能键。



图 11-9-2 趋势图配置对话框

11.10 报警表

在编辑视图时，可以在视图中插入报警表，如图 11-10-1 所示。

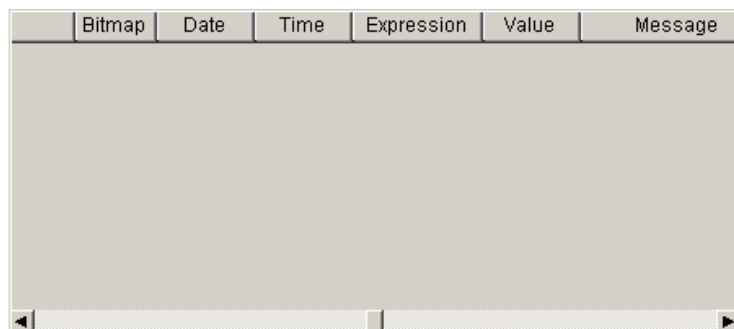


图 11-10-1 插入报警表

双击报警表对象，弹出报警表对象属性配置对话框，如图 11-10-2 所示，可以设置报警表 Alarm table、列 Columns、排序 Settings for sorting、选择 Selection settings、工具提示文本 Text for tooltip 和安全 Security 等属性。输入变量时可以使用 F2 功能键。

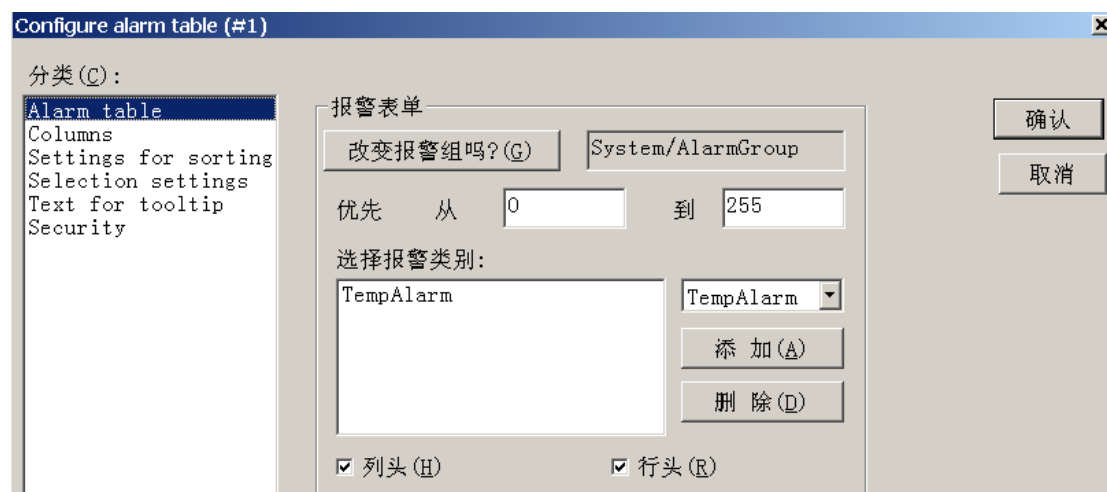


图 11-10-2 报警表配置对话框

11.11 ActiveX 控件

在编辑视图时，可以在视图中插入 ActiveX 控件，如图 11-11-1 所示。



图 11-11-1 插入 ActiveX 控件

双击 ActiveX 控件对象，弹出 ActiveX 控件对象属性配置对话框，如图 11-11-2 所示，可以设置控件 Control、方法调用 Method calls 和显示 Display 等属性。输入变量时可以使用 F2 功能键。

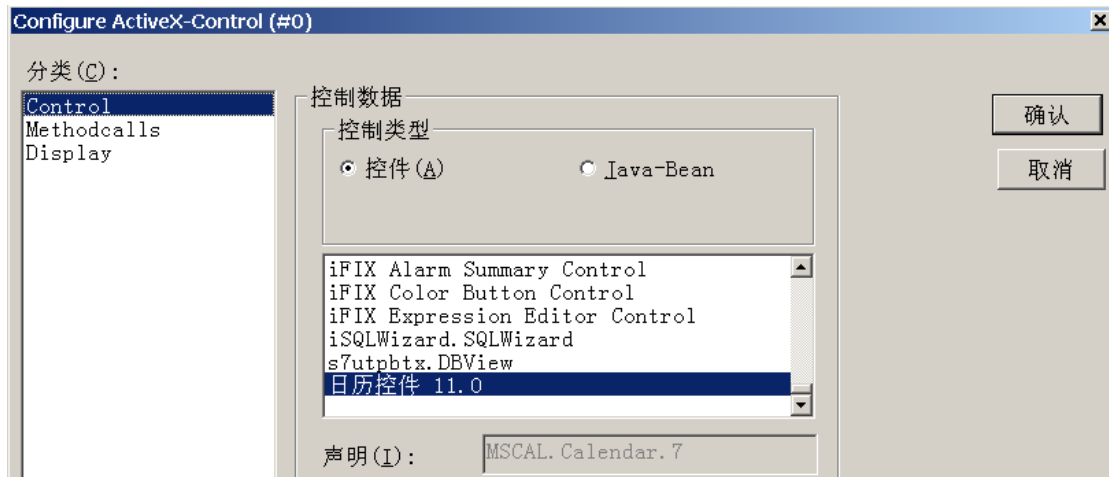


图 11-11-2 ActiveX 控件配置对话框

11.12 视图举例

下面以一个小程序为例来介绍视图的应用。此程序可以实现两个信号灯的交替闪烁。程序编写如图 11-12-1 所示。

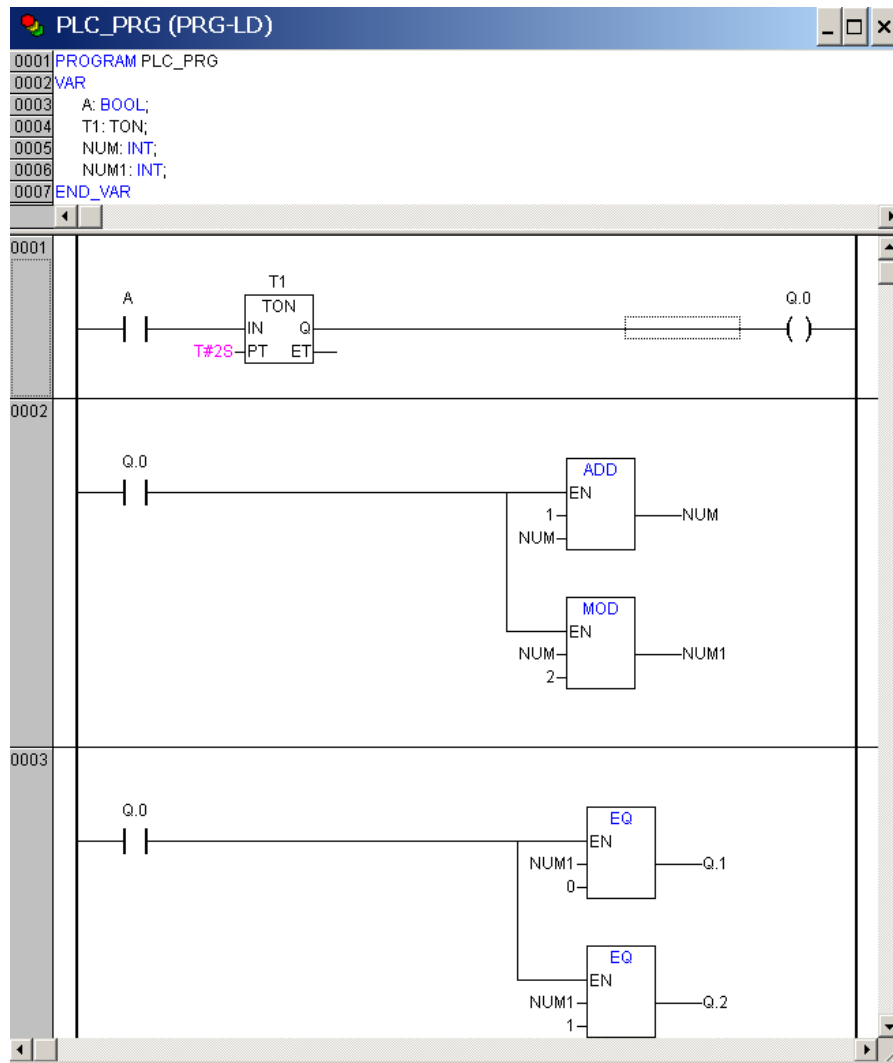


图 11-12-1 视图程序编写

为了能够进一步看清楚信号灯的闪烁情况，可以定义视图对象以不同颜色来显示信号灯的亮、灭情况。用绿色代表灯亮，红色代表灯灭。

首先创建视图 A。在视图 A 中，用两个圆分别表示两个信号灯，用两个矩形分别表示两个信号灯的名称“Q.1”和“Q.2”，如图 11-12-2 所示。

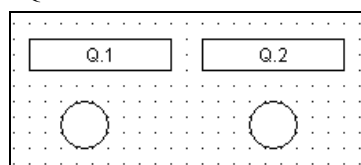


图 11-12-2 视图绘制

选中 Q.1 信号灯，并双击，则会出现“Regular Element Configuration”对话框。选择“Colors”设置“颜色”栏中的“内部”颜色为绿色，单击“确定”，如图 11-12-3 所示。同样，设置“报警色”栏中的“填充”颜色为红色。Q.2 信号灯的颜色设置方式相同。

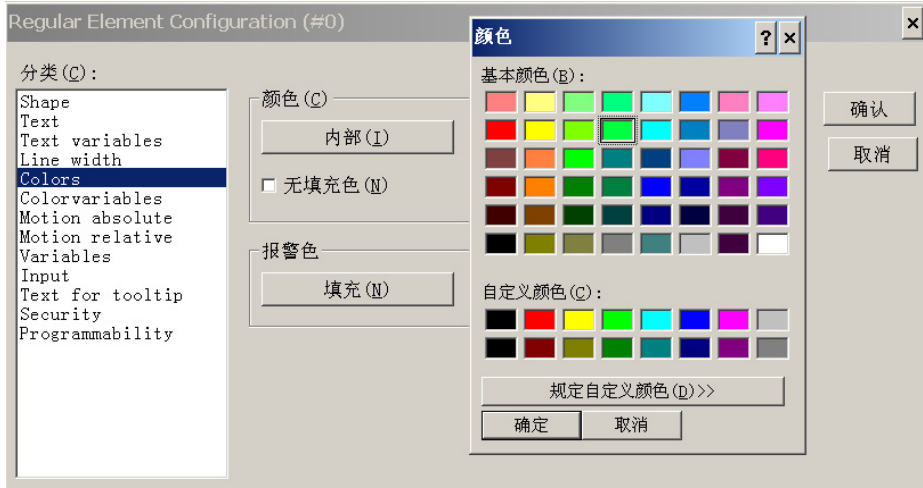


图 11-12-3 颜色设置

在“分类”栏中，选择“Variables”，在“改变颜色变量”选项中，填入信号灯“Q.1”，单击“确定”，如图 11-12-4 所示。同理，在“Q.2”的“改变颜色”项中填入“Q.2”。

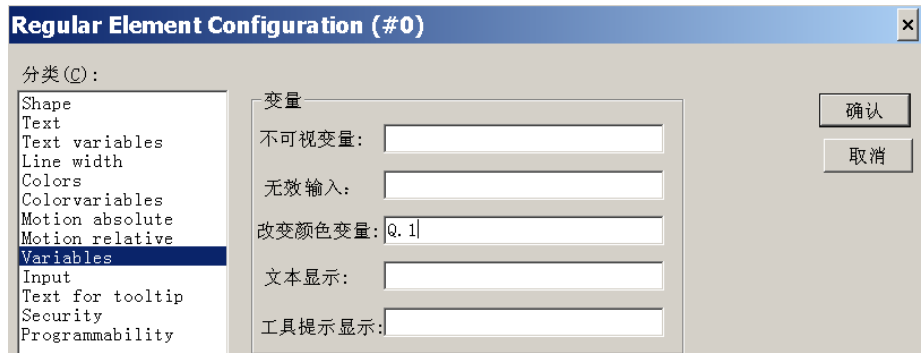


图 11-12-4 变量设置

至此，本程序的视图已经配置完毕。程序的运行情况如图 11-12-5 所示。Q.1 和 Q.2 两信号灯实现红、绿颜色的交替闪烁。

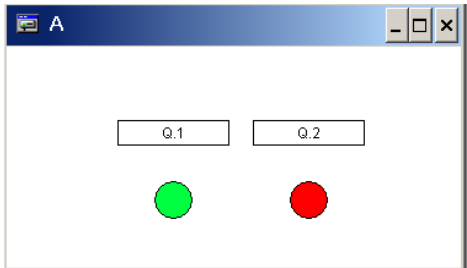


图 11-12-5 程序运行情况

本章小结

本章介绍了视图的开发和应用。

第12章 报警配置

12.1 报警概述

在 PowerPro 软件中集成了报警系统,可以用来检测、记录和查看生产过程的临界状态。

在“资源”选项卡中双击“报警配置”,弹出报警配置对话框,如图 12-1-1 所示。在报警配置对话框中可以定义报警类 Alarm classes 和报警组 Alarm groups。报警类用于定义报警的参数。报警组用于配置具体的报警。鼠标右键点击报警配置对话框中的“系统 System”,可以添加和配置不同的报警组。在视图可以使用报警表来查看和确认报警。在磁盘中可以记录报警文件。

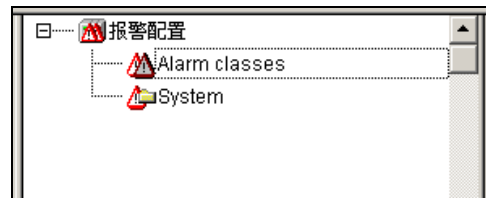


图 12-1-1 报警配置对话框

在 PowerPro 软件中集成了报警系统中通用的一些报警概念。

➤ 报警

变量或表达式的值达到了某一特定的状态,而相应触发一些事件称为报警。

➤ 优先级

优先级描述了报警的严重程度。最高级为“0”,最低级为“255”。

➤ 报警状态

控制报警所配置的变量或表达式的状态称为报警状态。报警状态分为 NORM（没有报警）、INTO（报警发生）、ACK（报警已经发生,并被确认）和 OUTOF（报警状态停止,报警消失,但是报警没有被确认）等几种情况。

➤ 子状态

产生报警的界限值称为报警限。报警限分为（高,低）和（高高,低低）两种,一般记作（Lo, Hi）和（LoLo, HiHi）。例如,表达式的值逐渐上升,第一次通过高限时,产生高报警。如果该值继续上升,在报警确认前通过了高高限,则产生高高报警,而高报警则会被系统自动确认。这里高报警被称为高高报警的子状态。

➤ 报警确认

报警的主要目的是在发生报警的情况下通知用户发生了报警。为了实现这一目的,就有必要确定用户是否注意到了这些信息。为了从报警列表中删除报警,用户必须确认这些报警。

➤ 报警事件

报警事件与报警状态不同。报警状态可以长期有效,报警事件描述了变化的发生,即从正常状态到报警状态的变化。

12.2 报警类

报警类用于对报警的总体参数进行设置，例如报警确认、报警动作和报警颜色等。报警类在报警配置中以全局方式来定义，作为报警组的基本配置。

在报警配置中选择“Alarm classes”，打开报警类对话框，如图 12-2-1 所示。

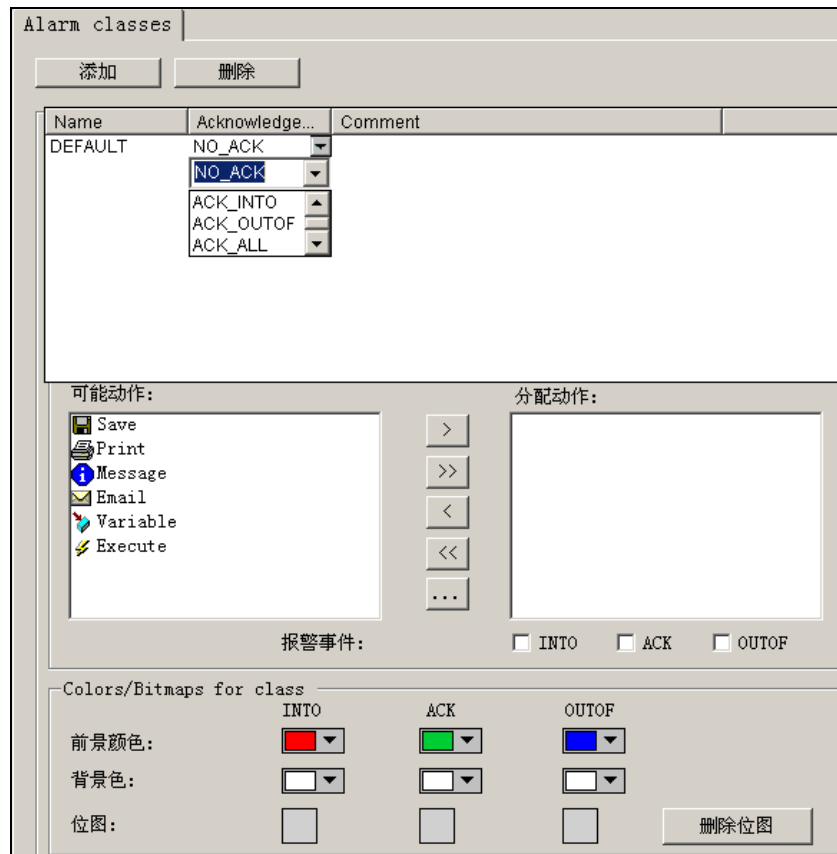


图 12-2-1 报警类对话框

➤ 添加报警类

点击“添加”按钮创建新的报警类。

Name: 在“Name”栏中，点击“DEFAULT”可以更改报警类的名称。

Acknowledgement: 在“Acknowledgement”栏中，点击“NO_ACK”（无确认），会弹出下拉菜单，可在“Acknowledgement”栏中修改确认类型，如图 12-2-2 所示。表 12-2-1 列出了报警确认的类型及含义。

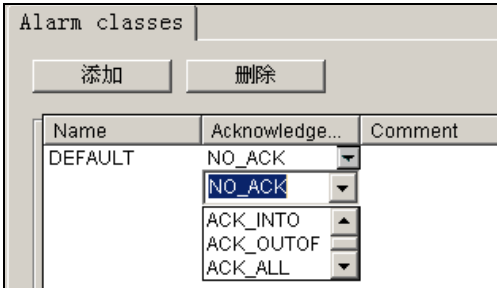


图 12-2-2 添加报警类

表 12-2-1 报警确认类型及含义

报警确认类型	含义
NO_ACK	报警到达，用户不进行确认。
ACK_INT0	报警来到，报警发生，用户需确认此报警。
ACK_OUTOF	报警已完成，用户已处理此报警，报警中止，用户需确认此报警。
ACK_ALL	对于来到的和已过去的报警，用户需要确认。

Comment: 在“Comment”中可以加入相应的注释。例如添加了两个报警类，分别命名为 TempAlarm 和 TempWarn，如图 12-2-3 所示。

Name	Acknowledge...	Comment
Temp Alarm	ACK_ALL	Alarms when temperature is too high
TempWarn	NO_ACK	Warnings when temperature is rising

图 12-2-3 报警类的定义

对于“TempAlarm”报警类，其描述为：当温度太高时报警（Alarm when temperature is too high），需要用户确认。

对于“TempWarn”报警类，其描述为：当温度升高时产生报警（Warning when temperature is rising），无需用户确认。

➤ 删除报警类

点击“删除”按钮，用于删除列表中当前选中的报警类。

➤ 为报警类分配动作

在报警类对话框中可以为报警类分配动作。一旦报警事件发生，便会执行这些动作。

在“可能动作”列表中，用鼠标点击其中一个所要分配的动作，例如“Message”，并按下“>”按钮，此时“Message”动作便被加入到“分配动作”区域，如图 12-2-4 所示。按钮“>>”表示增加所有动作。按钮“<”和“<<”分别表示删除一个和所有动作。选中分配的动作，例如“Message”，点击“...”按钮，或者直接双击“Message”，则会打开相应的对话框，可以输入“Message”的内容，则系统会在报警事件触发时，自动弹出报警提示来显示这些内容。表 12-2-4 给出了报警类所支持的动作。



图 12-2-4 为报警类分配 Message 动作

表 12-2-2 报警类所支持的动作

动作	描述	相应对话框的设置
Save	报警事件保存在报警文件中。此时报警文件必须在报警组配置中定义。	在报警组配置的报警文件对话框中定义报警文件。
Print	将消息文本发送到打印机。	打印机：在本地系统中选择打印机。 输出文本：被打印的消息文本
Message	在当前的报警视图中打开消息窗口，显示定义的文本。	消息：在消息窗口中显示的消息文本。
Email	发送带有消息的 Email。	从：发送者的 Email 地址。 到：接收者的 Email 地址。 主题：Email 主题。 消息：消息文本。 服务器：Email 服务器名称。
Variable	报警变量。	变量：变量名。 布尔型变量：显示报警状态 NORM=0 和 INTO=1。 整型变量：显示报警状态 NORM=0, INTO=1, ACK=2, OUTOF=4。 字符串型变量：显示消息文本。
Execute	当报警事件发生时启动可执行文件。	可执行文件：可执行的文件的名称，例如 notepad.exe， 点击“...”按钮选择可执行文件。 参数：可执行文件的参数。

➤ 定义消息文本

对于动作 Message、Print、Email 或 Variable，可以定义报警产生时需要输出的消息文本。按<Ctrl>+<Enter>组合键换行。在报警消息文本中，可以使用的占位符，如表 12-2-3 所示。占位符必须大写。

表 12-2-3 占位符及其含义

占位符	含义
MESSAGE	报警消息文本。
DATE	报警日期。
TIME	报警时间。
EXPRESSION	产生报警的表达式。
PRIORITY	报警优先级。
VALUE	报警的当前值。
TYPE	报警类型。
CLASS	报警类。
TARGETVALUE	报警类型 DEV+和 DEV-中的目标值。
DEADBAND	报警死区。
ALLDEFAULT	输出报警的所有信息，与报警文件的内容相同。

图 12-2-5 给出了定义报警消息文本的例子。双击分配的 message 动作，弹出对话框后输入报警消息文本内容，点击“确认”按钮即可完成报警消息文本的定义。当报警事件产生时，报警消息显示窗口如图 12-2-6 所示。表 12-2-4 给出了本例中占位符与报警显示内容的对应关系。

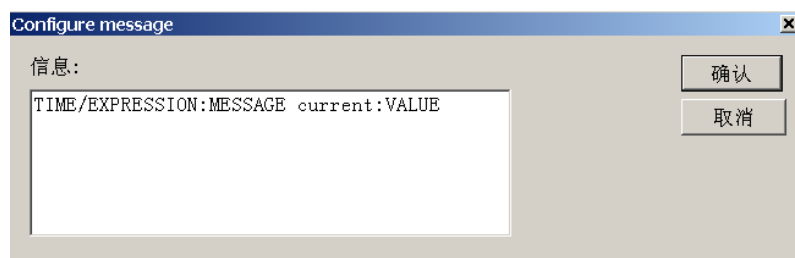


图 12-2-5 定义报警消息

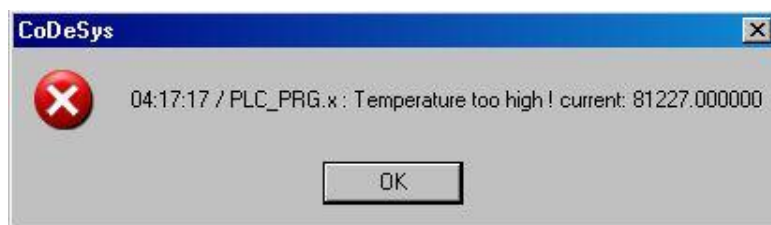


图 12-2-6 报警消息显示窗口

表 12-2-4 占位符与报警显示内容的对应关系

占位符	报警显示内容
TIME	报警时间 04:17:17
EXPRESSION	报警表达式 PLC_PRG.x
MESSAGE	报警消息文本 Temperature too high!
VALUE	报警的当前值 81227.000000

报警消息说明，在 04：17：17 时刻，程序 PLC_PRG 中变量 x 的值为 81227.000000，已经超过允许值，弹出消息文本“Temperature too high!”，表示此时温度太高。

➤ 动作的报警事件

对于每个动作，应当激活其所对应的报警事件。选中动作即可激活其所对应的报警事件。动作的报警事件有以下三个。

INTO：表示报警发生，Status = INTO

ACK：表示用户已经确认报警，Status = ACK

OUTOF：表示报警状态中断，Status = OUTOF

➤ 报警类的颜色与位图

可以给报警类设置用于视图报警表的颜色和位图，即为报警事件 INTO、ACK 和 OUTOF 选择前景颜色和背景颜色。点击颜色图标可以打开颜色对话框，点击灰色方框可以选择位图文件。

12.3 报警组

报警组 Alarm groups 用于组织报警。报警在报警组中配置和管理。在报警组中可以为报警配置变量和参数。可以为报警组建立文件夹。

右键点击报警配置中的“System”，弹出如图 12-3-1 所示的下拉菜单，选择“Append Subelement”中的“Alarm group”，可以添加报警组。

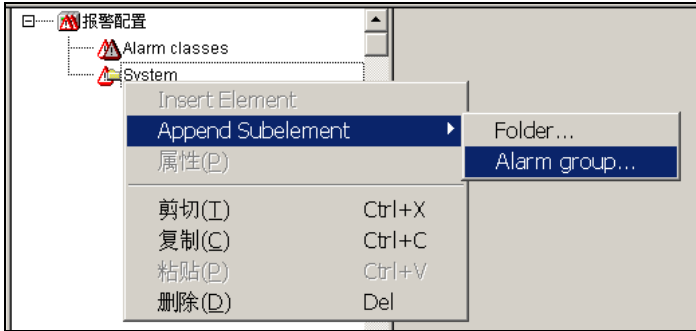


图 12-3-1 添加报警组

点击所添加的报警组，弹出如图 12-3-2 所示的报警组配置对话框，打开“报警组 Alarm group”选项卡，即可配置报警组。

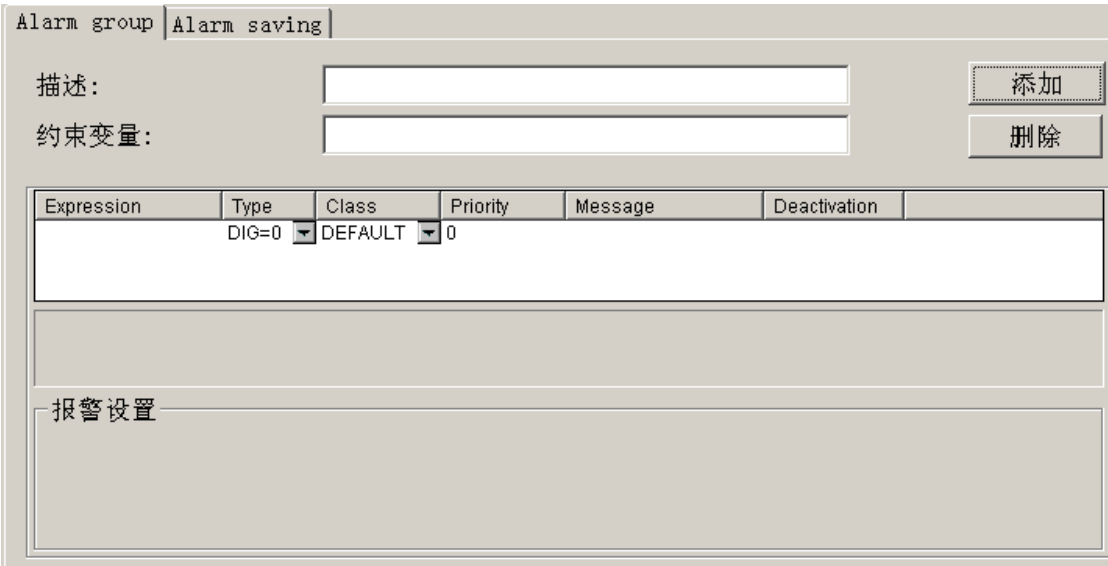


图 12-3-2 报警组配置对话框

➤ 描述

在“描述”后的文本框中输入报警组的名称。

➤ 约束变量

在“约束变量”后的文本框中可以输入布尔型变量。当约束变量的上升沿到达时，报警组中的所有报警无效。当约束变量的下降沿到达时，报警组中的所有报警被重新激活。如果不指定约束变量，则报警处于激活状态。

添加报警组后，报警组配置对话框中会显示表达式 Expression、报警类型 Type、报警类 Class、优先级 Priority、消息 Message、约束 Deactivation 的默认值。

➤ 表达式 Expression

在表达式 Expression 中输入产生报警的工程变量或表达式，可以使用输入助手 F2 功能键。如果输入了未定义的变量名，则编译时消息窗口中会提示“watch expression‘变量名’is not a numeric variable.”

➤ 报警类型 Type

可以从下拉式列表中选择不同的报警类型来定义报警组，如表 12-3-1 所示。

表 12-3-1 报警类型

报警类型 Type	含义
DIG=0	当表达式的值为 FALSE 时，产生数字量报警。
DIG=1	当表达式的值为 TRUE 时，产生数字量报警。
LOLO	低低限。当表达式的值小于低低限时，产生模拟量报警。
LO	低限。当表达式的值小于低限时，产生模拟量报警。
HI	高限。当表达式的值大于高限时，产生模拟量报警。
HIHI	高高限。当表达式的值大于高高限时，产生模拟量报警。
DEV-	负偏差。当表达式的变化量大于负偏差时，产生模拟量报警。
DEV+	正偏差。当表达式的变化量大于正偏差时，产生模拟量报警。
ROC (Rate Of Change)	变化率。当表达式的变化量大于变化率时，产生模拟量报警。

➤ 报警类 Class

可以从下拉式列表中选择已经配置的报警类来定义报警组。

➤ 优先级 Priority

可以设置 0~152 级报警优先级，0 级是最高的报警优先级。在报警表中，报警按照报警优先级的顺序排列。

➤ 消息 Message

可以输入消息文本。当报警产生时，在报警表中显示此消息文本。此消息文本可以记录在报警文件中。

➤ 约束 Deactivation

可以为每个报警类设置约束变量。当约束变量的上升沿到达时，约束报警组中的所有报警。注意，在此处设置的约束变量将会被在“约束变量 Deactivation variable”中设置的约束变量所覆盖。

12.4 报警文件

在报警类中配置“保存”动作后，对于每个报警组可以配置记录报警事件的报警文件。在报警配置对话框中，选择报警组，打开“报警保存 Alarm saving”选项卡，如图 12-4-1 所示，即可配置报警文件。

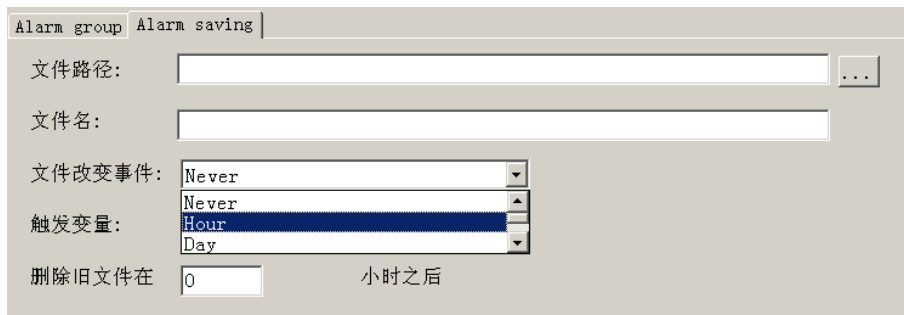


图 12-4-1 报警文件配置对话框

➤ 文件路径

在文件目录中定义报警文件的存储目录。点击按钮“...”，选择存储报警文件的目录。

➤ 文件名

在此处输入保存报警事件的文件名。当报警产生时，系统会自动创建以此文件名为前缀的报警文件，并且在此文件名前缀的后面加上数字，最后加上报警文件的扩展名“.alm”。数字表示报警文件的版本。第一个报警文件所加的数字是 0，以后的每个报警文件由所配置的“文件改变事件”来增加数字，依次为 1、2 等等。例如，如果输入报警文件名“alarmlog”，则报警文件依次为“alarmlog0.alm”、“alarmlog1.alm”等等。

➤ 文件改变事件

在此处配置创建新报警文件的触发事件。

Never: 从不创建新的报警文件

Hour: 每小时创建一个新的报警文件

Day: 每天创建一个新的报警文件

Week: 每星创建一个新的报警文件

Month: 每月创建一个新的报警文件

Variable: 触发变量

Record: 记录

➤ 触发变量

在此处可以配置布尔型变量作为报警文件的触发变量。在程序运行过程中，当触发变量的上升沿到达时，创建新的报警文件。

➤ 删除旧文件

在此处输入删除旧的报警文件的时间间隔。当报警文件的创建时间超过了所设置的时间间隔，系统将自动删除这些报警文件。

12.5 报警举例

本节介绍报警的应用开发过程。

变量声明如图 12-5-1 所示。

0001	PROGRAM PLC_PRG
0002	VAR
0003	bvar1:BOOL;
0004	wVar2:WORD;
0005	wVar3:WORD;
0006	M1: BOOL;
0007	T1: TON;
0008	byteVar4: BYTE;
0009	wVar4: WORD;
0010	M2: BOOL;
0011	END VAR

图12-5-1变量声明

程序编写如图 12-5-2 所示。

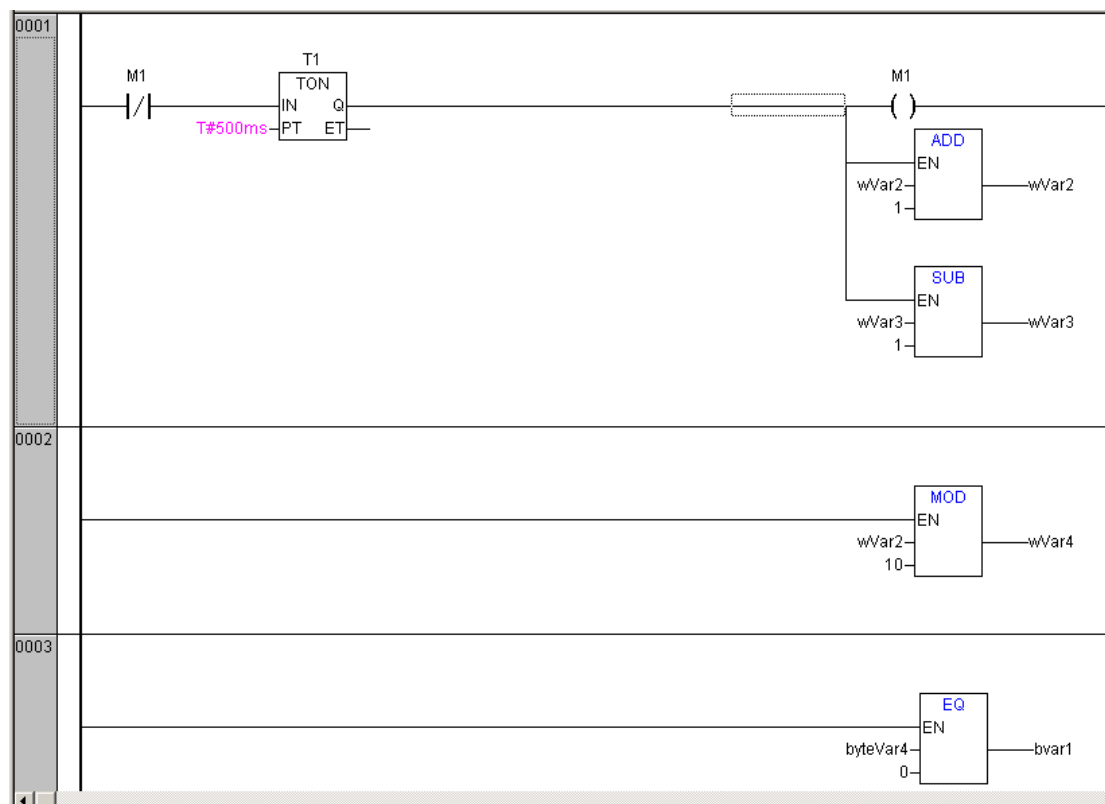


图12-5-2程序编写

视图报警表如图 12-5-3 所示。

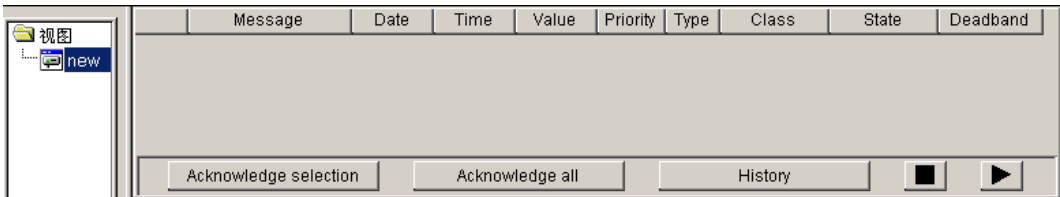


图12-5-3视图报警表

在 Alarm classes 中配置 DEFAULT 和 MYAlarm 两个报警类，如图 12-5-4 所示。MYAlarm 报警类的确认方式选择 ACK_ALL，并给 MYAlarm 报警类分配动作 Message。

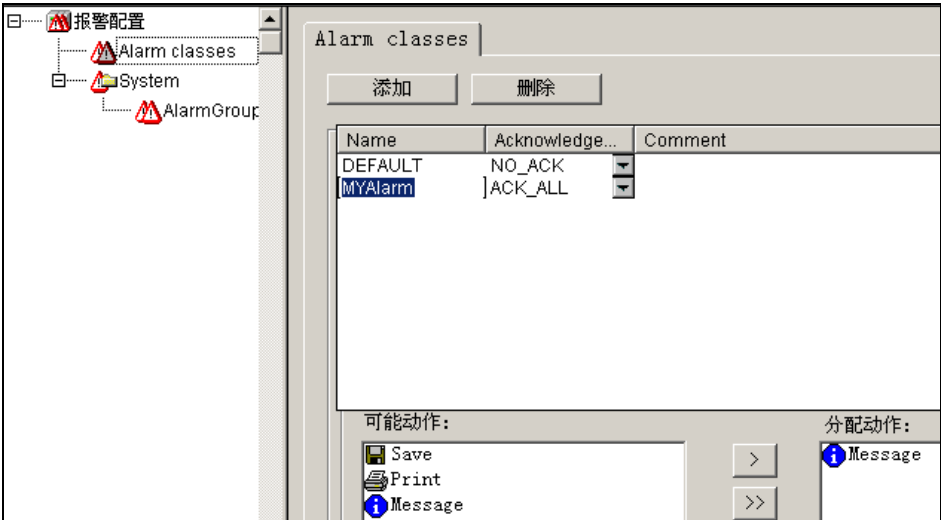


图12-5-4报警类配置

在 Alarm group 中配置报警组，参数配置如图 12-5-5 所示。

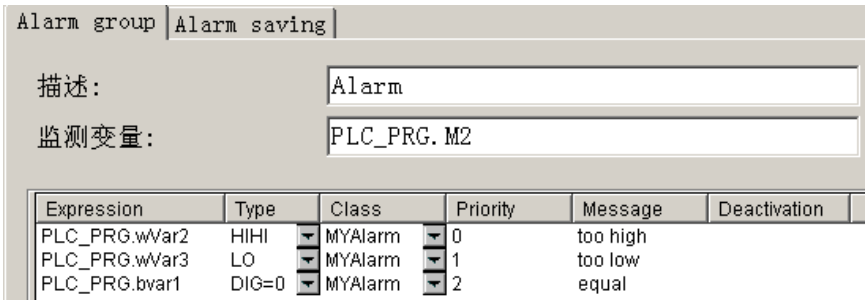


图12-5-5报警组变量参数配置

变量 wVar2 的类型为 HIHI，参数配置如图 12-5-6 所示。变量 wVar2 属于 MYAlarm 类，优先级为 0，显示信息“too high”。设置该变量值为 30。

Alarm type HIHI

Value: 30

图12-5-6变量wVar2

变量 wVar3 的类型为 LO，参数配置如图 12-5-7 所示。变量 wVar3 属于 MYAlarm 类，优先级为 1，显示信息“too low”。设置该变量值为 6500。

Alarm type LO

Value: 6500

图12-5-7 变量wVar3

对于 bvar1，其类型为 DIG=0，属于 MYAlarm 类，优先级为 2，显示信息“equal”。

程序运行后，图 12-5-8 所示为报警表的一种显示情况。

	Message	Date	Time	Value	Priority	Type	Class	State	Deadband
0	too high	20-12-2005	05:04:25	56.00	0	HIHI	MYAlarm	INTO	1
1	too low	20-12-2005	05:04:25	6458.00	1	LO	MYAlarm	INTO	0.2

Acknowledge selection

Acknowledge all

History

图12-5-8报警表显示

 本章小结

本章主要介绍了报警的配置，主要包括报警类、报警组和报警文件等的相关内容，还对报警进行了举例。

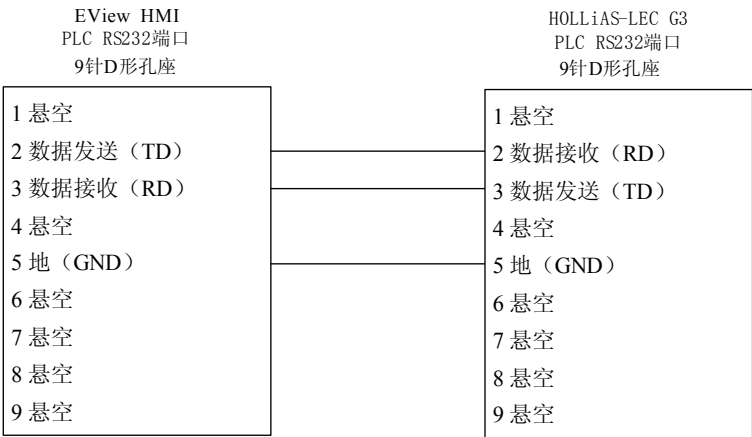
第13章 连接显示屏

HOLLIAS-LEC G3 系列 PLC 可与各种具有标准接口的显示屏进行连接。通过显示屏监视 PLC 的运行状况，并对 PLC 发出控制指令。本章简单描述 HOLLIAS-LEC G3 系列 PLC 与市场上主流的显示屏进行连接和通讯的实现方法。

HOLLIAS-LEC G3 系列 PLC 作为 Modbus 从站与显示屏通讯时，显示屏可读取 PLC 的 I 区、Q 区及 M 区数据，可写（或强制）Q 区及 M 区数据。

13.1 连接 EasyView 显示屏

13.1.1 物理连接



在 PLC 与显示屏进行物理连接后，需在 PLC 侧和显示屏侧分别设置通讯参数和地址。

13.1.2 PLC 侧参数设置

对于 PLC 侧，在编程软件 PowerPro 中将 PLC 的通讯波特率设置为 38400bps，数据位设为 8 位，停止位设为 1 位，奇偶校验设为无。同时，在软件中调用 HollySys_PLC_Ex.lib 库中的功能块 Set_Local_Address，设置作为 Modbus 从站的 PLC 的地址。

注意的是，PLC 的地址不要与显示屏的地址发生冲突。

13.1.3 显示屏侧参数设置

对于显示屏侧，通过软件 EasyBuilder 设置相关参数。通讯参数的设置应与 PLC 侧的设置相匹配。在下图的参数设置画面中，PLC 类型选择为 MODBUS RTU。同样，站地址要与 PowerPro 软件中设置的一致，图 8-1-1 中使用的地址为 51。



图 13-1-1 EasyView 显示屏参数设置界面

13.1.4 地址映射

显示屏侧所选择的 PLC 类型为 Modbus RTU，其地址操作范围为 1—9999，字存取模式。HOLLiAS-LEC G3 PLC 的 I/O 和 M 区地址从 0 开始，采用字存取模式，所以在 EasyBuild 中所添的地址与 HOLLiAS-LEC G3 PLC 中的 I/O 地址和 M 区地址具有一定的映射关系。在 PowerPro 软件中，在 M 区定义字型 and 双字变量时，地址一定是用偶地址。下面详细说明。

➤ 读写 Q/I 区的位数据

EasyBuild 中的位地址 $B = (\text{字节地址} \times 8 + \text{位地址}) + 1$ ，其中字节地址为 PowerPro 中相应于 %Q (I) Xm.n 中的 m，位地址为 PowerPro 中相应于 %Q (I) Xm.n 中的 n。如果想显示地址为 %QX1.1 的通道状态，则 EasyBuild 中的 B 地址为 $1(m) \times 8 + 1(n) + 1 = 10 (m=1, n=1)$ 。

➤ 读写 Q/I 区的字数据

因为 EasyBuild 中的地址是字存取模式，PowerPro PLC 为字节存取模式，所以 EasyBuild 中填写地址时，先要将 PowerPro 中相应通道的实际地址除以 2，转化为字偏移，然后再加 1（EasyBuild 中的起始地址为 1）。

例如在 PowerPro 中 AO 模块 LM3320 第一通道的地址为 QW2，如果想在显示屏上显示该模块第一通道的数值，则需填写该通道对应的实际地址/2（转化为字偏移）+1（补偿偏移），即 2。

➤ 读写 M 区数据

EasyBuild 中对 M 区数据的位存取地址 $B = (\text{字节地址} \times 8 + \text{位地址} + 1) + 3000$ ，其中字节地址为 PowerPro 中相应于 %MXm.n 中的 m，位地址为 PowerPro 中相应于 %MXm.n 中的 n。

对于 M 区的数据，在使用时应注意，MB0~MB99 被系统诊断占用，故用户只能使用 MB100 之后的 M 数据区。

✧ 举例

在 PowerPro 中，如果定义有布尔型变量 bMVar AT%MX32.1:BOOL。现欲在显示屏上读写该变量，则 B 地址为 $B = 32(m) \times 8 + 1(n) + 1 + 3000 = 3258 (m=32, n=1)$ ，即所填地址值为 3258。

EasyBuild 中对 M 区数据的字存取地址 W（字地址）为 PowerPro 中定义的 M 区变量的实际地址/2+1+3000。

✧ 举例

在 PowerPro 中，定义有字变量 wMVar AT%MW100:WORD。现欲在显示屏上读写该变量，则 W 地址为 $100/2 + 1 + 3000 = 3051$ ，故填写地址值 3051。

对 M 区操作时，不论是输入数据，还是输出数据，只要地址相同，则都指向同一数据区。对 M 区的“读”既可在输入数据区完成也可在输出数据区完成，但是如果要“强制”（或“写”）则只能在输出区完成。

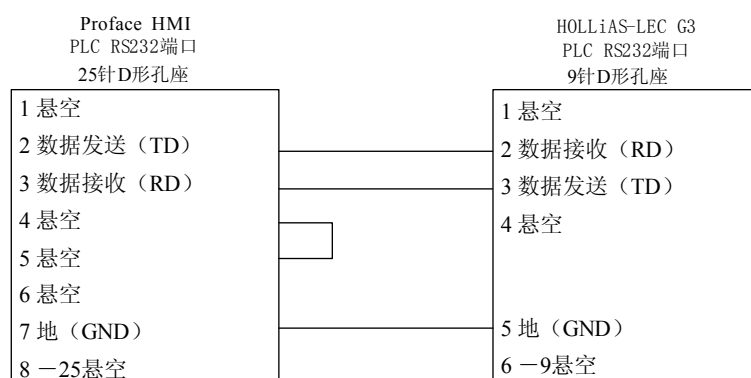
地址映射如表 13-1-1 所示。

表 13-1-1 地址映射表

Modbus RTU 地址类型	Modbus RTU 可操作范围	相应于 PLC 的地址类型	在 PowerPro 目标设置中设置 PLC 的可操作范围	相应于 EasyBuild 中实际可操作范围
0x	1—9999	DO	扩容型：0-4095	1-4096
			非扩容型：0-2047	1-2048
		布尔型 M 区变量 (前 100 个字节被系统占用)	扩容型：800-65536	3801—9999
			非扩容型：800-8192	即实际可访问到的布尔型 M 区的地址为：800-6998
1x	1—9999	DI	扩容型：0-4095	1-4096
			非扩容型：0-2047	1-2048
		布尔型 M 区变量 (前 100 个字节被系统占用)	扩容型：800-65536	3801—9999
			非扩容型：800-8192	即实际可访问到的布尔型 M 区的地址为：800—6998
3x	1—9999	AI	扩容型：0—510W	1-256W
			非为扩容型：0—254W	1—128W
		非布尔型 M 区变量 (前 100 个字节被系统占用)	扩容型：100-8190W	3051 (100/2+1+3000) - 7096 (8190/2+1+3000)
			非扩容型：100—1022W	3051 (100/2+1+3000) - 3512 (1022/2+1+3000)
4x	1—9999	AO	扩容型：0—510W	1-256W
			扩容型：0—254W	1-128W
		非布尔型 M 区变量 (前 100 个字节被系统占用)	扩容型：100-8190W	3051(100/2+1+3000)- 7096 (8190/2+1+300)
			非扩容型：100—1022W	3051 (100/2+1+3000) - 3512 (1022/2+1+3000)

13.2 连接 Proface 显示屏

13.2.1 物理连接



在 PLC 与显示屏进行物理连接后，需在 PLC 侧和显示屏侧分别设置通讯参数和地址。

13.2.2 PLC 侧参数设置

对于 PLC 侧，在编程软件 PowerPro 中将 PLC 的通讯波特率设置为 38400bps，数据位设为 8 位，停止位设为 1 位，奇偶校验设为无。同时，在软件中调用 HollySys_PLC_Ex.lib 库中的功能块 Set_Local_Address，设置作为 Modbus 从站的 PLC 的地址。

注意的是 PLC 的地址不要与显示屏的地址发生冲突。

13.2.3 显示屏侧参数设置

对于显示屏侧，通讯参数的设置与 PLC 侧设置一样。另外，需设置作为从站的 PLC 地址值，此处设置的 PLC 地址值必须与 PLC 编程软件 PowerPro 中所设地址相同。

由于该软件没有专门为 HOLLiAS-LEC G3 PLC 做驱动程序，所以在图 13-2-1 中选择 PLC 类型时，可以选择 CHINO Corp.的 CHINO CONTROLLER (MODBUS)，也可以选择 Schneider 的 PLC 类型 Modbus RTU。由于 Schneider 采用的是相近位字一次连续读取的方式，所以读取效率高，界面刷新快。CHINO Corp.采用每个变量分别读取的方式，效率相对低。选择 Schneider 的 Modbus RTU 时的通讯及数据设置方法与下述的 CHINO Corp.的设置方法相同。

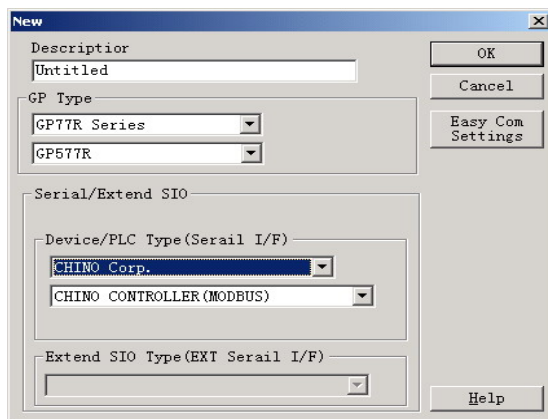


图 13-2-1 Proface 显示屏参数设置界面

对于 GP Setting，有 3 项主要设置。

➤ Communication Settings 通讯参数设置

首先是通讯参数的设定，如图 13-2-2 所示。

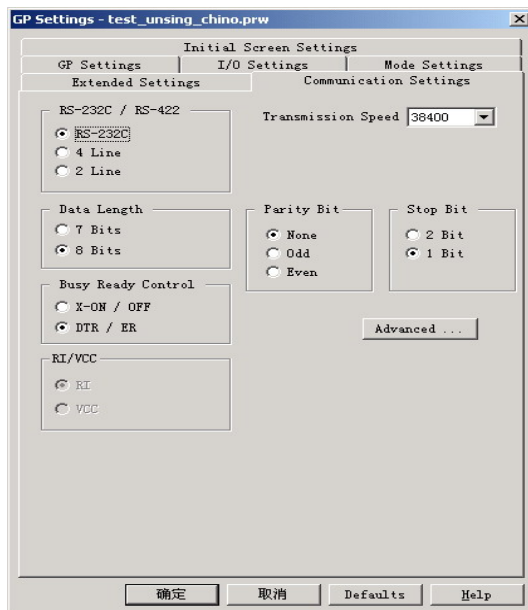


图 13-2-2 Proface 通讯参数设置

➤ Mode Settings 模式设置

其次是模式设置，如图 13-2-3 所示。其中需特别注意的是 System Start Address 和 Machine Number 的设置。System Start Address 是指系统数据域的起始地址，显示屏根据 System Start Address 设定的地址将数据传送给 PLC，所以这个地址一定不要和 PowerPro 中其它的数据地址冲突。Machine Number 即指 PLC 的地址，范围为 1~32。

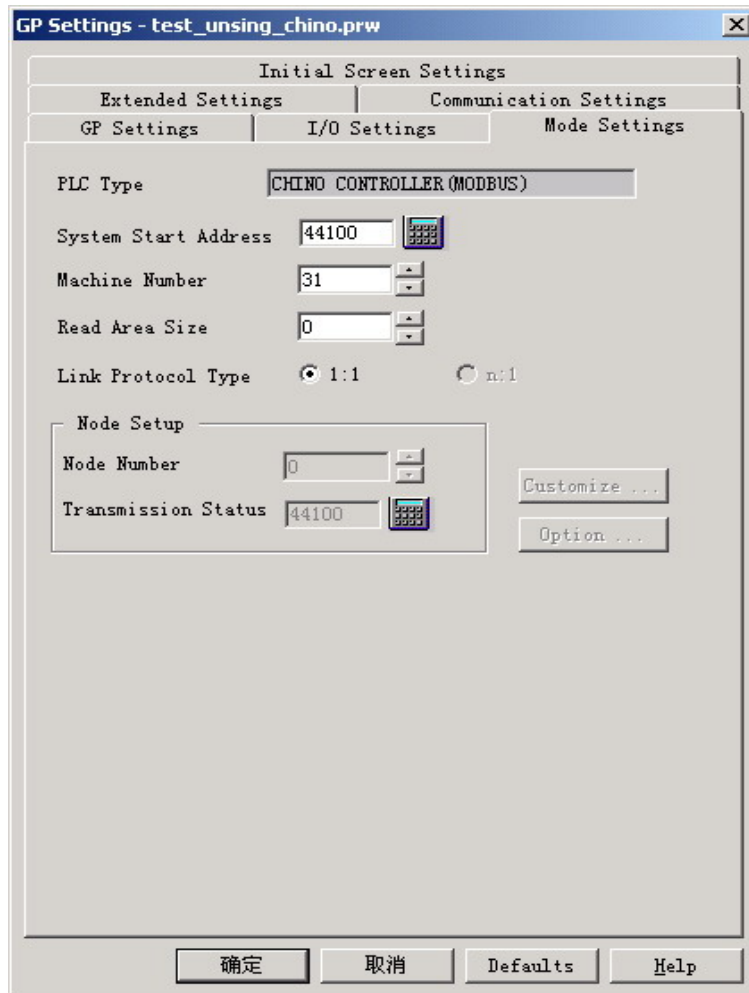


图 13-2-3 Proface 模式设置

➤ Extended Settings 扩展设置

最后是扩展设置，如图 13-4-4 所示。扩展设置中需要注意的是 WatchDog 的设置，同样这个地址一定不要和 PowerPro 中其它的数据地址冲突。

地址映射与 EasyView 显示屏的处理相同，如图 13-4-5 所示。

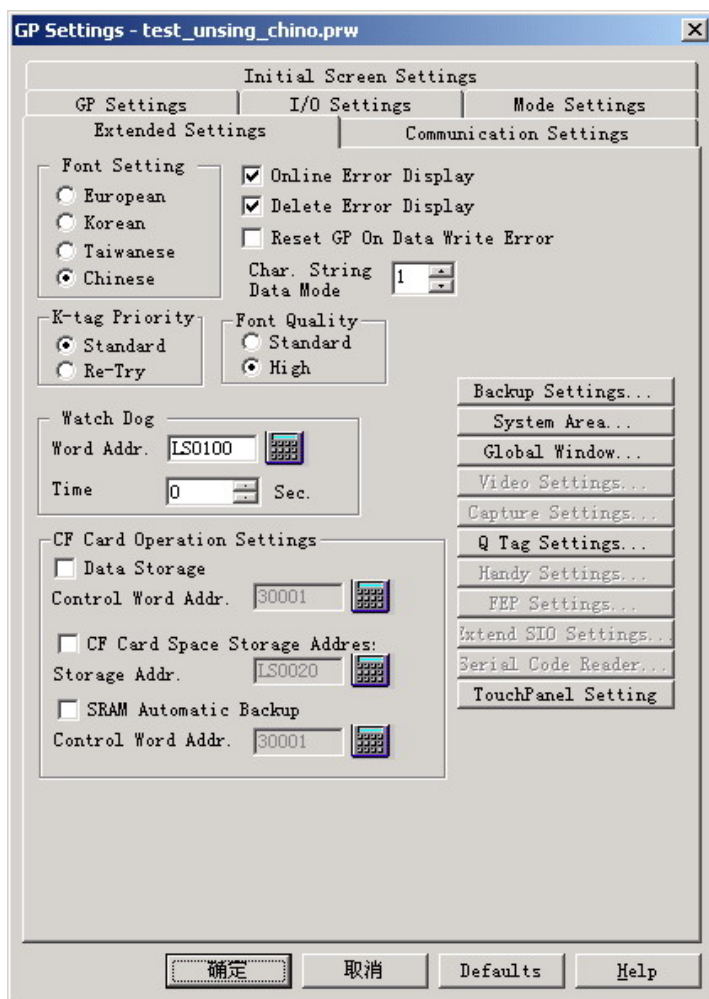


图 13-2-4 Proface 扩展设置

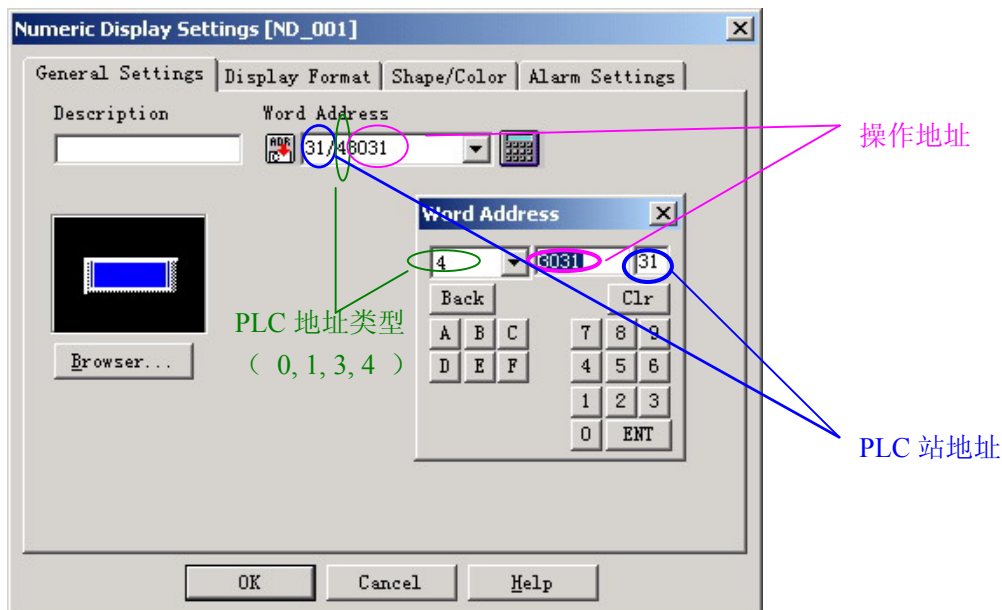
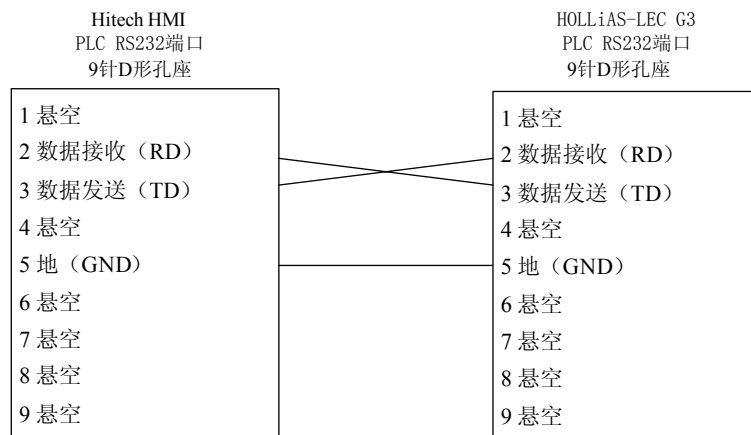


图 13-2-5 Proface 显示屏地址填写说明

13.3 连接 Hitech 显示屏

13.3.1 物理连接



在 PLC 与显示屏进行物理连接后，需在 PLC 侧和显示屏侧分别设置通讯参数和地址。

13.3.2 PLC 侧参数设置

对于 PLC 侧，在编程软件 PowerPro 中将 PLC 的通讯波特率设置为 38400bps，数据位设为 8 位，停止位设为 1 位，奇偶校验设为无。同时，在软件中调用 HollySys_PLC_Ex.lib 中的功能块 Set_Local_Address，设置作为 Modbus 从站的 PLC 的地址。

注意的是，PLC 的地址不要与显示屏的地址发生冲突。

13.3.3 显示屏侧参数设置

对于显示屏侧，通过 Hitech 提供的软件进行设置。首先，在工作参数/应用设定中，如图 13-3-1，选择好所用显示屏的型号，PLC 种类请选择 Modbus Slave。注意，在该界面中的控制区和状态区，不要填写任何数据。

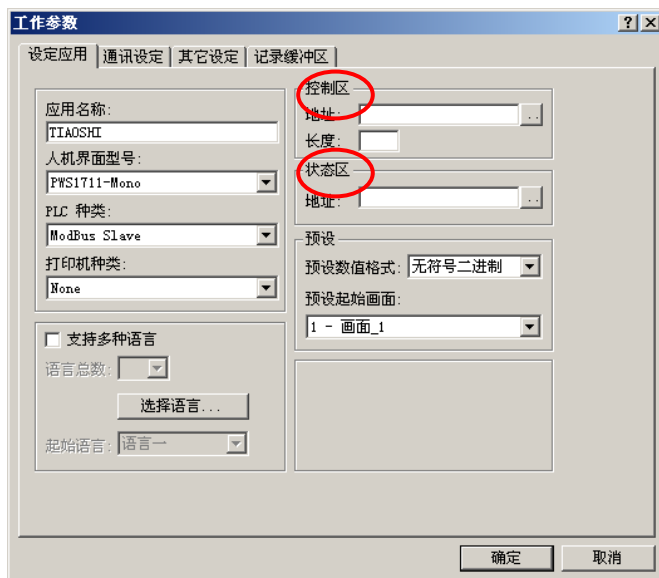


图 13-3-1 Hitech 显示屏设定应用界面

然后，在通讯设定中设置通讯参数，如图 13-3-2 所示。作为从站的 PLC 地址值，此处设置的 PLC 地址值必须与 PLC 编程软件 PowerPro 中所设地址相同，本例中将 PLC 地址设为 51。

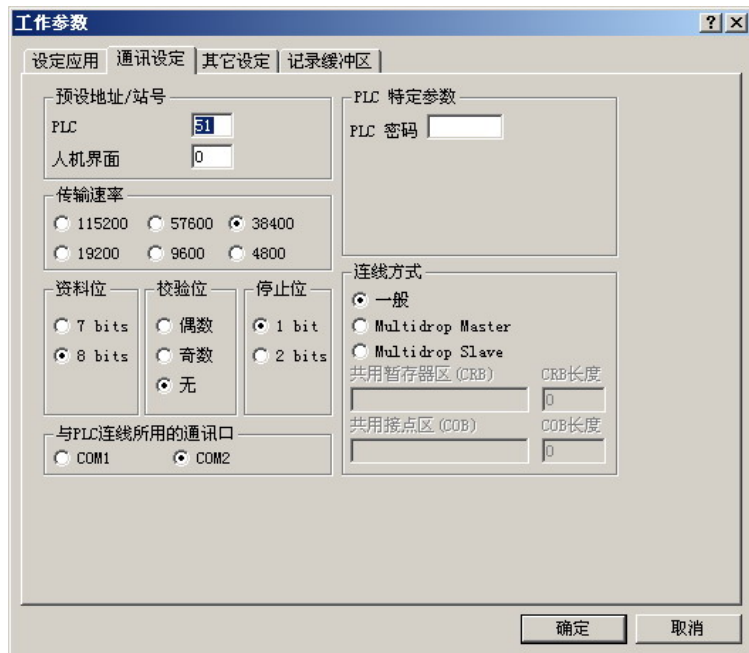
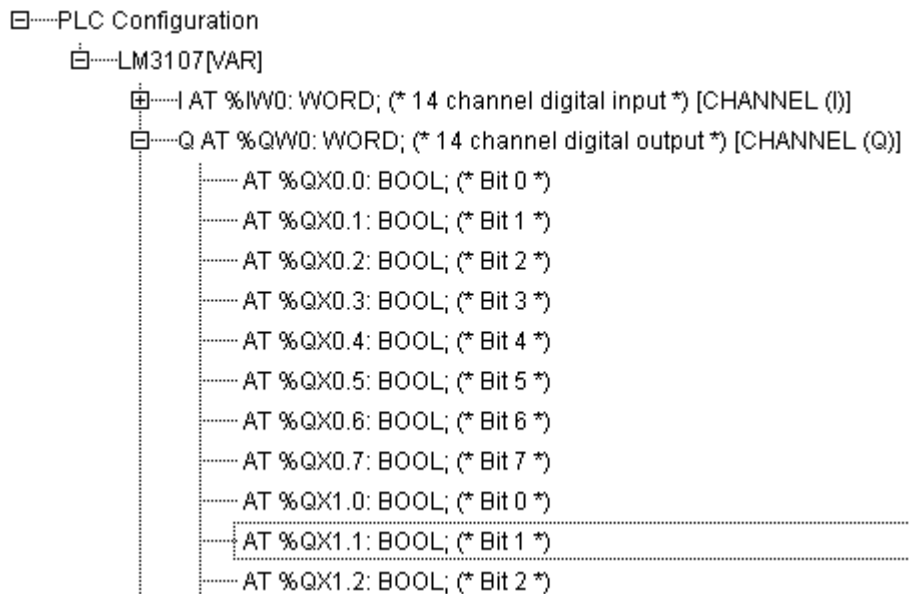


图 13-3-2 Hitech 通讯设定界面

下面介绍显示屏编程软件中地址的填写方法。

➤ 读写 Q/I 区的位数据

DI/DO 的存取地址 B 由式 $B(\text{位地址}) = (\text{字节地址} \times 8 + \text{位地址})$ 得到，其中字节地址为 PowerPro 中相应于 %Q (I) Xm.n 中的 m，位地址为 PowerPro 中相应于 %Q (I) Xm.n 中的 n。例如，想显示地址为 %QX1.1 的通道状态，则地址 $B = 1(m) \times 8 + 1(n) = 9(m=1, n=1)$ 。

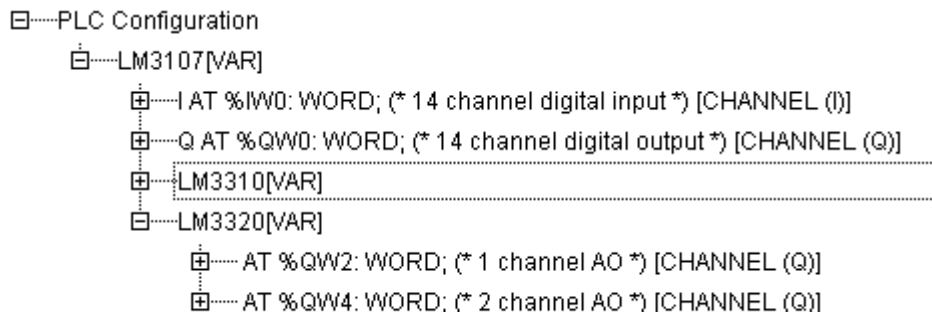


➤ 读写 Q/I 区的字型数据

因为 Hitech 中的地址是字存取模式，PowerPro PLC 为字节存取模式，所以在 Hitech 软件中填写地址时，先要将 PowerPro 中相应通道的实际地址除以 2，转化为字偏移。

◇ 举例

在下图中 AO 模块 LM3320 第一通道的地址为 QW2，第二通道的地址为 QW4。如果想在显示屏上显示第一通道的数值，则只需将该通道对应的实际地址除以 2，换算成字地址即可，填写 1。



➤ 读写 M 区数据

对于 M 区的数据，在使用时应注意，MB0~MB99 被系统诊断占用，故用户只能使用 MB100 之后的 M 区数据。

M 区数据的位存取地址 B（位地址）=（字节地址×8+位地址）+3000，其中字节地址为 PowerPro 中相应于%MXm.n 中的 m，位地址为 PowerPro 中相应于%MXm.n 中的 n。在 PowerPro 中，如果定义有布尔型变量 bMVar AT%MX32.1:BOOL。现欲在显示屏上读写该变量，则 B 地址为 $B=32(m) \times 8 + 1(n) + 3000 = 3257$ （m=32，n=1），即所填地址值为 3257。

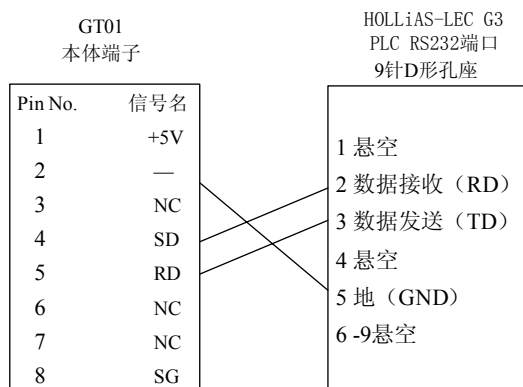
对于 M 区的数据，在 Hitech 软件中同样需要注意要将 PLC 中的 M 区的字节存取地址转化为字偏移地址，即 PowerPro 中定义的 M 区变量的实际地址先除以 2，转化为字偏移，再加 3000，得到在 Hitech 软件中所填地址值。

◇ 举例

在 PowerPro 中，定义有字变量 wMVar AT%MW100:WORD。现欲在显示屏上读写该变量，则 W 地址为 $100/2 + 3000 = 3050$ ，故填写地址值 3050。

13.4 连接 GT01 显示屏

13.4.1 物理连接



在 PLC 与显示屏进行物理连接后，需在 PLC 侧和显示屏侧分别设置通讯参数和地址。

13.4.2 PLC 侧参数设置

对于 PLC 侧，在编程软件 PowerPro 中将 PLC 的通讯波特率设置为 38400bps，数据位设为 8 位，停止位设为 1 位，奇偶校验设为无。调用 Hollysys_PLC_EX.lib 库中的功能块 Set_Local_Address，设置 PLC 的地址，与 GT01 中地址范围一致，设为 1~247 的数值。

13.4.3 显示屏侧参数设置

第一步，新建工程，进行模块选择，GT 模块选择 GT01，PLC 模块选择为 Modbus (RTU Mode)，如图 13-4-1 所示。

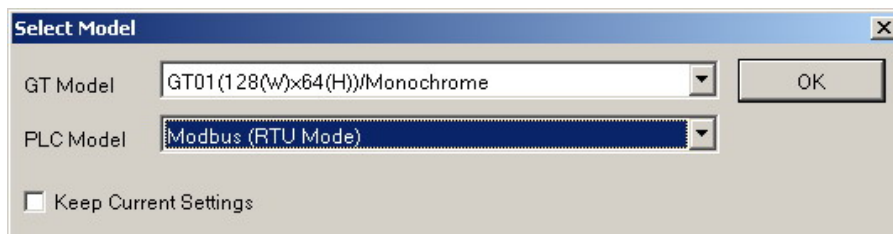


图 13-4-1 模块选择界面

第二步，进行配置项的设置。选择菜单 File->Configuration->GT Configuration 对通讯参数进行设置（GTWIN Configuration 是对该软件的一些设置，可以取默认值）。

GT 设置中，首先进行基本设置，如图 13-4-2 所示，将与 PLC 通讯的字区域(Word Area)和位区域 (Bit Area) 设置为本体区域 (WGR)，主要用来画面切换等功能的实现。如果 Device Setting 设置项中选择 0 或 4，则由设置地址指定的 PLC 中相应区域被占用，GT01 会对其进行读写操作，所以这种情况下用户一定要将此区域的地址选择好，避免读写重复。

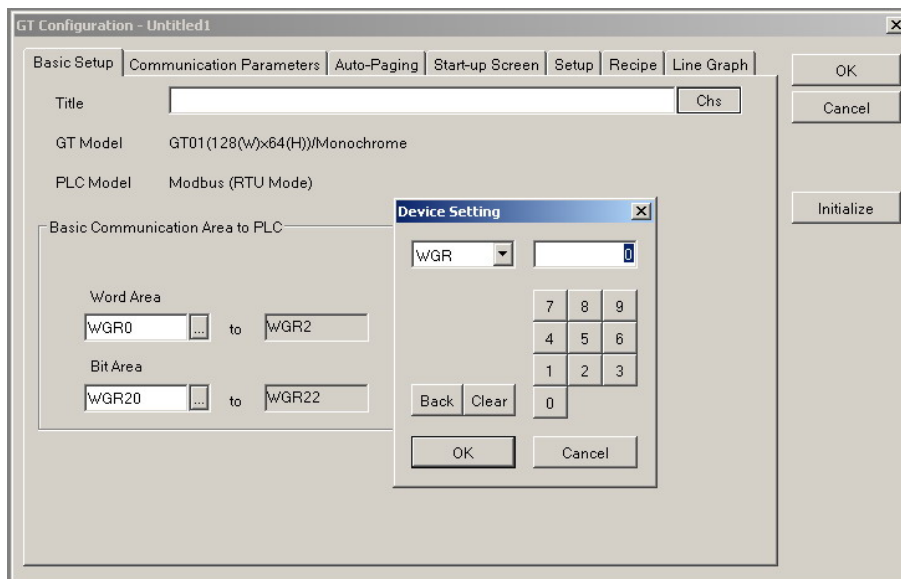


图 13-4-2 基本设置

然后要对通讯参数进行设置。此部分设置包括三部分内容：PLC 的设备地址（PLC UnitNo，范围 1~247）、与 PLC 进行通讯的参数设置（COM Port）、与上位软件通讯的参数设置（Tool Port）。PLC 的设备地址和 PLC 的通讯参数只要与 PowerPro 软件中设置一致即可，与上位软件的通讯设置随意，具体设置如图 13-4-3 所示。

其余各项设置参考 GT01 技术手册。

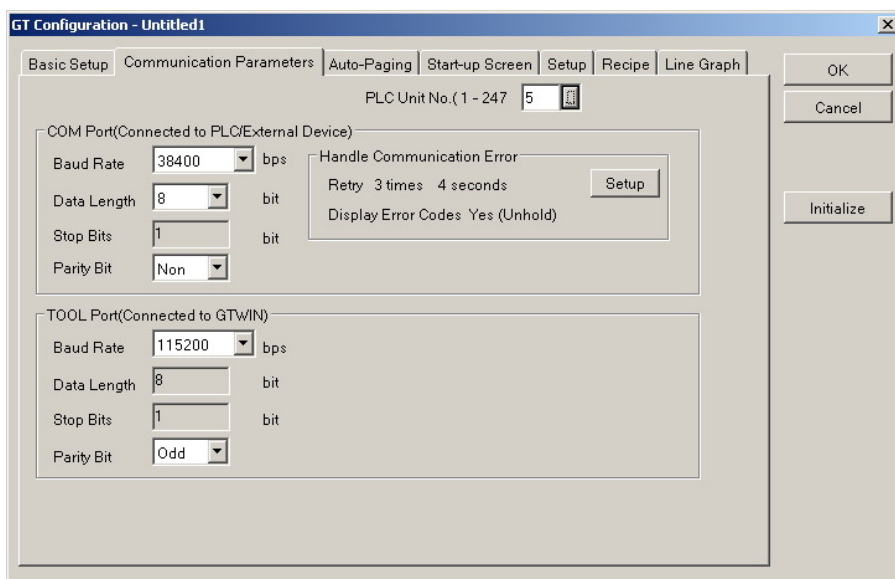


图 13-4-3 通讯参数设置

第三步，读写地址的填写方法。

数据地址类型分为四种：0、1、3、4

0：读写开关量输出、M 区位数据

1：读取开关量输入、M 区位数据

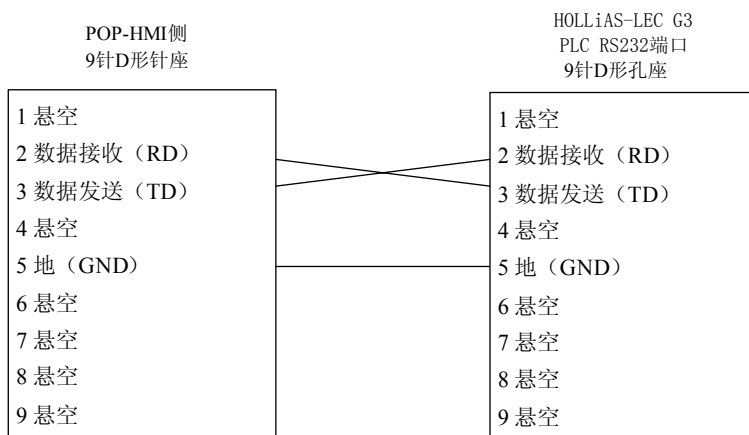
3：读取模拟量输入、M 区数据

4：读写模拟量输出、M 区数据

具体地址的填写方法与 Hitech 显示屏相同。

13.5 连接 POP 显示屏

13.5.1 物理连接



在 PLC 与显示屏进行物理连接后，需在 PLC 侧和显示屏侧分别设置通讯参数和地址。

13.5.2 PLC 侧参数设置

对于 PLC 侧，在编程软件 PowerPro 中将 PLC 的通讯波特率设置为 38400bps，数据位设为 8 位，停止位设为 1 位，奇偶校验设为无。同时，在软件中调用 HollySys_PLC_Ex.lib 中的功能块 Set_Local_Address，设置作为 Modbus 从站的 PLC 的地址。

13.5.3 显示屏侧参数设置

对于显示屏侧，主要在“工具/用户配置”中进行通讯参数的设置，如图 13-5-1 所示。其中关键设置为：设备类型，要选择“Modbus RTU Slave”；波特率：38400；数据位：8 Bits；停止位：1 Bits；校验方式：无；设备站号：请选择和 PLC 侧相同的设备站号。连接方式：单台设备。

组态软件中的地址映射关系与 EasyView 触摸屏相同。

图 13-5-1 用户设备属性设置

程序写好后，在下装前，要在“工具/下载配置”中根据 PC 机与显示屏相连的串口，选择好通讯端口，如图 13-5-2 所示。



图 13-5-2 下载端口配置



本章小结

本章主要介绍了 PLC 可与各种具有标准接口的主流显示屏进行连接的通信参数设置。在工程应用中，常常采用显示屏监视 PLC 的运行状况，并对 PLC 发出控制指令。

第14章 程序实例

14.1 LD 程序实例

本节介绍 LD 程序实例。

本程序的功能是控制 8 个彩灯实现循环移位闪烁。硬件只需配置一个 CPU 模块 LM3107 即可实现此功能。8 个彩灯的 8 位循环移位由输出点 QX0.0-QX0.7 即 QB0 控制。彩灯是否移位由输入点 IX0.0 控制。彩灯移位方向由输入点 IX0.1 来控制。首次运行时彩灯的初始值设置为 5。变量声明如图 14-1-1 所示。LD 程序如图 14-1-2 所示。

0001	PROGRAM PLC_PRG
0002	VAR
0003	SM01: BOOL;
0004	T1: TON;
0005	ET: TIME;
0006	END_VAR

图14-1-1变量声明

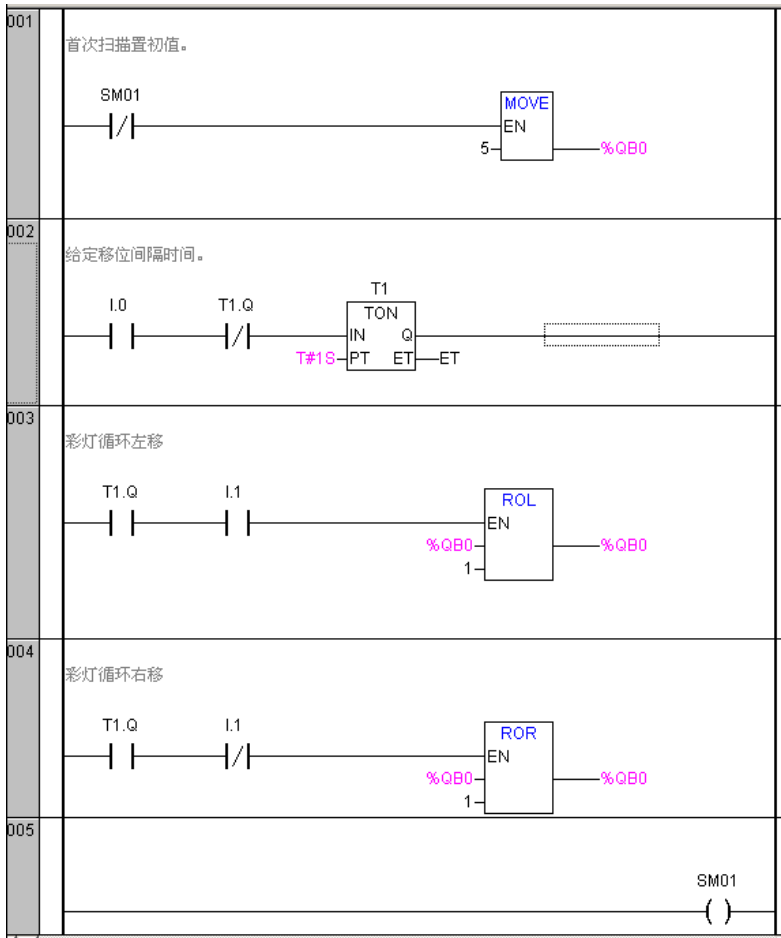


图14-1-2 LD程序

程序编译通过后，将程序下装到 PLC 中。程序运行时的 PLC 配置栏如图 14-1-3 所示。

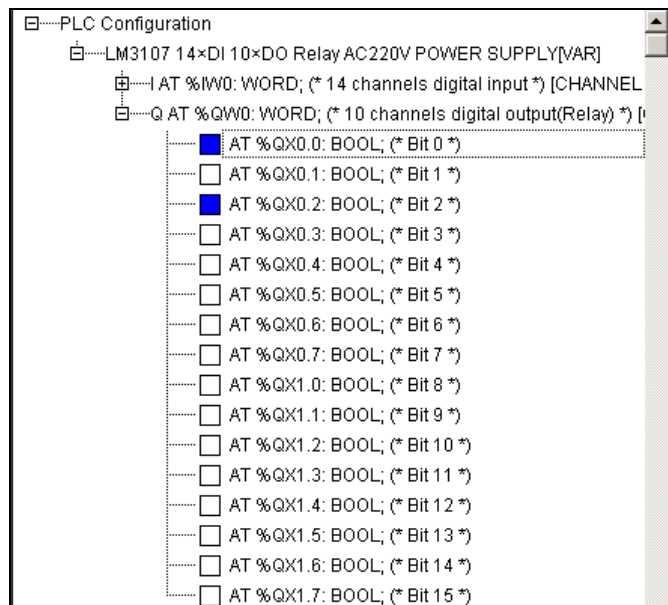


图14-1-3 程序运行时的PLC配置栏状态

给输入点 IX0.0 强制值后程序开始运行，且彩灯循环右移，从高位移至低位。在程序段 ROR 处，状态变化为：

```
10100000→  
01010000→  
00101000→  
00010100→  
00001010→  
00000101→  
10000010→  
01000001→  
10100000→
```

回到初始状态后继续循环右移。

在 PLC 配置栏处可以看到彩灯对应输出点处的点亮和熄灭情况。同时也可以看到 PLC 指示灯相应的点亮和熄灭情况。

14.2 FBD 程序实例

本节介绍 FBD 程序实例。

本程序的功能是控制 5 个彩灯实现有规律的闪烁。首先 I1、I2、I3、I4、I5 等 5 个彩灯依次点亮，且点亮后一直保持状态，直到 I5 点亮。然后从 I3 开始熄灭，1s 后 I2 和 I4 同时熄灭，再过 1s 后 I1 和 I5 同时熄灭。随后 I1、I2、I3、I4、I5 依次单独点亮，最后全熄灭，再进行往复循环。

硬件只需配置一个 CPU 模块 LM3107 即可实现此功能。

变量声明如图 14-2-1 所示。

FBD 程序如图 14-2-2、图 14-2-3 和图 14-2-4 所示。

视图如图 14-2-5 所示。

	VAR	VAR_INPUT	VAR_OUTPUT	VAR_IN_OUT	CONST
	名称	地址	类型	初始值	注释
0001	t1		TOF		
0002	on		BOOL		
0003	t2		TON		
0004	t3		TON		
0005	t4		TON		
0006	t5		TON		
0007	t6		TON		
0008	t7		TON		
0009	t8		TON		
0010	t9		TON		
0011	ta		TON		
0012	t11		TON		
0013	t13		TON		
0014	t14		TON		
0015	t15		TON		
0016	t16		TON		
0017	t17		TON		
0018	t18		TON		
0019	t19		ton		
0020	t20		ton		

图14-2-1变量声明

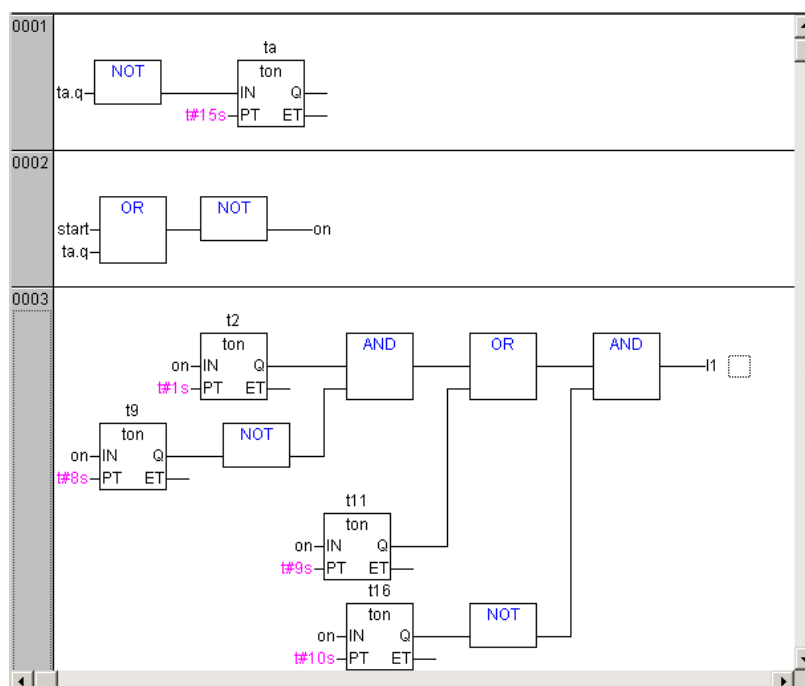


图14-2-2 FBD程序（0001-0003）

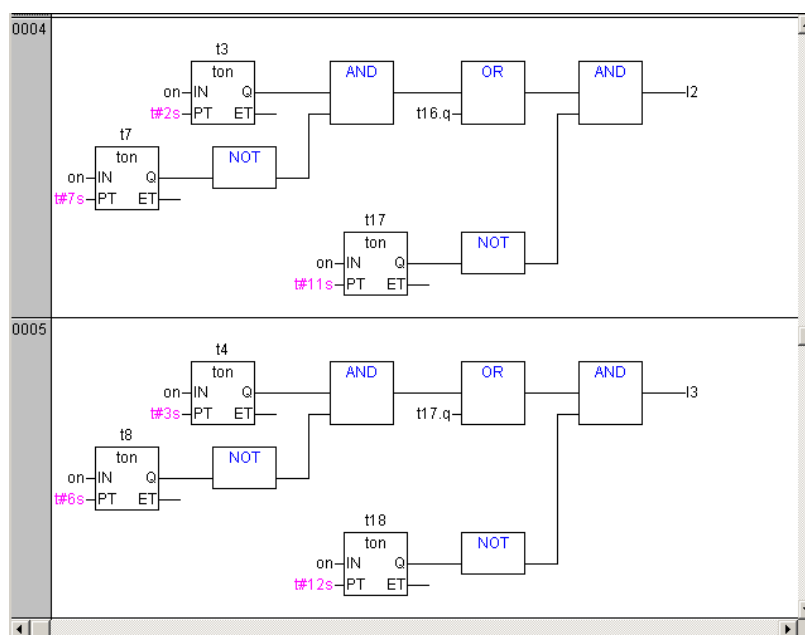


图14-2-3 FBD程序（0004-0005）

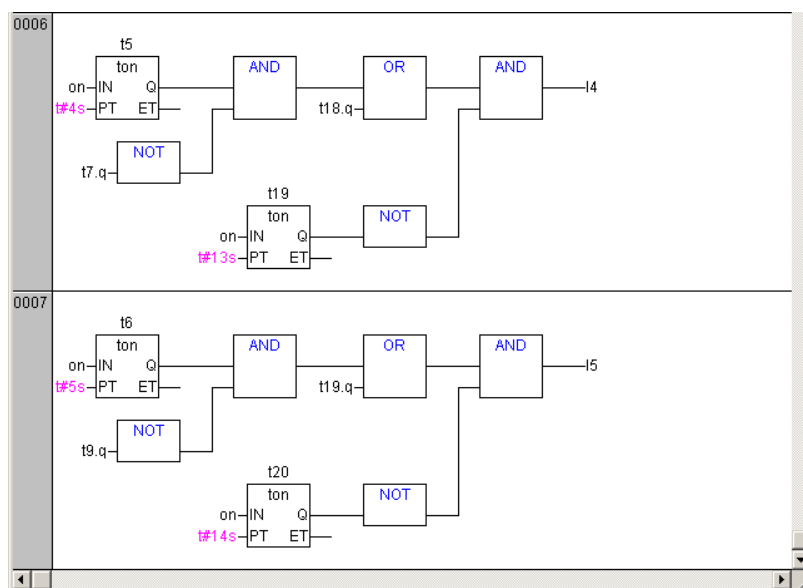


图14-2-4 FBD程序（0006-0007）

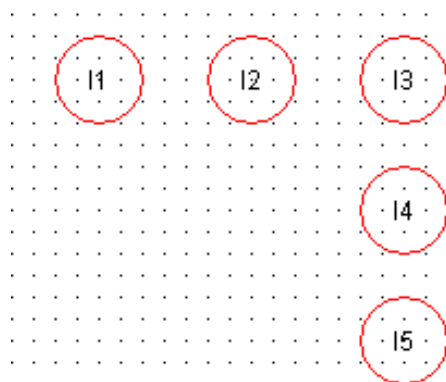


图14-2-5 视图

14.3 SFC 程序实例

本节介绍 SFC 程序实例。

本程序的功能是控制人行横道的交通信号灯。在一条单向道路上设置一个横过道路的人行横道，要求用交通信号灯自动控制，无人值守。在机动车道上设置红、黄、绿 3 个信号灯，在人行横道上设置红、黄、绿 3 个信号灯。红灯表示停止，黄灯表示准备，绿灯表示通行。

当有行人要求通过人行横道时，行人可按下通行按钮进行请求，机动车道上的信号灯则会作出相应的反应，执行相应的动作。

硬件只需配置一个 CPU 模块 LM3107 即可实现此功能。

变量声明如图 14-3-1 所示。

SFC 程序如图 14-3-2 所示。为了便于理解 SFC 程序，在相应步的右侧加入了注释语句。

视图如图 14-3-3 所示。视图对象的参数设置如表 14-3-1 和表 13-3-2 所示。

程序的运行状态如图 14-3-4、图 14-3-5、图 14-3-6、图 14-3-7 和图 14-3-8 所示。

0001	PROGRAM PLC_PRG
0002	VAR
0003	X1:BOOL;
0004	M1:BOOL;
0005	M2:BOOL;
0006	M3:BOOL;
0007	M4:BOOL;
0008	M5:BOOL;
0009	M6:BOOL;
0010	M7:BOOL;
0011	M8:BOOL;
0012	END_VAR

图14-3-1 变量声明

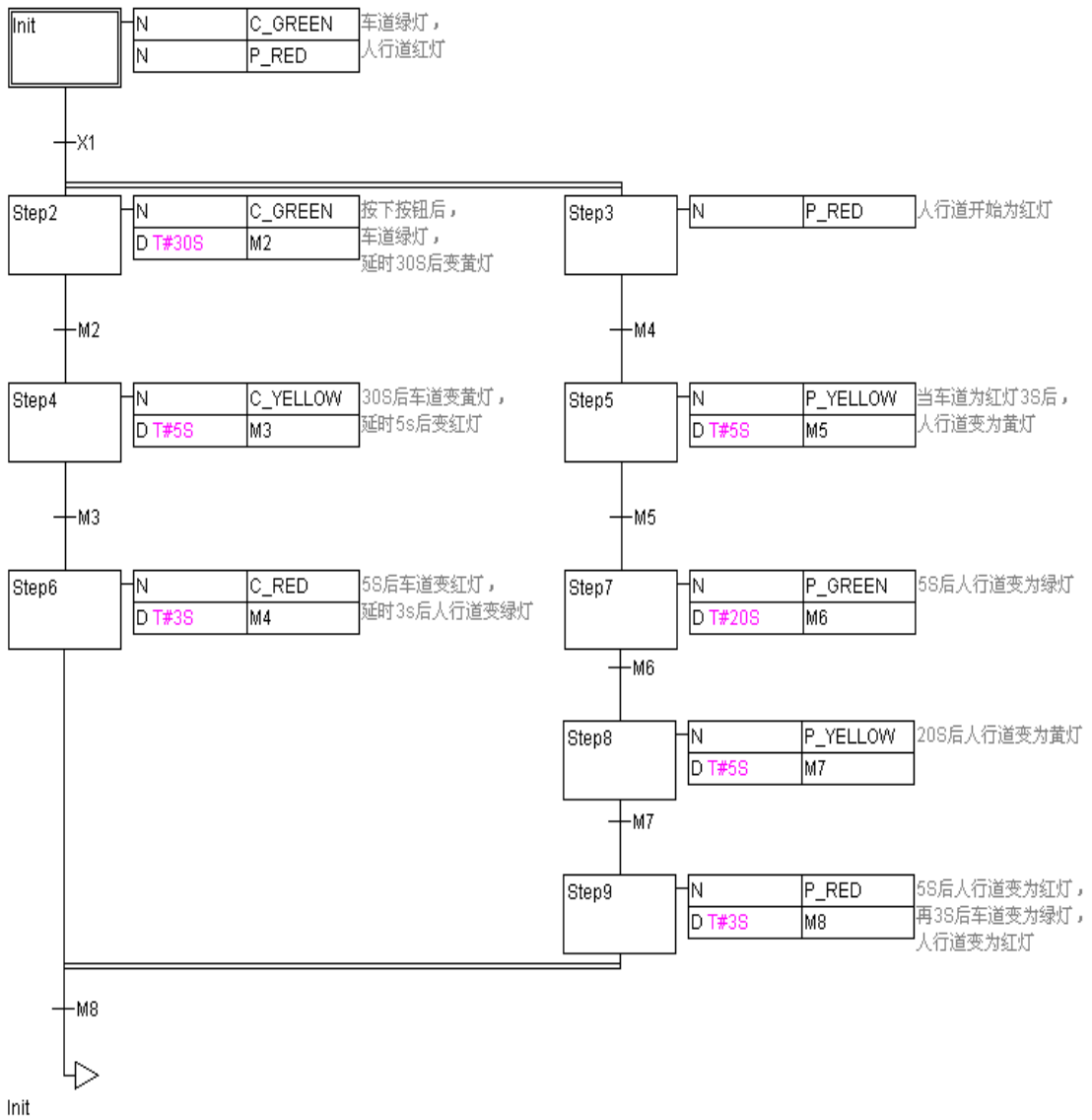


图14-3-2 SFC程序

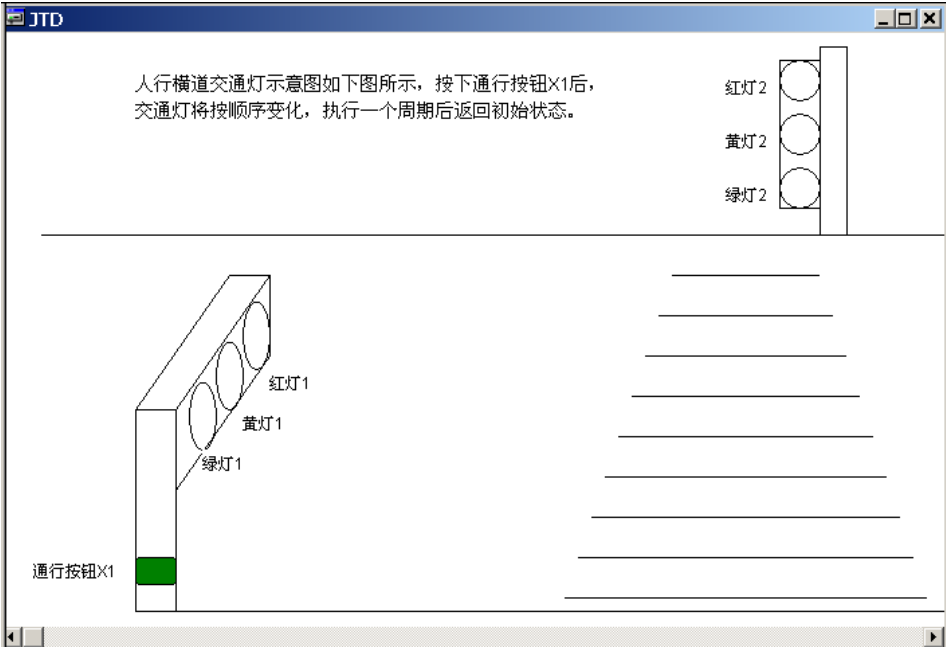


图14-3-3 视图

表 14-3-1 通行按钮 X1 参数设置

图形对象	属性设置	
	Input/输入/切换变量	Colors/颜色
通行按钮 X1	PLC_PRG.X1	绿色

表 14-3-2 交通信号灯参数设置

图形对象	属性设置	
	Variables/改变颜色变量	Colors/颜色
绿灯 1	C_GREEN	无填充色
黄灯 1	C_YELLOW	无填充色
红灯 1	C_RED	无填充色
绿灯 2	P_GREEN	无填充色
黄灯 2	P_YELLOW	无填充色
红灯 2	P_RED	无填充色

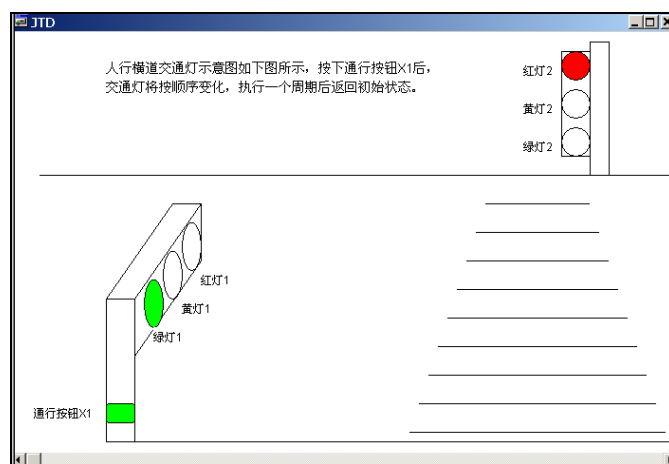


图14-3-4 运行状态1

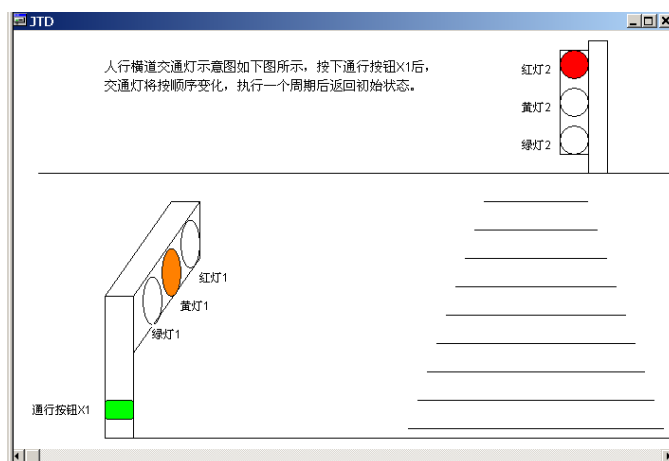


图14-3-5 运行状态2

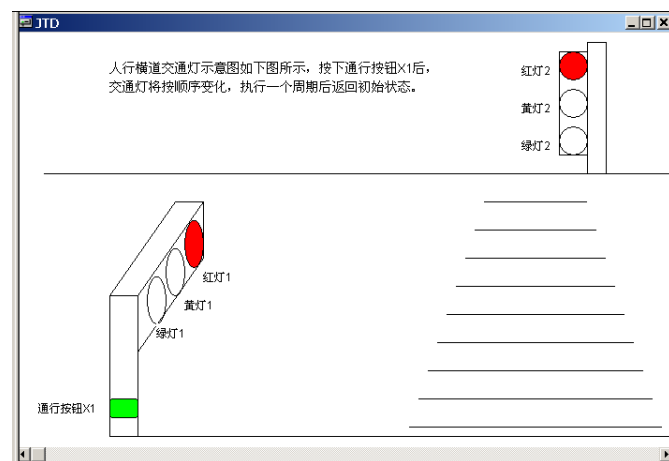


图14-3-6 运行状态3

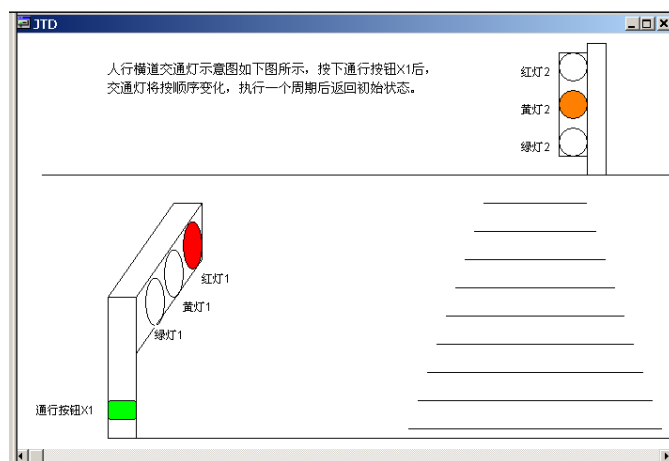


图14-3-7运行状态4

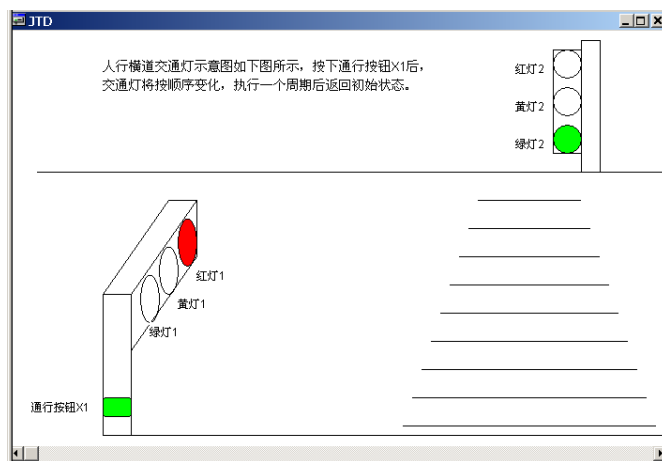


图14-3-8运行状态5



本章小结

本章主要介绍了 LD、FBD 和 SFC 的程序实例。

第15章 附录

15.1 PLC 的基本概念

15.1.1 PLC 的工作过程

PLC 以扫描方式工作。所谓扫描是指 CPU 连续执行用户程序和任务的循环过程。PLC 的工作过程一般可以分为输入采样、程序执行和输出刷新等三个阶段，如图 15-1-1 所示。

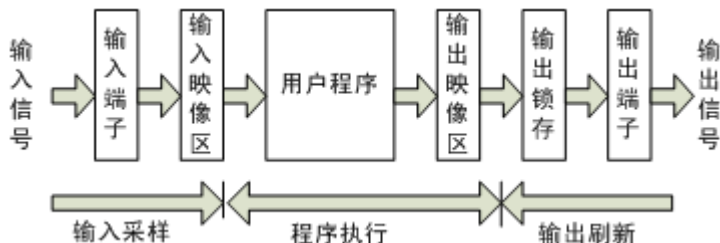


图15-1-1 PLC工作过程

➤ 输入采样阶段。

PLC 以扫描工作方式，按顺序将所有信号读入到寄存输入状态的输入映像寄存器中存储，这一过程称为采样。在本工作周期内，此采样结果的内容不会改变，而且采样结果将在 PLC 执行程序时被使用。

➤ 程序执行阶段。

PLC 按顺序对程序进行扫描，即从上到下和从左到右地扫描每条指令，并分别从输入映像寄存器和输出映像寄存器中获取所需的数据，进行运算和处理。再将程序执行结果写入寄存执行结果的输出映像寄存器中保存。注意，在整个程序未执行完毕之前，程序执行结果不会送到输出端口上。

➤ 输出刷新阶段。

在执行完所有用户程序后，PLC 将映像寄存器中的内容送入到寄存输出状态的输出锁存器中，再去驱动用户设备，这就是输出刷新。

PLC 重复执行上述三个阶段。每重复一次的时间称为一个扫描周期。在一个扫描周期中，PLC 的输入扫描时间和输出刷新时间一般小于 1ms，而程序执行时间因程序的长度不同而不同。PLC 的一个扫描周期一般在几十毫秒之内。

PLC 的一个工作扫描周期主要分为上述三个阶段。但是严格来说还应当包括下述三个过程，这三个过程都是在输入扫描过程之后进行的。

系统自检测。检查程序执行是否正确，如果超时则停止 CPU 工作。

与编程器 PowePro 交换信息。在使用编程器输入和调试程序时才执行这一过程。

网络通信。当 PLC 配置有网络通信模块时，与通信对象进行数据交换。

当 PLC 投入运行后, 重复完成以上三个阶段的工作, 即采用循环扫描工作过程, 如图 15-1-2 所示。PLC 工作的主要特点是输入信号集中批处理、执行过程集中批处理和输出控制集中批处理。PLC 的这种“串行”工作方式, 可以避免继电器-接触器控制系统中触点竞争和时序失配的问题, 这是 PLC 可靠性高的原因之一。但是, 循环扫描工作过程会导致输出相对输入在时间上的滞后, 这是 PLC 的缺点之一。

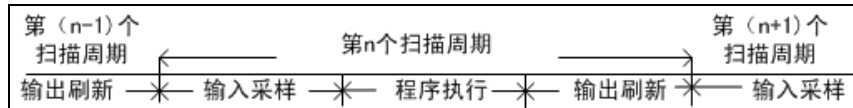


图15-1-2 PLC扫描工作方式

PLC 在执行程序时所使用的状态值不是直接从实际输入端口获得的, 而是来源于输入映像寄存器和输出映像寄存器。输入映像寄存器的状态值取决于上一个扫描周期从输入端子采样取得的数据, 并在程序执行阶段保持不变。输出映像寄存器中的状态值取决于执行程序输出指令的结果。输出锁存器中的状态值是上一个扫描周期的刷新阶段从输出映像寄存器转入的。

还需指出一点, 在 PLC 中经常采用一种称之为“看门狗”(Watch dog)的定时监视器来监视 PLC 的实际工作周期是否超出预定的时间, 以避免 PLC 在执行程序的过程中进入死循环, 或 PLC 执行非预定的程序而造成系统瘫痪。

15.1.2 CPU 的扫描周期

PLC 重复执行上述三个阶段, 每重复一次的时间称为一个扫描周期。在扫描周期中, CPU 执行下列任务。

- 读输入

每次扫描周期开始时, 先读输入点的当前值, 然后把这些值写到输入映像寄存器中。

- 执行程序

在扫描周期的执行程序阶段里, CPU 从第一条指令开始执行, 直到最后一条指令结束为止。注意, 如果在程序中使用中断, 则中断程序并不作为扫描周期的一部分, 只有当中断事件发生时才执行中断程序。

- 处理通讯请求

在扫描周期的信息处理阶段, CPU 处理从通讯接口接收到的任何信息。

- 执行 CPU 自诊断测试

CPU 自行检查硬件、用户程序存储器 (仅在 RUN 模式下) 和所有 I/O 模块的状态。

- 写输出

在每个扫描周期的结束阶段, CPU 把存储在输出映像寄存器的数据输出给输出点。

15.1.3 CPU 的工作方式

CPU 有 RUN 和 STOP 两种工作方式可供选择，如图 15-1-3 所示。具体设置由滑动开关来完成，详细说明见表 15-1-1 所示。

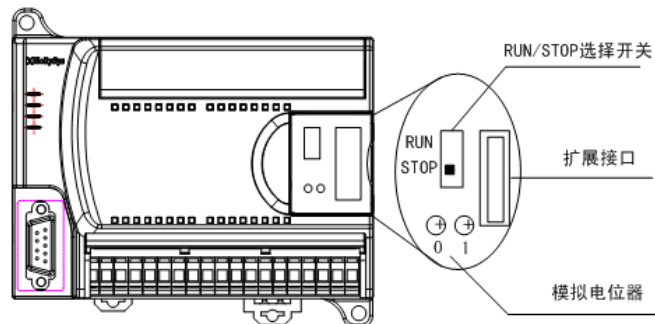


图15-1-3 PLC扫描工作方式

表 15-1-1 滑动开关说明

开关状态	说明
RUN	CPU 处于运行方式，执行用户程序。
STOP	CPU 不执行用户程序，此时用户可以向 CPU 装载程序或配置 CPU。

注意，如果在 RUN 方式下向 CPU 模块装载程序的过程不能完成，则必须把 CPU 模块置于 STOP 方式，再进行装载。

15.1.4 PLC 的运行方式

程序装载后，PLC 可以有联机运行和脱机运行两种运行方式。

联机运行是指 PLC 与编程计算机在连接编程电缆状态下的运行方式。

脱机运行是指 PLC 与编程计算机在断开编程电缆状态下的运行方式。

注意，在断开编程电缆前，应当先将 PLC 断电，然后再断开编程电缆，PLC 再重新上电，实现脱机运行。

15.1.5 PowerPro 的工作方式

PowerPro 软件中的 PLC 程序有在线模式和仿真模式两种工作方式。

将 PLC 程序下装到 PLC 的 CPU 硬件模块中，在 PLC 中运行用户程序，称为在线模式。

在没有连接硬件模块的情况下，在本地计算机模拟运行用户程序，称为仿真模式。

15.1.6 PowerPro 存储区分配

当在目标设定中选择“HollySys CoDeSys SP for C16x”时，存储区主要分为用户程序区（28K）和用户数据区（8K）。用户数据区又分为输入区（256 字节）、输出区（256 字节）、中间存储区（1K 字节）、全局区（2K 字节）和掉电保持区（512 字节）。

当在目标设定中选择“HollySys CoDeSys SP for C16x Extend”时，存储区主要分为用户程序区（120K）和用户数据区（48K）。用户数据区又分为输入区（512 字节）、输出区（512 字节）、中间存储区（8K 字节）、全局区（24K 字节）和掉电保持区（6K 字节）。

➤ 输入区（I）

在每次扫描周期的开始，CPU 对输入点进行采样，并将采样值存于输入区中。可以按位、字节、字来存取输入区。

➤ 输出区（Q）

在每次扫描周期的结尾，CPU 将输出区的数值复制到物理输出点上。可以按位、字节、字来存取输出区。

➤ 中间存储区（M）寻址

可以使用 M 存储器作为控制继电器存储中间操作状态或其它控制信息。可以按位、字节、字或双字来存取 M 存储器。其中，MB300-MB799 具有掉电保持功能。

➤ 全局区（G）

整个工程的全局变量存放在全局区中。

➤ 掉电保持区（R）

数据如果定义为保留型（RETAIN），则当 CPU 模块掉电时，数据会保存，待重新上电后，会自动恢复。

15.1.7 PowerPro 数据存储格式

以 M 区为例，PowerPro 软件中数据存储格式如图 15-1-4 所示。

字节：

MSB(最高有效位) LSB(最低有效位)

7 0
%MB100

% MB100

字：

MSB LSB
15 0
%MW100

%MB101	%MB100
--------	--------

双字：

MSB LSB
31 0
%MD100

%MB103	%MB102	%MB101	%MB100
--------	--------	--------	--------

图15-1-4 数据存储格式

15.1.8 采样滤波

在控制系统中，采样信号有模拟信号和离散信号。由于系统在工作过程中难免受到外界因素的干扰，这些干扰使得 PLC 所采集到的信号出现不真实性，从而造成系统工作紊乱和错误。为了消除干扰，准确获得真实信号，需要对采样输入的有关信号进行滤波处理。

在可编程控制器中，离散信号的采样主要由系统完成。但系统几乎没有滤波功能，只要有信号出现，系统就将其输入到内存中，供控制程序使用。离散量的滤波是由软件来完成的。实际上，对离散量的滤波过程就是对输入信号的再确认过程。如果信号状态被再确认为真，控制程序就采纳。否则，如再确认为假，就将其舍弃。这样做，既能保证能消除外界干扰，又能不影响系统的响应。所谓滤波值是指模拟量输入设定个数的采样值和平均值。

15.1.9 中断

“中断”，顾名思义就是中断或打断某一工作过程，去处理一些与本工作过程无关，或间接有关的事情。处理完后，则继续原工作过程。例如篮球比赛，要求暂停相当于申请中断，裁判同意相当于响应中断，商量对策相当于中断处理，回到场地继续比赛相当于中断返回。

在控制系统中，为了提高系统的响应速度，或者对某些应用程序进行有时间要求的控制处理，常常要求在系统中对某些过程信号进行中断信息处理。

PLC 的中断信息有多种，可以分为系统内部中断和由用户所引起的中断等两类。系统内部中断是由系统来处理的，例如编程器、数据处理器、某些智能模块等等，都可能在某些特殊条件下向中央处理器提出中断申请，这些中断的服务和处理都是由系统内部的处理程序自动完成的，不需要用户做过多的工作。由用户所引起的中断来自于控制过程，或者来自于程序内部的定时功能，这样的中断一般由用户来设定，可以通过程序设计来设定中断服务程序的入口，编制对中断信息的处理程序，即中断服务程序。另外，指令系统中都提供了相应的控制指令，可以通过程序设计对中断进行控制，以满足不同种类的生产工艺或控制过程的要求。

15.2 PowerPro 输入提示功能

对于不熟悉标准编程语言的用户，PowerPro 软件提供了有效的输入提示功能，帮助用户完成标准函数、功能块、运算符、操作数以及变量等的输入。使用主菜单中的“编辑”/“提示输入”命令，或者使用快捷功能键 F2，可以打开帮助管理器窗口，如图 15-2-1 所示。根据当前位置和选中对象的不同，可以列出不同的输入选项。

ST Keywords: ST 语言关键字;

ST operator: ST 语言操作符;

IL operator: IL 语言操作符;

Local Variables: 局部变量;

Global Variables: 全局变量;

System Variables: 系统变量;

Declarations: 变量声明关键字;

Standard Types: 标准变量声明数据类型;

User defined Types: 用户自定义数据类型;

Standard Functions: 标准函数，来自函数库中的函数;

User defined Functions: 用户自定义函数，由用户编写生成的函数;

Standard Function Blocks: 标准功能块，来自函数库中的功能块;

User defined Function Blocks: 用户自定义功能块，由用户编写生成的功能块;

FBD Operators: PowerPro 支持的标准 IEC 运算符;

Standard Programs: 标准程序;

User defined Programs: 用户自定义程序，由用户编写生成的程序;

Conversion Operators: 转换运算符，用于不同类型变量之间的数据类型转换。

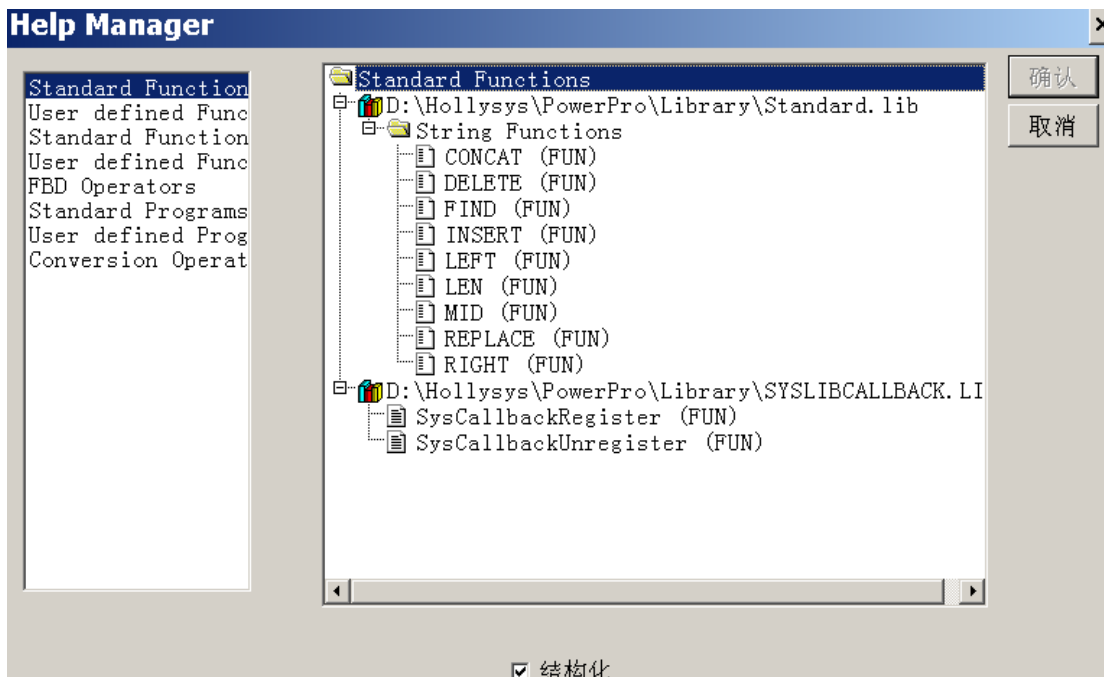


图15-2-1 输入提示帮助

15.3 PowerPro 文件存储目录

安装 PowerPro 软件时，系统在 D:\ 的根目录下生成一个默认的 Hollysys 文件夹，如图 15-3-1 所示。其中 PowerPro 子目录下的文件夹包括：

Documents 文件夹：存放《PowerPro 硬件手册》、《PowerPro 软件手册》和《PowerPro 指令与功能块手册》等电子文档。

Help 文件夹：存放英语、法语、德语等语言的帮助文件。

Library 文件夹：存放相关的库文件。

Project 文件夹：存放工程文件，其中 Compile 文件夹存放系统编译时产生的 LST 文件和 MAP 文件。

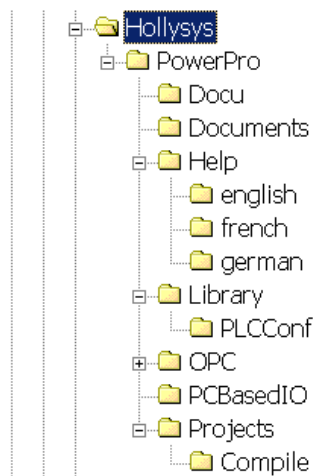


图15-3-1 Hollysys文件夹

在 C:\ 的根目录下生成一个默认的 TARGET 目标文件夹，如图 15-3-2 所示。TARGET 目标文件夹是在安装目标后自动生成的文件夹，软件卸载时不会被删除。PCBasedIO 文件夹是硬件配置文件夹。重新安装软件后，如果目标文件有变化，一定要重新安装目标文件 C16x_hollysys.tnf。



图15-3-2 TARGET目标文件夹

15.4 PowerPro 使用注意事项

- 正确安装中文 Windows2000 操作系统。系统设置在 1024x768 分辨率，小字体。
- 正确安装 MSXML3.0 及其补丁程序 MSXML3.0SP4。否则会导致一些不正常的运行情况发生，例如无法调用系统事件等。
- 如果要同时安装本公司的 SmartPro 和 PowerPro，必须首先安装 SmartPro，然后再安装 PowerPro。否则 PowerPro 中的 COM 口不能通讯。如果先前安装了 Smartpro，则无须卸载 Smartpro，可以直接安装 PowerPro。
- 在程序中，如果有与 RTS 上硬件相关的功能块时，程序不能使用仿真模式。例如系统事件等。与 RTS 上硬件相关的功能块包括定制库中的外部扩展功能块和外部功能块。对于详细功能块的应用，请参见《HOLLiAS-LEC G3 小型一体化 PLC 指令与功能块手册》。
- “上升沿使能”是指当功能块的使能端由低电平变为高电平并一直保持高电平时，执行功能块的相关功能。
- “上电/下装后首次使能有效”是指当使能端第一次由低电平变为高电平并一直保持高电平时，执行功能块的相关功能。以后再由低电平变为高电平将不会执行。如果要再次使用该功能块，需要重新下装或再次上电。
- 对于所有的功能块，只有当使能 EN 端、LOAD 端或 RSTLD 端在高电平时，才会执行相应功能。
- 使用 Analog_IN、Analog_OUT 和 DP_Slave 功能块时，Address 设置必须与 PLC 配置中相关模块的节点 id 一致，只有当 EN 输入使能后才启动功能块，进行相关的输入或输出。
- 使用 Set_COMM_PRMT 功能块设定 RS232 自由口参数后，要想恢复原编程系统的“下装/调试”功能，需将 RUN/STOP 开关拨到 STOP 位置，才可进行系统登录。
- 使用设置 Modbus 从站（SET_LOCAL_ADDRESS 功能块）时，如果所需通讯参数不是 38400bps、8 位、无校验，则需要使用 Reset_COMM_PRMT 功能块设置相应的通讯参数，然后再使用 SET_LOCAL_ADDRESS 功能块设置站地址。程序下装前需要将 RUN/STOP 开关拨到 STOP 位置，程序下装后再将 RUN/STOP 开关拨到 RUN 位置运行程序。
- 设置通信参数时，只有使用 Set_COMM_PRMT 功能块设定 RS232 自由口参数后，才可使用 COMM_RECEIVE 功能块进行数据接收，使用 COMM_SEND 功能块进行数据发送。
- 设置通信参数时，只有使用 Set_COMM2_PRMT 功能块设定 RS485 自由口参数后，才可使用 COMM2_RECEIVE 功能块进行数据接收，使用 COMM2_SEND 功能块进行数据发送。

- 存储区%MB0~%MB99 为系统使用，用户不能再使用，可以读取，但不能写入。
- 字 WORD 型变量(%MW)的地址必须定义在偶数地址，例如%MW200、%MW202、%MW204 等。
- 双字 DWORD 型变量(%MD)地址必须定义在偶数地址，例如%MD300、%MD304、%MD308 等。
- 在使用数学运算指令时，如果输出数据的定义范围小于运算结果，则高位部分会被删掉。
- 事件的调用需要使用 PowerPro2.0 以上的版本，通过任务配置中的 System events 调用相应的事件。关于事件调用的使用方法，请参考相应的程序实例。
- 对于部分功能块，因为使用了同一内部硬件，或者使用了相同的输入/输出端口，所以不可以同时使用。请参考《HOLLiAS-LEC G3 小型一体化 PLC 指令与功能块手册》附录中的 PowerPro 功能块关联冲突速查表。
- 如果功能块声明在 RETAIN 区（掉电保持区），该功能块的所有输入输出变量都要占用 RETAIN 内存区，所以建议不要在 RETAIN 内存区声明太多的功能块，以免超出 RETAIN 区的大小限制。
- 在梯形图（LD）编程环境下，使能运算符调用与功能块调用是两种不同的调用方式。其不同在于，如果采用使能运算符调用，当使能端低电平时，相应的指令或功能块不会执行。如果采用功能块调用，不论使能端低电平或高电平时，相应指令或功能块都会将使能端作为一个输入值来执行。
- 对于模拟量输入模块和模拟量输出模块，其参数设置正确与否将直接关系到整个工程运行正确与否。例如通道是否使能、通道参数值选取和通道输入类型选取等。
- 在“在线”菜单栏中，“输入值”和“强制值”均可以改变和输入变量的初始值。对于“输入值”，变量只被写入一次输入值，而且变量允许立刻被其它的程序赋值。对于“强制值”，变量在每一个循环之后被写入一次强制值，直到执行“解除强制”命令为止。
- 当用户改变了 CPU 模块的硬件配置（例如由非扩容 CPU 改为扩容 CPU）而程序不变时，只需将目标设置中的配置部分直接选成“Hollysys CoDeSys SP for C16x Extend”，再进行保存和编译即可。
- 对于全局变量而言，可以指定该变量定义在 I 区、Q 区、M 区、保留区，若无特殊指定，则默认定义在全局变量区。但全局变量不要与局部变量使用同一变量名，否则会由于变量名重名而引起混淆。

15.5 PowerPro 键盘命令

一般功能	
在程序的声明区和指令区之间移动	<F6>
在对象管理器、对象和消息窗口之间移动	<Alt>+<F6>
移动到下一个已经打开的编辑器窗口	<Ctrl>+<F6>
移动到上一个已经打开的编辑器窗口	<Ctrl>+<Shift>+<F6>
弹出右键菜单	<Shift>+<F10>
变量声明的快捷方式	<Ctrl>+<Enter>
从消息窗口返回到编辑器的初始位置	<Enter>
打开和关闭多层变量	<Enter>
打开或关闭文件夹	<Enter>
在对象管理器和库管理器的选项卡之间转换	<Arrow Keys>
移动到下一个字段	<Tab>
帮助	<F1>
一般命令	
"文件" / "打开"	<Ctrl>+<O>
"文件" / "保存"	<Ctrl>+<S>
"文件" / "打印"	<Ctrl>+<P>
"文件" / "退出"	<Alt>+<F4>
"编辑" / "恢复"	<Ctrl>+<Z>
"编辑" / "重复"	<Ctrl>+<Y>
"编辑" / "剪切"	<Ctrl>+<X>
"编辑" / "复制"	<Ctrl>+<C>
"编辑" / "粘贴"	<Ctrl>+<V>
"编辑" / "删除"	
"编辑" / "查找下一个"	<F3>
"编辑" / "替换"	<Ctrl>+<H>
"编辑" / "提示输入"	<F2>
"编辑" / "自动定义"	<Shift>+<F2>
"编辑" / "后错误"	<F4>
"编辑" / "前错误"	<Shift>+<F4>
"工程" / "全部编译"	<F11>
"工程" / "删除对象"	
"工程" / "添加"	<Ins>
"工程" / "重命名"	<Spacebar>
"工程" / "打开"	<Enter>

(续前表)

"在线" / "登录"	<Alt>+<F8>
"在线" / "退出"	<Ctrl>+<F8>
"在线" / "运行"	<F5>
"在线" / "停止"	<Shift>+<F8>
"在线" / "断点"	<F9>
"在线" / "跳出"	<F10>
"在线" / "跳进"	<F8>
"在线" / "单循环"	<Ctrl>+<F5>
"在线" / "输入值"	<Ctrl>+<F7>
"在线" / "强制值"	<F7>
"在线" / "解除强制"	<Shift>+<F7>
"在线" / "输入/强制对话框"	<Ctrl>+<Shift>+<F7>
"窗口" / "信息"	<Shift>+<Esc>
LD编辑器命令	
"插入" / "后节"	<Shift>+<T>
"插入" / "串联"	<Ctrl>+<O>
"插入" / "并联"	<Ctrl>+<R>
"插入" / "功能块"	<Ctrl>+
"插入" / "输出"	<Ctrl>+<L>
"其它" / "下方并联"	<Ctrl>+<U>
"其它" / "反向"	<Ctrl>+<N>
FBD编辑器命令	
"插入" / "后节"	<Shift>+<T>
"插入" / "输入"	<Ctrl>+<U>
"插入" / "运算符"	<Ctrl>+
"插入" / "赋值"	<Ctrl>+<A>
"插入" / "跳转"	<Ctrl>+<L>
"插入" / "返回"	<Ctrl>+<R>
"其它" / "反向"	<Ctrl>+<N>
"其它" / "放大"	<Alt>+<Enter>
SFC编辑器命令	
"插入" / "前步转移"	<Ctrl>+<T>
"插入" / "后步转移"	<Ctrl>+<E>
"插入" / "右选择分支"	<Ctrl>+<A>
"插入" / "右并行分支"	<Ctrl>+<L>
"插入" / "跳转"	<Ctrl>+<U>
"其它" / "编辑动作/转移"	<Alt>+<Enter>

15.6 扩展模块参数设置

15.6.1 4 通道热电偶输入模块 LM3311

LM3311 模块参数设置如图 15-6-1 所示。

基本参数		模块参数				
Index	Name	Value		Default	Min.	Max.
1	Filter_Parameter	1		1		
2	Cold_junction_Compensation	Yes		Yes		
3	Open_Wire_Detect_Direction	Holdi...		Holding		

图15-6-1 LM3311模块参数设置

➤ Filter_Parameter

“Filter_Parameter”即为“滤波参数”，其取值可为：1、2、4、8、16、32。

滤波参数与 RC 低通滤波的时间常数之间有对应关系，如表 4-4-1 所示。

➤ Cold_junction_Compensation

“Cold_junction_Compensation”即“该模块是否进行冷端补偿”，可以选择“Yes”（是）或“No”（否）。

➤ Open_Wire_Detect_Direction

开路故障检测方向：

Holding：当开路故障发生时，将会保持当前值。

Upscale：当开路故障发生时，将取正值极限。

Downscale：当开路故障发生时，将取负值极限。

LM3311 通道参数设置如图 15-6-2 所示。

基本参数		通道参数				
Index	Name	Value		Default	Min.	Max.
1	TC_Type	J		J		
2	Channel_Enable	Enable		Enable		

图 15-6-2 LM3311 通道参数设置

➤ TC_Type

选择“Value”值（J、K、T、N、E、R、S、B、-80~80mV），确定热电偶输入类型。

➤ Channel_Enable

选择通道使能，确定各个通道的运行情况。Enable 使能该通道。Disnable 禁用该通道。

当 TC_Type 选择 J、K、T、N、E、R、S、B 时，在使用某通道且 Channel_Enable 选择 Enable，但该通道未连入热电偶时（又称“断偶”），即超出量程范围，则该通道会显示-32768。当使能该通道时，该通道具备断偶检测功能，断偶时会设置相应状态值，同时该扩展模块 LM 3311 亮红灯。

当 TC_Type 选择-80~80mV 时，在使用该通道且 Channel_Enable 选择 Enable，但该通道没有信号输入，则该通道会显示 8000。

如图 15-6-3 所示，前两个通道信号输入类型-80~80mV，后两个通道信号输入类型为 J、K、T、N、E、R、S、B 中的输入类型。

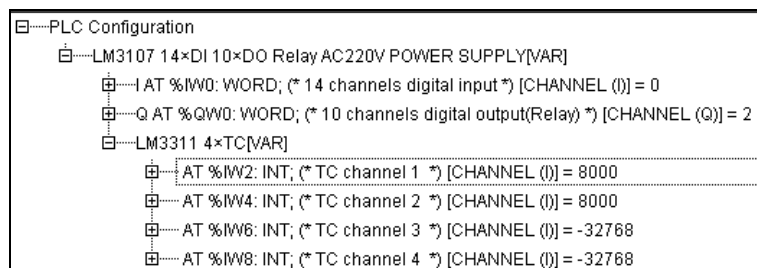


图 15-6-3 PLC 配置

另外，对于断偶、AD 芯片工作不正常或 24V 掉电，均会导致 ERROR 灯亮。有关 LM3311 模块的具体技术规格请参见《HOLLiAS-LEC G3 系列小型一体化 PLC 硬件手册》。表 15-6-1 给出了输入信号量程与输入数据范围的对应关系。

表 15-6-1 LM3311 输入信号的量程与输入数据范围的对应关系

（注：输入数据与模拟信号为非线性对应关系）

输入信号	温度范围（℃）	量程所对应的输入数据范围
J	-210~1200	-2100~12000
K	-270~1370	-2700~13700
E	-270~1000	-2700~10000
N	-270~1300	-2700~13000
T	-270~400	-2700~4000
R	-50~1768	-500~17680
S	-50~1768	-500~17680
B	0~1820	0~18200
±80mV	-	-8000~8000

15.6.2 4 通道热电阻输入模块 LM3312

LM3312 模块参数设置如图 15-6-4 所示。

Index	Name	Value	Default	Min.	Max.
1	Filter_Parameter	1	1		
2	Short_Circuit_Detect_Direction	Hold...	Holding		

图 15-6-4 LM3312 模块参数设置

➤ Filter_Parameter

“Filter_Parameter”即为“滤波参数”，其取值可为：1、2、4、8、16、32。

滤波参数与 RC 低通滤波的时间常数之间有对应关系，如表 4-4-1 所示。

➤ Short_Circuit_Detect_Direction

短路故障检测方向：

Holding：当短路故障发生时，将会保持当前值。

Upscale：当短路故障发生时，将取正值极限。

Downscale：当短路故障发生时，将取负值极限。

LM3312 通道参数设置如图 15-6-5 所示。

Index	Name	Value	Default	Min.	Max.
1	RTD_Type	Cu50 -50~140.1℃	Cu50 -50~140.1℃		
2	Channel_Enable	Enable	Enable		

图 15-6-5 LM3312 通道参数设置

➤ RTD_Type

可以根据实际需要，在“Value”中选择 Cu50（-50-140.1℃）、Cu50（-50-150℃）、Pt100（-150-157.2℃）或 Pt100（-150-619.6℃）等四种类型。

➤ Channel_Enable

选择通道使能，确定各个通道的运行情况。Enable 使能该通道。Disnable 禁用该通道。

在使用某通道且 Channel_Enable 选择 Enable 时，如果该通道连入热电阻<39Ω（即“短路”），超出量程范围，则该通道会出显示 32768。

有关 LM3312 模块的具体技术规格请参见《HOLLiAS-LEC G3 系列小型一体化 PLC 硬件手册》。表 15-6-2 给出了输入信号量程与输入数据范围的对应关系。

表 15-6-2 LM3312 输入信号的量程与输入数据范围的对应关系

输入信号范围	量程所对应的输入数据范围
Cu50（-50-140.1℃）	-500~1401
Cu50（-50-150℃）	-500~1500
Pt100(-150-157.2℃)	-1500~1572
Pt100(-150-619.6℃)	-1500~6196

15.6.3 8 通道模拟量输入模块 LM3313

LM3313 模块参数设置如图 15-6-6 所示。

基本参数 模块参数					
Index	Name	Value	Default	Min.	Max.
1	Filter_Factor	1	1		
2	Deadband	0	0	0	4080

图 15-6-6 LM3313 模块参数设置

➤ Filter_Factor
“Filter_Parameter”即为“滤波参数”，其取值可为：1、2、4、8、16、32。
滤波参数与 RC 低通滤波的时间常数之间有对应关系，如表 4-4-1 所示。

➤ Deadband
Deadband 表示用户设定的死区值，数值范围为 0-4080。在启动滤波功能时，当模拟量当前采集结果和上次滤波后的值之差超过设定死区值时，LM3313 直接输出当前采集结果；否则输出经过滤波后的转换结果。默认值=0 表示禁用死区参数。

LM3313 通道参数设置如图 15-6-7 所示。

基本参数 通道参数					
Index	Name	Value	Default	Min.	Max.
1	Channel_Enable	Enable	Enable		

图 15-6-7 LM3313 通道参数设置

➤ Channel_Enable
选择通道使能，确定各个通道的运行情况。Enable 使能该通道。Disnable 禁用该通道。
有关 LM3313 模块的具体技术规格请参见《HOLLiAS-LEC G3 系列小型一体化 PLC 硬件手册》。表 15-6-3 给出了输入信号的量程与输入数据范围的对应关系。

表 15-6-3 LM3313 输入信号的量程与输入数据范围的对应关系

输入信号范围	量程所对应的输入数据范围
-20mA~20mA	-32000~32000
-10V~10V	-32000~32000

15.6.4 8 通道热敏电阻输入模块 LM3314

LM3314 模块参数设置如图 15-6-8 所示。

基本参数

模块参数

Index	Name	Value	Default	Min.	Max.
1	B_Value	3950	3950	3000	4000
2	Filter_Parameter	1	1		
3	Open_Wire_Detect_Direction	Holding	Holding		

图 15-6-8 LM3314 模块参数设置

- B_Value
B 值：最小取值 3000，最大取值 4000，默认值：3950。
- Filter_Parameter
“Filter_Parameter”即为“滤波参数”，其取值可为：1、2、4、8、16、32。
滤波参数与 RC 低通滤波的时间常数之间有对应关系，如表 4-4-1 所示。
- Open_Wire_Detect_Direction
开路故障检测方向：
Holding：当开路故障发生时，将会保持当前值。
Upscale：当开路故障发生时，将取正值极限。
Downscale：当开路故障发生时，将取负值极限。

LM3314 通道参数设置如图 15-6-9 所示：

基本参数

通道参数

Index	Name	Value	Default	Min.	Max.
1	Channel_Enable	Enable	Enable		

图 15-6-9 LM3314 通道参数设置

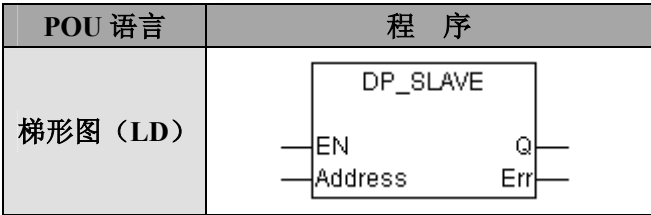
- Channel_Enable
选择通道使能，确定各个通道的运行情况。Enable 使能该通道。Disnable 禁用该通道。
在使用某通道且 Channel_Enable 选择 Enable 时，如果该通道连入热电阻<600Ω，（即“短路”），超出量程范围则该通道会出显示 32768。
在使用某通道且 Channel_Enable 选择 Enable 时，如果该通道连入热电阻>100KΩ，（即“断路”），超出量程范围则该通道会出显示-32768。
有关 LM3314 模块的具体技术规格请参见《HOLLIAS-LEC G3 系列小型一体化 PLC 硬件手册》。表 15-6-4 给出了输入信号的量程与输入数据范围的对应关系。
- 表 15-6-4 LM3314 输入信号的量程与输入数据范围的对应关系

测量温度信号范围	量程所对应的输入数据范围
-20℃～100℃	-200～1000

15.6.5 Profibus-DP 从站模块 LM3401

在进行参数设置之前，先介绍 DP_Slave 功能块及其应用。

➤ 功能块示例



➤ 输入输出说明

CPU 模块输入输出数据存放地址			
%IWxx %QWxx	PLC 配置中规定的 IW 和 QW 区地址		
输入	功能	数据类型	值
EN	使能	BOOL	0: 无效，不对 DP 模块进行扫描 1: 使能，对 DP 模块进行扫描
Address	该模块节点号	BYTE	0—7（与 PLC 配置里的节点号一致）
输出	功能	数据类型	值
Q	是否读到有效数据	BOOL	0: 未读到数据 1: 读到有效数据
Err	CPU 模块与从站模块通讯故障号	BYTE	0: 无故障 1: 等待 DP 从站模块 Ready 错 2: 写“读数据命令”错 3: 读应答错 4: 读数据错 5: 写数据错 6: 写数据命令错 7: 写 reg1=0 错 8: 写 reg1=0xFF 错 (0xFF: 硬件模块与所调功能块不匹配)

➤ DP_Slave 功能块举例（变量定义与梯形图）

DP_Slave 功能块的程序如图 15-6-10 所示。其中 Address 处的输入值 0 应与图 15-6-11 所示的 PLC 配置表中的节点号 0 一致。

当 EN 置位时，对 DP 模块进行扫描。当 EN 复位时，不对 DP 模块进行扫描。

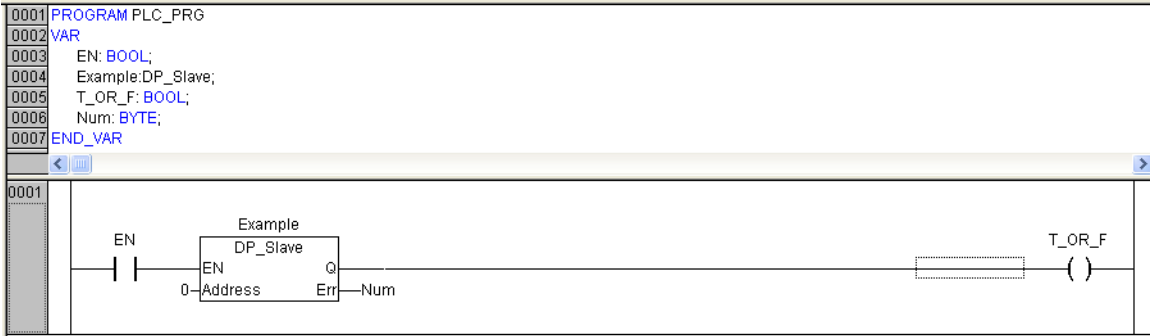


图 15-6-10 DP_Slave 功能块

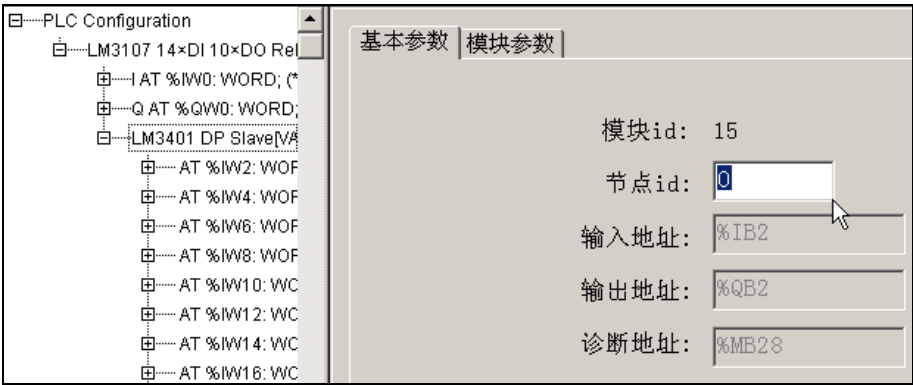


图 15-6-11 DP_Slave 配置表

LM3401 模块参数设置如图 15-6-12 所示。其中 Value 值可选 0-64。在图 15-6-12 中，Value 值为 64。有关 LM3401 模块的具体技术规格请参见《HOLLiAS-LEC G3 系列小型一体化 PLC 硬件手册》。

基本参数		模块参数			
Index	Name	Value	Def...	Min.	Max.
1	InputDataLen_Byte	64	0	0	64
2	OutputDataLen_Byte	64	0	0	64

图 15-6-12 LM3401 模块参数设置

下面作一个简单的 Profibus-DP 功能块的应用实例。

程序要求：PLC 通过 DP 从模块向 DP 主模块发送 8 个字节的数据，同时从 DP 主模块接收 8 个字节的数据。

变量定义：

```
PROGRAM PLC_PRG
VAR
    EN: BOOL;
    Example: DP_Slave;
    T_OR_F: BOOL;
    SendDataA: WORD;  PLC 向 DP 主站发送的数据 A
    SendDataB: WORD;  PLC 向 DP 主站发送的数据 B
    SendDataC: WORD;  PLC 向 DP 主站发送的数据 C
    SendDataD: WORD;  PLC 向 DP 主站发送的数据 D
    RecDataA: WORD;    DP 主站向 PLC 发送的数据 A
    RecDataB: WORD;    DP 主站向 PLC 发送的数据 B
    RecDataC: WORD;    DP 主站向 PLC 发送的数据 C
    RecDataD: WORD;    DP 主站向 PLC 发送的数据 D
END_VAR
```

软件配置：

配置 DP 从站模块 LM3401，如图 15-6-13 所示。
InputDataLen_Byte 为 DP 主站向 PLC 发送的数据长度，输入接收的字节数 8。
OutputDataLen_Byte 为 PLC 向 DP 主站发送的数据长度，输入发送的字节数 8。

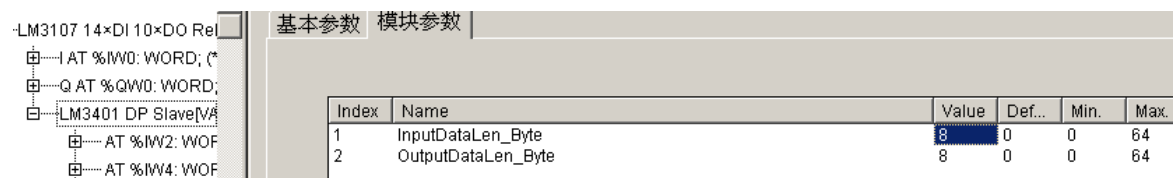


图 15-6-13 LM3401 模块参数设置

DP_Slave 模块中的 Address 与图 15-6-14 所示的节点号一致，DP 主站向 PLC 发送的数据 A、B、C、D 分别存放在图 15-6-14 所示的%IW2、%IW4、%IW6、%IW8 之中。



图 15-6-14 LM3401 配置表

PLC 向 DP 主站发送的数据 A、B、C、D 分别存放在图 15-6-15 所示的%QW2、%QW4、%QW6、%QW8 之中。

```
AT %IW64: WORD; (* channel 32 *) [CHANNEL (I)]
AT %QW2: WORD; (* channel 33 *) [CHANNEL (Q)]
AT %QW4: WORD; (* channel 34 *) [CHANNEL (Q)]
AT %QW6: WORD; (* channel 35 *) [CHANNEL (Q)]
AT %QW8: WORD; (* channel 36 *) [CHANNEL (Q)]
AT %QW10: WORD; (* channel 37 *) [CHANNEL (Q)]
```

图 15-6-15 LM3401 通道

配置 DP 主站的接收和发送区，DP 从站的 QW 区数据会自动传送到 DP 主站的接收区，DP 主站的发送区数据会自动传送到 DP 从站的 IW 区。梯形图如图 15-6-16 所示。

系统上电后，程序将 SendDataA、SendDataB、SendDataC、SendDataD 中的数据复制到%QW2、%QW4、%QW6、%QW8 中，%QW2、%QW4、%QW6、%QW8 中的数据会自动发送到 DP 主站的接收区。

DP 主站发送的数据自动存放在%IW2、%IW4、%IW6、%IW8 中，程序将%IW2、%IW4、%IW6、%IW8 中的数据分别复制到 RecDataA、RecDataB、RecDataC、RecDataD 中。

另外，DP 从站模块 LM3401 在配置过程中一定要临近 CPU 模块安装，作为第一个扩展模块。否则，LM3401 配置位置过后，会影响 DP 从站正常工作，甚至导致 DP 从站不工作。

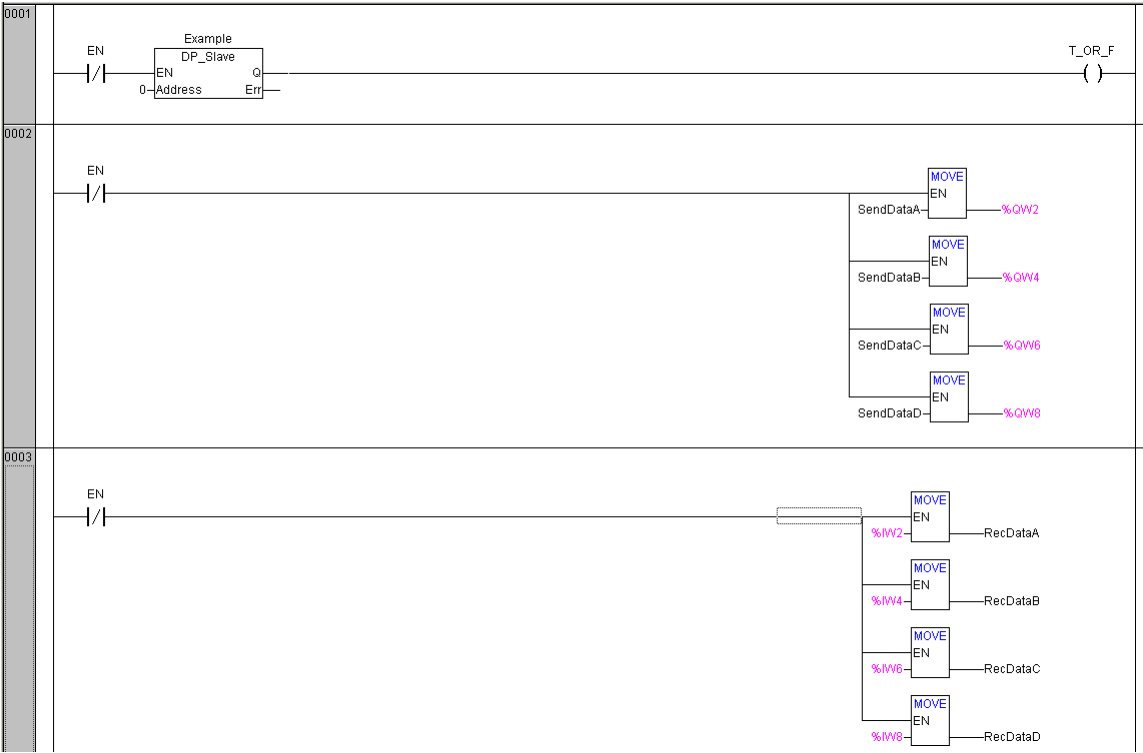


图 15-6-16 LM3401 梯形图

主站组态时，可以从和利时公司“网站资料下载\PLC 资料下载”中获取 HSLM3401.gsd 文件。也可以从软件安装默认路径“d: \Hollysys \PowerPro \PCBasedIO \HSLM3401.gsd”获取，如图 15-6-17 所示。只有在软件 PowerPro v2.0 对象组织器的“资源”选项卡 PLC 配置中添加 LM3401，才可以用此方式获取 HSLM3401.gsd。

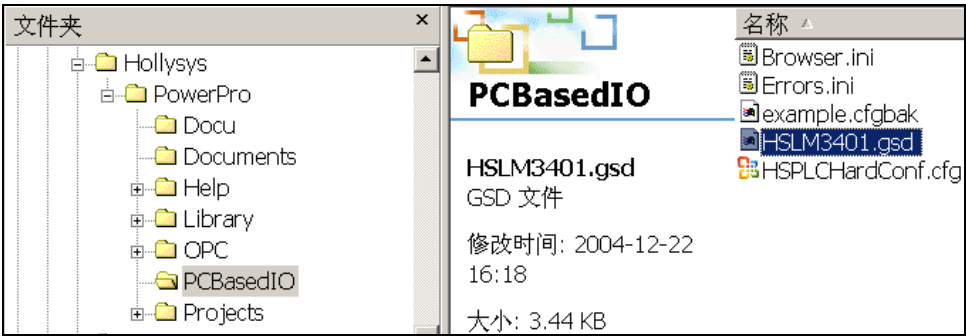


图 15-6-17 HSLM3401.gsd 文件

15.6.6 以太网模块 LM3403

在进行参数设置之前，先介绍 ETHERNET_TCP 功能块及其应用。

➤ 功能块示例

POU 语言	程 序
梯形图（LD）	

➤ 输入输出说明

CPU 模块输入输出数据存放地址			
%IWxx %QWxx	PLC 配置中规定的 IW 和 QW 区地址		
输入	功能	数据类型	值
EN	使能	BOOL	0: 无效，不对以太网模块进行扫描 1: 使能，对以太网模块进行扫描
Address	该模块节点号	BYTE	0—7（与 PLC 配置里的节点号一致）
输出	功能	数据类型	值
Q	是否读到有效数据	BOOL	0: 未读到数据 1: 读到有效数据
Err	CPU 模块与以太网模块 通讯故障号	BYTE	0: 无故障 1: 等待以太网模块 Ready 错 2: 写“读数据命令”错 3: 读应答错 4: 读数据错 5: 写数据错 6: 写数据命令错 7: 写 reg1=0 错 8: 写 reg1=0xFF 错 (0xFF: 硬件模块与所调功能块不匹配)

EtherNet_TCP 功能块举例。

变量定义与梯形图如图 15-6-18 所示。其中 Address 处的输入值 0 应与图 15-6-19 所示的 PLC 配置表中节点 id 号 0 一致。在图 15-6-20 中，分别配置 IP 地址 IP_Address、子网掩码 Subnet_Mask、网关 Gateway_Address、输入区大小 WriteDataLen_Byte 和输出区大小 ReadDataLen_Byte。无须设置 MAC_Address 参数。

有关以太网模块 LM3403 的具体技术规格请参见《HOLLIAS-LEC G3 系列小型一体化 PLC 硬件手册》。

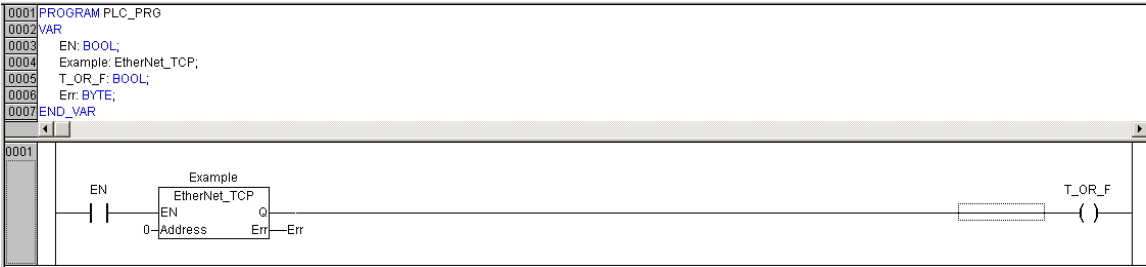


图 15-6-18 EtherNet_TCP 梯形图

PLC Configuration

LM3107 14×DI 10×DO Relay AC

I AT %IW0: WORD; (* 14 ch

Q AT %QW0: WORD; (* 10

LM3403 EtherNet[VAR]

AT %IW2: WORD; (*

AT %IW4: WORD; (*

AT %IW6: WORD; (*

AT %IW8: WORD; (*

AT %IW10: WORD; (

AT %IW12: WORD; (

AT %IW14: WORD; (

AT %IW16: WORD; (

基本参数

模块参数

模块id: 19

节点id: 0

输入地址: %IB2

输出地址: %QB2

诊断地址: %MB28

图 15-6-19 EtherNet_TCP 配置表

基本参数		模块参数				
Index	Name	Value	Defa...	Min.	Max.	
1	IP_Address	172.20.45.160				
2	Subnet_Mask	255.255.252.0				
3	Gateway_Address	172.20.45.1				
4	MAC_Address					
5	ReadDataLen_Byte	200	0	0	200	
6	WriteDataLen_Byte	200	0	0	200	

图 15-6-20 LM3403 模块参数设置

- 264 -

15.7 常见问题解答

下面列举在系统开发过程中常见的一些问题，并给出解答。

- 安装过程中会遇到哪些问题？

答：如果所使用的电脑从未安装过 MSXML3.0，而在进入 MSXML3.0 安装向导时，如果选择 Cancel，则会导致无法调用系统事件。

- 卸载时，如果不退出 gateway.exe，对安装 PowerPro 会不会产生影响？

答：卸载时，如果不退出 gateway.exe，会使与 gateway 相关的文件不能被删除，从而导致重新安装时，会弹出“文件不能被覆盖”等提示，安装失败。如果未退出 gateway，需要重启系统。

- 在输入变量时，如应输入%IX0.0 而误输入%I0.0 会出现什么情况？

答：在输入变量时，如应输入%IX0.0，而误输入%I0.0，则会弹出如图 15-7-1 所示的对话框。此时，应选择忽略。否则，会退出程序，而使程序终止。

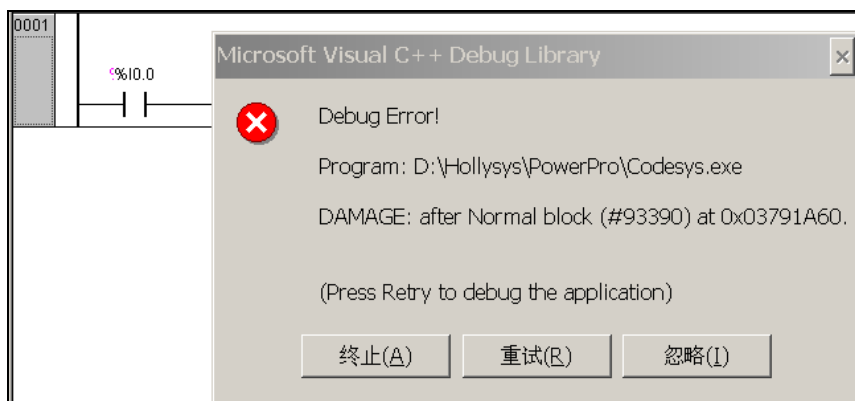


图 15-7-1 错误提示

- 什么情况会导致程序无法下装和通讯不正常？

答：未选择串口通讯、错误选择 COM 口或波特率设置不正确均会导致无法正常通讯，程序无法下装到 PLC。

- 在调试 PLC 程序时，有没有模拟器之类的东西，以降低调试成本？

答：在调试 PLC 程序时，使用“仿真模式”即可进行离线调试，以降低调试成本。

- 什么是 WAGO 接线技术？

答：WAGO 接线技术是德国 WAGO 公司的一项专利技术，它采用独特的弹簧压力技术，卡接方式接线，具有接线方便和牢固可靠等特点。采用 WAGO 接线技术不仅节省了大量的接线时间，更为用户提供了一种安全放心的接线方式。

➤ 如何选择小型一体化 PLC 的 CPU 模块？

答：要分以下几步来考虑：

计算系统所需的 IO 点数，并且必须考虑系统的扩展要求；

根据电源类型和负载不同，选择相应的 CPU；

根据系统的特殊需求选择型号，例如高速脉冲输出、存储区大小等等。

➤ HOLLiAS-LEC G3 系列 PLC 有多少种 CPU 类型？它们之间有什么区别？

答：有 3 种类型 7 款产品，分别为自带 14、24、40 点 I/O 三种类型。每种类型又包括晶体管和继电器两种输出方式，如表 15-7-1 所示。其中 LM3106A 与 LM3106 的区别是，LM3106A 是专用的运动控制模块，具备 100MHz 高速脉冲输出，而 LM3106 只具备 20MHz 高速脉冲输出。

表 15-7-1 HOLLiAS-LEC G3 系列 PLC 的 CPU 模块

基本性能	LM3104	LM3105	LM3106 LM3106A	LM3107	LM3108	LM3109
供电电压	DC24V	AC220V	DC24V	AC220V	DC24V	AC220V
DI	8×DC24V	8×DC24V	14×DC24V	14×DC24V	24×DC24V	24×DC24V
DO	6×晶体管	6×继电器	10×晶体管	10×继电器	16×晶体管	16×继电器
程序存储空间	28K 字节		28K 字节/120K 字节		120K 字节	
输入存储区	256 字节		256 字节		512 字节	
输出存储区	256 字节		256 字节		512 字节	
中间存储区	4K 字节		4K 字节/8K 字节		8K 字节	
全局区	12K 字节		12K 字节/24K 字节		24K 字节	
掉电保持区	无		4K 字节/6K 字节		6K 字节	
实时时钟	无		有			
实时时钟 掉电保持时间	无		10 天			
物理接口	RS232(非隔离)				RS232(非隔离) RS485(非隔离)	
中断输入 (上升下降沿可选)	2 点		3 点			
脉冲捕捉输入 (上升下降沿可选)	2 点		3 点			
高速计数器	单相高速计数器：3 点、100KHz					
	两相高速计数器：2 点、100KHz，支持加减双计数脉冲和 90 度正交计数脉冲					
高速输出 (仅限晶体管型)	输出点数：1 点		输出点数：2 点			
	输出类型：脉冲串输出 PTO、脉宽调制输出 PWM					
	输出频率：20KHz（LM3106A 为 100K）					
最多扩展	2 个模块		4 个模块		7 个模块	

- 系统中程序容量是多少？有掉电保持区吗？

答：程序容量根据 CPU 模块型号的不同而不同，LM3104 和 LM3105 没有掉电保持区，其他型号有掉电保持区。不同型号 CPU 模块的程序容量见表 15-7-2 所示。

表 15-7-2 不同型号 CPU 模块的程序容量

基本性能	LM3104	LM3105	LM3106	LM3107	LM3108	LM3109
程序存储空间	28K 字节		28K 字节/120K 字节		120K 字节	
输入存储区	256 字节		256 字节		512 字节	
输出存储区	256 字节		256 字节		512 字节	
中间存储区	4K 字节		4K 字节/8K 字节		8K 字节	
全局区	12K 字节		12K 字节/24K 字节		24K 字节	
掉电保持区	无		4K 字节/6K 字节		6K 字节	

- PLC 需要更换电池吗？多长时间，怎么更换？

答：PLC 不需要更换电池，因为 PLC 的程序区和掉电保持区采用不需要后备电源的 FLASH 硬件，保证 10 年以上数据不丢失。

- 能不能用 PLC 来监控以前的单片机设备？如何与这些旧设备通讯或者联网？

答：如果单片机支持 Modbus RTU 协议，则可以方便地与其进行通信。如果单片机采用专有协议，则可以通过自由口编程进行通信。HOLLiAS-LEC G3 系列 PLC 支持 RS232 与 RS485 的自由口通信，方便用户按需采用。

- 有一个客户正在使用贵公司的 DCS 系统，现在要求我方的设备必须能与此 DCS 系统连接，使用贵公司的 PLC 应该没问题吧？

答：当然没有问题。和利时公司的 DCS 系统可通过 Profibus-DP 相互进行网络通讯。G3 系列 PLC 产品有一款专用的 Profibus-DP 从站扩展模块 LM3401，不仅可以与和利时公司的 DCS 产品通讯，同时，只要是符合 Profibus-DP 协议标准的系统都可以连接。

- 从 Modbus 原始协议看，开关量所在的存储区从 H10000 开始编址，模拟量所在的存储区从 H30000 开始编址，这种理解是否正确？

答：不正确。G3 系列 PLC 产品的地址分为 I 区（输入区）、Q 区（输出区）和 M 区（中间存储区）。下面所介绍的编址计算以十进制数表示，在编程时请对应 16 进制数。

读写 I 区和 Q 区的开关量地址 = (字节地址 × 8 + 位地址) + 1，其中字节地址为 PowerPro 中相应于 %Q (I) Xm.n 中的 m，位地址为 PowerPro 中相应于 %Q (I) Xm.n 中的 n。例如，显示地址为 %QX1.1 通道的状态，则该开关量地址为 1(m) × 8 + 1(n) + 1 = 10 (m=1, n=1)。

读写 I 区和 Q 区的模拟量地址 (字) = PowerPro 中相应通道的实际地址除以 2，然后再加 1。例如，读写地址为 %QW2，则为实际地址/2 (转化为字偏移) + 1 (补偿偏移)，即 2。

读写 M 区的地址与读写 I 区和 Q 区的地址相似，采用上面方法得到地址后，在最前面加上 3000 即可。

- 称重传感器通过变送器送过来的标准信号,可以是 10V 电压信号,也可以是 20mA 电流信号,PLC 对此有没有要求? 这个信号的值在 PLC 寄存器中是如何反映的?

答: 4 通道模拟量输入模块 LM3310 可以接收 0~10V 电压信号、0~20mA 电流信号和 4~20mA 电流信号。根据 LM3310 上的拨码开关可以选择需要接收上述哪一种信号。拨码开关与接收信号的对应关系如表 15-7-3 所示。

例如,量程范围 0~1000 公斤对应 0~10V,而 PLC 中的 0~10V 对应十进制数 0~65535,则量程范围 0~1000 公斤在 PLC 中就对应十进制数 0~65535。

表 15-7-3 LM3310 模块拨码开关与接收信号的对应关系

拨码开关设置	信号模式	量程范围	量程所对应的输入数据范围	
			十进制值	十六进制值
OFF	电压信号	0~10V	0~65535	0x00~0xFFFF
OFF	电流信号	0~20mA	0~65535	0x00~0xFFFF
ON	电流信号	4~20mA	0~65535	0x00~0xFFFF

- 生产设备与 CPU 模块 LM3107 之间的输入信号的信号线长度有没有限制? 这个长度决定了生产设备与 PLC 的距离能够有多远。

答: 有关 CPU 模块 LM3107 的输入参数如表 15-7-4 所示。无论生产设备与 PLC 的距离能够有多远,或者对 PLC 有多少干扰,当输入信号到达 PLC 时,只要满足上述条件即可。具体情况可以进行实际测试,因为现场条件各不相同,而且不同设备送入 PLC 的输入信号也不相同。

表 15-7-4 CPU 模块 LM3107 的输入参数

输入特性	
输入类型	漏型/源型
输入电压额定值	24V DC
允许范围	0V DC~30V DC
逻辑 1 信号	15~30V DC, 允许最小电流 3mA
逻辑 0 信号	0~5V DC, 允许最大电流 1mA
输入延迟时间	<0.6ms (额定输入电压)
隔离方式	光电隔离
隔离组	2 组
隔离耐压	1500V AC

- CPU 模块 LM3107 与上位机通信的串口的参数，诸如波特率、停止位、数据位和奇偶校验等，在设置时有没有特殊的要求？

答：CPU 模块 LM3107 与上位机通信，如果采用 RS232 口与 PowerPro 通信，选择 PowerPro 的默认值即可。如果作为 Modbus 从站与上位机通信，可以选择如下参数。

校验：偶校验、无校验、奇校验；

数据位：8 位、7 位；

波特率：38400bps、19200bps、9600bps、4800bps、2400bps、1200bps、600bps、300bps；

建议参数：波特率 38400bps；数据位 8 位；无校验。

- CPU 模块 LM3106 与 LM3108 是否都可以通过自带的 RS232 编程口与上位机建立通信？编程电缆是否可以直接充当数据线？

答：CPU 模块 LM3106 与 LM3108 均可以通过自带的 RS232 编程口与上位机建立通信，编程电缆可以直接充当数据线。

- 当 PLC 作为从站时，怎样对贵公司的 PLC 设定从站地址号？

答：我公司的 PLC 支持 ModbusRTU 通信协议。通过调用 Set_Local_Address 功能块来设定从站地址号，该功能块位于 HOLLYSYS_PLC_EX.lib 库中。可以选择如下的通讯参数。

校验：奇校验、偶校验、无校验；

数据位：7 位、8 位；

波特率：300bps、600bps、1200bps、2400bps、4800bps、9600bps、19200bps、38400bps；

- 如果要将从站号为 01 的 %MX0.1 地址中的值置 1，应该发出怎样一个命令帧？请举例说明。

答：可以向 PLC 发送 16 进制数 0105 0BB9 FF00 5FFB，具体含义如下所示：

01	05	0BB9	FF	00	5FFB
站地址	功能码	站内地址	数据	开关原状态	CRC16 校验

- HOLLiAS-LEC G3 系列 PLC 产品的服务承诺是什么？

答：HOLLiAS-LEC G3 系列 PLC 产品的服务承诺：本着“真诚地为用户设想”的服务理念，我公司及下属 PLC 事业部始终站在用户的角度，考虑用户的利益。我公司可以为用户提供最完善的解决方案和最优质的系统配置。HOLLiAS-LEC G3 系列 PLC 产品一年保换，终身保修。在用户需要的时候，我公司的工程技术人员将在 24 小时之内到达，并提供高质量的服务。我公司的分支机构和服务网点遍布全国各大城市，随时满足用户需求。